Programación 3

Notas del curso

Manuel Sande

Facultad de Ingeniería — Universidad de la República

Segundo semestre, 2026

Índice general

1. Emparejamiento Estable **4**
- 1.1. Introducción . . . 4
- 1.2. Existencia de emparejamientos estables . . . 6
- 1.3. El algoritmo de Gale-Shapley . . . 7
- 1.4. Corrección del algoritmo . . . 10
- 1.5. Implementación eficiente . . . 12
- 1.6. Caracterización de la solución: optimalidad . . . 13
- 1.7. Extensiones del modelo . . . 15
- 1.8. Cinco problemas representativos . . . 16

2. Análisis de Algoritmos **24**
- 2.1. Tratabilidad computacional . . . 24
- 2.2. Orden asintótico de crecimiento . . . 26
- 2.3. Implementación con listas y arreglos . . . 34
- 2.4. Estudio de los tiempos de ejecución habituales . . . 38

3. Grafos **49**
- 3.1. Definiciones básicas y aplicaciones . . . 49
- 3.2. Conectividad y recorrido de grafos . . . 54
- 3.3. Implementación de los recorridos: colas y pilas . . . 61
- 3.4. Verificación de bipartición: una aplicación de BFS . . . 68
- 3.5. Conectividad en grafos dirigidos . . . 71
- 3.6. Grafos dirigidos acíclicos y orden topológico . . . 73

4. Algoritmos Greedy **79**
- 4.1. La estrategia greedy . . . 79
- 4.2. Planificación de intervalos: el greedy se mantiene adelante . . . 80
- 4.3. Minimizar la tardanza máxima: un argumento de intercambio . . . 88
- 4.4. Camino más corto en un grafo . . . 93
- 4.5. El árbol de cubrimiento mínimo . . . 97
- 4.6. Implementación de Kruskal: la estructura Union-Find . . . 104

5. Divide y Vencerás **111**
- 5.1. Una primera recurrencia: mergesort . . . 111
- 5.2. Otras relaciones de recurrencia . . . 115
- 5.3. Contar inversiones . . . 121

5.4. El par de puntos más cercano . . . 126
5.5. Multiplicación de enteros . . . 132

6. Programación Dinámica **137**
6.1. Planificación de intervalos con pesos: un procedimiento recursivo . . . 137
6.2. Principios de la programación dinámica: memorización o iteración sobre subproblemas . . . 144
6.3. Mínimos cuadrados segmentado: elección de múltiples vías . . . 148
6.4. Suma de subconjuntos y mochila: agregar una variable . . . 155
6.5. Estructura secundaria de ARN: programación dinámica sobre intervalos 163
6.6. Alineamiento de secuencias . . . 169
6.7. Caminos de costo mínimo en un grafo: el algoritmo de Bellman-Ford . . . 177
6.8. Resumen de la técnica . . . 185

7. Redes de Flujo **187**
7.1. El problema de flujo máximo y el algoritmo de Ford-Fulkerson . . . 187
7.2. Flujos máximos y cortes mínimos en una red . . . 201
7.3. Elección de buenos caminos de aumento . . . 207
7.4. Una primera aplicación: el problema de emparejamiento bipartito . . . 212
7.5. Caminos disjuntos en grafos dirigidos y no dirigidos . . . 220
7.6. Extensiones del problema de flujo máximo . . . 223
7.7. Diseño de encuestas . . . 229
7.8. Programación de vuelos . . . 231
7.9. Segmentación de imágenes . . . 234
7.10. Selección de proyectos . . . 236
7.11. Eliminación en béisbol . . . 238

8. Complejidad Computacional **242**
8.1. Reducciones de tiempo polinómico . . . 243
8.2. Reducciones mediante artilugios: el problema de satisfacibilidad . . . 253
8.3. Certificación eficiente y la clase $\mathcal{NP}$ . . . 259
8.4. Problemas $\mathcal{NP}$-completos . . . 264

Bibliografía **276**

Capítulo 1

Emparejamiento Estable

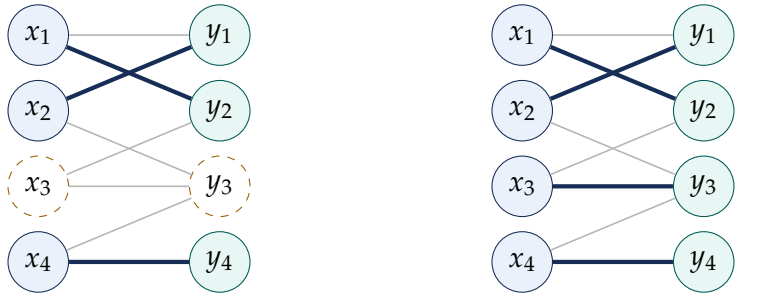
1.1 Introducción

El problema del emparejamiento estable aparece en cualquier situación en la que dos conjuntos disjuntos de agentes deben emparejarse de acuerdo con preferencias mutuas, de modo que el resultado sea resistente a desviaciones individuales. Un escenario clásico —y el que utilizaremos de manera recurrente— es la asignación de estudiantes de medicina a hospitales: cada hospital ordena a los estudiantes según su preferencia, y cada estudiante ordena a los hospitales. Se busca un proceso de admisión que se refuerce a sí mismo, es decir, que ninguna pareja hospital-estudiante tenga incentivo a abandonar el emparejamiento asignado.

Formalmente, consideremos dos conjuntos finitos disjuntos X e Y de igual cardinalidad $n \geq 1$. Cada elemento $x \in X$ posee una lista de preferencia total (orden lineal estricto) sobre Y, y cada elemento $y \in Y$ posee una lista de preferencia total sobre X. Las listas se asumen completas: todo elemento de un conjunto aparece exactamente una vez en la lista de cada elemento del otro.

Definición 1.1.1 (Emparejamiento)**.** Un emparejamiento M es un subconjunto de $X \times Y$ tal que ningún elemento de X aparece en más de un par de M y ningún elemento de Y aparece en más de un par de M. Equivalentemente, M es un conjunto de pares ordenados (x, y) en el que cada elemento de $X \cup Y$ interviene en a lo sumo un par.

Definición 1.1.2 (Emparejamiento perfecto)**.** Un emparejamiento M es perfecto si $|M| = n$, es decir, si todo elemento de X y todo elemento de Y aparecen exactamente en un par de M.



(a) emparejamiento: x_3 e y_3 quedan sin pareja

(b) emparejamiento perfecto: los $2n$ agentes están emparejados

Figura 1.1. Un emparejamiento es un conjunto de aristas sin extremos en común (en trazo grueso); las aristas finas representan los demás pares posibles. Es perfecto cuando cubre a todos los agentes de ambos lados, lo que exige $|M| = n$.

Ejemplo 1.1.3. Consideremos $n = 3$ con hospitales $H = \{A, B, C\}$ (Atlántida, Bella U., Canelones) y estudiantes $S = \{X, Y, Z\}$ (Xavier, Yolanda, Zeus). Las listas de preferencia son:

Hospital	1ª	2ª	3ª
A	X	Y	Z
B	Y	X	Z
C	X	Y	Z

Estudiante	1ª	2ª	3ª
X	B	A	C
Y	A	B	C
Z	A	B	C

El conjunto $M = \{(A, Z), (B, Y), (C, X)\}$ es un emparejamiento perfecto. Observemos, sin embargo, que A prefiere a Y sobre su pareja actual Z, y que Y prefiere a A sobre su pareja actual B. Esta situación es indeseable: A e Y podrían mejorar ambos abandonando a sus parejas y emparejándose entre sí.

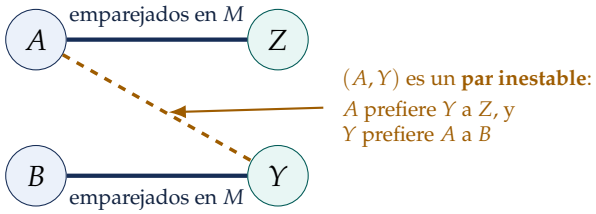


Figura 1.2. Una inestabilidad requiere que ambos agentes mejoren al abandonar sus parejas actuales. Si solo uno de los dos prefiere al otro, no hay incentivo conjunto para desviarse y el emparejamiento no se ve amenazado.

La observación del ejemplo anterior motiva la definición de par inestable.

Definición 1.1.4 (Par inestable)**.** Sea M un emparejamiento perfecto. Un par $(x, y) \notin M$ forma una inestabilidad (o se dice que es un par inestable) respecto de M si se cumplen simultáneamente las dos condiciones siguientes:

1. x prefiere a y sobre el elemento de Y con el que está emparejado en M;
2. y prefiere a x sobre el elemento de X con el que está emparejado en M.

Definición 1.1.5 (Emparejamiento estable)**.** Un emparejamiento perfecto M es estable si no admite ningún par inestable. El problema del emparejamiento estable consiste en, dadas las listas de preferencia de los elementos de X e Y, encontrar un emparejamiento estable (si existe).

Ejemplo 1.1.6. En la instancia del Ejemplo 1.1.3, el emparejamiento $M = \{(A, Z), (B, Y), (C, X)\}$ no es estable, pues (A, Y) es un par inestable. En cambio, el emparejamiento

$$M^* = \{(A, X), (B, Y), (C, Z)\}$$

sí es estable: puede verificarse exhaustivamente que ningún par fuera de M^* mejora mutuamente la situación de ambos agentes.

La noción de par inestable captura precisamente la posibilidad de que dos agentes (uno de cada conjunto) mejoren mutuamente su situación mediante una acción conjunta. Un emparejamiento estable es, por tanto, un estado de equilibrio: ningún par de agentes tiene incentivo a desviarse de la asignación.

1.2 Existencia de emparejamientos estables

Una pregunta natural es si, para toda colección de listas de preferencia, existe al menos un emparejamiento estable. La respuesta no es evidente a priori. Para ilustrar que la existencia no es trivial, consideremos primero una variante no bipartita del problema.

Ejemplo 1.2.1 (Problema del compañero de piso)**.** Sean $2n$ personas, cada una de las cuales ordena a las restantes $2n-1$ según su preferencia. Se desea emparejarlas en n parejas sin pares inestables (definidos análogamente). Consideremos $n=2$ con $\{A,B,C,D\}$ y preferencias:

Persona	1ª	2ª	3ª
A	B	C	D
B	C	A	D
C	A	B	D
D	A	B	C

Cualquier emparejamiento perfecto deja al menos un par inestable: $\{(A,B),(C,D)\}$ admite (B,C); $\{(A,C),(B,D)\}$ admite (A,B); y $\{(A,D),(B,C)\}$ admite (A,C). Por lo tanto, en este problema no siempre existe un emparejamiento estable.

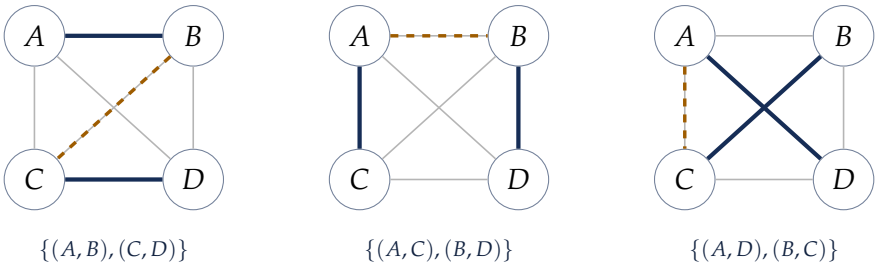


Figura 1.3. El problema del compañero de piso con $\{A,B,C,D\}$: los tres emparejamientos perfectos posibles admiten, cada uno, una inestabilidad. Sin bipartición, la existencia de un emparejamiento estable deja de estar garantizada.

El contraejemplo anterior muestra que la bipartición de los agentes es esencial para la existencia garantizada. En el modelo bipartito que nos ocupa, la existencia sí está asegurada, como se demostrará a continuación mediante la construcción explícita de un emparejamiento estable.

1.3 El algoritmo de Gale-Shapley

El algoritmo de Gale-Shapley (también llamado de aceptación diferida) proporciona un procedimiento constructivo que, para cualquier instancia del problema bipartito, produce un emparejamiento estable.

Proposición 1.3.1 (Algoritmo de Gale-Shapley)**.** El siguiente procedimiento termina y devuelve un emparejamiento estable. La versión que presentamos es aquella en la que los hospitales proponen (la versión simétrica, con los estudiantes proponiendo, es análoga).

GALE-SHAPLEY(H, S)

```
1  Inicializar M ← ∅
2  Marcar todo h ∈ H y s ∈ S como libre
3  while existe un hospital h libre que no se ha propuesto a todos do
4      s ← el estudiante preferido de h al que aún no se ha propuesto
5      if (s está libre) then
6          Añadir el par (h, s) a M
7      else if (s prefiere h a su pareja actual h′) then
8          Liberar h′
9          Eliminar (h′, s) de M
10         Añadir (h, s) a M
11     else
12         s rechaza h
13     end if
14 end while
15 return M
```

Estructuras de datos. Los hospitales y las estudiantes se numeran de 1 a n, lo que permite usar arreglos indexados por ellos. Las preferencias llegan en dos matrices $n \times n$, $HospPref[h, i]$ y $StudPref[s, i]$. Con eso:

- lista enlazada *Libres* con los hospitales que aún tienen plaza: tomar uno es leer la cabeza y sacarlo o insertarlo cuesta $O(1)$;
- arreglo $Next[h]$, inicializado en 1, con la posición de la próxima estudiante a la que h debe proponerse: se lee y se incrementa en $O(1)$;
- arreglo $Current[s]$ con el hospital actual de s, o null si está libre;
- arreglo inverso $Ranking[s, h]$ con la posición de h en la lista de s, precomputado en $O(n^2)$ al inicio: comparar dos hospitales pasa a ser comparar dos enteros.

Sin el arreglo inverso, la línea 4 tendría que recorrer la lista de preferencias de s y costaría $O(n)$, elevando el total a $O(n^3)$.

En cada iteración del `while` se elige el siguiente estudiante en la lista de preferencia de h (orden decreciente). El cuerpo del condicional implementa la regla de aceptación diferida: un estudiante solo cambia de pareja si el nuevo hospital le resulta estrictamente preferible. Existe una versión simétrica del algoritmo en la que proponen los estudiantes; ambas producen emparejamientos estables, aunque no necesariamente el mismo.

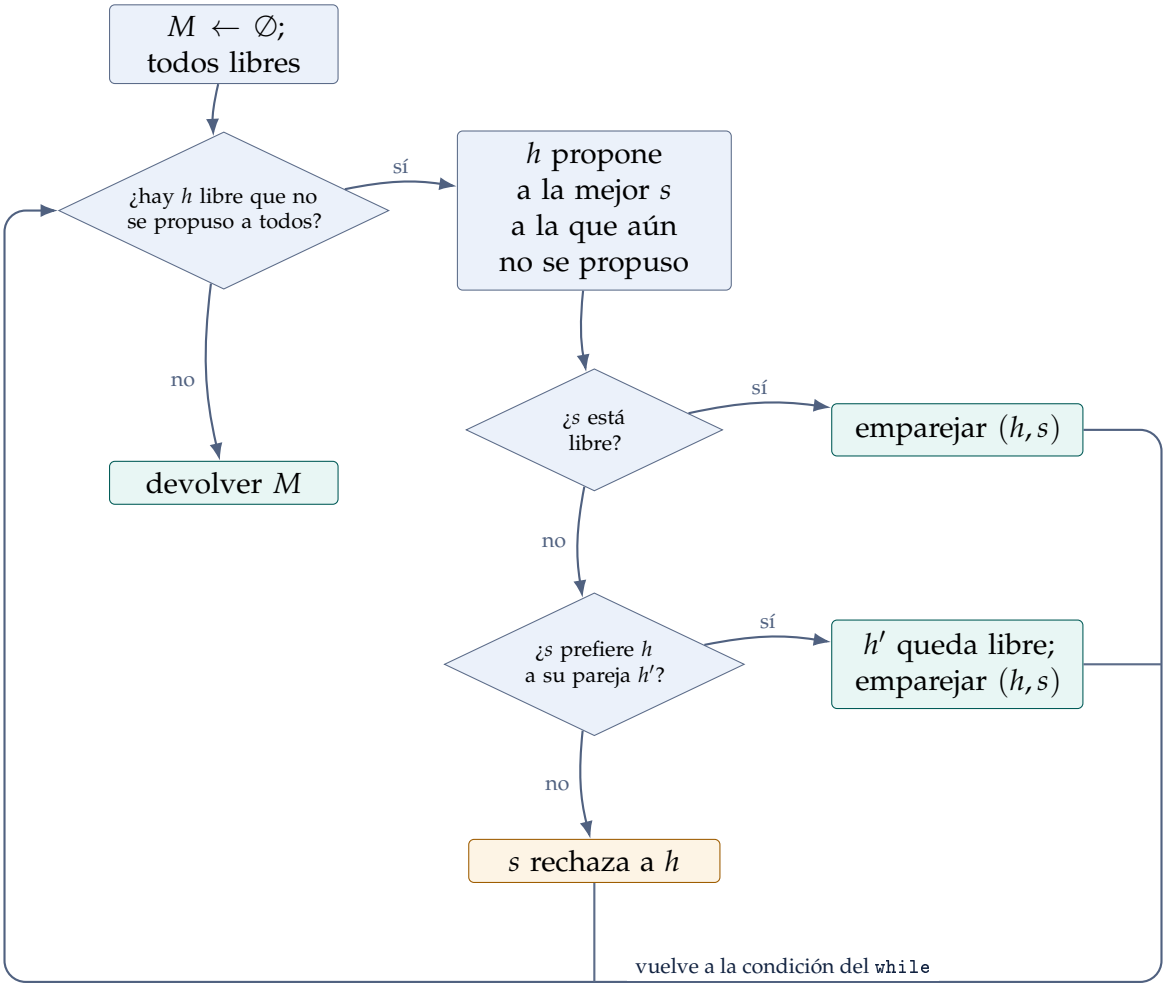


Figura 1.4. Estructura del algoritmo de Gale-Shapley. Cada vuelta del bucle consume exactamente una propuesta nueva, lo que acota el total en n^2; las tres salidas del cuerpo (aceptación, cambio de pareja y rechazo) regresan a la condición del `while`.

Ejemplo 1.3.2. Aplicamos el algoritmo a la instancia del Ejemplo 1.1.3, seleccionando en cada paso el hospital libre de menor índice según el orden A, B, C. La Figura 1.5 muestra la ejecución completa: los dos primeros hospitales obtienen su primera opción, mientras que C es rechazado sucesivamente por X y por Y —ambas ya comprometidas con hospitales que prefieren— antes de emparejarse con Z.

El algoritmo termina con el emparejamiento $M^* = \{(A, X), (B, Y), (C, Z)\}$, que es estable.

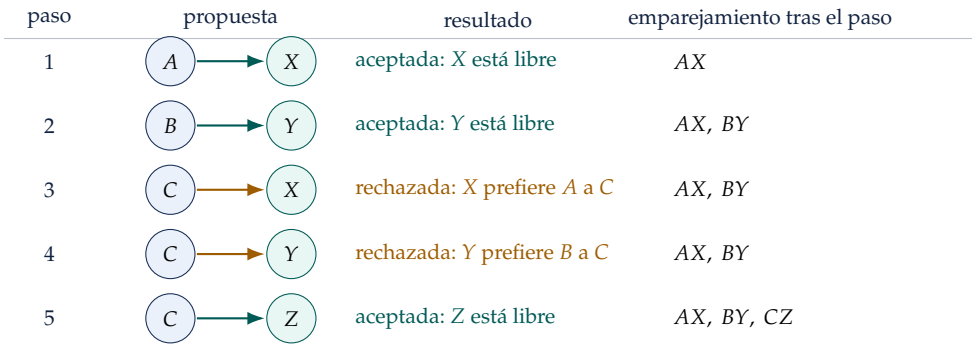


Figura 1.5. Ejecución del algoritmo sobre la instancia del Ejemplo 1.1.3 (proponen los hospitales). Un rechazo nunca deshace una pareja ya formada: solo hace que el hospital rechazado descienda una posición en su lista.

Ejemplo 1.3.3 (Otra instancia)**.** Consideremos $n = 2$ con hospitales $\{h_1, h_2\}$ y estudiantes $\{s_1, s_2\}$, y las siguientes preferencias:

	1^a	2^a
h_1	s_1	s_2
h_2	s_1	s_2
s_1	h_1	h_2
s_2	h_1	h_2

Si proponen los hospitales, h_1 se empareja con s_1 y h_2 con s_2. El emparejamiento $\{(h_1, s_1), (h_2, s_2)\}$ es el único estable: el otro emparejamiento perfecto $\{(h_1, s_2), (h_2, s_1)\}$ admite la inestabilidad (h_1, s_1).

1.4 Corrección del algoritmo

Proposición 1.4.1 (Terminación)**.** El algoritmo de Gale-Shapley termina tras a lo sumo n^2 iteraciones del bucle principal.

Demostración. Cada iteración del bucle consiste en una propuesta de un hospital a un estudiante al que aún no se había propuesto. Como hay n hospitales y n estudiantes, el número total de propuestas posibles es n^2. Por consiguiente el bucle no puede ejecutarse más de n^2 veces. □

Lema 1.4.2. Los hospitales proponen a los estudiantes en orden estrictamente decreciente de preferencia.

Lema 1.4.3. Una vez que un estudiante es emparejado, nunca vuelve a quedar libre; únicamente puede "subir de nivel", es decir, cambiar a un hospital que prefiere más.

Proposición 1.4.4 (Perfección)**.** Al terminar, el emparejamiento M producido por Gale-Shapley es perfecto.

Demostración. En primer lugar, M es un emparejamiento: un hospital solo propone (y por tanto solo puede emparejarse) cuando está libre, y un estudiante nunca acepta a más de un hospital a la vez.

Supongamos, por contradicción, que algún hospital h queda libre al final de la ejecución. Entonces h se ha propuesto a todos los estudiantes. Por el Lema 1.4.3, si algún estudiante hubiera recibido al menos una propuesta, permanecería emparejado. Por lo tanto ningún estudiante ha recibido propuesta alguna, lo cual contradice que h se haya propuesto a todos. En consecuencia todos los hospitales están emparejados. Como $|H| = |S| = n$, todos los estudiantes también están emparejados, y M es perfecto. □

Teorema 1.4.5 (Estabilidad)**.** El emparejamiento M producido por el algoritmo de Gale-Shapley es estable.

Demostración. Supongamos, por contradicción, que existe un par inestable (h, s) respecto de M. Sea s' el estudiante emparejado con h en M, y sea h' el hospital emparejado con s en M. Entonces se cumple:

- h prefiere s a s',
- s prefiere h a h'.

Como h prefiere s a s' y los hospitales proponen en orden decreciente de preferencia (Lema 1.4.2), h debió proponerse a s en algún momento anterior a proponerse a s'. Consideremos los dos casos posibles:

1. h nunca se propuso a s. Entonces, al proponerse a s', h aún no había alcanzado a s en su lista de preferencia, lo que contradice que s aparezca antes que s' en dicha lista.
2. s rechazó a h (o lo abandonó posteriormente) en favor de algún hospital h''. En el momento en que s se empareja definitivamente con h', o bien $h'' = h'$, o bien s prefiere h' a h'' (por el Lema 1.4.3). En cualquiera de los dos subcasos, s prefiere h' a h, contradiciendo la hipótesis de que (h, s) es inestable.

En ambos casos se obtiene una contradicción. Por lo tanto M no admite pares inestables y es estable. □

Corolario 1.4.6. Para toda instancia del problema de emparejamiento estable (listas de preferencia completas y estrictas sobre dos conjuntos de igual cardinalidad n) existe al menos un emparejamiento estable.

1.5 Implementación eficiente

Es posible implementar el algoritmo de Gale-Shapley de modo que su tiempo de ejecución sea $O(n^2)$.

Proposición 1.5.1. Existe una implementación del algoritmo de Gale-Shapley que termina en tiempo $O(n^2)$.

Demostración. Indexamos los elementos de H y de S con enteros de 1 a n. Se mantienen las siguientes estructuras:

- una cola (o pila) de hospitales libres;
- dos arreglos $estudiante[h]$ y $hospital[s]$ que almacenan la pareja actual (con valor 0 para indicar "libre");
- para cada hospital, la lista de estudiantes ordenada por preferencia y un puntero al siguiente estudiante al que proponer;
- para cada estudiante, la lista inversa de preferencias de hospitales: un arreglo $inverso_s$ tal que $inverso_s[h]$ es la posición de h en la lista de s. Esta estructura se construye en tiempo $O(n)$ por estudiante y permite responder en tiempo constante a la pregunta "¿prefiere s el hospital h al hospital h'?".

Cada propuesta se procesa en tiempo constante. Como hay a lo sumo n^2 propuestas, el tiempo total es $O(n^2)$. □

Ejemplo 1.5.2 (Lista inversa)**.** Sea la lista de preferencia de un estudiante s:

$$pref[1.,8] = (8, 3, 7, 1, 4, 5, 6, 2).$$

La lista inversa se construye mediante el siguiente recorrido:

$$\text{para } i = 1, \ldots, 8: \quad inverso[pref[i]] \leftarrow i.$$

Resulta

$$inverso[1.,8] = (4, 8, 2, 5, 6, 7, 3, 1).$$

Entonces s prefiere el hospital 3 al hospital 6 porque $inverso[3] = 2 < 7 = inverso[6]$.

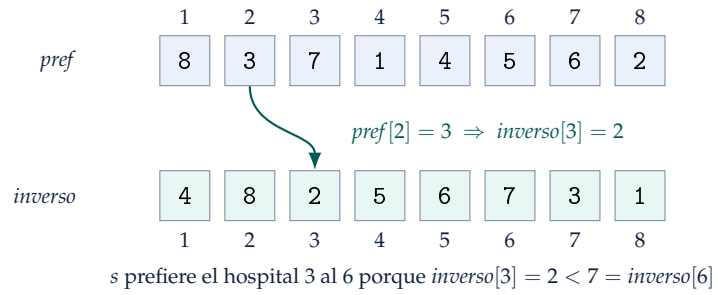


Figura 1.6. La lista inversa se obtiene con un único recorrido ($inverso[pref[i]] \leftarrow i$) y convierte la pregunta "¿a cuál de estos dos hospitales prefiere s?" en una comparación de dos enteros, es decir, en tiempo constante.

1.6 Caracterización de la solución: optimalidad

Una misma instancia puede admitir varios emparejamientos estables. Surge de inmediato la pregunta: ¿cuál de ellos produce el algoritmo de Gale-Shapley?

Definición 1.6.1 (Par válido)**.** Un elemento $y \in Y$ es un par válido (o socio válido) de un elemento $x \in X$ si existe al menos un emparejamiento estable en el que x e y están emparejados. Análogamente se define la noción simétrica.

Definición 1.6.2 (Asignación óptima para el lado que propone)**.** Supongamos que el algoritmo de Gale-Shapley se ejecuta con el conjunto X como lado que propone. La asignación óptima para X es el emparejamiento en el que cada elemento de X recibe su mejor par válido (el elemento de Y de mayor preferencia entre todos sus pares válidos).

Teorema 1.6.3 (Optimalidad del lado que propone)**.** Toda ejecución del algoritmo de Gale-Shapley (en la versión en que proponen los elementos de X) produce la asignación óptima para X. En particular, dicha asignación es un emparejamiento estable, y todas las ejecuciones del algoritmo producen el mismo emparejamiento.

Demostración. Trabajamos en la interpretación hospitales-estudiantes ($X = H$, $Y = S$) por concreción. Sea M^* el emparejamiento producido por Gale-Shapley. Supongamos, por contradicción, que algún hospital h no recibe en M^* a su mejor par válido. Como los hospitales proponen en orden decreciente de preferencia, debe existir un primer momento en el que un hospital es rechazado por un par válido. Sea h el primer hospital al que le ocurre esto, y sea s el primer par válido que rechaza a h.

Cuando s rechaza a h, forma (o reafirma) un compromiso con algún hospital h'. Por tanto s prefiere h' a h. Sea M un emparejamiento estable cualquiera en el que h y s están emparejados, y sea s' la pareja de h' en M. Como h es el primer hospital rechazado por un par válido, h' aún no ha sido rechazado por ningún par válido (en particular por s') en el instante en que h es rechazado por s. En consecuencia h' aún no se había propuesto a s' cuando se propuso a s, lo que implica que h' prefiere s a s'. Pero entonces el par (h', s) es inestable en M, contradicción.

Por lo tanto M^* es la asignación óptima para los hospitales. Como el mejor par válido de cada hospital está unívocamente determinado, todas las ejecuciones producen el mismo emparejamiento. □

Corolario 1.6.4. La asignación óptima para el lado que propone es siempre un emparejamiento estable.

Teorema 1.6.5 (Pesimalidad del lado que acepta)**.** El emparejamiento M^* producido por Gale-Shapley (con X como lado que propone) es el emparejamiento estable pesimal para los elementos de Y: cada elemento de Y recibe en M^* su peor par válido.

Demostración. Nuevamente en la interpretación hospitales-estudiantes. Supongamos, por contradicción, que existe un par (h, s) en M^* tal que h no es el peor par válido de s. Entonces existe un emparejamiento estable M en el que s está emparejado con un hospital h' al que s prefiere menos que h. En particular s prefiere h a h'. Sea s' la pareja de h en M. Por la optimalidad del lado que propone (Teorema 1.6.3), s es el mejor par válido de h, de modo que h prefiere s a s'. Entonces el par (h, s) es inestable en M, contradicción. □

La optimalidad para el lado que propone se obtiene a costa de la pesimalidad para el lado que acepta. Si se invierten los roles y proponen los elementos de Y, se obtiene el emparejamiento óptimo para Y (y pesimal para X).

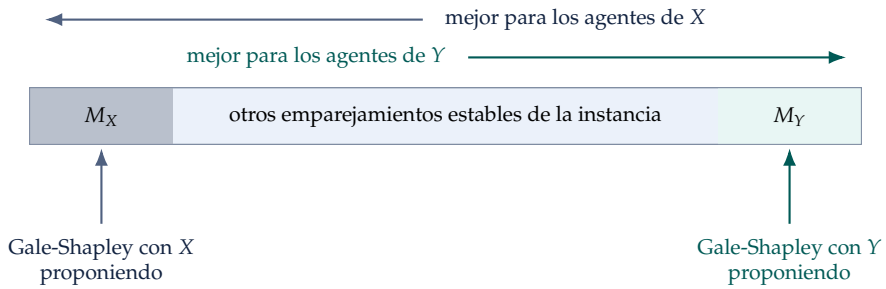


Figura 1.7. Los emparejamientos estables de una instancia forman un espectro con dos extremos. El lado que propone obtiene simultáneamente su mejor pareja válida, y el lado que acepta, la peor: optimalidad y pesimalidad son dos lecturas del mismo resultado.

Ejemplo 1.6.6. En la instancia del Ejemplo 1.1.3 existen exactamente dos emparejamientos estables:

$$M_1 = \{(A, X), (B, Y), (C, Z)\}, \qquad M_2 = \{(A, Y), (B, X), (C, Z)\}.$$

Xavier y Yolanda son pares válidos tanto de Atlántida como de Bella U.; Zeus es el único par válido de Canelones. El algoritmo de Gale-Shapley (hospitales proponen) produce M_1, que es óptimo para los hospitales: Atlántida recibe a su mejor par válido (X), Bella U. recibe a su mejor par válido (Y). Al mismo tiempo, M_1 es pesimal para los estudiantes: Xavier recibe su peor par válido entre los posibles.

1.7 Extensiones del modelo

El modelo básico admite varias extensiones naturales que aparecen en aplicaciones reales.

Proposición 1.7.1 (Inaceptabilidades)**.** Si algunos elementos de X declaran inaceptables a ciertos elementos de Y (y viceversa), el algoritmo de Gale-Shapley se adapta de manera inmediata: un elemento de X nunca se propone a un elemento de Y inaceptable, y un elemento de Y rechaza automáticamente a cualquier elemento de X inaceptable. El resultado sigue siendo un emparejamiento estable respecto de la noción de estabilidad restringida a los pares mutuamente aceptables. En este caso el emparejamiento puede no ser perfecto.

Proposición 1.7.2 (Capacidades mayores que uno)**.** Si un elemento $x \in X$ tiene capacidad $q_x \geq 1$ (por ejemplo, un hospital con varias plazas), la definición de inestabilidad se generaliza: un par (x, y) es inestable si x e y se aceptan mutuamente, y prefiere x a su asignación actual (o está libre), y x o bien tiene plazas libres o bien prefiere a y sobre al menos uno de sus elementos asignados. El algoritmo de Gale-Shapley se extiende de forma natural permitiendo que x proponga hasta llenar sus q_x plazas.

Proposición 1.7.3 (Cardinalidades distintas)**.** Cuando las cardinalidades de X e Y difieren (o, en el modelo con capacidades, cuando el número total de plazas difiere del número de elementos de Y), el algoritmo sigue produciendo un emparejamiento estable (posiblemente con agentes libres). La existencia de un emparejamiento estable perfecto ya no está garantizada.

Estas extensiones son las que se utilizan, por ejemplo, en el sistema nacional de asignación de residentes médicos en Estados Unidos (NRMP), que procesa anualmente decenas de miles de estudiantes y plazas.

1.8 Cinco problemas representativos

El problema del emparejamiento estable ilustra un patrón que se repite a lo largo del diseño de algoritmos: formular el problema con precisión matemática, diseñar un algoritmo, y analizarlo demostrando su corrección y acotando su tiempo de ejecución. Para anticipar el resto del curso conviene presentar, a un nivel introductorio, cinco problemas que servirán de hilo conductor: los primeros tres se resuelven de manera eficiente mediante técnicas cada vez más sofisticadas (algoritmos golosos, programación dinámica y flujo en redes); el cuarto marca un punto de quiebre, pues se cree que no admite solución eficiente; y el quinto sugiere una clase de problemas aún más difícil. Varios de ellos se formulan naturalmente en el lenguaje de grafos.

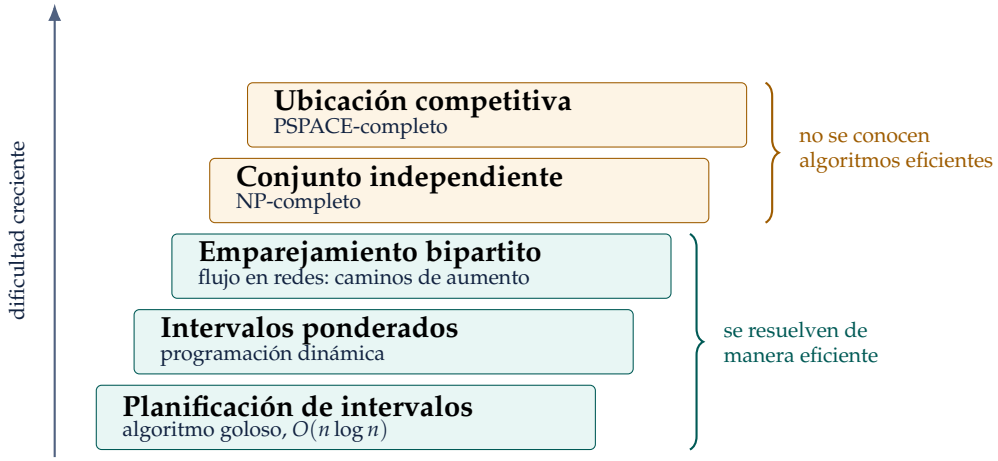


Figura 1.8. Los cinco problemas representativos del curso, ordenados por dificultad. Los tres primeros admiten algoritmos eficientes mediante técnicas cada vez más elaboradas; el cuarto marca el punto de quiebre y el quinto sugiere una clase aún más difícil.

Definición 1.8.1 (Grafo)**.** Un grafo $G = (V, E)$ consiste en un conjunto V de nodos (o vértices) y un conjunto E de aristas, donde cada arista $e \in E$ es un subconjunto de dos elementos $e = \{u, v\}$ con $u, v \in V$; se dice que u y v son los extremos de e.

Definición 1.8.2 (Grafo bipartito)**.** Un grafo $G = (V, E)$ es bipartito si el conjunto de nodos V admite una partición $V = X \cup Y$, con $X \cap Y = \emptyset$, tal que todo elemento de E tiene un extremo en X y el otro en Y.

El modelo del emparejamiento estable de las secciones anteriores corresponde al caso particular en que G es el grafo bipartito completo sobre $X = H$ e $Y = S$ (existe una arista entre todo par (h, s)), enriquecido con listas de preferencia sobre las aristas incidentes a cada nodo.

1.8.1 Planificación de intervalos

Consideremos un recurso único (una sala, un supercomputador, un microscopio electrónico) que múltiples solicitudes desean reservar por un intervalo de tiempo, y que solo puede ser usado por una solicitud a la vez.

Definición 1.8.3 (Problema de Planificación de Intervalos)**.** Se tienen n solicitudes numeradas $1, \ldots, n$; la solicitud i especifica un horario de inicio s_i y uno de fin f_i, con $s_i < f_i$. Dos solicitudes i, j son compatibles si sus intervalos no se superponen, es decir, $f_i \leq s_j$ o $f_j \leq s_i$. Un subconjunto A de solicitudes es compatible si lo son todos sus pares. El problema consiste en hallar un subconjunto compatible de cardinalidad máxima.

Ejemplo 1.8.4. Sean cinco solicitudes con horarios $[1,4]$, $[3,5]$, $[0,6]$, $[5,7]$ y $[3,8]$ (en las unidades que sean). El subconjunto $\{[1,4],[5,7]\}$ es compatible. Puede verificarse que no existe ningún subconjunto compatible de cardinalidad 3 en esta instancia (cualquier tercer intervalo se superpone con alguno de los dos primeros o con alguno de los restantes), de modo que $\{[1,4],[5,7]\}$, de cardinalidad 2, resulta óptimo.

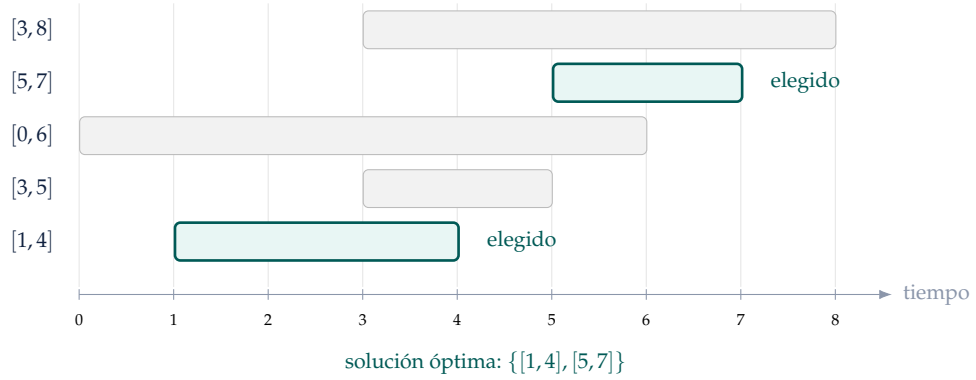


Figura 1.9. Instancia de Planificación de Intervalos. Dos solicitudes son compatibles si sus intervalos no se superponen; se busca el subconjunto compatible más grande. Aquí ningún subconjunto compatible alcanza cardinalidad 3.

Este problema se resuelve de manera óptima mediante un algoritmo goloso: se ordenan las solicitudes según un criterio adecuado (por ejemplo, tiempo de finalización creciente) y se las procesa una única vez, aceptando cada solicitud compatible con las ya aceptadas. Cuando un algoritmo goloso —miope, sin mirar hacia adelante— resulta óptimo para todas las instancias, suele revelar algo profundo sobre la estructura combinatoria del problema.

1.8.2 Planificación de intervalos ponderados

Definición 1.8.5 (Problema de Planificación de Intervalos Ponderados)**.** Es la generalización del problema anterior en la que cada solicitud i tiene, además de s_i y f_i, un valor $v_i > 0$. Se busca un subconjunto compatible A que maximice $\sum_{i \in A} v_i$.

El caso $v_i = 1$ para todo i es exactamente el problema no ponderado, pero permitir valores arbitrarios cambia radicalmente la naturaleza de la optimización: si v_1 excede la suma de todos los demás valores, la solución óptima debe incluir la solicitud 1 sin importar el resto de la configuración. No existe, en general, una regla golosa simple que recorra los intervalos una única vez y tome siempre la decisión correcta en presencia de valores arbitrarios. Este problema se resuelve mediante programación dinámica, técnica que construye el valor óptimo de manera tabular a partir de subproblemas más pequeños, degenerando en el algoritmo del caso no ponderado cuando todos los v_i son iguales.

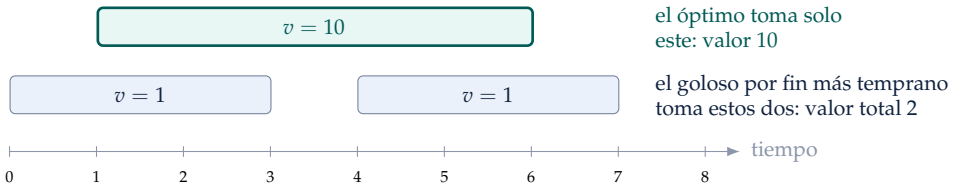


Figura 1.10. Con valores arbitrarios, la regla golosa deja de ser óptima: un único intervalo puede valer más que todos los demás juntos. La solución exige comparar el valor de incluir cada intervalo contra el de descartarlo, que es precisamente la estructura de la programación dinámica.

1.8.3 Emparejamiento bipartito

Podemos formular la noción de emparejamiento en términos de un grafo bipartito arbitrario, sin exigir que toda solicitud esté conectada con todo elemento del otro lado.

Definición 1.8.6 (Emparejamiento en un grafo)**.** Sea $G = (V, E)$ un grafo. Un emparejamiento en G es un subconjunto $N \subseteq E$ tal que cada nodo de V es extremo de a lo sumo una arista de N. N es un emparejamiento perfecto si cada nodo de V es extremo de exactamente una arista de N.

Si G es el grafo bipartito completo entre un conjunto X de n hombres y un conjunto Y de n mujeres (una arista entre cada par), los emparejamientos y emparejamientos perfectos de G coinciden exactamente con las nociones de la Definición 1.1.1 y la Definición 1.1.2. La diferencia esencial es que, en un grafo bipartito arbitrario, no toda pareja $(x, y) \in X \times Y$ está unida por una arista: es como si solo ciertos pares de agentes estuvieran dispuestos a emparejarse.

Ejemplo 1.8.7. Sea $X = \{x_1, \ldots, x_5\}$ el conjunto de tareas y $Y = \{y_1, \ldots, y_5\}$ el conjunto de máquinas, con una arista (x_i, y_j) si la máquina y_j puede procesar la tarea x_i. Un emparejamiento perfecto corresponde a una asignación de cada tarea a una máquina capaz de procesarla, de modo que cada máquina recibe exactamente una tarea. Análogamente, en un departamento universitario, X puede representar a los docentes, Y a los cursos ofrecidos, y una arista (x_i, y_j) indica que el docente x_i está habilitado para dictar el curso y_j; un emparejamiento perfecto asigna a cada docente un curso que puede dictar, cubriendo todos los cursos.

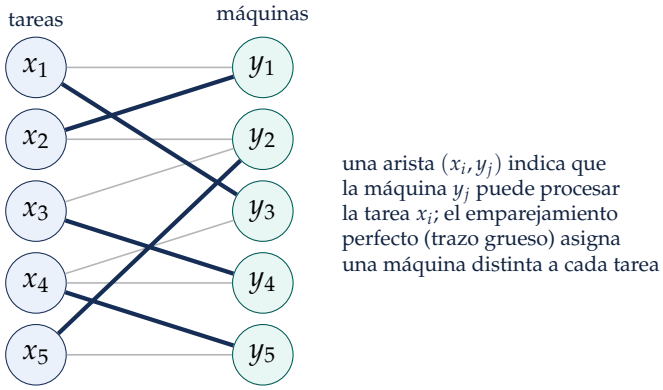


Figura 1.11. Emparejamiento bipartito sobre un grafo arbitrario: a diferencia del modelo del emparejamiento estable, no todo par está unido por una arista, y por eso la existencia de un emparejamiento perfecto deja de estar garantizada.

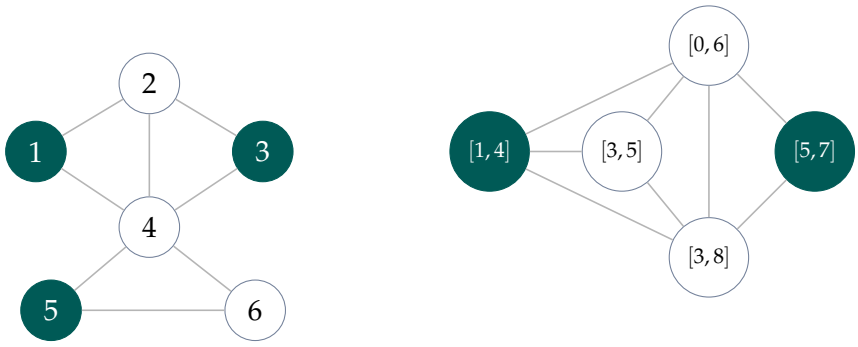
Definición 1.8.8 (Problema de Emparejamiento Bipartito)**.** Dado un grafo bipartito arbitrario $G = (X \cup Y, E)$, hallar un emparejamiento de cardinalidad máxima. Si $|X| = |Y| = n$, existe un emparejamiento perfecto si y solo si el emparejamiento máximo tiene tamaño n.

A diferencia de los dos problemas anteriores, no existe un algoritmo goloso evidente para el Emparejamiento Bipartito. La técnica estándar para resolverlo de manera eficiente construye emparejamientos cada vez más grandes mediante caminos de aumento, procedimiento que retrocede selectivamente sobre decisiones previas. Esta técnica —augmentación— es la base de una familia más amplia de problemas eficientemente resolubles: los problemas de flujo en redes.

1.8.4 Conjunto independiente

Definición 1.8.9 (Conjunto independiente)**.** Dado un grafo $G = (V, E)$, un subconjunto $I \subseteq V$ es independiente si ningún par de nodos de I está unido por una arista de E. El Problema del Conjunto Independiente consiste en, dado G, hallar un conjunto independiente de cardinalidad máxima.

Ejemplo 1.8.10. Supongamos que se tiene un grupo de n amigos, y que ciertos pares de ellos no se llevan bien entre sí. Formamos el grafo G cuyos nodos son los amigos y cuyas aristas unen a cada par que no se lleva bien; el mayor grupo de amigos que se puede invitar a una cena sin generar tensiones interpersonales es exactamente el mayor conjunto independiente de G.



(a) $I = \{1, 3, 5\}$ es independiente: ningún par de I está unido

(b) la instancia de la Figura 1.9 como grafo: un nodo por intervalo y una arista por cada par que se superpone; $\{[1, 4], [5, 7]\}$ es independiente

Figura 1.12. El Conjunto Independiente generaliza a los problemas anteriores: (a) la definición sobre un grafo cualquiera; (b) la traducción de una instancia de Planificación de Intervalos a un grafo de conflictos, en la que "compatible" se vuelve "independiente".

Proposición 1.8.11 (Generalidad del Conjunto Independiente)**.** Tanto el problema de Planificación de Intervalos (Definición 1.8.3) como el de Emparejamiento Bipartito (Definición 1.8.8) son casos particulares del Conjunto Independiente.

Idea de demostración. Para Planificación de Intervalos, se define un grafo G cuyos nodos son los intervalos, con una arista entre cada par de intervalos que se superponen; los conjuntos independientes de G son exactamente los subconjuntos compatibles de intervalos.

Para Emparejamiento Bipartito, dado $G' = (V', E')$, se define un grafo $G = (V, E)$ cuyo conjunto de nodos V es el conjunto de aristas E' de G', uniendo mediante una arista

de E a todo par de elementos de V que correspondan a aristas de G' con un extremo en común (pues esas son, precisamente, las aristas que no pueden pertenecer a un mismo emparejamiento). Los conjuntos independientes de G son entonces exactamente los emparejamientos de G'. □

No se conoce ningún algoritmo eficiente para el Conjunto Independiente, y se conjetura que no existe: el algoritmo de fuerza bruta —probar todos los subconjuntos de nodos, verificar cuáles son independientes y recordar el mayor— podría ser esencialmente lo mejor posible. El Conjunto Independiente pertenece a la clase de problemas llamados NP-completos: no se conoce un algoritmo eficiente para ninguno de ellos, pero todos son equivalentes en el sentido de que una solución eficiente para uno implicaría una solución eficiente para todos. Sin embargo, existe una asimetría notable: si alguien nos muestra un grafo G con 1000 nodos y afirma que contiene un conjunto independiente I de tamaño 100, verificar la afirmación es sencillo (basta comprobar que ningún par de nodos de I está unido por una arista), aunque encontrar ese conjunto pueda ser computacionalmente muy difícil. Esta distinción entre la dificultad de encontrar una solución y la de verificarla resulta central para entender esta clase de problemas.

1.8.5 Ubicación competitiva de las instalaciones

Consideremos el siguiente juego de dos jugadores. Dos cadenas de cafeterías compiten por el mercado en una región dividida en n zonas $1, \ldots, n$; cada zona i tiene un valor b_i (el ingreso que obtiene quien abra allí una sucursal). Ciertas normas de zonificación impiden que dos sucursales, de cualquiera de las dos cadenas, se ubiquen en zonas adyacentes; modelamos estas adyacencias mediante un grafo $G = (V, E)$ donde V es el conjunto de zonas y $(i, j) \in E$ si las zonas i y j son adyacentes. Las cadenas abren sucursales alternadamente (la primera cadena mueve primero), y en todo momento el conjunto de zonas ocupadas debe ser un conjunto independiente de G.

Definición 1.8.12 (Problema de Ubicación Competitiva de las Instalaciones)**.** Dos jugadores P_1 y P_2 seleccionan alternadamente nodos de un grafo $G = (V, E)$ con valores b_i en cada nodo, comenzando por P_1, de modo que el conjunto de nodos seleccionados sea en todo momento un conjunto independiente de G. Dado un valor objetivo B, el problema consiste en determinar si P_2 dispone de una estrategia que le garantice, sin importar cómo juegue P_1, seleccionar un conjunto de nodos cuyo valor total sea al menos B.

Ejemplo 1.8.13. Con diez zonas dispuestas en línea y valores 10,1,5,15,5,1,5,1,15,10, si el objetivo de P_2 es $B = 20$, existe una estrategia ganadora para P_2; si en cambio $B = 25$, no la hay. Determinar esto requiere, en general, analizar exhaustivamente las posibles respuestas de cada jugador ante cada jugada del otro.

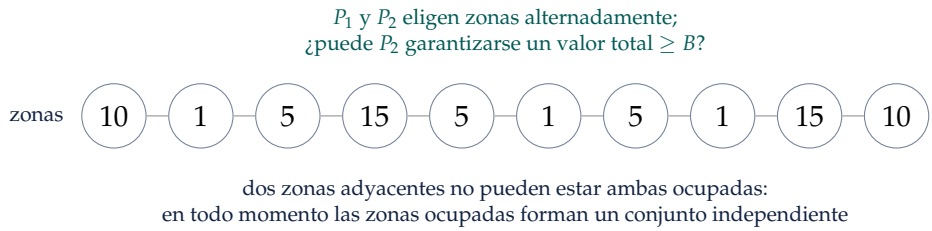


Figura 1.13. Ubicación competitiva sobre diez zonas en línea. Con $B = 20$ el segundo jugador tiene estrategia ganadora, y con $B = 25$ no la tiene; decidirlo exige analizar la respuesta de cada jugador a cada jugada del otro, y no parece existir ningún certificado corto de la respuesta.

A diferencia del Conjunto Independiente, aquí no solo es difícil (en apariencia) determinar si P_2 tiene una estrategia ganadora, sino que tampoco parece existir una manera corta de convencer a alguien de que tal estrategia existe: haría falta exhibir, para cada posible jugada de P_1, la respuesta correspondiente de P_2, en un análisis potencialmente exhaustivo caso por caso. Este problema es un ejemplo de la clase PSPACE-completa, que se conjetura estrictamente más difícil que la clase NP-completa: mientras que para el Conjunto Independiente creemos que hallar una solución es difícil pero verificarla es fácil, aquí ni siquiera parece existir una demostración corta y verificable de que una respuesta sea correcta. La noción de PSPACE-completitud captura una amplia colección de problemas de juegos y de planificación, muchos de ellos centrales en inteligencia artificial.

Capítulo 2

Análisis de Algoritmos

2.1 Tratabilidad computacional

Analizar algoritmos consiste en estudiar cómo escalan sus requerimientos de recursos —tiempo y espacio— a medida que crece el tamaño de la entrada. El objetivo de esta sección es transformar la noción intuitiva de algoritmo "eficiente" en una definición matemática precisa, que sea independiente de la plataforma de ejecución, de la instancia particular considerada, y que tenga valor predictivo a medida que el tamaño de entrada aumenta.

2.1.1 Hacia una definición de eficiencia

Una primera propuesta, ingenua, es la siguiente.

Un algoritmo es eficiente si, al implementarlo, corre rápido en instancias reales.

Esta definición tiene problemas serios: no dice nada sobre la calidad de la implementación (un mal algoritmo puede correr rápido en una computadora muy veloz sobre instancias pequeñas, y un buen algoritmo puede correr lento si está mal codificado), no precisa qué es una instancia "real", y sobre todo no dice nada sobre cómo escala el algoritmo cuando el tamaño de la entrada crece a niveles no anticipados. Necesitamos, entonces, adoptar una visión más matemática: analizar el tiempo de ejecución como una función del tamaño n de la entrada.

Un segundo intento, más útil, compara el algoritmo con la búsqueda por fuerza bruta sobre el espacio de soluciones del problema.

Un algoritmo es eficiente si logra, a nivel analítico, un desempeño en el peor caso cualitativamente mejor que la búsqueda por fuerza bruta.

Para el problema del Emparejamiento Estable, el espacio de búsqueda de fuerza bruta consiste en las $n!$ biyecciones posibles entre H y S. El algoritmo de Gale-Shapley resuelve el problema en $O(n^2)$ iteraciones (Proposición 1.4.1), una mejora cualitativa enorme frente a examinar las $n!$ biyecciones una por una.

Esta segunda definición sigue siendo vaga: ¿qué significa "cualitativamente mejor"? Esto motiva cuantificar con precisión qué tiempo de ejecución consideramos razonable.

2.1.2 Tiempo polinómico

Una propiedad de escala deseable sería la siguiente: cuando el tamaño de la entrada se duplica, un algoritmo razonable debería ralentizarse, como máximo, en un factor constante C (que puede depender del algoritmo, pero no de n).

Definición 2.1.1 (Tiempo polinómico)**.** Un algoritmo tiene tiempo de ejecución polinómico si existen constantes $c > 0$ y $d > 0$ tales que, para toda entrada de tamaño n, el tiempo de ejecución del algoritmo está acotado superiormente por $c\,n^d$ pasos computacionales primitivos. Un algoritmo con esta propiedad se llama un algoritmo de tiempo polinómico.

Verifiquemos que esta definición cumple la propiedad de escala deseada: si el tamaño de entrada pasa de n a $2n$, la cota pasa de cn^d a $c(2n)^d = c \cdot 2^d n^d$, es decir, el algoritmo se ralentiza en un factor constante $C = 2^d$ (independiente de n, aunque sí depende de d).

Definición 2.1.2 (Eficiencia)**.** Decimos que un algoritmo es eficiente si tiene tiempo de ejecución polinómico.

La justificación de la Definición 2.1.2 es fundamentalmente empírica: en la práctica, los problemas que admiten algoritmos de tiempo polinómico casi siempre admiten algoritmos con exponentes y constantes moderados (tiempos como $O(n)$, $O(n \log n)$, $O(n^2)$, $O(n^3)$), mientras que los problemas sin algoritmos polinómicos conocidos suelen ser genuinamente difíciles en la práctica. Existen excepciones en ambas direcciones: hay algoritmos de tiempo exponencial que funcionan bien en la práctica porque los casos verdaderamente malos son raros, y hay algoritmos polinómicos con constantes o exponentes tan altos que resultan inútiles (por ejemplo, $20n^{120}$ frente a $n^{1+0{,}02 \ln n}$, donde para valores razonables de n el segundo, pese a no ser estrictamente polinómico en apariencia, resulta muchísimo más rápido).

Otra ventaja crucial de definir la eficiencia en términos de tiempo polinómico es que la propiedad se vuelve negable: tiene sentido preguntar si un problema no admite ningún algoritmo eficiente, una pregunta que las definiciones anteriores, subjetivas, no permitían formular con rigor.

2.1.3 Análisis del peor caso

Definición 2.1.3 (Peor caso)**.** El tiempo de ejecución en el peor caso de un algoritmo, sobre entradas de tamaño n, es una cota que vale para cualquier entrada de tamaño n, no solo para las instancias "típicas" o "esperables".

El análisis del peor caso suele capturar bien la eficiencia práctica de un algoritmo, y tiene la ventaja de no requerir supuestos sobre la distribución de las entradas. El análisis del caso promedio (*average-case*) es una alternativa natural, pero exige definir con precisión qué es una entrada "al azar", y esa elección puede cambiar completamente la conclusión: un mismo algoritmo puede comportarse muy bien sobre una familia de entradas aleatorias y muy mal sobre otra. En este curso, salvo que se indique lo contrario, "tiempo de ejecución" significa tiempo de ejecución en el peor caso.

Existen otros tipos de análisis que aparecerán ocasionalmente a lo largo del curso:

- Peor caso: tiempo de ejecución garantizado para cualquier entrada de tamaño n. Ejemplo: heapsort realiza a lo sumo $2n \log_2 n$ comparaciones para ordenar n elementos.
- Probabilístico: tiempo de ejecución esperado de un algoritmo que usa aleatorización internamente (no de la entrada). Ejemplo: el número esperado de comparaciones de quicksort aleatorizado es $\sim 2n \ln n$.
- Amortizado: cota en el peor caso, pero promediada sobre cualquier secuencia de n operaciones. Ejemplo: partiendo de una pila vacía implementada con un arreglo redimensionable, cualquier secuencia de n inserciones y extracciones toma $O(n)$ pasos en total.
- Caso medio: tiempo de ejecución esperado sobre una entrada aleatoria de tamaño n, según alguna distribución fijada de antemano.

Ejemplo 2.1.4. Consideremos la búsqueda secuencial de un elemento x en un arreglo A de n elementos sin ordenar, recorriendo A de izquierda a derecha y deteniéndose al encontrar x. En el mejor caso (si $x = A[1]$), el algoritmo hace 1 comparación. En el peor caso (si x no está en A, o si $x = A[n]$), el algoritmo hace n comparaciones. Decimos que el tiempo de ejecución en el peor caso de la búsqueda secuencial es n comparaciones, sin importar cuán "afortunada" pudiera ser una instancia particular.

2.2 Orden asintótico de crecimiento

Dar el número exacto de pasos que ejecuta un algoritmo (por ejemplo, "a lo sumo $1{,}62n^2 + 3{,}5n + 8$ pasos") suele ser una tarea ardua y, además, poco informativa: el número exacto de pasos depende del nivel de abstracción elegido (pseudocódigo, código intermedio, instrucciones de máquina), y ese nivel introduce factores multiplicativos constantes que no reflejan nada esencial del algoritmo. Lo que sí es robusto frente a estos cambios de nivel es la tasa de crecimiento de la función, salvo por factores constantes. Formalizamos esto mediante las notaciones O, Ω y Θ.

2.2.1 Notación O grande: cotas superiores

Definición 2.2.1 (Notación O)**.** Sea $T(n)$ una función no negativa (típicamente, el tiempo de ejecución en el peor caso de un algoritmo sobre entradas de tamaño n) y sea $f(n)$ otra función. Decimos que $T(n)$ es $O(f(n))$ si existen constantes $c > 0$ y $n_0 \geq 0$ tales que

$$T(n) \leq c \cdot f(n) \qquad \text{para todo } n \geq n_0.$$

En tal caso decimos que T está asintóticamente acotada superiormente por f. Es esencial que la constante c sea fija: no puede depender de n.

Nota 2.2.2. Aunque $O(f(n))$ es, formalmente, un conjunto de funciones (el conjunto de todas las funciones acotadas superiormente por un múltiplo constante de f, a partir de algún n_0), es una práctica estándar en informática escribir $T(n) = O(f(n))$ en lugar de $T(n) \in O(f(n))$. Este abuso de notación es aceptado, pero hay que tener cuidado: de $f(n) = O(n^3)$ y $g(n) = O(n^3)$ no se deduce que $f(n) = g(n)$.

Ejemplo 2.2.3. Sea $T(n) = 32n^2 + 17n + 1$. Veamos que $T(n)$ es $O(n^2)$. Para todo $n \geq 1$ tenemos $17n \leq 17n^2$ y $1 \leq n^2$, de modo que

$$T(n) = 32n^2 + 17n + 1 \leq 32n^2 + 17n^2 + n^2 = 50n^2.$$

Esto verifica la Definición 2.2.1 con $c = 50$ y $n_0 = 1$. Notemos que $T(n)$ también es $O(n^3)$ (pues $n^2 \leq n^3$ para $n \geq 1$, luego $T(n) \leq 50n^3$), pero $T(n)$ no es $O(n)$ ni $O(n \log n)$: intuitivamente, $32n^2$ crece más rápido que cualquier múltiplo constante de n o de $n \log n$ (esto se sigue de los hechos generales que probaremos en la Sección 2.2.5).

En el uso típico, cuando decimos, por ejemplo, que "la ordenación por inserción hace $O(n^2)$ comparaciones", estamos afirmando una cota superior sobre el peor caso: el algoritmo nunca hace más que un múltiplo constante de n^2 comparaciones. La notación O no dice nada, por sí sola, sobre si esa cota es ajustada.

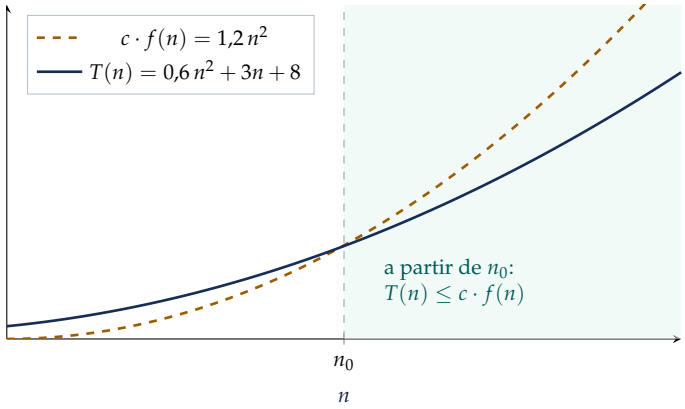


Figura 2.1. Lectura gráfica de la Definición 2.2.1. La cota superior debe valer solo a partir de un umbral n_0 y salvo un factor constante c: ambas licencias son las que permiten ignorar el comportamiento en instancias pequeñas y los detalles de implementación.

2.2.2 Notación Ω: cotas inferiores

Definición 2.2.4 (Notación Ω)**.** Decimos que $T(n)$ es $\Omega(f(n))$ si existen constantes $\varepsilon > 0$ y $n_0 \geq 0$ tales que

$$T(n) \geq \varepsilon \cdot f(n) \qquad \text{para todo } n \geq n_0.$$

En tal caso decimos que T está asintóticamente acotada inferiormente por f.

Ejemplo 2.2.5. Retomando $T(n) = 32n^2 + 17n + 1$: para todo $n \geq 0$ tenemos $T(n) \geq 32n^2$, lo que verifica la Definición 2.2.4 con $\varepsilon = 32$ y $n_0 = 0$. Así, $T(n)$ es $\Omega(n^2)$, y también es $\Omega(n)$ (pues $32n^2 \geq 32n$ para $n \geq 1$). Sin embargo $T(n)$ no es $\Omega(n^3)$.

En el uso típico, una afirmación como "todo algoritmo de ordenación basado en comparaciones requiere $\Omega(n \log n)$ comparaciones en el peor caso" es una cota inferior sobre cualquier algoritmo de esa familia: expresa una limitación intrínseca del problema, no de un algoritmo particular.

2.2.3 Notación Θ: cotas ajustadas

Definición 2.2.6 (Notación Θ)**.** Decimos que $T(n)$ es $\Theta(f(n))$ si $T(n)$ es tanto $O(f(n))$ como $\Omega(f(n))$. Equivalentemente, existen constantes $c_1 > 0$, $c_2 > 0$ y $n_0 \geq 0$ tales que

$$c_1 \cdot f(n) \leq T(n) \leq c_2 \cdot f(n) \qquad \text{para todo } n \geq n_0.$$

En este caso decimos que $f(n)$ es una cota asintóticamente ajustada para $T(n)$.

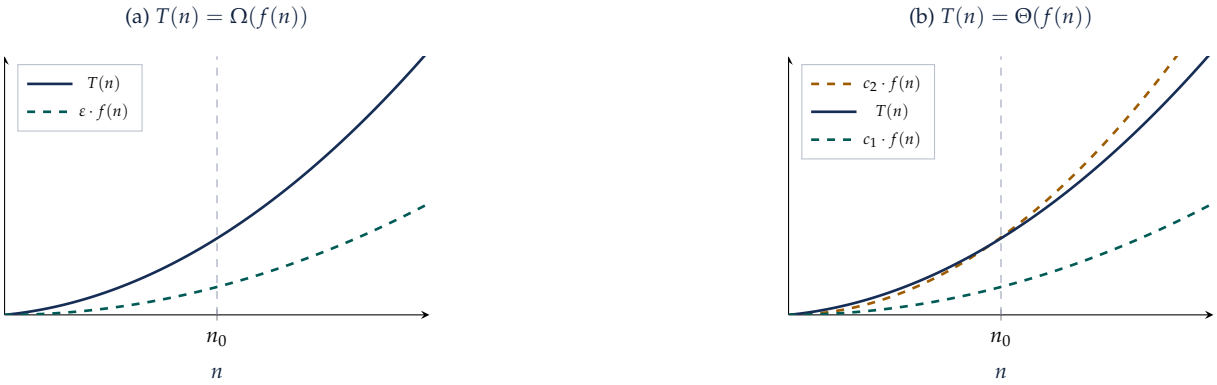


Figura 2.2. (a) Una cota inferior exige que T quede por encima de un múltiplo de f a partir de cierto punto. (b) Una cota ajustada encierra a T entre dos múltiplos de la misma función: Θ es, exactamente, tener a la vez O y Ω.

Ejemplo 2.2.7. Combinando los Ejemplos 2.2.3 y 2.2.5: como $T(n) = 32n^2 + 17n + 1$ es tanto $O(n^2)$ como $\Omega(n^2)$, concluimos que $T(n)$ es $\Theta(n^2)$.

Proposición 2.2.8. Sean f y g funciones tales que

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = c$$

existe y $c > 0$. Entonces $f(n) = \Theta(g(n))$.

Demostración. Por definición de límite, existe n_0 tal que para todo $n \geq n_0$ se cumple

$$\tfrac{1}{2}c \leq \frac{f(n)}{g(n)} \leq 2c.$$

De la desigualdad de la derecha se obtiene $f(n) \leq 2c \cdot g(n)$ para todo $n \geq n_0$, lo que implica $f(n) = O(g(n))$; de la desigualdad de la izquierda se obtiene $f(n) \geq \frac{1}{2}c \cdot g(n)$ para todo $n \geq n_0$, lo que implica $f(n) = \Omega(g(n))$. Luego $f(n) = \Theta(g(n))$. □

Proposición 2.2.9. Si $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$, entonces $f(n) = O(g(n))$.

Demostración. Tomando $c = 1$ en la definición de límite, existe n_0 tal que $f(n)/g(n) \leq 1$ para todo $n \geq n_0$, es decir, $f(n) \leq g(n)$ para todo $n \geq n_0$. Esto verifica la Definición 2.2.1 con $c = 1$. □

Ejemplo 2.2.10. Sean $f(n) = 5n^3 + 2n^2 - n$ y $g(n) = n^3$. Como

$$\lim_{n\to\infty} \frac{5n^3 + 2n^2 - n}{n^3} = \lim_{n\to\infty} \left(5 + \frac{2}{n} - \frac{1}{n^2}\right) = 5 > 0,$$

la Proposición 2.2.8 nos dice directamente que $f(n) = \Theta(n^3)$, sin necesidad de acotar manualmente cada término como en el Ejemplo 2.2.3.

2.2.4 Propiedades del orden asintótico

Proposición 2.2.11 (Transitividad)**.**

(a) Si $f = O(g)$ y $g = O(h)$, entonces $f = O(h)$.

(b) Si $f = \Omega(g)$ y $g = \Omega(h)$, entonces $f = \Omega(h)$.

Demostración. Probamos (a); la prueba de (b) es análoga. Por hipótesis existen c, n_0 tales que $f(n) \leq c\, g(n)$ para todo $n \geq n_0$, y existen c', n_0' tales que $g(n) \leq c'\, h(n)$ para todo $n \geq n_0'$. Entonces, para todo $n \geq \max(n_0, n_0')$,

$$f(n) \leq c\, g(n) \leq c\, c'\, h(n),$$

lo que muestra $f = O(h)$ con constante cc' y umbral $\max(n_0, n_0')$. □

Corolario 2.2.12. Si $f = \Theta(g)$ y $g = \Theta(h)$, entonces $f = \Theta(h)$.

Demostración. De $f = \Theta(g)$ se sigue $f = O(g)$ y $f = \Omega(g)$; de $g = \Theta(h)$ se sigue $g = O(h)$ y $g = \Omega(h)$. Por la Proposición 2.2.11(a), $f = O(g)$ y $g = O(h)$ implican $f = O(h)$; por la parte (b), $f = \Omega(g)$ y $g = \Omega(h)$ implican $f = \Omega(h)$. Luego $f = \Theta(h)$. □

Proposición 2.2.13 (Suma de funciones)**.** Si $f = O(h)$ y $g = O(h)$, entonces $f + g = O(h)$. Más generalmente, si k es una constante fija y $f_1, \ldots, f_k$ son funciones con $f_i = O(h)$ para todo i, entonces $f_1 + \cdots + f_k = O(h)$.

Demostración. Existen c, n_0 con $f(n) \leq c\,h(n)$ para $n \geq n_0$, y c', n_0' con $g(n) \leq c'\,h(n)$ para $n \geq n_0'$. Entonces, para $n \geq \max(n_0, n_0')$,

$$f(n) + g(n) \leq c\,h(n) + c'\,h(n) = (c + c')\,h(n),$$

lo que prueba $f + g = O(h)$. El caso general (con k funciones) se obtiene aplicando este argumento $k - 1$ veces (o, equivalentemente, tomando como constante la suma de las k constantes individuales). □

Proposición 2.2.14. Si f y g son funciones no negativas con $g = O(f)$, entonces $f + g = \Theta(f)$. Es decir, el término dominante de una suma es una cota asintóticamente ajustada para toda la suma.

Demostración. Como $f(n) \geq 0$ y $g(n) \geq 0$, tenemos $f(n) + g(n) \geq f(n)$ para todo n, lo que muestra $f + g = \Omega(f)$. Por otro lado, $f = O(f)$ trivialmente (con $c = 1$), y por hipótesis $g = O(f)$; por la Proposición 2.2.13, $f + g = O(f)$. Luego $f + g = \Theta(f)$. □

La Proposición 2.2.14 formaliza una situación muy frecuente: cuando un algoritmo tiene dos fases claramente separadas y una de ellas es asintóticamente más lenta que la otra, el tiempo total del algoritmo es del mismo orden que el de la fase más lenta.

Ejemplo 2.2.15. Un algoritmo primero ordena un arreglo de n elementos en tiempo $O(n \log n)$ y luego realiza una única pasada lineal, en tiempo $O(n)$, para procesar el resultado ordenado. Como $n = O(n \log n)$ (Sección 2.2.5), la Proposición 2.2.14 garantiza que el tiempo total del algoritmo es $\Theta(n \log n)$: la fase de ordenamiento domina asintóticamente.

2.2.5 Cotas asintóticas para funciones comunes

Proposición 2.2.16 (Polinomios)**.** Sea $f(n) = a_0 + a_1 n + \cdots + a_d n^d$ un polinomio de grado d con $a_d > 0$. Entonces $f(n) = \Theta(n^d)$.

Demostración. Como $\lim_{n \to \infty} \dfrac{f(n)}{n^d} = a_d > 0$ (los términos de grado menor que d se anulan al dividir por n^d y tomar el límite), el resultado se sigue directamente de la Proposición 2.2.8. □

Proposición 2.2.17 (Logaritmos y polinomios)**.** Para todo $b > 1$ y todo $x > 0$, se cumple $\log_b n = O(n^x)$.

Esbozo. Se verifica que $\lim_{n\to\infty} \dfrac{\log_b n}{n^x} = 0$ (por ejemplo, mediante la regla de L'Hôpital, derivando numerador y denominador respecto de n), y se aplica la Proposición 2.2.9. □

Un corolario inmediato, particularizando $x = 1$: para todo $b > 1$, $\log_b n = O(n)$, y en particular $n \log n = O(n \cdot n) = O(n^2)$; esto es lo que justifica que todo algoritmo con tiempo $O(n \log n)$ sea, en particular, de tiempo polinómico.

Proposición 2.2.18 (Cambio de base)**.** Para cualesquiera constantes $a, b > 0$, se cumple $\log_a n = \Theta(\log_b n)$.

Demostración. Por la identidad de cambio de base, $\log_a n = \dfrac{\log_b n}{\log_b a}$, y $\log_b a$ es una constante positiva (si $a > 1$; el caso $0 < a < 1$ es análogo salvo signos, y queda fuera del interés práctico pues consideramos bases mayores que 1). Luego $\log_a n$ es un múltiplo constante de $\log_b n$, lo que da directamente $\log_a n = \Theta(\log_b n)$ (tomando $c_1 = c_2 = 1/\log_b a$ en la Definición 2.2.6). □

La Proposición 2.2.18 es la razón por la que en análisis de algoritmos se suele escribir simplemente $O(\log n)$, sin especificar la base: cambiar de base solo multiplica por una constante, algo que la notación O ya ignora.

Proposición 2.2.19 (Exponenciales y polinomios)**.** Para todo $r > 1$ y todo $d > 0$, se cumple $n^d = O(r^n)$.

Demostración. Se verifica que $\lim_{n\to\infty} \dfrac{n^d}{r^n} = 0$ (nuevamente por L'Hôpital, o comparando el crecimiento multiplicativo de r^n frente al crecimiento polinómico de n^d), y se aplica la Proposición 2.2.9. □

En particular, ninguna función exponencial r^n (con $r > 1$) es $O(n^d)$ para ningún d fijo: toda exponencial crece, eventualmente, más rápido que cualquier polinomio. Además, para bases distintas $r > s > 1$, nunca ocurre $r^n = \Theta(s^n)$ (pues $(r/s)^n \to \infty$), de modo que, a diferencia de los logaritmos, las funciones exponenciales de distinta base no son asintóticamente equivalentes entre sí. Por eso decir simplemente "tiempo exponencial", sin más precisión, es una afirmación más informal que decir "tiempo $O(\log n)$".

Ejemplo 2.2.20. Ordenemos de menor a mayor tasa de crecimiento las funciones $f_1(n) = \log_2 n$, $f_2(n) = \sqrt{n}$, $f_3(n) = 10n$, $f_4(n) = n \log_2 n$, $f_5(n) = 2^n$. Por la Proposición 2.2.17 (con $x = 1/2$), $f_1 = O(f_2)$. Como $\sqrt{n} = O(n)$ trivialmente ($\sqrt{n} \leq n$ para $n \geq 1$) y $n = O(n \log_2 n)$ (pues $\log_2 n \geq 1$ para $n \geq 2$), tenemos $f_2 = O(f_3) = O(f_4)$ salvo el factor constante 10, que la notación O absorbe; más precisamente, $f_3 = O(f_4)$ pues $10n \leq 10\, n \log_2 n$ para $n \geq 2$. Finalmente, por la Proposición 2.2.19 (con $d = 1$, $r = 2$), $f_4 = O(n^2) = O(2^n)$ (ya que todo polinomio es $O(2^n)$). En definitiva:

$$\log_2 n \;=\; O(\sqrt{n}) \;=\; O(n) \;=\; O(n \log_2 n) \;=\; O(2^n),$$

una jerarquía que resume el orden relativo de crecimiento entre logaritmos, raíces, polinomios y exponenciales.

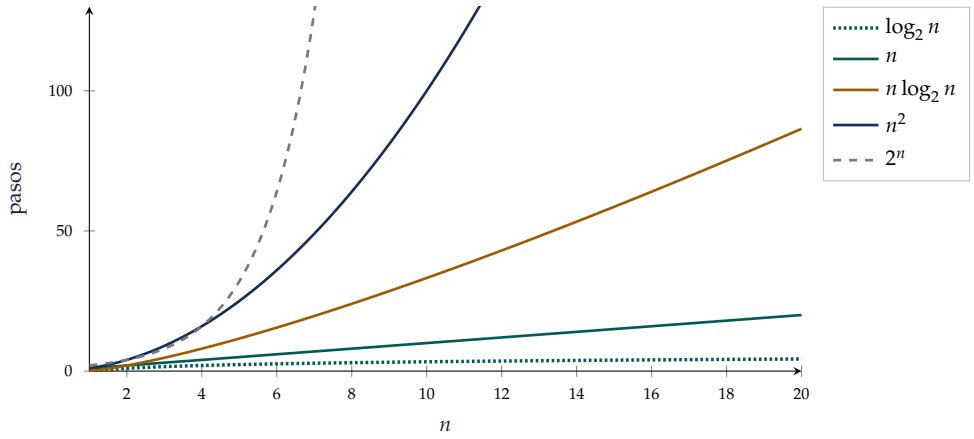


Figura 2.3. Las tasas de crecimiento más frecuentes, dibujadas a la misma escala. Lo que distingue a un algoritmo eficiente de uno que no lo es no es el valor de la función para un n pequeño, sino la forma en que crece: la exponencial supera a todas las demás apenas n pasa de 4, y ninguna constante multiplicativa puede compensarlo.

2.2.6 Notación O con varias variables

Definición 2.2.21 (Notación O multivariable)**.** Decimos que $T(m, n)$ es $O(f(m, n))$ si existen constantes $c > 0$, $m_0 \geq 0$ y $n_0 \geq 0$ tales que

$$T(m, n) \leq c \cdot f(m, n) \qquad \text{para todo } m \geq m_0 \text{ y todo } n \geq n_0.$$

Ejemplo 2.2.22. Sea $T(m,n) = 32mn^2 + 17mn + 32n^3$. Se verifica que $T(m,n)$ es tanto $O(mn^2 + n^3)$ como $O(mn^3)$ (pues $n^2 \leq n \cdot n^2 \leq mn^3/m$... más directamente, $mn^2 \leq mn^3$ y $n^3 \leq mn^3$ cuando $m \geq 1$), pero $T(m,n)$ no es $O(n^3)$ (el término $32mn^2$ puede ser arbitrariamente mayor que cualquier múltiplo constante de n^3 si m crece mucho más rápido que n) ni $O(mn^2)$ (el término $32n^3$ puede dominar si n crece mucho más rápido que m).

En el uso típico, la notación multivariable es habitual al analizar algoritmos sobre grafos, donde el tamaño de la entrada tiene naturalmente dos parámetros: el número de nodos $n = |V|$ y el número de aristas $m = |E|$. Por ejemplo, la búsqueda en anchura (*breadth-first search*) sobre un grafo dirigido encuentra el camino más corto entre dos nodos s y t en tiempo $O(m + n)$.

2.3 Implementación con listas y arreglos

Para analizar el tiempo de ejecución de un algoritmo expresado a alto nivel —como hicimos con Gale-Shapley en el Capítulo 1— no hace falta programarlo y ejecutarlo, pero sí hay que pensar con cuidado en cómo se representan y manipulan los datos, de modo de poder acotar el número de pasos computacionales. En la Sección 2.1 del Capítulo 1 ya adelantamos, en la Proposición 1.5.1, que el algoritmo de Gale-Shapley admite una implementación en tiempo $O(n^2)$. Ahora que contamos con el vocabulario de la Sección 2.2, conviene retomar esa implementación con más detalle, precisando qué estructuras de datos —arreglos y listas— hacen falta y por qué cada operación involucrada cuesta tiempo constante.

2.3.1 Arreglos y listas enlazadas

Las dos estructuras más simples para mantener una lista de n elementos son el arreglo y la lista enlazada, y cada una tiene ventajas y desventajas complementarias.

Nota 2.3.1 (Arreglos). Un arreglo A de tamaño n almacena sus elementos en posiciones consecutivas de memoria, de modo que $A[i]$ es el i-ésimo elemento de la lista.

- Acceder al i-ésimo elemento cuesta tiempo $O(1)$: basta calcular la dirección de memoria correspondiente.
- Determinar si un elemento e pertenece al arreglo cuesta, en general, tiempo $O(n)$: hay que recorrer los elementos uno por uno, salvo que se sepa algo sobre su orden.
- Si el arreglo está ordenado, se puede determinar la pertenencia de e en tiempo $O(\log n)$ mediante búsqueda binaria (Ejemplo 2.4.11).

Los arreglos son incómodos para mantener un conjunto que cambia dinámicamente de tamaño (como el conjunto de hospitales libres en Gale-Shapley, que crece y decrece a lo largo de la ejecución), porque insertar o eliminar un elemento en el medio del arreglo obliga, en general, a desplazar los elementos siguientes.

Nota 2.3.2 (Listas doblemente enlazadas). En una lista doblemente enlazada, cada elemento e tiene un campo $e.val$ con su valor, un puntero $e.sig$ al elemento siguiente (o *NULL* si e es el último) y un puntero $e.ant$ al elemento anterior (o *NULL* si e es el primero). Se mantienen además punteros globales al primer y al último elemento.

- Eliminación: para quitar e de la lista, basta enlazar directamente a $e.ant$ con $e.sig$ ("empalmar" la lista alrededor de e). Esto cuesta tiempo $O(1)$, dado un puntero a e.
- Inserción: para insertar e entre d y f, se actualizan $d.sig$ y $f.ant$ para que apunten a e, y los punteros de e para que apunten a d y a f. También cuesta $O(1)$.
- Recorrer la lista completa toma tiempo $O(n)$ (proporcional a su longitud), pero acceder directamente al i-ésimo elemento cuesta $O(i)$: a diferencia del arreglo, no hay acceso en tiempo constante a una posición arbitraria, porque hay que seguir los punteros *sig* desde el principio.

Convertir una lista enlazada en un arreglo, o viceversa, toma tiempo $O(n)$: basta recorrer la estructura de origen una vez, copiando cada elemento a la estructura de destino. Esto permite elegir, para cada parte de un algoritmo, la estructura que más convenga, sin quedar atado a la forma en que se recibió la entrada originalmente.

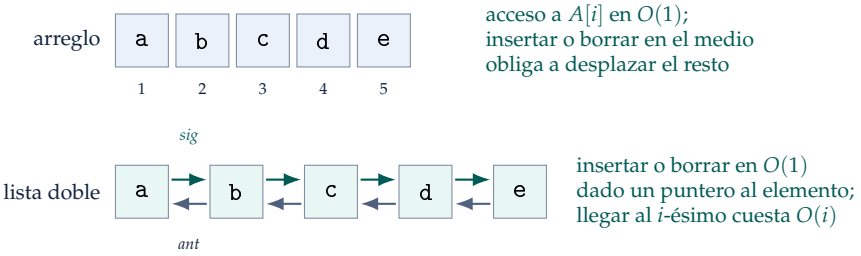


Figura 2.4. Arreglos y listas doblemente enlazadas tienen ventajas complementarias: el arreglo da acceso directo por índice, la lista da modificaciones locales baratas. Convertir una en otra cuesta $O(n)$, de modo que puede elegirse la más conveniente para cada fase del algoritmo.

Ejemplo 2.3.3. Supongamos una lista doblemente enlazada de hospitales libres $\mathit{Inicio} = h_3 \leftrightarrow h_1 \leftrightarrow h_5 \leftrightarrow \mathit{Final}$ (es decir, $h_3.\mathit{sig} = h_1$, $h_1.\mathit{sig} = h_5$, y los punteros *ant* análogos en sentido inverso). Si h_1 deja de estar libre, se lo elimina en tiempo $O(1)$: se actualiza $h_3.\mathit{sig} \leftarrow h_5$ y $h_5.\mathit{ant} \leftarrow h_3$, quedando la lista $h_3 \leftrightarrow h_5$. Si más tarde un hospital h_7 vuelve a quedar libre, se lo puede insertar al principio en tiempo $O(1)$, actualizando *Inicio*, sin tener que recorrer ni reordenar el resto de la lista.

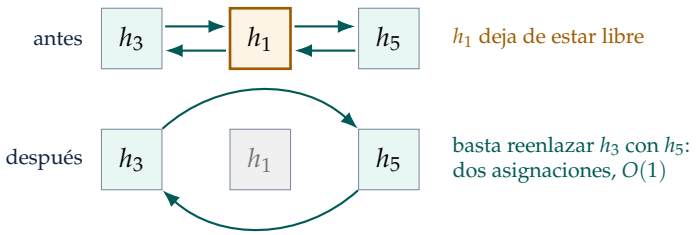


Figura 2.5. Eliminación por empalme: no hay que recorrer ni reordenar nada, solo redirigir los dos punteros que rodean al elemento. Esta es la operación que permite mantener el conjunto de hospitales libres de Gale-Shapley con costo constante por cambio.

2.3.2 Revisitando la implementación de Gale-Shapley

Para lograr el tiempo $O(n^2)$ de la Proposición 1.5.1 hace falta poder ejecutar cada iteración del bucle principal en tiempo constante. Numeramos a los hospitales y estudiantes con $\{1, \ldots, n\}$, y usamos dos matrices $\mathit{HospPref}[h, i]$ y $\mathit{StudPref}[s, i]$ (de tamaño $n \times n$ cada una, construidas en tiempo $O(n^2)$ a partir de la entrada) que almacenan, respectivamente, la i-ésima estudiante en la lista de preferencia del hospital h, y el i-ésimo hospital en la lista de preferencia de la estudiante s. Cada iteración del bucle debe resolver cuatro tareas, cada una en tiempo $O(1)$:

1. Identificar un hospital libre. Se mantiene el conjunto de hospitales libres como una lista enlazada (Nota anterior); se toma siempre el primero, y se lo inserta o elimina de la lista en $O(1)$ cuando cambia su condición (como en el Ejemplo 2.3.3).

2. *Identificar, para un hospital h, la estudiante de mayor preferencia a la que aún no se propuso.* Se mantiene un arreglo $Next[h]$, inicializado en 1, que indica la posición (en $HospPref[h, \cdot]$) de la próxima estudiante a proponer. Tras cada propuesta de h, se incrementa $Next[h]$ en 1, sin importar si la propuesta fue aceptada o rechazada.

3. *Determinar si una estudiante s está comprometida y, en tal caso, con quién.* Se mantiene un arreglo $Current[s]$ que almacena el hospital actual de s (con un símbolo especial "*null*" si s está libre).

4. *Dada una estudiante s y dos hospitales h, h', decidir cuál prefiere s.* Recorrer la lista de preferencias $StudPref[s, \cdot]$ de s cada vez que se necesita esta comparación costaría $O(n)$ por comparación —correcto pero demasiado lento para nuestro objetivo—. En cambio, se precomputa, para cada estudiante s, un arreglo inverso $Ranking[s, \cdot]$ tal que $Ranking[s, h]$ es la posición de h en la lista de preferencias de s (exactamente la construcción del Ejemplo del Capítulo 1 sobre la lista inversa, aplicada aquí a las n estudiantes). Esta precomputación toma tiempo $O(n)$ por estudiante, es decir, $O(n^2)$ en total, pero se hace una única vez al principio del algoritmo. Luego, comparar h contra h' se reduce a comparar $Ranking[s, h]$ contra $Ranking[s, h']$, en tiempo $O(1)$.

Este es exactamente el mecanismo que ya se describió, de manera más resumida, en la demostración de la Proposición 1.5.1 del Capítulo 1 (la "cola de hospitales libres", los arreglos $estudiante[h]/hospital[s]$, y la lista inversa de preferencias). Lo que agregamos ahora es la justificación de por qué cada una de esas cuatro operaciones cuesta exactamente $O(1)$, y no simplemente $O(n)$ o algo peor: es precisamente el tipo de análisis, apoyado en la elección correcta de estructuras de datos, que perseguimos formalizar en este capítulo.

estructura	qué guarda	consulta que resuelve	costo
Libres	los hospitales con plaza (lista enlazada)	tomar un h libre	$O(1)$
$Next[h]$	posición de la próxima estudiante en la lista de h	elegir a quién proponer	$O(1)$
$Current[s]$	hospital actual de s, o *null*	¿está s comprometida?	$O(1)$
$Ranking[s,h]$	posición de h en la lista de s	¿prefiere s a h o a h'? (precomputada en $O(n^2)$)	$O(1)$

las cuatro consultas cuestan $O(1) \Rightarrow$ cada propuesta cuesta $O(1) \Rightarrow$ el total es $O(n^2)$

Figura 2.6. Las cuatro estructuras que hacen que cada iteración del bucle principal cueste tiempo constante. La tabla *Ranking* es la única que requiere precomputación ($O(n^2)$ en total, una sola vez), y es justamente la que evita recorrer una lista de preferencias en cada comparación.

Ejemplo 2.3.4. Vale la pena notar por qué no basta con mantener las preferencias de las estudiantes únicamente en el arreglo $StudPref[s,\cdot]$ (sin la tabla *Ranking* auxiliar). Supongamos que s debe comparar a los hospitales $h = 4$ y $h' = 9$, y que su lista de preferencias es $StudPref[s, 1..n] = (7, 2, 9, \ldots, 4, \ldots)$. Para saber cuál de los dos aparece primero, habría que recorrer el arreglo desde la posición 1 hasta encontrar a 4 o a 9, lo cual cuesta $O(n)$ en el peor caso (por ejemplo, si ambos están cerca del final de la lista). Como esta comparación se repite hasta n^2 veces a lo largo de la ejecución (una por cada propuesta), el costo total pasaría de $O(n^2)$ a $O(n^3)$ si no se usara *Ranking*. La tabla $Ranking[s,\cdot]$ evita este recorrido: $Ranking[s,4]$ y $Ranking[s,9]$ se leen directamente, en $O(1)$, y s prefiere al hospital cuya posición en *Ranking* es menor.

2.4 Estudio de los tiempos de ejecución habituales

Con las herramientas de la Sección 2.2, conviene familiarizarse con el "paisaje" de los tiempos de ejecución que aparecen recurrentemente al diseñar algoritmos, junto con los patrones de diseño que típicamente los producen.

2.4.1 Tiempo lineal: $O(n)$

Un algoritmo de tiempo lineal procesa la entrada, en esencia, en una única pasada, gastando tiempo constante por cada elemento.

Ejemplo 2.4.1 (Cálculo del máximo)**.** Dado un arreglo $a_1, \ldots, a_n$, el siguiente algoritmo calcula su máximo en tiempo $O(n)$:

$$max \leftarrow a_1$$
$$\textbf{for } i = 2 \textbf{ to } n :$$
$$\quad \textbf{if } a_i > max \textbf{ then } max \leftarrow a_i$$

Cada iteración del `for` realiza trabajo constante (una comparación y, a lo sumo, una asignación), y hay $n - 1$ iteraciones. Luego el tiempo total es $O(n)$.

Ejemplo 2.4.2 (Combinar dos listas ordenadas)**.** Dadas dos listas ordenadas $A = a_1, \ldots, a_n$ y $B = b_1, \ldots, b_n$, se desea producir una única lista ordenada con los $2n$ elementos. El algoritmo mantiene un puntero a la "cabeza" de cada lista, compara los dos elementos señalados, agrega el menor a la salida, y avanza el puntero correspondiente; al vaciarse una lista, se agrega el resto de la otra directamente.

A primera vista podría pensarse que cada elemento participa en tiempo constante, pero eso no es cierto en general: un elemento puede quedar "en espera" en la cabeza de su lista durante muchas iteraciones mientras se extraen elementos de la otra lista (por ejemplo, si $A = 1, 3, 5, \ldots, 2n - 1$ y $B = n, n + 2, \ldots, 3n - 2$, el elemento $b_1 = n$ permanece en la cabeza de B durante $\sim n/2$ iteraciones).

El argumento correcto es de cobro (*accounting*): se le cobra el costo de cada iteración del `while` al elemento que se agrega a la salida en esa iteración. Cada uno de los $2n$ elementos se agrega exactamente una vez, así que se cobra a lo sumo $2n$ veces en total, y por lo tanto hay a lo sumo $2n$ iteraciones del `while`, cada una de costo constante. El tiempo total es $O(n)$.

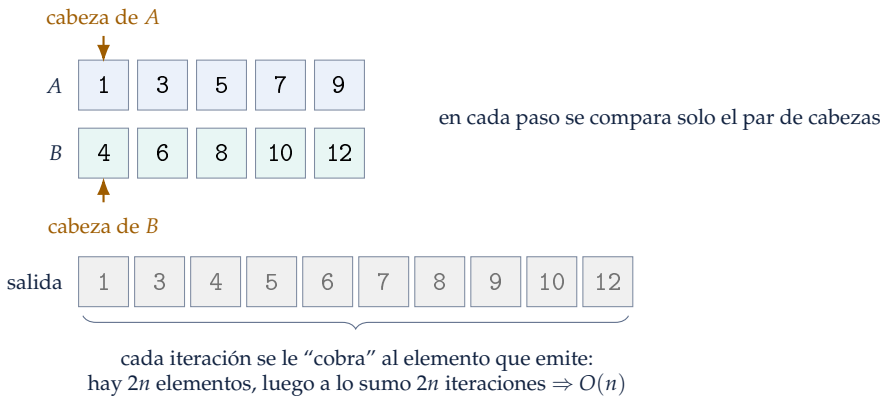


Figura 2.7. Combinación de dos listas ordenadas. Un elemento puede permanecer muchas iteraciones en la cabeza de su lista, de modo que no es cierto que cada elemento cueste tiempo constante "mientras espera"; el argumento correcto cobra cada iteración al elemento que sale hacia la salida.

El Ejemplo 2.4.2 ilustra que un tiempo de ejecución lineal no siempre se debe a que cada elemento cueste, individualmente, tiempo constante; a veces hace falta un argumento de cobro (o, más generalmente, un argumento amortizado) que distribuya el costo total entre los elementos de una manera no evidente a primera vista.

2.4.2 Tiempo $O(n \log n)$

Este tiempo de ejecución surge típicamente de dos maneras: algoritmos de divide y vencerás que dividen la entrada en dos mitades, resuelven cada mitad recursivamente, y combinan las dos soluciones en tiempo lineal (como Mergesort, que se estudiará en detalle más adelante en el curso); y algoritmos cuyo paso más costoso es ordenar la entrada.

Ejemplo 2.4.3 (Mayor intervalo vacío)**.** Se tienen n marcas de tiempo $x_1, \ldots, x_n$ en las que llegan copias de un archivo a un servidor, y se quiere hallar el mayor intervalo de tiempo, entre la primera y la última marca, durante el cual no llega ninguna copia. Basta ordenar las marcas de tiempo (tiempo $O(n \log n)$) y luego recorrer la lista ordenada una vez, calculando la distancia entre cada marca y su sucesora, quedándonos con la mayor (tiempo $O(n)$ adicional). Por la Proposición 2.2.14, el tiempo total es $\Theta(n \log n)$, dominado por la etapa de ordenamiento.

2.4.3 Tiempo cuadrático: $O(n^2)$

Surge típicamente al enumerar todos los pares de elementos de una entrada de tamaño n, o mediante dos bucles anidados, cada uno de $O(n)$ iteraciones.

Ejemplo 2.4.4 (Par de puntos más cercano)**.** Dados n puntos $(x_1, y_1), \ldots, (x_n, y_n)$ en el plano, se busca el par a menor distancia entre sí. El algoritmo de fuerza bruta enumera los $\binom{n}{2} = O(n^2)$ pares posibles, calcula la distancia de cada uno en tiempo constante, y se queda con el mínimo:

```
PAR-MÁS-CERCANO(P)
 1  mejor ← ∞
 2  for i = 1 to n do
 3      for j = i + 1 to n do
 4          d ← √((x_i − x_j)² + (y_i − y_j)²)
 5          if (d < mejor) then
 6              mejor ← d
 7          end if
 8      end for
 9  end for
10  return mejor
```

> **Estructuras de datos.** Los puntos van en un arreglo de largo n con las dos coordenadas por posición, de modo que leer (x_i, y_i) cuesta $O(1)$ y los dos bucles recorren todos los pares sin ninguna búsqueda.

El bucle interno tiene $O(n)$ iteraciones y el externo también, dando un tiempo total $O(n^2)$.

El algoritmo del Ejemplo 2.4.4 es, precisamente, la enumeración por fuerza bruta del espacio de búsqueda natural de tamaño $O(n^2)$; podría parecer inevitable examinar todos los pares, pero esto es una ilusión: más adelante en el curso veremos un algoritmo de divide y vencerás que resuelve este problema en tiempo $O(n \log n)$.

Ejemplo 2.4.5 (Ordenación por inserción)**.** La ordenación por inserción procesa el arreglo de izquierda a derecha; en el paso i (para $i = 2, \ldots, n$), inserta el elemento a_i en la posición correcta dentro del prefijo ya ordenado $a_1, \ldots, a_{i-1}$, desplazando los elementos mayores que a_i un lugar a la derecha. En el peor caso (arreglo en orden estrictamente decreciente), el paso i requiere desplazar los $i - 1$ elementos del prefijo, es decir, $O(i)$ trabajo. El tiempo total en el peor caso es

$$\sum_{i=2}^{n} O(i) = O\left(\sum_{i=2}^{n} i\right) = O\left(\frac{n(n+1)}{2} - 1\right) = O(n^2),$$

donde usamos la fórmula de la suma de los primeros n enteros y la Proposición 2.2.16 (con $d = 2$) para concluir que $\frac{n(n+1)}{2} - 1 = \Theta(n^2)$.

2.4.4 Tiempo cúbico: $O(n^3)$

Surge típicamente al enumerar todas las ternas de elementos, o mediante tres bucles anidados de $O(n)$ iteraciones cada uno.

Ejemplo 2.4.6 (Conjuntos disjuntos)**.** Dados n conjuntos $S_1, \ldots, S_n$, cada uno subconjunto de $\{1, \ldots, n\}$ (de modo que cada conjunto tiene tamaño $O(n)$), se quiere saber si existe algún par de ellos disjunto. Suponiendo que se puede recorrer cada conjunto en tiempo constante por elemento, y verificar pertenencia en tiempo constante:

```
SON-DISJUNTOS(S_1, ..., S_k)
 1  for cada conjunto S_i do
 2      for cada conjunto S_j con j ≠ i do
 3          for cada elemento p ∈ S_i do
 4              if (p ∈ S_j) then
 5                  return false
 6              end if
 7          end for
 8      end for
 9  end for
10  return true
```

> **Estructuras de datos.** Cada S_j se guarda en una tabla de hash, de modo que la consulta $p \in S_j$ de la línea 4 cuesta $O(1)$ en promedio en vez de recorrer S_j entera. Con listas simples, esa misma consulta costaría $O(|S_j|)$ y el total subiría un factor más.

El bucle interno recorre $O(n)$ elementos de S_i; el bucle sobre S_j aporta otro factor $O(n)$; el bucle sobre S_i aporta un tercer factor $O(n)$. El tiempo total es $O(n^3)$.

2.4.5 Tiempo polinómico de grado mayor: $O(n^k)$

Así como enumerar pares de elementos da $O(n^2)$, enumerar todos los subconjuntos de tamaño fijo k de un conjunto de n elementos da $O(n^k)$, para cualquier constante k (no dependiente de n).

Ejemplo 2.4.7 (Conjunto independiente de tamaño k)**.** Dado un grafo $G = (V, E)$ con n nodos y una constante k fija, se quiere determinar si G tiene un conjunto independiente (Definición 1.8.9) de exactamente k nodos. El algoritmo de fuerza bruta enumera todos los subconjuntos S de k nodos, y para cada uno verifica si es independiente:

INDEPENDIENTE-DE-TAMAÑO(G, k)

```
1 for cada subconjunto S ⊆ V con |S| = k do
2     if (S es un conjunto independiente) then
3         return S
4     end if
5 end for
6 return "no existe"
```

El número de subconjuntos de tamaño k es

$$\binom{n}{k} = \frac{n(n-1)\cdots(n-k+1)}{k!} \leq \frac{n^k}{k!} = O(n^k)$$

(tratando k como constante). Verificar si un conjunto de k nodos es independiente requiere examinar $\binom{k}{2} = O(k^2)$ pares, es decir, tiempo $O(k^2)$ (constante, ya que k no depende de n). El tiempo total es $O(k^2 n^k) = O(n^k)$.

Nota 2.4.8. Aunque $O(n^{17})$ es, formalmente, tiempo polinómico (Definición 2.1.1), un algoritmo con ese exponente es completamente impráctico para valores de n moderadamente grandes. Esto ilustra el límite de usar "tiempo polinómico" como sinónimo exacto de "eficiente en la práctica".

2.4.6 Más allá del tiempo polinómico

Dos tiempos de ejecución superpolinómicos aparecen con frecuencia: 2^n y $n!$.

Ejemplo 2.4.9 (Conjunto independiente máximo)**.** A diferencia del Ejemplo 2.4.7 (tamaño k fijo), si se busca el conjunto independiente de cardinalidad máxima en un grafo de n nodos, el algoritmo de fuerza bruta debe enumerar los 2^n subconjuntos posibles de V:

```
INDEPENDIENTE-MÁXIMO(G)
1  mejor ← ∅
2  for cada subconjunto S ⊆ V do
3      if (S es independiente y |S| > |mejor|) then
4          mejor ← S
5      end if
6  end for
7  return mejor
```

Verificar cada subconjunto (de hasta n nodos) toma $O(n^2)$ tiempo, dando un total de $O(n^2 2^n)$. No se conoce ningún algoritmo sustancialmente mejor para este problema (Nota tras la Proposición 1.8.11, Capítulo 1).

El espacio de búsqueda de tamaño 2^n (todos los subconjuntos) aparece en muchísimos problemas, pero eso no significa que todos ellos requieran tiempo exponencial: el problema de Planificación de Intervalos (Definición 1.8.3, Capítulo 1) tiene, en apariencia, el mismo tipo de espacio de búsqueda por fuerza bruta (subconjuntos de intervalos), pero admite un algoritmo goloso que lo resuelve en tiempo $O(n \log n)$. Distinguir cuándo se puede evitar la fuerza bruta y cuándo no es, en gran medida, el contenido central de este curso.

Ejemplo 2.4.10 (Espacios de búsqueda de tamaño $n!$)**.** El valor $n!$ aparece como tamaño del espacio de búsqueda en dos situaciones típicas: cuando se buscan todas las formas de emparejar n elementos con otros n (como las $n!$ biyecciones del problema de Emparejamiento Estable, Capítulo 1), y cuando se buscan todos los órdenes posibles de n elementos (como en el Problema del Viajante de Comercio, donde se buscan todos los $(n-1)!$ órdenes posibles de recorrer $n-1$ ciudades partiendo y llegando a una ciudad fija). Pese a que el Emparejamiento Estable tiene un espacio de búsqueda de tamaño $n!$, el algoritmo de Gale-Shapley lo resuelve en $O(n^2)$ iteraciones (Proposición 1.4.1); en cambio, no se conoce ningún algoritmo eficiente para el Problema del Viajante de Comercio, que —como el Conjunto Independiente— es NP-completo.

2.4.7 Tiempo sublineal

Un algoritmo puede correr en tiempo asintóticamente menor que n únicamente si no necesita leer toda la entrada, sino que puede "consultarla" de manera indirecta.

Ejemplo 2.4.11 (Búsqueda binaria)**.** Dado un arreglo ordenado A de n números y un valor x, se quiere determinar si $x \in A$. En lugar de recorrer A linealmente, se consulta el elemento central: si coincide con x, se termina; si es mayor que x, se descarta la mitad superior y se repite la búsqueda en la mitad inferior; si es menor, análogamente con la mitad superior.

BÚSQUEDA-BINARIA(A, x)

```
1  lo ← 1
2  hi ← n
3  while lo ≤ hi do
4      mid ← ⌊(lo + hi)/2⌋
5      if (x < A[mid]) then
6          hi ← mid − 1
7      else if (x > A[mid]) then
8          lo ← mid + 1
9      else
10         return true
11     end if
12 end while
13 return false
```

> **Estructuras de datos.** La entrada tiene que ser un arreglo ordenado, no una lista enlazada: el algoritmo salta al elemento central en $O(1)$, y con una lista ese salto costaría $O(n)$ y arruinaría la idea. Los tres índices son enteros y no hace falta ninguna estructura auxiliar.

Tras k consultas, la región donde x podría estar tiene tamaño a lo sumo $n/2^k$. Esta región se reduce a tamaño constante cuando $n/2^k = O(1)$, es decir, cuando $k = \log_2 n$. Luego el algoritmo termina en $O(\log n)$ iteraciones, cada una de costo constante: el tiempo total es $O(\log n)$.

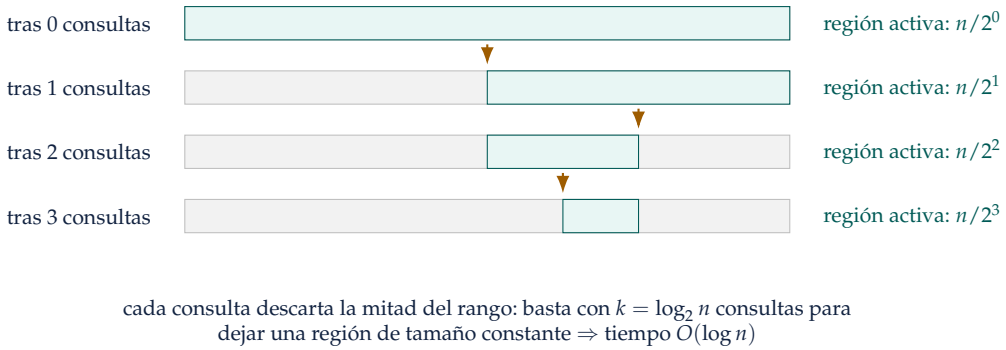


Figura 2.8. Búsqueda binaria. El algoritmo corre en tiempo sublineal porque nunca necesita leer toda la entrada: le alcanza con consultar un elemento por paso, siempre que cada consulta le permita descartar una fracción constante de lo que queda.

Ejemplo 2.4.12 (Otra búsqueda sublineal)**.** Consideremos un arreglo ordenado A de n números distintos, todos positivos salvo posiblemente un prefijo negativo (es decir, A tiene primero algunos valores negativos, en orden creciente, seguidos de valores no negativos, también en orden creciente). Se quiere hallar el índice del primer elemento no negativo de A (o determinar que no existe). Una adaptación directa de la búsqueda binaria del Ejemplo 2.4.11 —comparando $A[mid]$ contra 0 en lugar de contra x, y buscando la frontera entre negativos y no negativos en vez de un valor puntual— resuelve este problema también en tiempo $O(\log n)$, por el mismo argumento de reducción geométrica de la región activa.

Nota 2.4.13 (Panorama). En general, $O(\log n)$ aparece como cota de tiempo siempre que un algoritmo realiza trabajo constante para descartar una fracción constante de la entrada en cada paso: basta con $O(\log n)$ pasos de ese tipo para reducir la entrada a tamaño constante.

tiempo de ejecución	mayor n tratable en $\approx$ 1 segundo	patrón de diseño típico
$O(\log n)$	astronómico	descartar una fracción constante
$O(n)$	$\sim 10^9$	una pasada sobre la entrada
$O(n \log n)$	$\sim 3 \times 10^7$	dividir y vencer; ordenar
$O(n^2)$	$\sim 3 \times 10^4$	enumerar todos los pares
$O(n^3)$	$\sim 10^3$	enumerar todas las ternas
$O(2^n)$	~ 30	enumerar todos los subconjuntos
$O(n!)$	~ 12	enumerar todas las permutaciones

(estimación con 10^9 operaciones elementales por segundo;
sirve para comparar órdenes de magnitud, no como medida exacta)

Figura 2.9. El "paisaje" de los tiempos de ejecución habituales. La diferencia entre $O(n^2)$ y $O(2^n)$ no es de grado sino de naturaleza: mejorar la computadora corre la frontera de lo tratable apenas unas unidades en el caso exponencial, mientras que en el polinómico la multiplica.

Capítulo 3

Grafos

3.1 Definiciones básicas y aplicaciones

Un grafo es la estructura combinatoria que utilizamos para codificar relaciones entre pares de objetos. Su interés no proviene de su definición —que es de una simplicidad casi decepcionante— sino de la enorme cantidad de situaciones que admiten ser modeladas con ella: redes de transporte, redes de comunicaciones, relaciones de precedencia entre tareas, vínculos entre personas. Una vez traducido un problema al lenguaje de los grafos, quedan disponibles los algoritmos generales que estudiaremos en este capítulo.

Definición 3.1.1 (Grafo)**.** Un grafo es un par $G = (V, E)$, donde V es un conjunto finito cuyos elementos se llaman nodos (o vértices) y E es un conjunto de aristas: cada arista $e \in E$ es un subconjunto de dos elementos de V, es decir $e = \{u, v\}$ con $u, v \in V$ y $u \neq v$. Decimos que u y v son los extremos de e, y que u y v son adyacentes (o vecinos).

Las aristas de un grafo representan una relación simétrica entre sus extremos. Cuando la relación que se quiere modelar es asimétrica se recurre a la noción, estrechamente emparentada, de grafo dirigido.

Definición 3.1.2 (Grafo dirigido)**.** Un grafo dirigido es un par $G = (V, E)$ donde cada arista $e \in E$ es un par ordenado $e = (u, v)$ de nodos distintos. En ese caso u se denomina cola y v cabeza de la arista, y decimos que e sale de u y entra en v.

Cuando se quiere enfatizar que un grafo no es dirigido se lo llama grafo no dirigido; por omisión, en estas notas la palabra “grafo” significa grafo no dirigido.

Nota 3.1.3 (Dos advertencias de notación). Primero: aunque una arista no dirigida debería escribirse como el conjunto $\{u, v\}$, es habitual (también en estas notas) escribirla (u, v), con la notación de los pares ordenados; el contexto indica si el grafo es dirigido o no. Segundo: las palabras nodo y vértice son sinónimos exactos en este contexto.

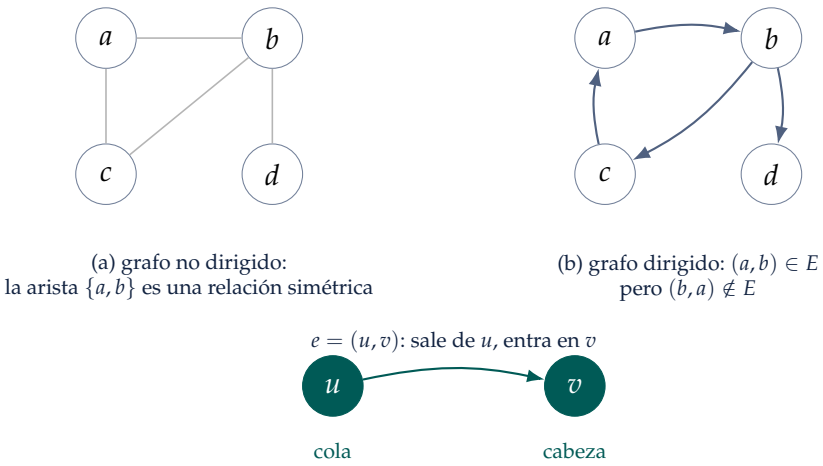


Figura 3.1. Un grafo no dirigido y uno dirigido sobre el mismo conjunto de nodos. La dirección de las aristas no es un detalle cosmético: cambia por completo qué nodos son alcanzables desde cuáles, y por lo tanto qué preguntas tienen sentido sobre el grafo.

3.1.1 Ejemplos de grafos

La lista siguiente no pretende ser exhaustiva; su propósito es ofrecer un conjunto de situaciones concretas contra las cuales contrastar las definiciones y los problemas algorítmicos del capítulo. Conviene, en cada caso, preguntarse qué representan los nodos y qué representan las aristas.

1. Redes de transporte. El mapa de rutas de una compañía aérea es un grafo: los nodos son aeropuertos y hay una arista de u a v si existe un vuelo directo de u a v. En principio es dirigido, pero como casi siempre que existe el vuelo (u, v) existe también (v, u), se lo suele tratar como no dirigido. Lo mismo ocurre con una red ferroviaria o con el mapa de líneas de ómnibus de una ciudad.

2. Redes de comunicaciones. Un nodo por computadora y una arista entre u y v si hay un enlace físico entre ambas. En redes inalámbricas conviene un modelo dirigido: puede ocurrir que v reciba la señal de u pero no al revés, si u tiene mayor potencia de transmisión.

3. Redes de información. La Web es un grafo dirigido: los nodos son páginas y hay una arista de u a v si u contiene un enlace a v. Aquí la dirección es esencial: muchas páginas enlazan a un diario, y el diario no las enlaza de vuelta. La red de citas bibliográficas entre artículos científicos tiene la misma estructura y es varias décadas anterior.

4. Redes sociales. Los nodos son personas y las aristas representan algún vínculo: amistad (no dirigido), "u le pide consejo a v" (dirigido), o pertenencia de una persona a una organización (grafo bipartito).

5. Redes de dependencias. Un nodo por tarea y una arista dirigida de u a v si u debe realizarse antes que v. Por ejemplo, las materias de una carrera con sus previaturas, o las funciones de un programa con la relación "u invoca a v". Volveremos sobre este caso en la Sección 3.6.

3.1.2 Caminos y conectividad

La operación fundamental sobre un grafo es recorrer una sucesión de nodos unidos por aristas: un usuario navegando la Web enlace a enlace, un rumor propagándose de boca en boca, un pasajero encadenando vuelos.

Definición 3.1.4 (Camino)**.** Un camino en un grafo $G = (V, E)$ es una sucesión de nodos $P = v_1, v_2, \ldots, v_k$ tal que cada par consecutivo v_i, v_{i+1} está unido por una arista de G. Decimos que P es un camino de v_1 a v_k. El camino es simple si todos sus nodos son distintos entre sí. En un grafo dirigido se exige además que $(v_i, v_{i+1}) \in E$ para todo i, es decir, el camino debe respetar la dirección de las aristas.

Definición 3.1.5 (Ciclo)**.** Un ciclo es un camino $v_1, v_2, \ldots, v_{k-1}, v_k$ con $k > 2$ en el que los primeros $k - 1$ nodos son distintos y $v_1 = v_k$; es decir, la sucesión regresa al nodo de partida. Su longitud es la cantidad de aristas que lo componen.

Definición 3.1.6 (Conexo, fuertemente conexo)**.** Un grafo no dirigido es conexo si para todo par de nodos u, v existe un camino de u a v. Un grafo dirigido es fuertemente conexo si para todo par de nodos u, v existe un camino de u a v y un camino de v a u.

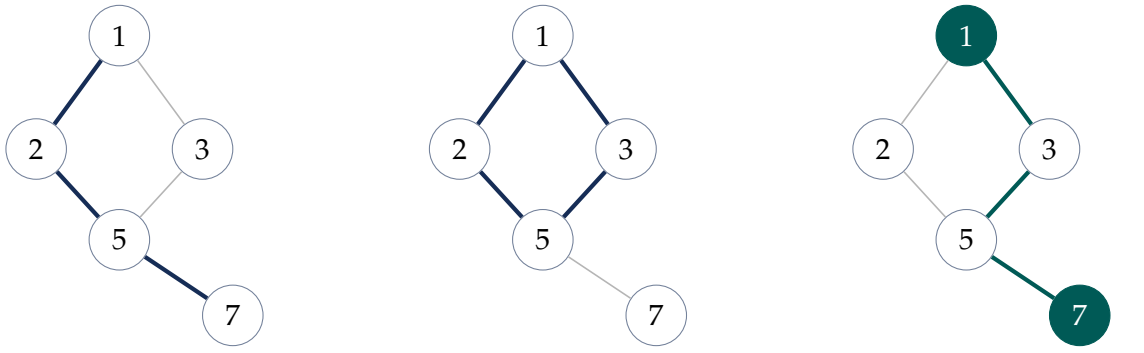
La definición de conectividad para grafos dirigidos requiere este cuidado adicional porque puede ocurrir que u alcance a v sin que v alcance a u.

Un grafo cualquiera no tiene por qué ser conexo, pero siempre se descompone en pedazos que sí lo son. Cada uno de esos pedazos debe ser lo más grande posible: si un nodo de afuera estuviera unido por una arista a un nodo de adentro, no habría razón para dejarlo afuera.

Definición 3.1.7 (Componente conexa)**.** Un conjunto de nodos $R \subseteq V$ es una componente conexa de G si para todo par $u, v \in R$ existe un camino de u a v, y R es maximal con esa propiedad: no existe un conjunto conexo de nodos que contenga estrictamente a R.

Nota 3.1.8 (Una formulación equivalente de la maximalidad). Un conjunto conexo R es maximal si y solo si ninguna arista de G une un nodo de R con un nodo de $V \setminus R$. En efecto, si existiera una arista (u, v) con $u \in R$ y $v \notin R$, el conjunto $R \cup \{v\}$ seguiría siendo conexo —para llegar a v desde cualquier nodo de R basta ir hasta u y cruzar la arista— y R no sería maximal. Esta caracterización es la que usaremos en los algoritmos: la condición de parada será, precisamente, que no quede ninguna arista con un extremo adentro y el otro afuera.

Definición 3.1.9 (Distancia)**.** La distancia entre dos nodos u y v, notada $\text{dist}(u, v)$, es la mínima cantidad de aristas de un camino de u a v. Si no existe tal camino, se define $\text{dist}(u, v) = \infty$.



(a) camino simple $1, 2, 5, 7$

(b) ciclo $1, 2, 5, 3, 1$ (longitud 4)

(c) $\text{dist}(1, 7) = 3$: ningún camino de 1 a 7 usa menos de tres aristas

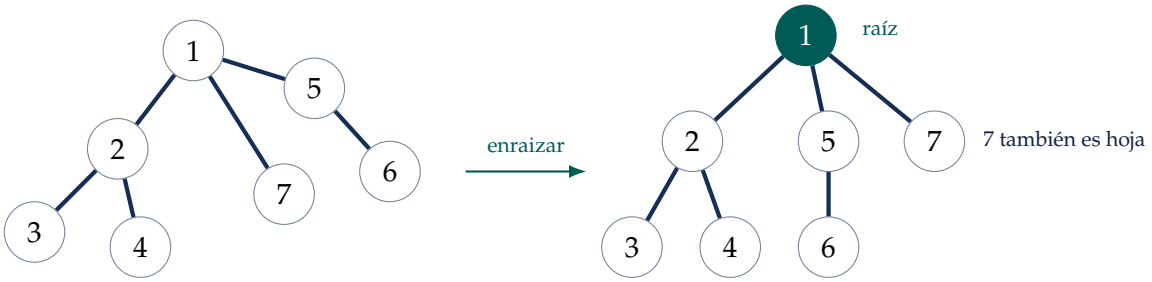
Figura 3.2. Sobre un mismo grafo: un camino simple, un ciclo y la noción de distancia. Obsérvese que la distancia no depende del dibujo sino de la cantidad mínima de aristas que hay que atravesar.

3.1.3 Árboles

Definición 3.1.10 (Árbol)**.** Un grafo no dirigido es un árbol si es conexo y no contiene ciclos.

En un sentido preciso, los árboles son los grafos conexos más simples posibles: eliminar cualquier arista de un árbol lo desconecta. Para razonar sobre la estructura de un árbol T resulta útil enraizarlo en un nodo r: intuitivamente, se toma T del nodo r y se deja que el resto cuelgue por su propio peso. Formalmente, se orienta cada arista en sentido contrario a r; para cada nodo $v \neq r$ se define su padre como el nodo u que lo precede inmediatamente en el camino de r a v, y se dice que v es hijo de u. Más

generalmente, w es descendiente de v (y v es ancestro de w) si v está en el camino de la raíz a w; un nodo sin hijos se llama hoja.



(a) un árbol: conexo y sin ciclos

(b) el mismo árbol, enraizado en 1: hojas 3, 4, 6 y 7

Figura 3.3. Enraizar un árbol no cambia el grafo —son los mismos nodos y las mismas aristas— pero impone una jerarquía: padre, hijos, ancestros y descendientes. Esa jerarquía es lo que vuelve fáciles muchas preguntas sobre árboles.

Enraizar permite, por ejemplo, contar las aristas de un árbol sin esfuerzo.

Proposición 3.1.11. Todo árbol de n nodos tiene exactamente $n - 1$ aristas.

Demostración. Enraicemos el árbol en un nodo cualquiera r. Cada nodo distinto de r tiene exactamente una arista que lo une con su padre; recíprocamente, cada arista une a exactamente un nodo (el de abajo) con su padre. La correspondencia entre nodos distintos de r y aristas es entonces biyectiva, de modo que hay $n - 1$ aristas. □

Proposición 3.1.12. Sea G un grafo no dirigido de n nodos. Dos cualesquiera de las tres afirmaciones siguientes implican la tercera:

(I) G es conexo;

(II) G no contiene ciclos;

(III) G tiene $n - 1$ aristas.

(Omitimos la demostración de esta última proposición, más fuerte que la anterior.)

La Definición 3.1.10 pide que un árbol no contenga ciclos, y la Proposición 3.1.11 muestra que tiene la menor cantidad posible de aristas para ser conexo. Ambas cosas son la misma observación vista desde dos ángulos, y tienen una consecuencia que usaremos más adelante: un árbol está al borde de tener un ciclo, y cualquier arista extra lo empuja.

Proposición 3.1.13 (Agregar una arista a un árbol crea un ciclo)**.** Sea T un árbol y sean $x \neq y$ dos de sus nodos tales que (x, y) no es una arista de T. Entonces el grafo $T + (x, y)$ contiene exactamente un ciclo, formado por la arista (x, y) junto con el camino de x a y en T.

Demostración. Como T es conexo, existe un camino P de x a y en T, y ese camino es único: si hubiera dos caminos simples distintos entre x e y, recorriendo uno de ida y el otro de vuelta se obtendría un ciclo en T, que por definición no tiene. Cerrando P con la arista (x, y) se obtiene un ciclo de $T + (x, y)$.

Recíprocamente, todo ciclo C de $T + (x, y)$ debe usar la arista (x, y), ya que en caso contrario sería un ciclo de T. Al quitarle esa arista, C queda reducido a un camino simple de x a y en T, que por unicidad es P. Luego C es el ciclo ya descrito y no hay otro. □

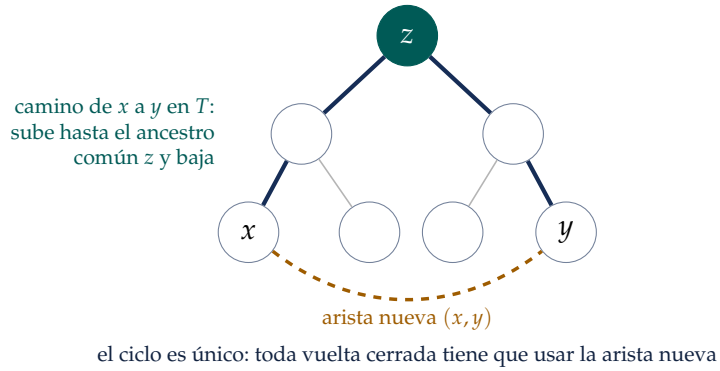


Figura 3.4. Una arista que no pertenece al árbol cierra un ciclo con el camino que une sus extremos, camino que pasa por el ancestro común de ambos. Cuando el árbol provenga de una recorrida (Sección 3.2), esta observación permitirá deducir propiedades del grafo entero mirando solo el árbol; el Teorema 3.4.4 es el ejemplo central.

3.2 Conectividad y recorrido de grafos

Planteemos el problema algorítmico más básico sobre grafos. Dados un grafo $G = (V, E)$ y dos nodos s y t, queremos decidir si existe un camino de s a t. Lo llamaremos problema de la conectividad s-t.

Para grafos pequeños la pregunta se responde a simple vista; para grafos grandes no. Conviene pensarlo como el problema de resolver un laberinto: si imaginamos cada nodo como una habitación y cada arista como un pasillo, se trata de partir de la habitación s y encontrar el camino hasta t. En esta sección describimos dos algoritmos naturales —búsqueda en anchura (BFS) y búsqueda en profundidad (DFS)—; en la Sección 3.3 veremos cómo implementarlos eficientemente.

Usaremos como ejemplo recurrente el grafo G_1 de la Figura 3.5.

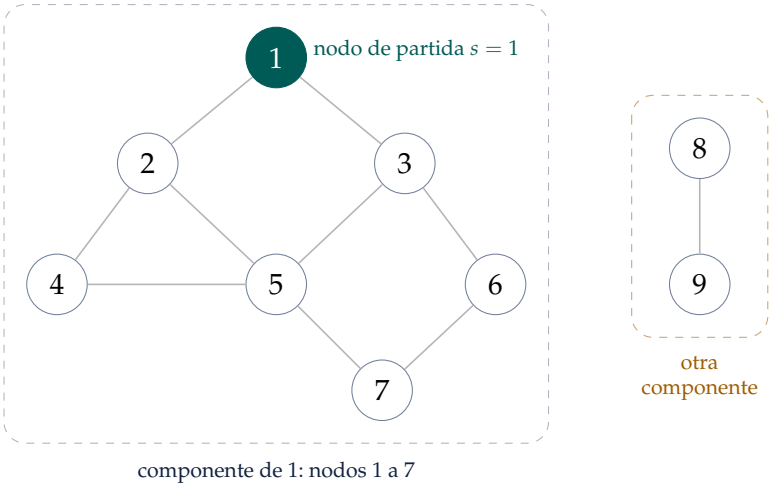


Figura 3.5. El grafo G_1 que usaremos a lo largo del capítulo: $n = 9$ nodos y $m = 10$ aristas. Desde el nodo 1 se alcanzan los nodos 2 a 7, pero no los nodos 8 y 9: ningún recorrido que arranque en 1 los verá jamás.

3.2.1 Búsqueda en anchura (BFS)

La idea de la búsqueda en anchura es explorar desde s en todas las direcciones a la vez, agregando nodos de a una "capa" por vez: primero los vecinos de s, luego los vecinos de estos que aún no fueron vistos, y así sucesivamente hasta que no se descubran nodos nuevos. La imagen física es la de una onda que se expande desde s e inunda todo lo que puede alcanzar; la capa en que aparece un nodo indica el instante en que la onda llega a él.

Definición 3.2.1 (Capas de BFS)**.** Sea $s \in V$. Se define $L_0 = \{s\}$ y, para $j \geq 0$, la capa L_{j+1} es el conjunto de todos los nodos que no pertenecen a ninguna capa anterior y que tienen una arista a algún nodo de L_j.

Ejemplo 3.2.2. En el grafo G_1 de la Figura 3.5, partiendo de $s = 1$: la capa L_1 es $\{2, 3\}$; la capa L_2 es $\{4, 5, 6\}$; la capa L_3 es $\{7\}$. En ese punto la búsqueda se detiene, porque no queda ningún nodo nuevo por descubrir: los nodos 8 y 9 nunca son alcanzados.

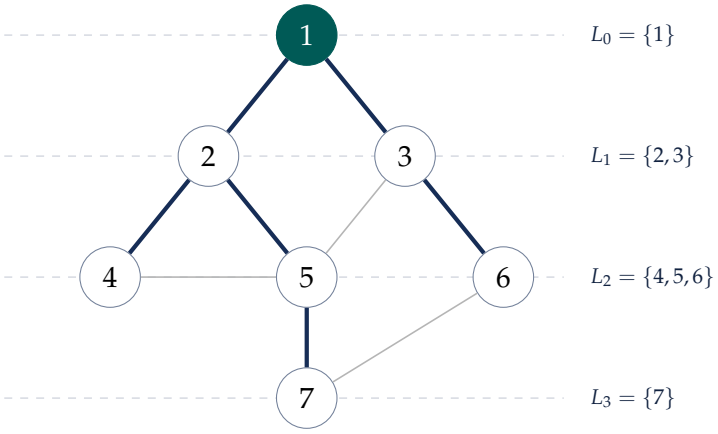


Figura 3.6. BFS sobre G_1 desde $s = 1$, con los nodos dispuestos por capas. La capa L_j contiene exactamente los nodos a distancia j de s; las aristas que no entran en el árbol unen nodos de la misma capa o de capas consecutivas, nunca de capas más lejanas.

Recordando que la distancia entre dos nodos es la mínima cantidad de aristas de un camino que los une, la capa L_1 es el conjunto de nodos a distancia 1 de s y, en general, L_j es el conjunto de nodos a distancia exactamente j. Un nodo no aparece en ninguna capa si y solo si no existe camino hasta él. Es decir: BFS no solo determina qué nodos son alcanzables desde s, sino que además calcula los caminos más cortos hacia ellos.

Proposición 3.2.3. Para cada $j \geq 1$, la capa L_j producida por BFS consiste exactamente en los nodos a distancia j de s. En particular, existe un camino de s a t si y solo si t aparece en alguna capa.

BFS produce además, de manera natural, un árbol T enraizado en s sobre el conjunto de nodos alcanzables: cuando se examina un nodo $u \in L_j$ y se descubre a través suyo un nodo v no visto antes, se agrega la arista (u, v) a T y u pasa a ser el padre de v, reflejando que u fue el "responsable" de completar el camino hasta v. El árbol resultante se llama árbol de búsqueda en anchura.

Ejemplo 3.2.4 (Construcción del árbol BFS sobre G_1)**.** Partiendo de $s = 1$ y recorriendo las listas de vecinos en orden creciente:

1. Se descubren 2 y 3: ambos son hijos de 1 y forman L_1.
2. Al examinar 2 se descubren 4 y 5, que pasan a ser sus hijos. Al examinar 3 se descubre 6; la arista $(3, 5)$ no entra en el árbol, porque para cuando se la mira el nodo 5 ya había sido descubierto.
3. Al examinar la capa L_2 solo aparece un nodo nuevo, 7, descubierto desde 5. Las aristas $(4, 5)$ y $(6, 7)$ no entran en el árbol: no descubren nada.
4. Al examinar 7 no se descubre nada nuevo, la capa L_4 queda vacía y el algoritmo termina.

En el ejemplo anterior, todas las aristas que quedaron fuera del árbol unen nodos de una misma capa o de capas consecutivas. Esto no es una casualidad del ejemplo.

Proposición 3.2.5. Sea T un árbol BFS, sean x e y nodos de T pertenecientes a las capas L_i y L_j respectivamente, y sea (x, y) una arista de G. Entonces i y j difieren a lo sumo en 1.

Demostración. Supongamos por el absurdo que difieren en más de 1; sin pérdida de generalidad, $i < j - 1$. Consideremos el momento en que BFS examina las aristas incidentes a x. Como x pertenece a L_i, todo nodo descubierto desde x queda en la capa L_{i+1} o en una anterior. Siendo y vecino de x, a más tardar en ese momento y debe haber sido descubierto, de modo que y pertenece a L_{i+1} o antes, lo que contradice $j > i + 1$. □

3.2.2 Explorar una componente conexa

El conjunto de nodos descubiertos por BFS desde s es exactamente el conjunto de nodos alcanzables desde s. Lo llamaremos componente conexa de G que contiene a s (Definición 3.1.7), y lo notaremos R. Una vez conocida R, la conectividad s-t se responde verificando si $t \in R$.

Ahora bien, BFS es solo una manera posible de construir R. En un nivel más general, podemos hacer crecer la componente explorando G en cualquier orden: se comienza con $R = \{s\}$ y, mientras exista una arista (u, v) con $u \in R$ y $v \notin R$, se agrega v a R. La justificación es inmediata: si hay un camino P de s a u, entonces hay un camino de s a v que consiste en recorrer P y luego la arista (u, v).

COMPONENTE(G, s)

```
1  R ← {s}
2  while exista una arista (u, v) con u ∈ R y v ∉ R do
3      Agregar v a R
4  end while
5  return R
```

> **Estructuras de datos.** El grafo se recibe en listas de adyacencia: un arreglo Adj de largo n donde $Adj[u]$ es la lista enlazada de los vecinos de u. El conjunto R no se guarda como conjunto abstracto sino como un arreglo de booleanos $enR[1..n]$, que responde "¿$v \in R$?" en $O(1)$, más una lista con sus elementos para poder recorrerlos al final. Las aristas con un extremo en R y otro afuera se mantienen en una bolsa de pendientes: cada vez que un nodo entra a R se agregan sus aristas incidentes. Qué estructura se use para esa bolsa es exactamente lo que distingue a BFS de DFS.

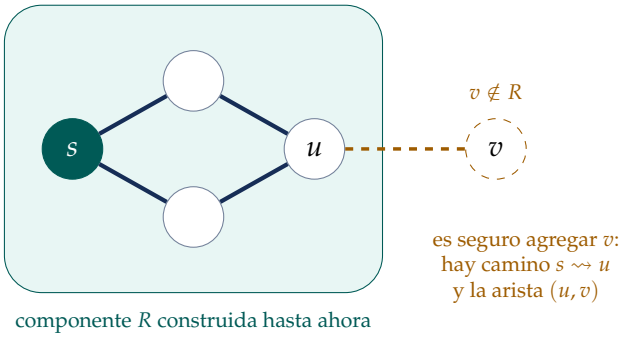


Figura 3.7. El paso básico del crecimiento de una componente. Cualquier orden de elección de la arista (u, v) da un algoritmo correcto; BFS y DFS son dos maneras particulares de decidir cuál mirar primero.

Proposición 3.2.6. El conjunto R producido al finalizar el algoritmo es exactamente la componente conexa de G que contiene a s.

Demostración. Ya observamos que todo nodo $v \in R$ es alcanzable desde s. Supongamos ahora, por el absurdo, que existe un nodo $w \notin R$ y un camino P de s a w en G. Como $s \in R$ y $w \notin R$, existe un primer nodo v sobre P que no pertenece a R, y ese nodo no es s. Sea u el nodo inmediatamente anterior a v en P, de modo que (u, v) es una arista. Por ser v el primero fuera de R, se tiene $u \in R$. Entonces (u, v) es una arista con $u \in R$ y $v \notin R$, lo que contradice la condición de parada del bucle. □

Nota 3.2.7 (Recuperar el camino). Para recuperar el camino de s a un nodo $t \in R$ alcanza con registrar, para cada nodo v, la arista (u, v) que se consideró en la iteración en que v fue agregado a R. Siguiendo esas aristas hacia atrás desde t se recorren nodos agregados en iteraciones cada vez más tempranas, hasta llegar a s.

3.2.3 Búsqueda en profundidad (DFS)

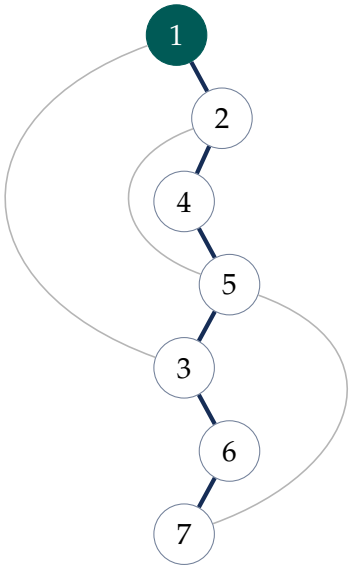
El algoritmo genérico anterior está subespecificado: no dice qué arista mirar primero. BFS resuelve esa indeterminación por capas. La otra respuesta natural es la que daría alguien que camina efectivamente por el laberinto: desde s se sigue la primera arista disponible hasta un nodo v; desde v, la primera arista disponible; y así hasta llegar a un callejón sin salida, es decir, a un nodo cuyos vecinos ya fueron todos explorados. Entonces se retrocede hasta el último nodo con un vecino sin explorar y se continúa desde allí. Este algoritmo se llama búsqueda en profundidad (DFS), porque explora yendo tan lejos como puede y solo retrocede cuando no le queda alternativa.

DFS(u)
1 Marcar u como "explorado"
2 Agregar u a R
3 **for** cada arista (u, v) incidente a u **do**
4 **if** (v no está marcado "explorado") **then**
5 Invocar recursivamente **DFS**(v)
6 **end if**
7 **end for**

> **Estructuras de datos.** Listas de adyacencia para G y un arreglo de booleanos $Explorado[1..n]$. La pila de llamadas recursivas hace de estructura de pendientes; su profundidad es $O(n)$ en el peor caso, lo que conviene tener presente en grafos grandes: la versión iterativa de la Sección 3.3 usa una pila explícita y evita ese consumo.

Para resolver la conectividad s-t se marcan todos los nodos como no explorados y se invoca DFS(s).

DFS visita exactamente el mismo conjunto de nodos que BFS —ambos construyen la componente conexa de s— pero típicamente en un orden muy distinto: se aleja de s por caminos largos antes de volver a mirar nodos cercanos. También DFS induce un árbol enraizado T: se hace s la raíz y se declara a u padre de v cuando la llamada DFS(v) se produce directamente dentro de DFS(u). El árbol resultante se llama árbol de búsqueda en profundidad y, como muestra la figura siguiente, tiende a ser angosto y profundo, en lugar de ancho y bajo como el de BFS.



orden de descubrimiento:
1, 2, 4, 5, 3, 6, 7

aristas fuera del árbol: $(1,3)$, $(2,5)$, $(5,7)$.
En los tres casos un extremo es ancestro del otro:
eso es lo que afirma la Proposición 3.2.9.

compárese con la Figura 3.6:
mismos nodos, mismas aristas,
árboles de forma opuesta

el árbol DFS es angosto y profundo

Figura 3.8. Árbol de búsqueda en profundidad de G_1 enraizado en 1, con las listas de vecinos recorridas en orden creciente. DFS avanza por una única rama larga $(1,2,4,5,3,6,7)$ y solo retrocede cuando no encuentra vecinos nuevos.

Proposición 3.2.8. Para una llamada recursiva DFS(u), todos los nodos marcados "explorados" entre el comienzo y el fin de esa llamada son descendientes de u en T.

Proposición 3.2.9. Sea T un árbol DFS, sean x e y nodos de T y sea (x,y) una arista de G que no pertenece a T. Entonces uno de los dos nodos es ancestro del otro.

Demostración. Supongamos, sin pérdida de generalidad, que x es alcanzado primero por el algoritmo. Cuando la arista (x,y) es examinada durante la ejecución de DFS(x), no se agrega a T porque y ya está marcado "explorado". Como y no estaba marcado al comenzar DFS(x), fue descubierto entre el comienzo y el fin de esa llamada; por la Proposición 3.2.8, y es descendiente de x. □

Nota 3.2.10 (BFS y DFS, en una frase). Ambos construyen la misma componente conexa y ambos lo hacen en tiempo lineal, pero con estructuras de árbol opuestas: BFS descubre por cercanía y sus aristas no arbóreas unen capas iguales o consecutivas; DFS descubre por profundidad y sus aristas no arbóreas unen ancestros con descendientes.

3.2.4 El conjunto de todas las componentes conexas

Hasta aquí hablamos de la componente conexa de un nodo particular s; pero hay una componente asociada a cada nodo del grafo. La relación entre ellas es muy rígida.

Proposición 3.2.11. Para todo par de nodos s y t de un grafo, sus componentes conexas son idénticas o disjuntas.

Demostración. Supongamos primero que existe un camino entre s y t. Para todo nodo v de la componente de s, v es alcanzable desde t: basta ir de t a s y luego de s a v. El mismo razonamiento vale intercambiando los papeles de s y t, de modo que un nodo está en la componente de uno si y solo si está en la del otro: las componentes coinciden.

Supongamos ahora que no hay camino entre s y t. Si existiera un nodo v perteneciente a ambas componentes, podríamos ir de s a v y luego de v a t, obteniendo un camino de s a t, lo que es absurdo. Luego las componentes son disjuntas. □

La demostración sugiere un algoritmo para hallar todas las componentes: se toma un nodo cualquiera s y se genera su componente con BFS (o DFS); luego se busca un nodo v no visitado y se repite desde v, obteniendo una componente que, por la Proposición 3.2.11, es disjunta de la anterior; se continúa hasta visitar todos los nodos. En G_1 este procedimiento produce dos componentes: $\{1, \ldots, 7\}$ y $\{8, 9\}$.

3.3 Implementación de los recorridos: colas y pilas

Hasta aquí describimos los recorridos sin comprometernos con una representación concreta del grafo. En esta sección discutimos cómo representar un grafo con arreglos y listas, y luego implementamos BFS y DFS en tiempo lineal. Veremos que ambos algoritmos son esencialmente el mismo, y que difieren solo en la estructura de datos que usan para decidir a qué nodo mirar después: una cola en un caso, una pila en el otro.

Un grafo $G = (V, E)$ tiene dos parámetros naturales de tamaño: la cantidad de nodos $n = |V|$ y la cantidad de aristas $m = |E|$. Los tiempos de ejecución se expresarán en función de ambos. Como hay a lo sumo una arista por par de nodos, vale $m \leq \binom{n}{2} \leq n^2$; y si el grafo es conexo, por la Proposición 3.1.11 vale $m \geq n - 1$. En esta sección apuntamos a implementaciones de tiempo $O(m + n)$, al que llamaremos tiempo lineal, porque leer la entrada ya cuesta $O(m + n)$.

3.3.1 Matriz de adyacencia y listas de adyacencia

Definición 3.3.1 (Matriz de adyacencia)**.** Sea $G = (V, E)$ con $V = \{1, \ldots, n\}$. Su matriz de adyacencia es la matriz A de $n \times n$ tal que $A[u, v] = 1$ si $(u, v) \in E$ y $A[u, v] = 0$ en caso contrario. Si G no es dirigido, A es simétrica.

Definición 3.3.2 (Listas de adyacencia)**.** La representación por listas de adyacencia consiste en un arreglo Ady de largo n, donde $Ady[v]$ es una lista con todos los nodos adyacentes a v. En un grafo no dirigido cada arista $e = (v, w)$ aparece en dos listas: w en la de v y v en la de w.

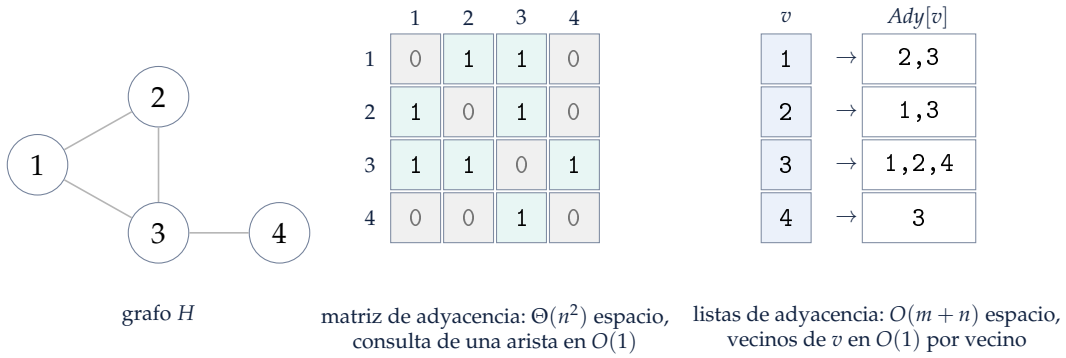


Figura 3.9. Las dos representaciones del mismo grafo. La matriz responde rápido a "¿existe la arista (u, v)?"; las listas responden rápido a "¿quiénes son los vecinos de v?". Como los recorridos hacen exactamente esta segunda pregunta, usaremos listas de adyacencia.

Llamamos grado de un nodo v, notado n_v, a la cantidad de aristas incidentes a v. El largo de la lista $Ady[v]$ es entonces n_v, y el largo total de todas las listas es $\sum_{v \in V} n_v$. Esta suma aparece constantemente en el análisis de algoritmos sobre grafos, así que conviene calcularla de una vez.

Proposición 3.3.3 (Suma de grados)**.** $\sum_{v \in V} n_v = 2m$.

Demostración. Cada arista $e = (v, w)$ contribuye exactamente dos veces a la suma: una vez en n_v y otra en n_w. Como la suma total es la suma de las contribuciones de todas las aristas, vale $2m$. □

Proposición 3.3.4. La matriz de adyacencia requiere $\Theta(n^2)$ espacio, mientras que las listas de adyacencia requieren $O(m + n)$.

Demostración. La matriz es de $n \times n$. Para las listas se necesita un arreglo de n punteros más el espacio de las listas, cuyo largo total es $\sum_{v \in V} n_v = 2m$ por la Proposición 3.3.3; en total $O(m+n)$. □

Como $m \leq n^2$, la cota $O(m+n)$ nunca es peor que $O(n^2)$, y es mucho mejor cuando el grafo es ralo, es decir, cuando m es bastante menor que n^2. En contrapartida, con listas de adyacencia consultar si una arista (u, v) existe cuesta $O(n_u)$ en lugar de $O(1)$. Para explorar un grafo el intercambio es claramente favorable a las listas: un algoritmo situado en u lee sus vecinos a costo constante por vecino, se traslada a uno de ellos y repite. La lista de adyacencia corresponde así a la idea física de exploración: uno se entera de los vecinos de u recién cuando llega a u.

3.3.2 Colas y pilas

Definición 3.3.5 (Cola y pila)**.** Una cola es un conjunto del que se extraen los elementos en orden FIFO (*first-in, first-out*): se selecciona siempre el que fue agregado hace más tiempo. Una pila es un conjunto del que se extraen los elementos en orden LIFO (*last-in, first-out*): se selecciona siempre el agregado más recientemente.

Ambas se implementan con una lista doblemente enlazada. En los dos casos se extrae el primer elemento de la lista; la diferencia está en dónde se inserta: en una cola el elemento nuevo va al final, en una pila va al principio. Como la lista doblemente enlazada mantiene punteros explícitos al primer y al último elemento, ambas inserciones cuestan $O(1)$.

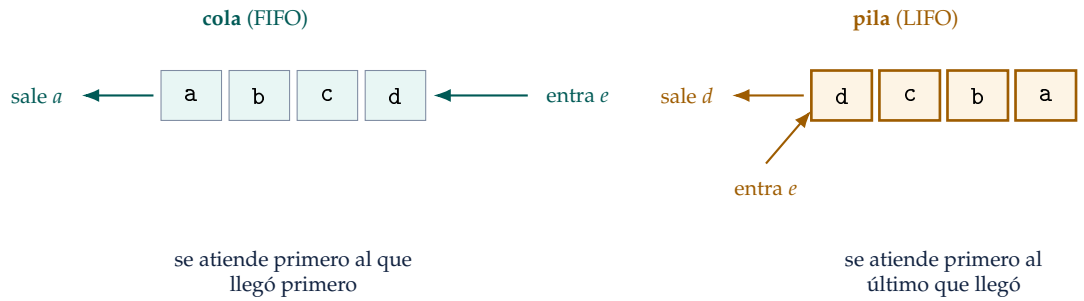


Figura 3.10. Cola y pila difieren solo en el extremo por el que entran los elementos. Esa diferencia mínima es exactamente la que separa BFS de DFS: con cola, la exploración avanza por capas; con pila, se hunde por una rama.

3.3.3 Implementación de BFS

La estructura de listas de adyacencia es ideal para BFS. Al recorrer las aristas que salen de u y llegar a una arista (u, v) necesitamos saber si v ya fue descubierto; para ello mantenemos un arreglo *Descubierto* de largo n y ponemos $Descubierto[v] =$ **true** apenas

la búsqueda ve a v por primera vez. Los nodos de la capa L_i se mantienen en una lista $L[i]$.

```
BFS(s)
 1  Descubierto[v] ← false para todo v ∈ V
 2  Descubierto[s] ← true
 3  L[0] ← {s}
 4  i ← 0
 5  T ← ∅
 6  while L[i] no esté vacía do
 7      Inicializar una lista vacía L[i + 1]
 8      for cada nodo u ∈ L[i] do
 9          for cada arista (u, v) incidente a u do
10              if (Descubierto[v] = false) then
11                  Descubierto[v] ← true
12                  Agregar la arista (u, v) al árbol T
13                  Agregar v a la lista L[i + 1]
14              end if
15          end for
16      end for
17      i ← i + 1
18  end while
```

> **Estructuras de datos.** Listas de adyacencia para G, y un arreglo de booleanos *Descubierto*$[1..n]$ indexado por nodo. Cada capa $L[i]$ es una lista enlazada, de modo que agregar un nodo al final cuesta $O(1)$ y recorrer la capa cuesta proporcional a su largo. El arreglo de capas puede a su vez ser un arreglo dinámico de listas, ya que la cantidad de capas no se conoce de antemano pero nunca supera n.

En esta implementación da lo mismo administrar cada lista $L[i]$ como cola o como pila, porque el algoritmo puede considerar los nodos de una capa en cualquier orden.

Proposición 3.3.6. La implementación anterior de BFS corre en tiempo $O(m + n)$ si el grafo está dado por listas de adyacencia.

Demostración. Primero, una cota más débil. Hay a lo sumo n listas $L[i]$ para inicializar, lo que cuesta $O(n)$. Cada nodo aparece en a lo sumo una lista, así que el cuerpo del for de la línea 5 se ejecuta a lo sumo n veces en total; y al considerar un nodo u hay que recorrer sus a lo sumo n aristas incidentes, a costo $O(1)$ cada una. Esto da $O(n^2)$.

Para obtener $O(m + n)$ observemos que procesar un nodo u cuesta $O(n_u)$ y no $O(n)$, donde n_u es el grado de u. Entonces el tiempo total dedicado a examinar aristas es

$$O\Big(\sum_{u \in V} n_u\Big) = O(2m) = O(m),$$

por la Proposición 3.3.3. Sumando el $O(n)$ necesario para inicializar las listas y el arreglo *Descubierto*, el total es $O(m+n)$. □

Nota 3.3.7 (Una sola cola). En lugar de mantener una lista por capa, se puede usar una única lista L administrada como cola: cada nodo descubierto se agrega al final, y el algoritmo procesa siempre las aristas del nodo que está primero. Como todos los nodos de L_i entran a la cola antes que los de L_{i+1}, se los procesa en bloques contiguos por capa y el resultado es idéntico al de la versión anterior.

Esta variante es la que suele escribirse en la práctica. Al desaparecer las listas $L[i]$ hay que registrar de otro modo en qué capa quedó cada nodo, y para eso se agrega un arreglo *nivel*: cuando v se descubre desde u, se le asigna el nivel de u más uno. También registramos explícitamente $padre[v]$, que es lo que construye el árbol de recorrida.

BFS-COLA(s)

```
1  Descubierto[v] ← false para todo v ∈ V
2  padre[v] ← null para todo v ∈ V
3  nivel[v] ← 0 para todo v ∈ V
4  Descubierto[s] ← true
5  Inicializar una cola Q que contenga únicamente a s
6  while Q no esté vacía do
7      Retirar de Q el primer nodo u      /* acá se explora u */
8      for cada arista (u, v) incidente a u do
9          if (Descubierto[v] = false) then
10             Descubierto[v] ← true
11             padre[v] ← u
12             nivel[v] ← nivel[u] + 1
13             Agregar v al final de Q
14         end if
15     end for
16 end while
```

> **Estructuras de datos.** Listas de adyacencia, arreglos *Descubierto*, *padre* y *nivel* indexados por nodo, y una cola Q. La cola se implementa con una lista doblemente enlazada con punteros a primero y último, o bien con un arreglo circular de largo n: como cada nodo entra a lo sumo una vez, ese tamaño alcanza y evita todo pedido dinámico de memoria. Encolar y desencolar cuestan $O(1)$ en las dos variantes.

Al terminar, $nivel[v]$ es el número de capa de v y, por la Proposición 3.2.3, coincide con $\text{dist}(s, v)$; las aristas $(v, padre[v])$ son exactamente las del árbol T. El análisis es el mismo que el de la Proposición 3.3.6: cada nodo entra y sale de Q a lo sumo una vez, y al salir se recorre su lista de adyacencia, de modo que el total es $O(m+n)$.

3.3.4 Implementación de DFS

DFS puede verse como casi idéntico a BFS, con la diferencia de que mantiene los nodos pendientes en una pila en lugar de una cola. Conviene distinguir dos actos: descubrir un nodo v —la primera vez que se lo ve, al recorrer una arista que llega a él— y explorarlo —cuando se recorren todas sus aristas incidentes, descubriendo posiblemente nodos nuevos—. BFS y DFS difieren en cómo intercalan ambos actos: BFS descubre todos los vecinos de la capa actual y difiere su exploración; DFS es más impulsivo, y apenas encuentra un vecino no explorado se muda inmediatamente a explorarlo.

```
DFS-ITERATIVO(s)
 1  Inicializar la pila S con el único elemento s
 2  while S no esté vacía do
 3      Extraer un nodo u de S
 4      if (Explorado[u] = false) then
 5          Explorado[u] ← true
 6          for cada arista (u, v) incidente a u do
 7              Apilar v en S
 8              if (Explorado[v] = false) then
 9                  padre[v] ← u
10              end if
11          end for
12      end if
13  end while
```

> **Estructuras de datos.** Listas de adyacencia, arreglos *Explorado* y *padre*, y una pila S. Acá la pila no puede ser un arreglo de largo n: un mismo nodo se apila una vez por cada vecino explorado que lo alcanza, de modo que llegan a convivir hasta $2m$ elementos. Conviene entonces una lista enlazada usada como pila, o un arreglo dinámico que duplique su tamaño al llenarse; en ambos casos apilar y desapilar cuestan $O(1)$ amortizado.

Obsérvese la diferencia con BFS en el uso de las marcas: aquí $Explorado[v]$ se pone en true recién cuando se recorren las aristas incidentes a v, mientras que BFS marca $Descubierto[v]$ apenas ve a v por primera vez. Como consecuencia, un mismo nodo v puede figurar varias veces en la pila —una por cada vecino explorado que lo apiló—, pero solo la copia que se extraiga primero lo explora: las demás encuentran $Explorado[v] =$ **true** y se descartan. Al marcar $u \neq s$ como explorado se agrega la arista $(u, padre[u])$ al árbol T.

Nota 3.3.8 (Sobre la asignación de *padre*). La guarda de la línea 8 evita tocar $padre[v]$ una vez que v ya fue explorado, que es cuando su arista al árbol quedó fijada; sin ella se estropearía un árbol ya construido. Mientras v sigue sin explorar, en cambio, las sucesivas apiladas sobrescriben $padre[v]$ sin consecuencias: la última escritura anterior a la exploración de v corresponde al nodo que apiló la copia que está más arriba en la pila, es decir, exactamente el nodo desde el cual v va a ser explorado.

Nota 3.3.9. Este algoritmo apila todos los vecinos antes de considerar a cualquiera de ellos, por lo que recorre cada lista de adyacencia en orden inverso respecto de la versión recursiva. Visita los nodos en el mismo orden que aquella, salvo por esa inversión.

Proposición 3.3.10. La implementación anterior de DFS corre en tiempo $O(m+n)$ si el grafo está dado por listas de adyacencia.

Demostración. El trabajo del algoritmo consiste en apilar y desapilar nodos, a costo $O(1)$ cada operación; basta entonces acotar la cantidad de nodos apilados, ya que cada uno se desapila a lo sumo una vez por cada vez que se apiló. Un nodo v se apila cada vez que se explora uno de sus n_v vecinos, de modo que la cantidad total de apiladas es a lo sumo $\sum_{v \in V} n_v = 2m$. Sumando el $O(n)$ de la inicialización, el total es $O(m+n)$. □

3.3.5 Todas las componentes conexas

Las dos implementaciones anteriores exploran la componente de un único nodo s. Para obtener todas las componentes de G se envuelve la búsqueda en un bucle que recorre los nodos $1, \ldots, n$ y arranca una búsqueda nueva cada vez que encuentra un nodo todavía no alcanzado, tal como sugiere la Proposición 3.2.11. Los arreglos de marcas, de padres y de niveles se inicializan una sola vez, fuera del bucle —por eso las búsquedas invocadas omiten su propia línea de inicialización—: son globales a todas las búsquedas, y es justamente eso lo que impide volver a recorrer una componente ya visitada.

COMPONENTES-BFS(G)

```
1  Descubierto[u] ← false para todo u ∈ V
2  padre[u] ← null para todo u ∈ V
3  nivel[u] ← 0 para todo u ∈ V
4  for s ← 1 to n do
5      if (Descubierto[s] = false) then
6          BFS-COLA(s)      /* nueva componente */
7      end if
8  end for
```

> **Estructuras de datos.** Las mismas estructuras de BFS-COLA, con la diferencia de que los arreglos se reservan e inicializan una sola vez, fuera del bucle. Si además se quiere saber a qué componente pertenece cada nodo, se agrega un arreglo $comp[1..n]$ que se escribe al descubrir cada nodo.

La versión con DFS es idéntica salvo por la marca que consulta, que ahora es *Explorado*, y por la búsqueda que invoca.

COMPONENTES-DFS(G)

```
1  Explorado[u] ← false para todo u ∈ V
2  padre[u] ← null para todo u ∈ V
3  for s ← 1 to n do
4      if (Explorado[s] = false) then
5          DFS-ITERATIVO(s)      /* nueva componente */
6      end if
7  end for
```

> **Estructuras de datos.** Idénticas a las de DFS-ITERATIVO, con los arreglos reservados una sola vez. La pila se puede reutilizar entre componentes: al terminar una búsqueda queda vacía.

Para saber a qué componente pertenece cada nodo alcanza con llevar un contador c que se incremente en cada invocación de la línea 6 (respectivamente, de la línea 5) y escribir $comp[v] \leftarrow c$ al descubrir cada v. Nótese que el conjunto de aristas $(v, padre[v])$ ya no forma un árbol sino un bosque: un árbol de recorrida por componente.

Proposición 3.3.11. Ambos procedimientos calculan todas las componentes conexas de G en tiempo $O(m + n)$.

Demostración. Que el resultado es correcto se desprende de la Proposición 3.2.6 —cada búsqueda devuelve la componente de su nodo de partida— junto con la Proposición 3.2.11: cuando el bucle llega a un nodo s ya marcado, su componente ya fue generada por una búsqueda anterior, y las componentes son idénticas o disjuntas, de modo que ninguna se genera dos veces ni queda sin generar.

En cuanto al tiempo, el bucle externo cuesta $O(n)$ y lo mismo las inicializaciones. Cada búsqueda solo trabaja sobre los nodos y las aristas de su propia componente, y las componentes son disjuntas; si la componente C_i tiene n_i nodos y m_i aristas, la búsqueda correspondiente cuesta $O(m_i + n_i)$ y el total es $O\big(\sum_i (m_i + n_i)\big) = O(m + n)$. □

3.4 Verificación de bipartición: una aplicación de BFS

Definición 3.4.1 (Grafo bipartito)**.** Un grafo $G = (V, E)$ es bipartito si su conjunto de nodos puede particionarse en dos conjuntos X e Y de modo que toda arista tenga un extremo en X y el otro en Y.

Para hacer más fluida la discusión, imaginemos que los nodos de X se pintan de rojo y los de Y de azul. Entonces G es bipartito si y solo si es posible pintar sus nodos de rojo y azul de manera que toda arista tenga un extremo de cada color.

Ejemplo 3.4.2. Un grupo de n estudiantes debe repartirse en dos turnos de laboratorio. Algunos pares de estudiantes no pueden ir en el mismo turno (por ejemplo, porque comparten computadora en el trabajo). Construimos un grafo con un nodo por estudiante y una arista por cada par incompatible. Repartir a todos en dos turnos respetando las incompatibilidades es exactamente colorear el grafo con dos colores, es decir, decidir si el grafo es bipartito.

¿Cuándo es imposible? Un triángulo no es bipartito: se pinta un nodo de rojo, otro de azul, y el tercero no admite ningún color. Más en general, consideremos un ciclo C de longitud impar, con nodos numerados $1, 2, \ldots, 2k, 2k+1$. Si pintamos el nodo 1 de rojo, el 2 debe ser azul, el 3 rojo, y así sucesivamente: los impares quedan rojos y los pares azules. Pero entonces el nodo $2k+1$ debe ser rojo, y tiene una arista al nodo 1, que también es rojo. Queda establecido lo siguiente.

Proposición 3.4.3. Si un grafo G es bipartito, entonces no contiene ningún ciclo de longitud impar.

La pregunta natural es si los ciclos impares son el único obstáculo, o si existen otros más sutiles. Veremos que son el único, y la demostración vendrá acompañada de un algoritmo.

3.4.1 El algoritmo

Supongamos G conexo (si no lo es, se calculan sus componentes y se analiza cada una por separado). Elegimos un nodo cualquiera s y lo pintamos de rojo, lo cual no quita generalidad porque s debe recibir algún color. Entonces todos los vecinos de s deben ser azules; los vecinos de estos, rojos; y así hasta pintar todo el grafo. Al terminar, o bien tenemos un coloreo válido, o bien existe una arista con ambos extremos del mismo color.

Este procedimiento es idéntico a BFS: avanzamos hacia afuera desde s pintando cada nodo apenas lo encontramos. Concretamente, ejecutamos BFS y pintamos s y todas las capas pares de rojo, y todas las capas impares de azul. Basta agregar a la implementación de BFS un arreglo *Color*: cuando se agrega un nodo v a la lista $L[i+1]$ se asigna $Color[v]$

según la paridad de $i+1$. Al final se recorren todas las aristas y se verifica si alguna tiene ambos extremos del mismo color. El tiempo total es $O(m+n)$, igual que BFS.

Teorema 3.4.4. Sea G un grafo conexo y sean $L_0, L_1, L_2, \ldots$ las capas producidas por BFS desde un nodo s. Entonces vale exactamente una de las dos afirmaciones siguientes:

(I) Ninguna arista de G une dos nodos de una misma capa. En ese caso G es bipartito: los nodos de las capas pares pueden pintarse de rojo y los de las impares de azul.

(II) Alguna arista de G une dos nodos de una misma capa. En ese caso G contiene un ciclo de longitud impar y por lo tanto no es bipartito.

Demostración. Caso (i). Por la Proposición 3.2.5, toda arista de G une nodos de la misma capa o de capas consecutivas. La hipótesis del caso descarta la primera alternativa, de modo que toda arista une capas consecutivas. Como el coloreo asigna colores opuestos a capas consecutivas, toda arista tiene extremos de colores opuestos y el coloreo prueba que G es bipartito.

Caso (ii). Sea $e = (x, y)$ una arista con $x, y \in L_j$. Sea T el árbol BFS producido por el algoritmo y sea z el nodo de mayor número de capa entre los ancestros comunes de x e y en T, es decir, el ancestro común más bajo; digamos $z \in L_i$ con $i < j$. Consideremos el ciclo C formado por el camino de z a x en T, la arista e, y el camino de y a z en T. Su longitud es

$$(j-i) + 1 + (j-i) = 2(j-i) + 1,$$

que es impar. Por la Proposición 3.4.3, G no es bipartito. □

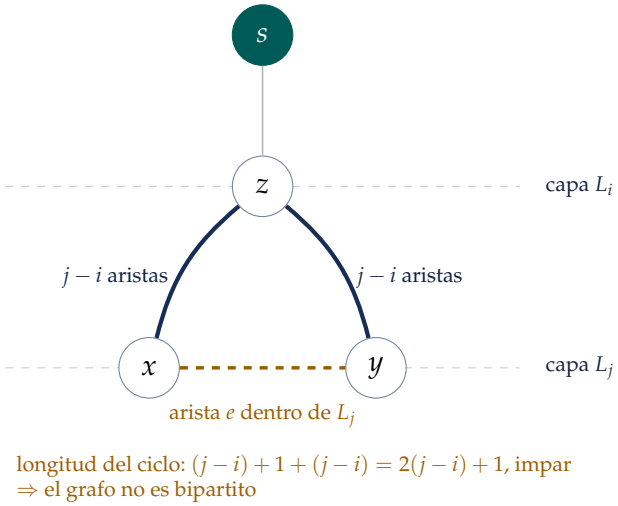


Figura 3.11. Si dos nodos de una misma capa están unidos por una arista, el ciclo que pasa por ambos y por su ancestro común más bajo tiene longitud impar. Los dos tramos que bajan de z miden lo mismo porque terminan en la misma capa; la arista e agrega la unidad que rompe la paridad.

Ejemplo 3.4.5 (G_1 no es bipartito)**.** Volvamos al grafo G_1 de la Figura 3.5. Las capas desde $s = 1$ son $L_1 = \{2, 3\}$, $L_2 = \{4, 5, 6\}$ y $L_3 = \{7\}$. La arista $(4, 5)$ une dos nodos de la capa L_2, de modo que estamos en el caso (ii) del Teorema 3.4.4. El ancestro común más bajo de 4 y 5 en el árbol BFS es $z = 2$, y el ciclo $2, 4, 5, 2$ tiene longitud 3: impar, como anuncia la demostración. En cambio, si se elimina la arista $(4, 5)$ el grafo resultante sí es bipartito, con $X = \{1, 4, 5, 6\}$ (capas pares) e $Y = \{2, 3, 7\}$ (capas impares).

3.5 Conectividad en grafos dirigidos

Veamos en qué medida las ideas anteriores se trasladan al caso dirigido. La asimetría de la relación tiene efectos cualitativos: navegar la Web consiste en seguir aristas de un grafo dirigido, y en general no es posible "navegar hacia atrás" siguiendo enlaces en sentido inverso.

3.5.1 Representación y recorridos

Para representar un grafo dirigido se usa una variante de las listas de adyacencia: cada nodo tiene asociadas dos listas, una con los nodos hacia los que tiene aristas y otra con los nodos desde los que recibe aristas. Así, un algoritmo situado en u puede leer tanto los nodos alcanzables dando un paso hacia adelante como los que lo alcanzarían dando un paso hacia atrás.

BFS y DFS son prácticamente idénticos al caso no dirigido: se parte de s, la primera capa son los nodos a los que s tiene una arista, la segunda los nodos nuevos a los que estos tienen una arista, y así sucesivamente. Los nodos de la capa j son aquellos cuyo camino más corto desde s tiene exactamente j aristas, y el tiempo sigue siendo $O(m+n)$.

Lo que hay que entender bien es qué calcula esta versión dirigida: el conjunto de nodos t tales que s tiene un camino hasta t. Esos nodos pueden tener o no un camino de regreso a s. Si lo que se quiere es el conjunto de nodos que tienen un camino hasta s, basta definir el grafo G^{rev}, obtenido de G invirtiendo la dirección de todas las aristas, y correr BFS o DFS sobre G^{rev}: un nodo tiene camino desde s en G^{rev} si y solo si tiene camino hacia s en G.

3.5.2 Conectividad fuerte

Definición 3.5.1 (Mutuamente alcanzables, componente fuerte)**.** Dos nodos u y v de un grafo dirigido son mutuamente alcanzables si existe un camino de u a v y también un camino de v a u. La componente fuerte que contiene a un nodo s es el conjunto de todos los nodos v tales que s y v son mutuamente alcanzables.

Con esta terminología, un grafo dirigido es fuertemente conexo si y solo si todo par de nodos es mutuamente alcanzable.

Proposición 3.5.2. Si u y v son mutuamente alcanzables, y v y w son mutuamente alcanzables, entonces u y w son mutuamente alcanzables.

Demostración. Para construir un camino de u a w vamos primero de u a v y luego de v a w, usando los caminos garantizados por ambas hipótesis. Para el camino de w a u invertimos el razonamiento: de w a v y luego de v a u. □

Esta proposición da un algoritmo de tiempo lineal para decidir si G es fuertemente conexo: se elige un nodo cualquiera s, se corre BFS en G desde s y luego BFS en G^{rev} desde s. Si alguna de las dos búsquedas no alcanza todos los nodos, G no es fuertemente conexo. Si ambas los alcanzan, entonces s y v son mutuamente alcanzables para todo v, y por la Proposición 3.5.2 todo par u, v lo es. Nótese que el mismo par de búsquedas calcula la componente fuerte de s: es el conjunto de nodos alcanzados por ambas.

Proposición 3.5.3. Para todo par de nodos s y t de un grafo dirigido, sus componentes fuertes son idénticas o disjuntas.

Demostración. Si s y t son mutuamente alcanzables, para todo v vale: si s y v son mutuamente alcanzables entonces, por la Proposición 3.5.2, t y v también; y simétricamente. Luego las componentes coinciden.

Si s y t no son mutuamente alcanzables, no puede existir v en ambas componentes: de existir, s y v serían mutuamente alcanzables y v y t también, de donde s y t lo serían, contra lo supuesto. □

Ejemplo 3.5.4 (Calles de sentido único)**.** Un barrio tiene seis esquinas y todas sus calles son de sentido único. Modelamos las esquinas con nodos y cada calle con una arista dirigida. El grafo resultante (Figura 3.12) no es fuertemente conexo: desde la esquina f no se puede volver a ninguna otra. Sus componentes fuertes son $\{a, b, c\}$, $\{d, e\}$ y $\{f\}$: dentro de cada una se puede ir y volver, pero entre ellas el tránsito solo fluye en una dirección.

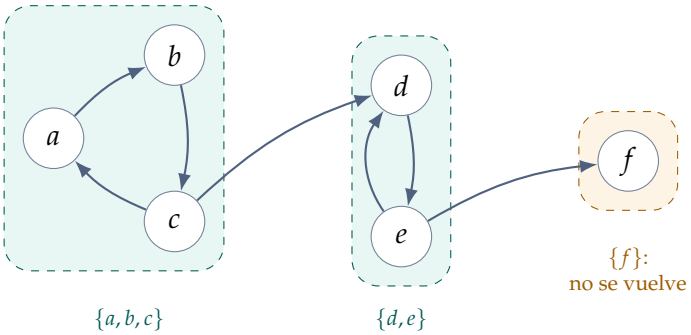


Figura 3.12. Componentes fuertes de un grafo dirigido. Dentro de una componente se puede ir y volver; entre componentes distintas, el flujo es en un solo sentido. Como en el caso no dirigido, las componentes particionan el conjunto de nodos.

Nota 3.5.5. Con más trabajo del que haremos aquí, es posible calcular las componentes fuertes de todos los nodos en tiempo total $O(m + n)$.

3.6 Grafos dirigidos acíclicos y orden topológico

Si un grafo no dirigido no tiene ciclos, su estructura es muy pobre: cada componente conexa es un árbol. En cambio, un grafo dirigido puede carecer de ciclos dirigidos y aun así tener una estructura muy rica; por ejemplo, puede tener muchísimas aristas: si tomamos $V = \{1, \ldots, n\}$ e incluimos la arista (i, j) cada vez que $i < j$, el grafo resultante tiene $\binom{n}{2}$ aristas y ningún ciclo.

Definición 3.6.1 (DAG)**.** Un grafo dirigido sin ciclos dirigidos se llama grafo dirigido acíclico, o DAG (por *directed acyclic graph*).

Los DAG son ubicuos en computación porque las redes de dependencias del tipo que mencionamos en la Sección 3.2 son acíclicas. Supongamos que tenemos un conjunto de tareas $\{1, \ldots, n\}$ y ciertas restricciones de precedencia que estipulan, para algunos pares i, j, que i debe realizarse antes que j. Representamos las tareas con nodos y agregamos la arista (i, j) cuando i debe hacerse antes que j. Si la relación de precedencia ha de tener algún sentido, el grafo debe ser un DAG: si contuviera un ciclo C, ninguna tarea de C podría comenzar nunca, porque cada una esperaría a otra del mismo ciclo.

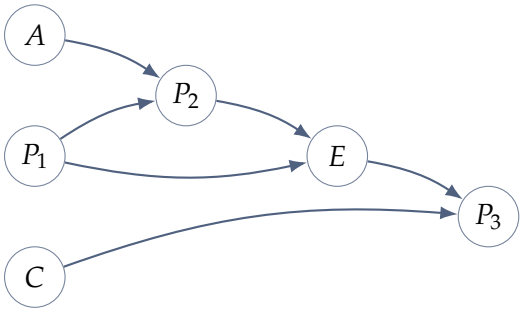
Definición 3.6.2 (Orden topológico)**.** Un orden topológico de un grafo dirigido G es una enumeración $v_1, v_2, \ldots, v_n$ de sus nodos tal que para toda arista (v_i, v_j) se cumple $i < j$. Es decir, todas las aristas apuntan "hacia adelante" en el orden.

Un orden topológico sobre un conjunto de tareas es precisamente un orden en el que pueden ejecutarse sin violar ninguna dependencia: al llegar a la tarea v_j, todas las tareas que debían precederla ya fueron realizadas.

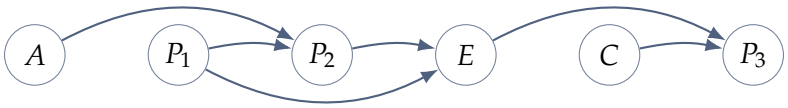
Ejemplo 3.6.3 (Previaturas)**.** Una carrera tiene seis materias: Álgebra (A), Cálculo (C), Programación 1 (P_1), Programación 2 (P_2), Estructuras de Datos (E) y Programación 3 (P_3), con las previaturas

$$P_1 \to P_2, \qquad P_1 \to E, \qquad A \to P_2, \qquad P_2 \to E, \qquad E \to P_3, \qquad C \to P_3.$$

El orden A, P_1, P_2, E, C, P_3 es topológico: cada materia aparece después de todas sus previas. No es el único: C, P_1, A, P_2, E, P_3 también lo es. En cambio P_1, P_2, E, P_3, A, C no lo es, porque A aparece después de P_2 pese a ser previa suya.



(a) el DAG de previaturas



(b) los mismos nodos ordenados como A, P_1, P_2, E, C, P_3:
todas las aristas apuntan hacia la derecha

Figura 3.13. Un orden topológico es una forma de "aplanar" un DAG sobre una recta de modo que ninguna arista apunte hacia atrás. Visualmente, el dibujo (b) es una prueba inmediata de que el grafo no tiene ciclos.

Proposición 3.6.4. Si G admite un orden topológico, entonces G es un DAG.

Demostración. Supongamos por el absurdo que G tiene un orden topológico $v_1, \ldots, v_n$ y además un ciclo C. Sea v_i el nodo de menor índice de C y sea v_j el nodo de C inmediatamente anterior a v_i, de modo que (v_j, v_i) es una arista. Por la elección de i vale $j > i$, lo que contradice que $v_1, \ldots, v_n$ sea un orden topológico. □

3.6.1 Cálculo de un orden topológico

La pregunta interesante es la recíproca: ¿todo DAG admite un orden topológico y, en tal caso, cómo se lo calcula eficientemente? La clave está en saber cómo empezar: el primer nodo del orden no puede tener aristas entrantes, pues cualquiera de ellas apuntaría hacia atrás.

Proposición 3.6.5. En todo DAG G existe un nodo v sin aristas entrantes.

Demostración. Sea G un grafo dirigido en el que todo nodo tiene al menos una arista entrante; mostraremos que G tiene un ciclo. Tomemos un nodo cualquiera v y sigamos aristas hacia atrás: como v tiene una arista entrante (u, v), caminamos hasta u; como u tiene una arista entrante (x, u), caminamos hasta x; y así indefinidamente, ya que todo

nodo visitado tiene una arista entrante. Tras $n+1$ pasos habremos visitado algún nodo w dos veces. La sucesión de nodos recorrida entre ambas visitas a w forma un ciclo. □

Teorema 3.6.6. Si G es un DAG, entonces G admite un orden topológico.

Demostración. Por inducción en la cantidad de nodos. El enunciado es evidente para DAG de uno o dos nodos. Supongámoslo cierto para DAG de hasta n nodos y sea G un DAG de $n+1$ nodos. Por la Proposición 3.6.5 existe un nodo v sin aristas entrantes; lo colocamos primero, lo cual es seguro porque todas las aristas incidentes a v salen de él y por lo tanto apuntarán hacia adelante. El grafo $G - \{v\}$ sigue siendo un DAG (borrar un nodo no puede crear ciclos) y tiene n nodos, así que por hipótesis inductiva admite un orden topológico. Concatenando v con ese orden se obtiene un orden de G en el que todas las aristas apuntan hacia adelante. □

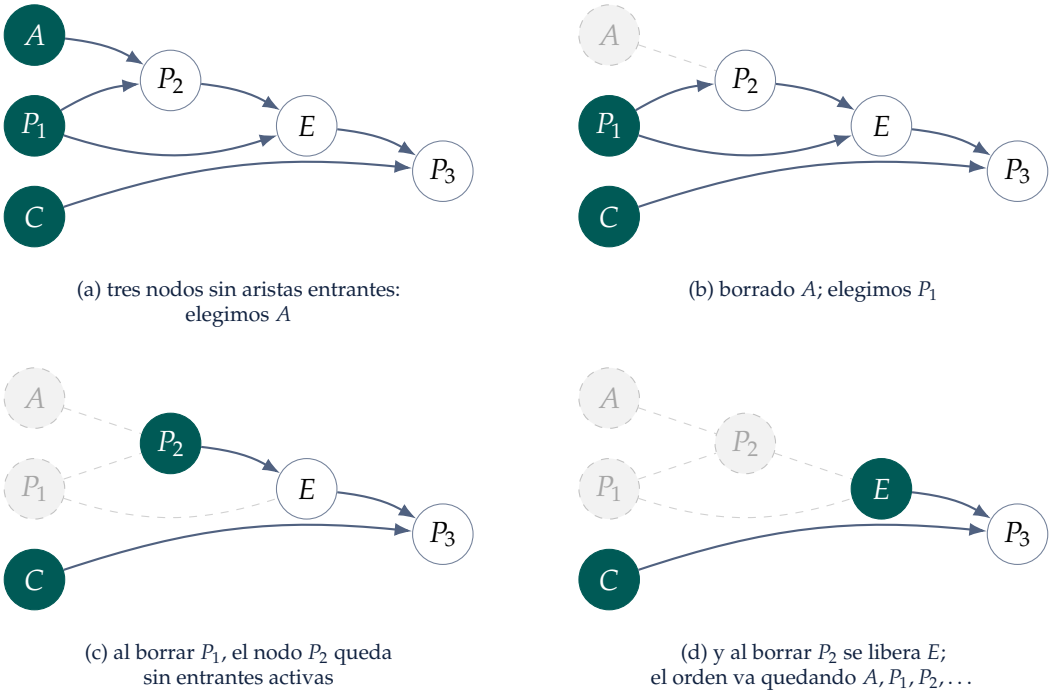
La demostración es constructiva y da el algoritmo siguiente.

ORDEN-TOPOLÓGICO(G)

1 Hallar un nodo v sin aristas entrantes y ubicarlo primero
2 Borrar v de G
3 Calcular recursivamente un orden topológico de $G - \{v\}$
4 y concatenarlo después de v

Estructuras de datos. Listas de adyacencia salientes para poder recorrer las aristas que salen de cada nodo borrado. Además:

- arreglo $ge[1..n]$ con la cantidad de aristas entrantes activas de cada nodo, que se llena con un barrido inicial en $O(m+n)$ y se decrementa en $O(1)$ por arista;
- pila o cola S con los nodos activos de grado de entrada cero. Cualquiera de las dos sirve, porque el orden en que se sacan de S no afecta la validez del resultado: cada elección produce un orden topológico posiblemente distinto, pero todos correctos;
- arreglo dinámico para el orden de salida, al que se agrega al final en $O(1)$.



(a) tres nodos sin aristas entrantes: elegimos A

(b) borrado A; elegimos P_1

(c) al borrar P_1, el nodo P_2 queda sin entrantes activas

(d) y al borrar P_2 se libera E; el orden va quedando $A, P_1, P_2, \ldots$

Figura 3.14. Primeros pasos del algoritmo sobre el DAG de previaturas. Los nodos resaltados son los que no tienen aristas entrantes desde nodos activos: la Proposición 3.6.5 garantiza que en cada etapa hay al menos uno, y borrarlo puede liberar nodos nuevos.

3.6.2 Análisis y refinamiento del tiempo

Identificar un nodo sin aristas entrantes y borrarlo de G puede hacerse en tiempo $O(n)$; como el algoritmo realiza n iteraciones, el tiempo total es $O(n^2)$. No es un mal tiempo —si el grafo es denso, con $\Theta(n^2)$ aristas, es lineal en el tamaño de la entrada— pero cuando m es mucho menor que n^2 querríamos algo mejor.

Podemos alcanzar $O(m+n)$ con el mismo esquema de alto nivel, simplemente siendo más cuidadosos al buscar esos nodos. Digamos que un nodo está activo si todavía no fue borrado, y mantengamos explícitamente:

(a) para cada nodo w, la cantidad de aristas entrantes que w recibe desde nodos activos;

(b) el conjunto S de nodos activos sin aristas entrantes desde otros nodos activos.

Al comienzo todos los nodos están activos, de modo que (a) y (b) se inicializan con una única pasada sobre nodos y aristas, en tiempo $O(m+n)$. Cada iteración consiste en tomar un nodo v de S y borrarlo; luego se recorren los nodos w a los que v tenía aristas y se resta uno a su contador. Si el contador de algún w llega a cero, se agrega w a S. De este modo se conocen en todo momento los nodos elegibles, y el trabajo total dedicado a los contadores es constante por arista.

Proposición 3.6.7. Un orden topológico de un DAG dado por listas de adyacencia puede calcularse en tiempo $O(m + n)$.

Nota 3.6.8 (Detectar ciclos). Este esquema también sirve para grafos dirigidos arbitrarios: si en algún momento quedan nodos activos pero S está vacío, entonces todos los nodos restantes tienen alguna arista entrante activa y, por la demostración de la Proposición 3.6.5, el grafo restante contiene un ciclo. Es decir, el mismo algoritmo o bien produce un orden topológico —certificando que G es un DAG— o bien detecta que no lo es.

Capítulo 4

Algoritmos Greedy

4.1 La estrategia greedy

Es difícil, quizás imposible, dar una definición precisa de qué es un algoritmo greedy. Diremos que un algoritmo es greedy si construye la solución en pasos pequeños, eligiendo en cada paso, de manera miope, la opción que optimiza algún criterio local. La palabra clave es miope: la decisión se toma mirando únicamente el estado actual, sin considerar las consecuencias futuras y sin la posibilidad de volver atrás.

Para un mismo problema suelen existir muchos algoritmos greedy distintos, uno por cada criterio local que se nos ocurra. Inventarlos es fácil; lo interesante —y lo difícil— es determinar en qué casos funcionan y demostrar que efectivamente lo hacen. Cuando un algoritmo greedy resuelve óptimamente un problema no trivial, eso dice algo sobre la estructura del problema: existe una regla de decisión local con la que se pueden construir soluciones globalmente óptimas.

En este capítulo veremos dos técnicas generales para demostrar que un algoritmo greedy produce una solución óptima.

Definición 4.1.1 (El greedy se mantiene adelante)**.** La primera técnica consiste en medir el progreso del algoritmo greedy paso a paso y probar que, en cada paso, va al menos tan bien como cualquier otra solución. De ahí se sigue que la solución final es óptima.

Definición 4.1.2 (Argumento de intercambio)**.** La segunda técnica, más general, consiste en tomar una solución óptima cualquiera y transformarla gradualmente —mediante intercambios que no empeoran su calidad— en la solución que produce el algoritmo greedy. De ahí se sigue que la solución greedy es al menos tan buena como cualquier otra.

Aparecerá también una tercera variante, muy útil en la práctica: encontrar una cota estructural que toda solución posible debe respetar, y mostrar que el algoritmo greedy siempre alcanza esa cota. La veremos en la Subsección 4.2.4.

Con estas herramientas estudiaremos cinco problemas: la planificación de intervalos, la minimización de la tardanza máxima, el camino más corto en un grafo, el árbol

de cubrimiento mínimo y la estructura Union-Find que hace eficiente al algoritmo de Kruskal.

4.2 Planificación de intervalos: el greedy se mantiene adelante

Retomemos el problema de planificación de intervalos, el primero de los cinco problemas representativos de la Subsección 1.8.1. Tenemos un conjunto de trabajos $\{1, 2, \ldots, n\}$; el trabajo i corresponde a un intervalo de tiempo que comienza en $s(i)$ y termina en $f(i)$. Cambiamos levemente la notación respecto de aquel planteo: escribimos $s(i)$ y $f(i)$ en lugar de s_i y f_i, porque esto facilita hablar de los índices en las demostraciones.

Definición 4.2.1 (Conjunto compatible)**.** Un subconjunto de trabajos es compatible si no hay dos de ellos que se solapen en el tiempo. Un conjunto compatible de cardinalidad máxima se dice óptimo.

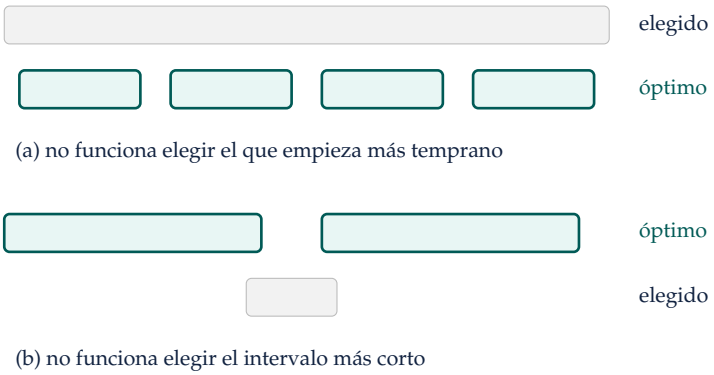
El objetivo es aceptar un conjunto compatible tan grande como sea posible. Hay un único recurso —una sala, una máquina, un procesador— y cada trabajo aceptado lo ocupa en exclusiva durante su intervalo.

4.2.1 Diseño del algoritmo

La idea de un algoritmo greedy para este problema es usar una regla simple para elegir un primer trabajo i_1. Una vez aceptado i_1, se rechazan todos los trabajos incompatibles con él; luego se elige un segundo trabajo i_2 con la misma regla, se rechazan los incompatibles con i_2, y así hasta quedarnos sin trabajos. Todo el problema está en elegir la regla, y varias reglas naturales fallan.

- La regla más obvia es tomar siempre el trabajo disponible que empieza más temprano, es decir, el de mínimo $s(i)$: así el recurso se empieza a usar cuanto antes. No es óptima. Si el trabajo más temprano ocupa un intervalo muy largo, aceptarlo obliga a rechazar muchos trabajos cortos. En el peor caso ese trabajo ocupa el recurso durante todo el horizonte y el algoritmo acepta uno solo, mientras que el óptimo acepta muchos.
- Esto sugiere aceptar primero el trabajo más corto, el de mínimo $f(i) - s(i)$. Es una regla mejor que la anterior, pero tampoco es óptima: un intervalo corto ubicado en el medio puede impedir aceptar los dos intervalos que lo rodean, que juntos formaban la solución óptima.
- El problema del caso anterior es que el intervalo corto compite con dos trabajos a la vez. Podríamos entonces contar, para cada trabajo, cuántos trabajos incompatibles tiene, y aceptar el que tenga menos conflictos. Esta regla resuelve bien el ejemplo

anterior y es bastante más difícil de romper. También falla, pero los contraejemplos conocidos necesitan instancias bastante más grandes que las de la Figura 4.1, con muchos intervalos apilados alrededor de cada extremo para inflar los conteos de conflictos; por eso no los dibujamos acá.



(a) no funciona elegir el que empieza más temprano

(b) no funciona elegir el intervalo más corto

Figura 4.1. Dos instancias en las que reglas greedy naturales fallan. En (a) el intervalo que empieza primero bloquea a otros cuatro, de modo que elegirlo deja una solución de tamaño uno donde el óptimo acepta cuatro; en (b) el intervalo más corto impide aceptar los dos que lo rodean, que juntos forman el óptimo.

La regla que sí conduce al óptimo es una cuarta idea: aceptar primero el trabajo que termina antes, es decir, el de mínimo $f(i)$. La intuición es clara: liberamos el recurso lo antes posible habiendo satisfecho un trabajo, con lo que maximizamos el tiempo disponible para atender a los demás.

Escribamos el algoritmo con un poco más de formalidad. Usaremos R para el conjunto de trabajos que todavía no fueron ni aceptados ni rechazados, y A para el conjunto de trabajos aceptados.

INTERVALOS(s, f, n)

```
1  R ← {1, 2, ..., n};    A ← ∅
2  while R ≠ ∅ do
3      Elegir i ∈ R con el menor tiempo de finalización f(i)
4      A ← A ∪ {i}
5      Borrar de R todos los trabajos incompatibles con i
6  end while
7  return A
```

Estructuras de datos. Los trabajos llegan en dos arreglos $ini[1..n]$ y $fin[1..n]$ con los valores $s(i)$ y $f(i)$. La versión eficiente que veremos más abajo ordena los índices por *fin* —por ejemplo con mergesort— y guarda el resultado en un arreglo de índices $orden[1..n]$. Con eso, ni R ni la operación de borrado necesitan existir: se recorre *orden* una sola vez y alcanza con una variable entera *ultimoFin*. El resultado A se acumula en una lista enlazada de índices.

Ejemplo 4.2.2 (Reserva de una sala de reuniones)**.** Una sala recibe nueve trabajos en el día, expresados en horas:

i	1	2	3	4	5	6	7	8	9
$s(i)$	8	8	11	12	14	9	15	17	19
$f(i)$	10	13	14	16	17	20	19	20	21

El algoritmo elige primero el trabajo 1, que termina a las 10, y descarta el 2 y el 6. Entre los que quedan, el que termina antes es el 3; lo acepta y descarta el 4. Luego acepta el 5, que termina a las 17, y descarta el 7. Finalmente acepta el 8 y descarta el 9. El resultado es $A = \{1, 3, 5, 8\}$, de tamaño cuatro. Ninguna otra selección compatible tiene cinco trabajos.

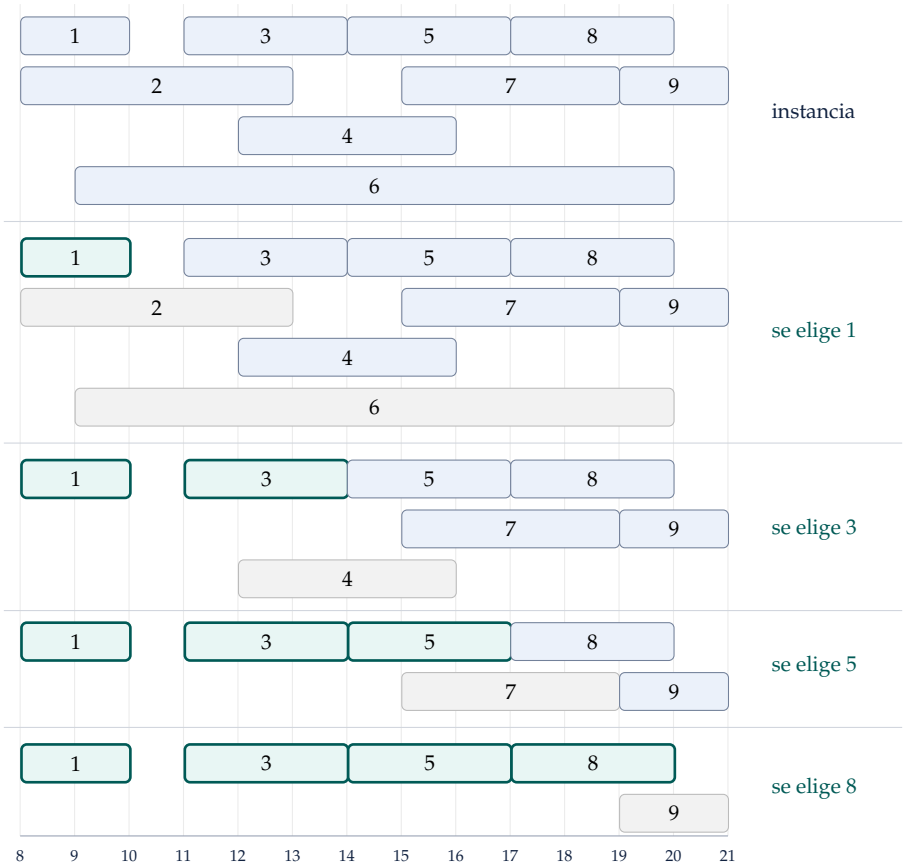


Figura 4.2. Corrida del algoritmo sobre la instancia del Ejemplo 4.2.2. En cada paso el intervalo elegido se marca en trazo grueso y los que quedan descartados por incompatibilidad se atenúan. El resultado final acepta cuatro trabajos.

4.2.2 Análisis del algoritmo

Aunque la regla es natural, no es obvio que devuelva un conjunto óptimo; conviene desconfiar, porque las reglas descartadas antes también parecían prometedoras. Lo primero, en cambio, es inmediato.

Proposición 4.2.3. El conjunto A devuelto por el algoritmo es un conjunto compatible de trabajos.

Demostración. Cada vez que se agrega un trabajo i a A, la línea 5 elimina de R todos los trabajos incompatibles con i. Por lo tanto, cualquier trabajo agregado después de i es compatible con i, y esto vale para todo par de elementos de A. □

Falta lo importante: que A sea óptimo. Sea O un conjunto óptimo cualquiera. Sería demasiado pedir que $A = O$, porque puede haber muchas soluciones óptimas y A coincidirá a lo sumo con una de ellas; alcanza con probar que $|A| = |O|$.

Introduzcamos notación. Sean $i_1, \ldots, i_k$ los trabajos de A en el orden en que fueron agregados, de modo que $|A| = k$, y sean $j_1, \ldots, j_m$ los trabajos de O ordenados de izquierda a derecha según sus intervalos. Como los trabajos de O son compatibles entre sí, sus tiempos de inicio quedan en el mismo orden que sus tiempos de finalización. El objetivo es probar $k = m$.

La idea es encontrar un sentido en el que el algoritmo se mantiene adelante de O: compararemos las soluciones parciales que construye el greedy con los segmentos iniciales de O y veremos que, paso a paso, el greedy va mejor. Concretamente, su r-ésimo intervalo termina no más tarde que el r-ésimo intervalo de O.

Lema 4.2.4. Para todo índice $r \leq k$ se cumple $f(i_r) \leq f(j_r)$.

Demostración. Por inducción en r. Para $r = 1$ la afirmación es inmediata: el algoritmo comienza eligiendo el trabajo i_1 de mínimo tiempo de finalización entre todos.

Sea ahora $r > 1$ y supongamos como hipótesis inductiva que $f(i_{r-1}) \leq f(j_{r-1})$. Como O está formado por intervalos compatibles y j_{r-1} precede a j_r, vale $f(j_{r-1}) \leq s(j_r)$. Combinando ambas desigualdades,

$$f(i_{r-1}) \;\leq\; f(j_{r-1}) \;\leq\; s(j_r),$$

de donde el intervalo j_r todavía pertenece al conjunto R de intervalos disponibles en el momento en que el algoritmo elige i_r. Como el algoritmo elige entre los disponibles el de menor tiempo de finalización, y j_r es uno de ellos, resulta $f(i_r) \leq f(j_r)$. □

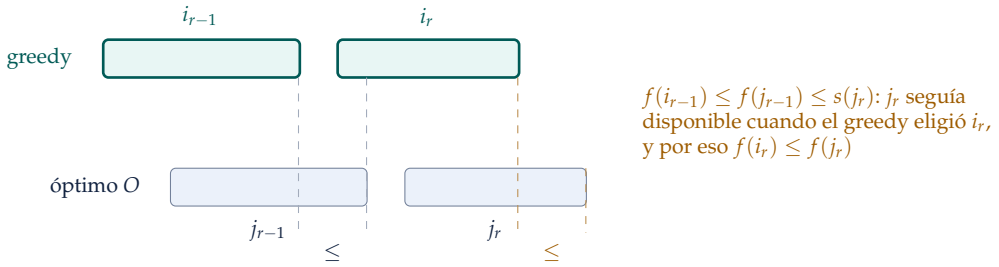


Figura 4.3. El paso inductivo del argumento. Como el greedy venía terminando antes que O, cuando llega el turno de elegir su r-ésimo intervalo todavía tiene disponible al intervalo j_r; eligiendo el de menor finalización no puede quedar por detrás de él.

Teorema 4.2.5. El algoritmo devuelve un conjunto óptimo A.

Demostración. Por el absurdo. Si A no es óptimo, el conjunto óptimo O tiene más trabajos, es decir $m > k$. Aplicando el Lema 4.2.4 con $r = k$ obtenemos $f(i_k) \leq f(j_k)$. Como $m > k$, existe en O un trabajo j_{k+1}, que empieza después de que termina j_k y, por lo tanto, después de que termina i_k. Entonces, tras eliminar todos los trabajos incompatibles con $i_1, \ldots, i_k$, el conjunto R todavía contiene a j_{k+1}. Pero el algoritmo se detuvo después de elegir i_k, y solo se detiene cuando R está vacío: contradicción. □

4.2.3 Implementación y tiempo de ejecución

El algoritmo puede implementarse en tiempo $O(n \log n)$. Se comienza ordenando los n trabajos por tiempo de finalización y renombrándolos en ese orden, de modo que $f(i) \leq f(j)$ cuando $i < j$; esto cuesta $O(n \log n)$. En $O(n)$ adicional se arma un arreglo $S[1..n]$ con $S[i] = s(i)$.

Luego se seleccionan los trabajos recorriendo los intervalos en orden creciente de $f(i)$. Siempre se acepta el primero; después se avanza hasta encontrar el primer j con $S[j] \geq f(1)$, y se lo acepta también. En general, si el último intervalo aceptado termina en f, se sigue avanzando hasta el primer j con $S[j] \geq f$. De esta manera todo el greedy se ejecuta en una única pasada, con trabajo constante por intervalo, es decir en tiempo $O(n)$.

Proposición 4.2.6. El problema de planificación de intervalos se resuelve óptimamente en tiempo $O(n \log n)$, dominado por el ordenamiento inicial.

Nota 4.2.7 (Dos extensiones). El modelo supone que todos los trabajos son conocidos de antemano. Es natural preguntarse qué pasa si hay que aceptar o rechazar cada trabajo sin conocer los que vendrán: de eso se ocupan los algoritmos online. También podría ocurrir que cada trabajo i tenga un valor v_i y que el objetivo sea maximizar la suma de los valores aceptados en lugar de la cantidad; ese es el problema de planificación de intervalos ponderados de la Subsección 1.8.2, para el cual ningún greedy funciona.

4.2.4 Un problema relacionado: partición de intervalos

Cambiemos el planteo. Ahora tenemos muchos recursos idénticos disponibles y queremos atender todos los trabajos usando la menor cantidad posible de recursos. Como el objetivo es repartir todos los intervalos entre varios recursos, lo llamaremos problema de partición de intervalos.

El ejemplo natural es el de una facultad: cada trabajo es una clase que necesita un salón durante cierto intervalo, se quieren dictar todas las clases y se busca usar la menor cantidad de salones. La restricción básica es que dos clases que se solapan en el tiempo deben ir a salones distintos.

Definición 4.2.8 (Profundidad)**.** La profundidad de un conjunto de intervalos es la máxima cantidad de intervalos que pasan por un mismo punto de la línea de tiempo.

Proposición 4.2.9. En toda instancia de partición de intervalos, la cantidad de recursos necesarios es al menos la profundidad del conjunto de intervalos.

Demostración. Supongamos que el conjunto tiene profundidad d y sean $I_1, \ldots, I_d$ intervalos que pasan todos por un mismo punto de la línea de tiempo. Como se solapan de a pares, cada uno debe programarse en un recurso distinto, de modo que la instancia entera necesita al menos d recursos. □

Esta es una cota estructural: vale para toda solución posible, sin importar cómo se la construya. Quedan entonces dos preguntas, que resultan estar íntimamente relacionadas. ¿Existe un algoritmo eficiente que use la mínima cantidad de recursos? ¿Existe siempre una solución que use exactamente la profundidad? Una respuesta afirmativa a la segunda diría que los únicos obstáculos son locales, esto es, montones de intervalos apilados sobre un mismo punto, y que no hay obstáculos de largo alcance que fuercen a usar más recursos.

Veremos que un greedy muy simple usa siempre exactamente d recursos, lo que por la Proposición 4.2.9 lo vuelve óptimo de inmediato. El esquema de la demostración

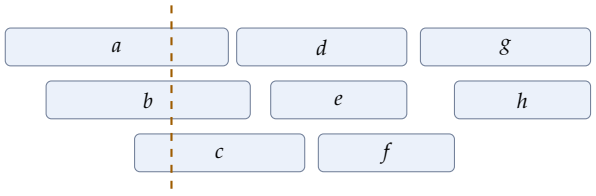
es el tercero que anunciamos: se exhibe una cota que toda solución debe respetar y se muestra que el algoritmo la alcanza.

Sea d la profundidad del conjunto. Asignaremos a cada intervalo una etiqueta del conjunto $\{1, 2, \ldots, d\}$ de manera que dos intervalos que se solapan reciban etiquetas distintas; interpretando cada etiqueta como el nombre de un recurso, eso es exactamente una solución con d recursos. La regla es de una sola pasada: se ordenan los intervalos por tiempo de inicio y se le da a cada uno una etiqueta que no haya sido usada por ningún intervalo anterior que lo solape.

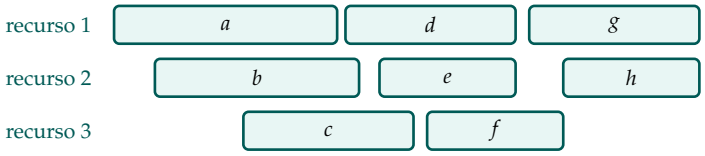
PARTICIÓN$(I_1, \ldots, I_n)$

```
1  Ordenar los intervalos por tiempo de inicio, desempatando arbitrariamente
2  Sean I_1, I_2, ..., I_n los intervalos en ese orden
3  for j ← 1 to n do
4      for cada intervalo I_i anterior a I_j que lo solapa do
5          Excluir la etiqueta de I_i de las candidatas para I_j
6      end for
7      Asignar a I_j alguna etiqueta de {1, ..., d} no excluida
8  end for
```

Estructuras de datos. Los intervalos se guardan en un arreglo ordenado por inicio. Las etiquetas libres se mantienen en una pila de enteros inicializada con $d, d-1, \ldots, 1$: asignar una etiqueta es un desapilado, y devolverla es un apilado. Para saber cuándo devolver etiquetas se usa una cola de prioridad con los intervalos actualmente en curso, con clave el tiempo de finalización: antes de atender a I_j se extraen de ella todos los intervalos que ya terminaron y se reintegran sus etiquetas a la pila. Con esto cada intervalo cuesta $O(\log n)$ y el total es $O(n \log n)$.



(a) instancia: la profundidad es $d = 3$



(b) solución con tres recursos

Figura 4.4. Una instancia de partición de intervalos y una solución. Como hay un punto por el que pasan tres intervalos, dos recursos no alcanzan; el algoritmo greedy logra exactamente tres, que es la profundidad.

Teorema 4.2.10. El algoritmo anterior asigna una etiqueta a todo intervalo y nunca asigna la misma etiqueta a dos intervalos que se solapan. En consecuencia, programa todos los intervalos usando una cantidad de recursos igual a la profundidad del conjunto, que es la cantidad óptima.

Demostración. Veamos primero que ningún intervalo queda sin etiqueta. Consideremos un intervalo I_j y supongamos que hay t intervalos anteriores en el orden que lo solapan. Esos t intervalos empezaron antes que I_j y siguen abiertos cuando I_j comienza, así que junto con I_j forman un conjunto de $t + 1$ intervalos que pasan todos por un mismo punto de la línea de tiempo, el instante de inicio de I_j. Por lo tanto $t + 1 \leq d$, es decir $t \leq d - 1$, y esos t intervalos no pueden excluir las d etiquetas: siempre queda al menos una para I_j.

Veamos ahora que dos intervalos que se solapan reciben etiquetas distintas. Sean I e I' dos intervalos que se solapan, con I anterior a I' en el orden. Cuando el algoritmo considera a I', el intervalo I está entre los anteriores que lo solapan, de modo que su etiqueta queda excluida y I' recibe otra.

Como el algoritmo usa d etiquetas y, por la Proposición 4.2.9, ninguna solución puede usar menos de d recursos, la solución obtenida es óptima. □

Nota 4.2.11. La demostración anterior es sorprendentemente simple, y conviene retener la imagen: si uno dispone de d etiquetas y barre los intervalos de izquierda a derecha asignando cualquier etiqueta libre, nunca puede llegar a un punto en que todas estén en uso, porque eso significaría $d+1$ intervalos apilados sobre un mismo instante.

4.3 Minimizar la tardanza máxima: un argumento de intercambio

Pasemos a un problema de scheduling emparentado con el anterior. A pesar de la similitud del planteo y de la simplicidad del algoritmo greedy que lo resuelve, la demostración de optimalidad requerirá un análisis de otro tipo.

Tenemos un único recurso y n trabajos que quieren usarlo durante un intervalo de tiempo. El recurso está disponible a partir de un instante s. A diferencia del problema anterior, ahora los trabajos son flexibles: en lugar de un tiempo de inicio y uno de fin, el trabajo i tiene un plazo d_i y requiere un intervalo contiguo de duración t_i, pero acepta ser programado en cualquier momento. Cada trabajo aceptado recibe un intervalo de largo t_i y los intervalos asignados a trabajos distintos no se solapan.

Definición 4.3.1 (Tardanza)**.** Programado el trabajo i en el intervalo $[s(i), f(i)]$ con $f(i) = s(i) + t_i$, decimos que i es tardío si $f(i) > d_i$. Su tardanza es $l_i = f(i) - d_i$ si es tardío, y $l_i = 0$ en caso contrario. La tardanza máxima de una programación es $L = \max_i l_i$.

El objetivo es programar todos los trabajos, con intervalos que no se solapen, minimizando la tardanza máxima L. Notemos la diferencia con el problema anterior: allí había que elegir qué trabajos aceptar, aquí hay que aceptarlos todos y elegir en qué orden.

Ejemplo 4.3.2 (Tres trabajos)**.** Sean tres trabajos con $t_1 = 1$, $d_1 = 2$; $t_2 = 2$, $d_2 = 4$; y $t_3 = 3$, $d_3 = 6$, con $s = 0$. Programándolos en el orden $1, 2, 3$, el primero termina en el instante 1, el segundo en $1 + 2 = 3$ y el tercero en $1 + 2 + 3 = 6$. Ninguno excede su plazo, de modo que la tardanza máxima es 0 y esa programación es claramente óptima.

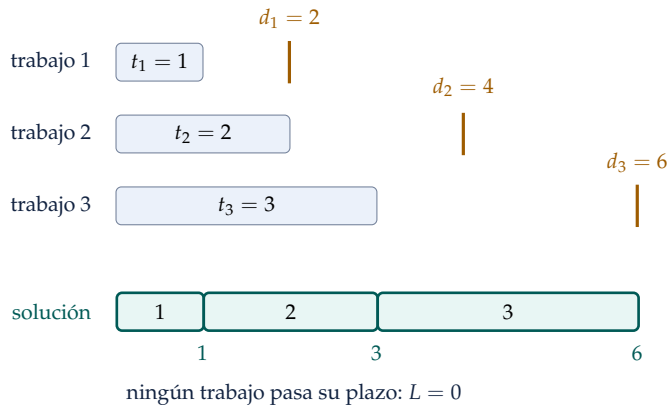


Figura 4.5. La instancia del Ejemplo 4.3.2. Cada trabajo se dibuja con su duración y su plazo marcado con una barra; abajo, la programación en el orden 1, 2, 3, en la que los tres terminan a tiempo.

4.3.1 Diseño del algoritmo

Como antes, hay varias reglas greedy naturales que ordenan los trabajos a partir de los datos (t_i, d_i), y las primeras que se nos ocurren fallan.

- Ordenar por duración creciente, para sacarse de encima rápido los trabajos cortos. Ignora por completo las fechas límite, y eso se paga: con dos trabajos, el primero con $t_1 = 1$ y $d_1 = 100$ y el segundo con $t_2 = 10$ y $d_2 = 10$, la única forma de lograr $L = 0$ es empezar por el segundo.
- El ejemplo anterior sugiere preocuparse por los trabajos con poca holgura $d_i - t_i$, que son los que necesitan empezar sin demora. Ordenar por holgura creciente también falla: con $t_1 = 1$, $d_1 = 2$ y $t_2 = 10$, $d_2 = 10$, la holgura manda al segundo trabajo primero y el primero termina en el instante 11, con tardanza 9; haciendo el primero antes, en cambio, termina a tiempo y el segundo acumula tardanza 1.

La regla que sí funciona es todavía más simple: ordenar los trabajos por plazo creciente y programarlos en ese orden. Se la conoce como Earliest Deadline First. La intuición es razonable —conviene terminar antes los trabajos cuyo plazo vence antes— pero cuesta creer que sea óptima, porque no mira las duraciones. Antes desconfiábamos de la regla que ordenaba por duración porque tiraba a la basura la mitad de los datos; ahora estamos considerando una que tira la otra mitad. Y sin embargo es óptima.

Renombrando los trabajos si hace falta, supongamos $d_1 \leq d_2 \leq \cdots \leq d_n$ y programémoslos en ese orden a partir del instante s.

EDF(t, d, n, s)

```
1  Ordenar los trabajos por plazo creciente
2  Supongamos, para simplificar la notación, d_1 ≤ d_2 ≤ ··· ≤ d_n
3  f ← s
4  for i ← 1 to n do
5      Asignar al trabajo i el intervalo [s(i), f(i)] con s(i) = f y f(i) = f + t_i
6      f ← f + t_i
7  end for
8  return los intervalos [s(i), f(i)] para i = 1, ..., n
```

> **Estructuras de datos.** Los datos vienen en dos arreglos $dur[1..n]$ y $lim[1..n]$. Se ordena un arreglo de índices $orden[1..n]$ por lim con mergesort, en $O(n \log n)$; el resto es una pasada con un acumulador entero f, de modo que el algoritmo completo es $O(n \log n)$ y no necesita ninguna estructura auxiliar más.

4.3.2 Análisis del algoritmo

La programación que produce el algoritmo no tiene huecos: no hay instantes en los que la máquina esté ociosa habiendo trabajos pendientes. Al tiempo transcurrido durante un hueco lo llamaremos tiempo ocioso. Es fácil convencerse de que existe también una programación óptima sin tiempo ocioso: si la hubiera, adelantar los trabajos posteriores para tapar el hueco no puede aumentar la tardanza de ninguno.

Proposición 4.3.3. Existe una programación óptima sin tiempo ocioso.

Para probar la optimalidad partiremos de una programación óptima O y la modificaremos gradualmente, preservando su optimalidad en cada paso, hasta transformarla en la programación A que produce el greedy. Este es el argumento de intercambio de la Definición 4.1.2.

Definición 4.3.4 (Inversión)**.** Una programación tiene una inversión si un trabajo i con plazo d_i está programado antes que otro trabajo j con plazo anterior, $d_j < d_i$.

Por construcción, la programación A no tiene inversiones. Si hay trabajos con plazos iguales puede haber muchas programaciones sin inversiones distintas entre sí, pero todas tienen la misma tardanza máxima.

Proposición 4.3.5. Todas las programaciones sin inversiones y sin tiempo ocioso tienen la misma tardanza máxima.

Demostración. Dos programaciones sin inversiones ni tiempo ocioso pueden no producir exactamente el mismo orden de trabajos, pero solo pueden diferir en el orden relativo de

trabajos con plazos idénticos. Consideremos un plazo d. En ambas programaciones, los trabajos con plazo d se ejecutan de manera consecutiva, después de todos los de plazo anterior y antes de todos los de plazo posterior. Entre ellos, el que termina último es el de mayor tardanza, y esa tardanza no depende del orden interno, porque el instante en que termina el bloque es el mismo. □

El paso central es mostrar que existe una programación óptima sin inversiones y sin tiempo ocioso.

Lema 4.3.6. Existe una programación óptima que no tiene inversiones ni tiempo ocioso.

Demostración. Por la Proposición 4.3.3 existe una programación óptima O sin tiempo ocioso. La demostración consiste en tres afirmaciones.

► (a) Si O tiene una inversión, entonces existe un par de trabajos i y j tales que j está programado inmediatamente después de i y $d_j < d_i$. En efecto, consideremos una inversión en la que un trabajo a está programado antes que un trabajo b con $d_a > d_b$. Si avanzamos de a hacia b mirando los trabajos de a uno, en algún momento el plazo que vemos disminuye por primera vez, y eso corresponde a un par de trabajos consecutivos que forman una inversión.

► (b) Sean i y j un par de trabajos invertidos y consecutivos. Al intercambiarlos, la inversión (i, j) desaparece y no se crea ninguna nueva, porque el orden relativo de todos los demás pares queda intacto. Es decir, el intercambio reduce en uno la cantidad de inversiones.

► (c) La programación resultante del intercambio tiene tardanza máxima no mayor que la de O. Esta es la parte delicada y la probamos aparte, más abajo.

Con (c) terminamos: la programación inicial O tiene a lo sumo $\binom{n}{2}$ inversiones, de modo que después de a lo sumo $\binom{n}{2}$ intercambios llegamos a una programación óptima sin inversiones.

Probemos entonces (c). Escribamos O con la notación siguiente: cada trabajo r está programado en el intervalo $[s(r), f(r)]$ y tiene tardanza l_r, y sea $L = \max_r l_r$ la tardanza máxima de O. Notemos $\widehat{O}$ la programación intercambiada y $\widehat{s}(r)$, $\widehat{f}(r)$, $\widehat{l}_r$, $\widehat{L}$ las cantidades correspondientes.

Sean i y j los dos trabajos adyacentes e invertidos, con $d_i > d_j$. El instante en que termina j antes del intercambio es exactamente el instante en que termina i después del intercambio, así que todos los trabajos distintos de i y j terminan en el mismo momento en ambas programaciones. Además, j termina antes en la programación nueva, con lo cual el intercambio no aumenta su tardanza.

Lo único que puede preocuparnos es i, cuya tardanza sí puede haber aumentado. Después del intercambio, i termina en el instante $f(j)$, que es donde terminaba j en O. Si i es tardío en la programación nueva, su tardanza es $\widehat{l}_i = \widehat{f}(i) - d_i = f(j) - d_i$. El punto

crucial es que i no puede llegar más tarde en $\widehat{O}$ de lo que llegaba j en O: como $d_i > d_j$,

$$\widehat{l_i} \;=\; f(j) - d_i \;<\; f(j) - d_j \;=\; l_j.$$

Y como la tardanza máxima de O cumple $L \geq l_j > \widehat{l_i}$, el intercambio no aumenta la tardanza máxima, que era lo que había que probar. □

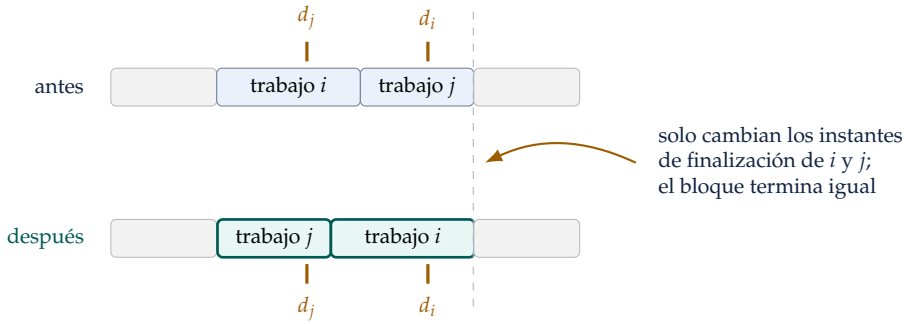


Figura 4.6. El efecto de intercambiar dos trabajos invertidos y consecutivos. El trabajo j pasa a terminar antes, y el trabajo i pasa a terminar donde terminaba j; como $d_i > d_j$, la tardanza que hereda i es menor que la que tenía j.

Teorema 4.3.7. La programación A producida por el algoritmo Earliest Deadline First tiene tardanza máxima L óptima.

Demostración. Por el Lema 4.3.6 existe una programación óptima sin inversiones ni tiempo ocioso. Por la Proposición 4.3.5, todas las programaciones sin inversiones ni tiempo ocioso tienen la misma tardanza máxima. Como A es una de ellas, A es óptima. □

Nota 4.3.8 (Tiempos de liberación). Supusimos que todos los trabajos están disponibles desde el instante común s. Una variante natural, y mucho más difícil, agrega a cada trabajo i un instante mínimo de comienzo r_i, llamado tiempo de liberación; aparece apenas los trabajos toman la forma "quiero la sala dos horas, en algún momento entre las 13 y las 17". La demostración anterior usa de manera esencial que todos los trabajos están disponibles desde el principio: si no lo estuvieran, el trabajo j podría no poder adelantarse al lugar de i. Esa versión general del problema no admite una solución greedy simple.

4.4 Camino más corto en un grafo

Varios de los algoritmos básicos sobre grafos responden a un diseño greedy. Aplicaremos la estrategia al problema de encontrar caminos mínimos, y en la sección siguiente a la construcción de árboles de cubrimiento de costo mínimo.

Los grafos modelan redes en las que uno se traslada de un punto a otro: una secuencia de rutas que pasan por empalmes, o una secuencia de enlaces de comunicación que atraviesan routers intermedios. Un problema algorítmico básico es entonces determinar el camino más corto entre dos nodos. Podemos plantearlo punto a punto —dados u y v, ¿cuál es el camino u-v más corto?— o pedir más información: dado un nodo de partida s, ¿cuál es el camino más corto de s a cada uno de los demás nodos?

El planteo concreto es el siguiente. Se da un grafo dirigido $G = (V, E)$ con un nodo de partida distinguido s, y se supone que desde s hay camino a todos los demás nodos. Cada arista e tiene un largo $\ell_e \geq 0$, que representa el tiempo, la distancia o el costo de recorrerla. Para un camino P, su largo $\ell(P)$ es la suma de los largos de sus aristas. El objetivo es determinar el camino más corto de s a cada nodo. Aunque el problema está enunciado para grafos dirigidos, el caso no dirigido se resuelve reemplazando cada arista $e = (u, v)$ de largo ℓ_e por las dos aristas dirigidas (u, v) y (v, u), ambas de largo ℓ_e.

4.4.1 Diseño del algoritmo

En 1959 Edsger Dijkstra propuso un algoritmo greedy muy simple para el problema de caminos mínimos desde un origen. Describamos primero la versión que solo calcula el largo del camino mínimo de s a cada nodo; recuperar los caminos es después inmediato.

El algoritmo mantiene un conjunto S de nodos u para los que ya se determinó la distancia mínima $d(u)$ desde s; es la parte explorada del grafo. Inicialmente $S = \{s\}$ y $d(s) = 0$. En cada paso, para cada nodo $v \in V - S$ se considera el mejor camino que se puede formar viajando por la parte explorada hasta algún $u \in S$ y saliendo por una única arista (u, v), es decir, la cantidad

$$d'(v) \;=\; \min_{e=(u,v)\,:\,u\in S} d(u) + \ell_e.$$

Se elige el nodo $v \in V - S$ que minimiza esa cantidad, se lo agrega a S y se define $d(v) = d'(v)$.

DIJKSTRA(G, ℓ, s)

```
1  S ← {s};   d(s) ← 0
2  while S ≠ V do
3      Elegir v ∉ S con al menos una arista desde S que minimice
4          d'(v) = mín_{e=(u,v) : u∈S} d(u) + ℓ_e
5      S ← S ∪ {v};   d(v) ← d'(v)
6      Registrar la arista (u, v) que alcanzó ese mínimo
7  end while
8  return d y las aristas registradas
```

> **Estructuras de datos.** El grafo se recibe en listas de adyacencia, con cada nodo de la lista guardando el vecino y el largo de la arista. Las distancias d se guardan en un arreglo $d[1..n]$ de reales y la pertenencia a S en un arreglo de booleanos $enS[1..n]$. Los nodos de $V - S$ se mantienen en una cola de prioridad implementada con heap binario, con clave $d'(v)$; se necesita `ExtraerMin` y `CambiarClave`, y para esta última hace falta un arreglo $pos[1..n]$ que diga en qué posición del heap vive cada nodo. Las aristas registradas se guardan en un arreglo $padre[1..n]$.

Producir los caminos es simple. Cada vez que un nodo v entra a S se registra la arista (u, v) con la que alcanzó el mínimo. El camino P_v queda representado implícitamente por esas aristas: si (u, v) es la arista guardada para v, entonces P_v es el camino P_u seguido de la arista (u, v). Para construirlo explícitamente se parte de v, se sigue hacia atrás la arista guardada para v hasta u, luego la guardada para u hasta su predecesor, y así sucesivamente. El recorrido termina siempre en s, porque cada paso hacia atrás visita nodos que entraron a S cada vez más temprano.

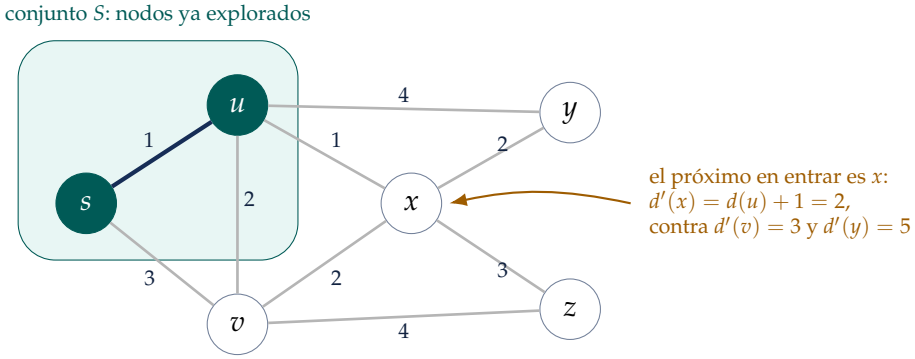


Figura 4.7. Una foto de la ejecución de Dijkstra. Ya entró u; el próximo nodo en entrar es x, porque logra el menor valor de d'. Agregar y en este momento daría un valor incorrecto (5): terminará entrando por su arista desde x, con $d(y) = 4$.

4.4.2 Análisis del algoritmo

El ejemplo muestra que Dijkstra evita la trampa recurrente: hacer crecer S por el nodo equivocado lleva a sobrestimar la distancia a ese nodo. La pregunta es si siempre que el

algoritmo agrega un nodo v obtiene su verdadera distancia mínima.

La respuesta usa una variante del primer estilo de análisis: el algoritmo se mantiene adelante de todas las demás soluciones, en el sentido de que cada vez que fija un camino hacia un nodo v, ese camino no es más largo que ningún otro camino posible hasta v.

Teorema 4.4.1. Consideremos el conjunto S en cualquier momento de la ejecución del algoritmo. Para cada $u \in S$, el camino P_u es un camino mínimo de s a u.

Demostración. Por inducción en el tamaño de S. El caso $|S| = 1$ es inmediato, porque entonces $S = \{s\}$ y $d(s) = 0$. Supongamos que la afirmación vale cuando $|S| = k$ para algún $k \geq 1$ y hagamos crecer S a tamaño $k+1$ agregando el nodo v. Sea (u, v) la última arista del camino P_v.

Por hipótesis inductiva, P_u es un camino mínimo de s a u para todo $u \in S$. Consideremos ahora cualquier otro camino P de s a v; queremos ver que es al menos tan largo como P_v. Para llegar a v, el camino P tiene que salir del conjunto S en algún momento: sea y el primer nodo de P que no está en S, y sea $x \in S$ el nodo inmediatamente anterior.

El núcleo del argumento es simple: P no puede ser más corto que P_v porque ya es al menos tan largo como P_v en el momento en que abandona S. En efecto, en la iteración $k+1$ el algoritmo consideró agregar el nodo y a través de la arista (x, y) y descartó esa opción en favor de v. Eso significa que ningún camino de s a y que pase por x es más corto que P_v. Pero el tramo inicial de P hasta y es un camino de esa forma, así que ese tramo es al menos tan largo como P_v. Como los largos de las aristas son no negativos, el camino completo P es al menos tan largo como P_v.

La misma cuenta, escrita con desigualdades: sea P' el tramo de P de s a x. Como $x \in S$, por hipótesis inductiva P_x es un camino mínimo de s a x, de largo $d(x)$, y por lo tanto $\ell(P') \geq \ell(P_x) = d(x)$. Entonces el tramo de P hasta y tiene largo

$$\ell(P') + \ell_{(x,y)} \;\geq\; d(x) + \ell_{(x,y)} \;\geq\; d'(y),$$

y el camino completo P es al menos tan largo como ese tramo. Finalmente, como el algoritmo eligió v en esta iteración, vale $d'(y) \geq d'(v) = \ell(P_v)$. Combinando, $\ell(P) \geq \ell(P_v)$. □

Este teorema establece la corrección del algoritmo: basta aplicarlo al terminar, cuando S contiene a todos los nodos.

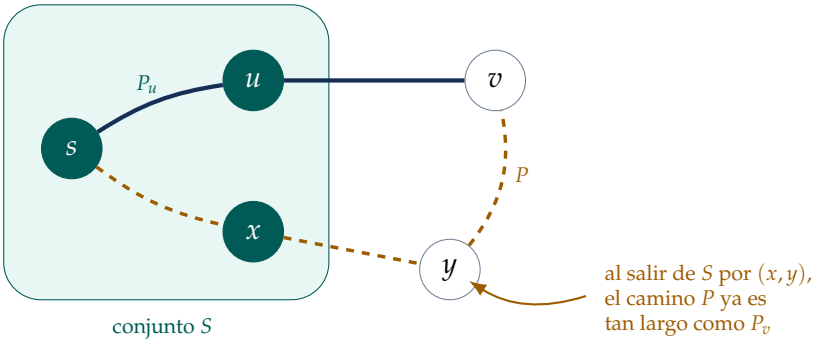


Figura 4.8. El camino mínimo P_v y un camino alternativo P que pasa por el nodo y. La demostración compara ambos en el instante en que P abandona el conjunto explorado.

Nota 4.4.2 (Largos negativos). El algoritmo no encuentra caminos mínimos si algunas aristas pueden tener largo negativo. Conviene ver dónde se rompe la demostración: en el último paso usamos que el tramo restante de P solo puede agregar largo, y con aristas negativas un camino puede acortarse después de haber salido de S. Para ese caso hace falta un algoritmo más complejo, el de Bellman y Ford.

Nota 4.4.3 (Dijkstra como BFS continuo). El algoritmo de Dijkstra es, en un sentido preciso, una versión continua del recorrido en anchura del Capítulo 3. Imaginemos que las aristas de G son caños llenos de agua unidos en los nodos, donde la arista e tiene largo ℓ_e y sección constante. Si dejamos caer una gota extra en el nodo s, la onda que se expande desde s a velocidad constante alcanza los nodos en orden creciente de distancia a s, y el camino que sigue el frente de onda hasta cada nodo v es exactamente el camino que encuentra Dijkstra.

4.4.3 Implementación y tiempo de ejecución

Hay $n - 1$ iteraciones del bucle, porque cada una agrega un nodo a S. La sutileza está en elegir eficientemente el nodo v. La primera idea sería recorrer, en cada iteración, todos los nodos $v \notin S$ y todas las aristas entre S y v para calcular el mínimo. Eso cuesta $O(m)$ por iteración y da una implementación en $O(mn)$.

Podemos hacerlo bastante mejor con las estructuras adecuadas. Primero, mantenemos explícitamente los valores $d'(v)$ para cada $v \in V - S$ en lugar de recalcularlos en cada iteración. Segundo, guardamos los nodos de $V - S$ en una cola de prioridad con clave $d'(v)$. Necesitamos dos operaciones: `ExtraerMin`, para seleccionar el nodo que entra a S, y `CambiarClave`, para actualizar las claves.

Veamos cómo se actualizan. Cuando el nodo v entra a S, sea $w \notin S$ un nodo que sigue en la cola. Si (v, w) no es arista, no hay nada que hacer: el conjunto de aristas que intervienen en el mínimo de w es el mismo antes y después. Si en cambio $e = (v, w) \in E$,

la nueva clave es mín$\big(d'(w),\, d(v)+\ell_e\big)$, de modo que si $d'(w) > d(v)+\ell_e$ hay que invocar `CambiarClave` para bajar la clave de w. Esta operación ocurre a lo sumo una vez por arista, en el momento en que el nodo del que sale la arista entra a S.

Proposición 4.4.4. Con una cola de prioridad, el algoritmo de Dijkstra se implementa sobre un grafo con n nodos y m aristas en tiempo $O(m)$ más el tiempo de n operaciones `ExtraerMin` y m operaciones `CambiarClave`. Usando un heap binario, cada operación cuesta $O(\log n)$ y el tiempo total es $O(m \log n)$.

Ejemplo 4.4.5 (Una red de rutas)**.** Consideremos la red de la Figura 4.7 y sigamos la ejecución desde s. Al principio $S = \{s\}$ y las claves son $d'(u) = 1$ y $d'(v) = 3$; el resto vale ∞. Entra u con $d(u) = 1$ y se actualizan $d'(v) = \text{mín}(3, 1+2) = 3$, $d'(x) = 2$ y $d'(y) = 5$. Entra x con $d(x) = 2$ y se actualizan $d'(y) = \text{mín}(5, 2+2) = 4$ y $d'(z) = 5$. Entra v con $d(v) = 3$ y se actualiza $d'(z) = \text{mín}(5, 3+4) = 5$. Entran finalmente y con $d(y) = 4$ y z con $d(z) = 5$. Notemos que la clave de y bajó dos veces antes de que y entrara a S: eso es exactamente una operación `CambiarClave` por arista entrante.

4.5 El árbol de cubrimiento mínimo

Aplicaremos ahora un argumento de intercambio a un segundo problema fundamental sobre grafos: el del árbol de cubrimiento de costo mínimo.

Supongamos que tenemos un conjunto de ubicaciones $V = \{v_1, v_2, \ldots, v_n\}$ y queremos construir una red de comunicaciones sobre ellas. La red debe ser conexa —tiene que haber un camino entre cada par de nodos— pero, sujeto a eso, queremos construirla lo más barata posible.

Para ciertos pares (v_i, v_j) podemos tender un enlace directo a un costo $c(v_i, v_j) > 0$. El conjunto de enlaces posibles se representa entonces con un grafo $G = (V, E)$ que tiene un costo positivo c_e asociado a cada arista $e = (v_i, v_j)$. El problema es encontrar un subconjunto de aristas $T \subseteq E$ tal que el grafo (V, T) sea conexo y el costo total $\sum_{e \in T} c_e$ sea mínimo. Supondremos siempre que G es conexo; si no lo fuera, no habría solución.

Proposición 4.5.1. Sea T una solución de costo mínimo al problema anterior. Entonces (V, T) es un árbol.

Demostración. Por definición (V, T) es conexo; veamos que no tiene ciclos. Supongamos que contuviera un ciclo C y sea e cualquier arista de C. Afirmamos que $(V, T - \{e\})$ sigue siendo conexo, porque todo camino que antes usaba la arista e puede ahora dar la vuelta larga por el resto del ciclo C. Entonces $(V, T - \{e\})$ también es solución del problema y es más barata, lo que contradice la minimalidad de T. □

Si se permitieran aristas de costo 0, una solución de costo mínimo podría tener aristas de más, que se podrían borrar sin cambiar el costo; aun así, siempre existe una solución de costo mínimo que es un árbol, porque partiendo de cualquier solución óptima se pueden ir borrando aristas de ciclos sin aumentar el costo.

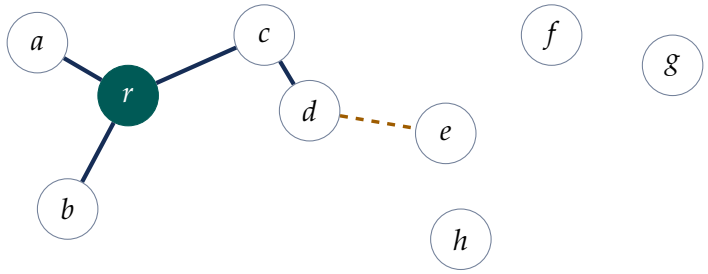
Definición 4.5.2 (Árbol de cubrimiento)**.** Un subconjunto $T \subseteq E$ es un árbol de cubrimiento de G si (V, T) es un árbol. El problema de encontrar el árbol de cubrimiento de costo mínimo se llama problema del árbol de cubrimiento mínimo.

Salvo que G sea muy simple, tiene una cantidad exponencial de árboles de cubrimiento, con estructuras muy distintas entre sí, de modo que no es para nada evidente cómo encontrar el más barato de manera eficiente.

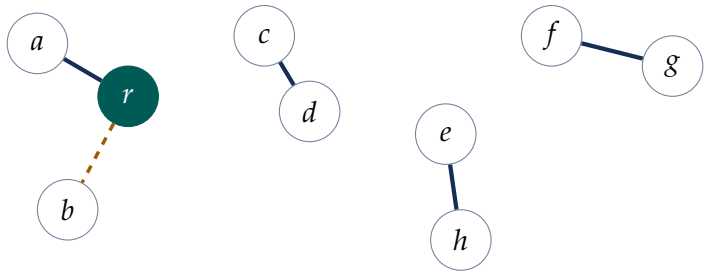
4.5.1 Tres algoritmos greedy

Como en los problemas anteriores, es fácil inventar algoritmos greedy. La curiosidad —y la suerte— es que aquí varios de los primeros que se nos ocurren resultan correctos: los tres resuelven el problema óptimamente.

- Empezar sin ninguna arista e ir insertando aristas de E en orden creciente de costo, agregando cada arista e siempre que no forme un ciclo con las ya insertadas, y descartándola en caso contrario. Es el algoritmo de Kruskal.
- Por analogía con Dijkstra, y aun más simple de enunciar que él: empezar con un nodo raíz s y hacer crecer un árbol desde s hacia afuera, agregando en cada paso el nodo que se puede enganchar más barato al árbol parcial que ya tenemos. Concretamente, se mantiene un conjunto $S \subseteq V$ sobre el que ya se construyó un árbol, con $S = \{s\}$ al principio, y en cada iteración se agrega el nodo v que minimiza el costo de enganche $\min_{e=(u,v)\,:\,u \in S} c_e$, incluyendo en el árbol la arista $e = (u, v)$ que alcanza ese mínimo. Es el algoritmo de Prim.
- Correr una especie de Kruskal hacia atrás: empezar con el grafo completo (V, E) e ir borrando aristas en orden decreciente de costo, borrando cada arista siempre que hacerlo no desconecte el grafo. A falta de un nombre mejor, se lo llama algoritmo de borrado inverso.



(a) Prim: crece desde la raíz r



(b) Kruskal: componentes sueltas que después se unen

Figura 4.9. Las primeras cuatro aristas que agregan Prim y Kruskal sobre la misma instancia geométrica, en la que el costo de cada arista es proporcional a la distancia en el plano. Las aristas ya agregadas van en trazo grueso y la próxima en trazo punteado. Prim mantiene siempre un único árbol; Kruskal trabaja con varias componentes a la vez.

Que los tres algoritmos produzcan una solución óptima sugiere cierta robustez del problema: hay muchas maneras de llegar a la respuesta. Veamos por qué.

4.5.2 Análisis: propiedad del corte y propiedad del ciclo

Los tres algoritmos insertan o borran aristas de una solución parcial. Para analizarlos conviene tener a mano hechos básicos que digan cuándo es seguro incluir una arista en el árbol de cubrimiento mínimo y, simétricamente, cuándo es seguro descartarla porque no puede pertenecer a ninguno. Para el análisis supondremos que todos los costos son distintos entre sí; al final de la sección veremos que ese supuesto se elimina sin esfuerzo.

Teorema 4.5.3 (Propiedad del corte)**.** Supongamos que todos los costos de las aristas son distintos. Sea S un subconjunto de nodos que no es ni vacío ni igual a todo V, y sea $e = (v, w)$ la arista de mínimo costo con un extremo en S y el otro en $V - S$. Entonces e pertenece a todo árbol de cubrimiento mínimo.

Demostración. Sea T un árbol de cubrimiento que no contiene a e; hay que mostrar que T no tiene el costo mínimo posible. Lo haremos con un argumento de intercambio: identificaremos una arista e' de T más cara que e, tal que intercambiar e' por e da otro árbol de cubrimiento, que resultará más barato que T.

Todo está en encontrar la arista correcta para intercambiar. Los extremos de e son v y w. Como T es un árbol de cubrimiento, existe en T un camino P de v a w. Recorramos P desde v: hay un primer nodo w' de P que está en $V - S$. Sea $v' \in S$ el nodo inmediatamente anterior a w' en P y sea $e' = (v', w')$ la arista que los une. Así, e' es una arista de T con un extremo en S y el otro en $V - S$.

Consideremos $T' = T - \{e'\} \cup \{e\}$. Afirmamos que T' es un árbol de cubrimiento. Es conexo, porque cualquier camino que en (V, T) usaba la arista $e' = (v', w')$ puede reencaminarse en (V, T') recorriendo el tramo de P de v' a v, luego la arista e, y luego el tramo de P de w a w'. Y no tiene ciclos, porque el único ciclo de $(V, T \cup \{e\})$ es el formado por e y el camino P, y ese ciclo desaparece al borrar e'.

Finalmente, e' tiene un extremo en S y otro en $V - S$, y e es la arista más barata con esa propiedad, así que $c_e < c_{e'}$, con desigualdad estricta porque no hay dos aristas del mismo costo. El costo total de T' es entonces menor que el de T. □

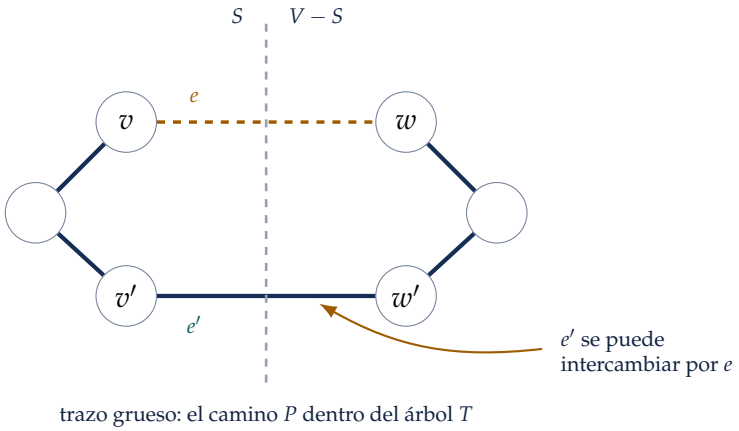


Figura 4.10. El intercambio de la arista e' por la arista e en el árbol generador T. La clave es que e' no es una arista cualquiera que cruce el corte, sino la primera que lo cruza al recorrer P desde v.

Nota 4.5.4 (Un argumento más corto pero incorrecto). La demostración anterior es más sutil de lo que parece. Podría tentarnos el siguiente razonamiento breve: sea T un árbol de cubrimiento que no contiene a e; como T es generador, debe contener alguna arista f con un extremo en S y otro en $V - S$; como e es la más barata con esa propiedad, $c_e < c_f$ y entonces $T - \{f\} \cup \{e\}$ es un árbol de cubrimiento más barato que T. El problema no está en la existencia de f ni en la comparación de costos, sino en que $T - \{f\} \cup \{e\}$ puede no ser un árbol de cubrimiento. Hay que elegir la arista con cuidado, y por eso recorremos el camino P.

Teorema 4.5.5. Los algoritmos de Kruskal y de Prim producen un árbol de cubrimiento mínimo de G.

Demostración. Para Kruskal, consideremos cualquier arista $e = (v, w)$ que el algoritmo agrega, y sea S el conjunto de los nodos a los que v tiene camino justo antes de agregar e. Claramente $v \in S$ y $w \notin S$, porque agregar e no crea un ciclo. Además, todavía no se encontró ninguna arista de S a $V - S$: cualquier arista así podría haberse agregado sin crear un ciclo y, por lo tanto, Kruskal ya la habría agregado. Entonces e es la arista más barata con un extremo en S y el otro en $V - S$, y por la propiedad del corte pertenece a todo árbol de cubrimiento mínimo.

Falta ver que la salida (V, T) de Kruskal es un árbol de cubrimiento. No tiene ciclos, porque el algoritmo está diseñado explícitamente para evitarlos. Y si no fuera conexo, existiría un subconjunto no vacío de nodos $S \neq V$ sin ninguna arista de T hacia $V - S$; pero como G es conexo hay al menos una arista entre S y $V - S$ en G, y el algoritmo habría agregado la primera de ellas que encontrara.

Para Prim, en cada iteración hay un conjunto S sobre el que se construyó un árbol parcial, y se agregan un nodo v y una arista e que minimizan $\min_{e=(u,v)\,:\,u \in S} c_e$. Por definición, e es la arista más barata con un extremo en S y el otro en $V - S$, así que por la propiedad del corte pertenece a todo árbol de cubrimiento mínimo. Que Prim produce un árbol de cubrimiento es inmediato de su construcción. □

Veamos ahora el hecho simétrico, que dice cuándo una arista es descartable.

Teorema 4.5.6 (Propiedad del ciclo)**.** Supongamos que todos los costos de las aristas son distintos. Sea C un ciclo cualquiera de G y sea $e = (v, w)$ la arista más cara de C. Entonces e no pertenece a ningún árbol de cubrimiento mínimo de G.

Demostración. Sea T un árbol de cubrimiento que contiene a e; hay que mostrar que T no tiene costo mínimo. Igual que antes, lo haremos con un intercambio, cambiando e por una arista más barata de manera que siga siendo un árbol de cubrimiento.

Borremos e de T; esto parte los nodos en dos componentes: S, que contiene a v, y $V - S$, que contiene a w. La arista que usemos en lugar de e debe tener un extremo en S y el otro en $V - S$, para volver a coser el árbol.

Una arista así se encuentra recorriendo el ciclo C. Las aristas de C distintas de e forman, por definición, un camino P con un extremo en v y el otro en w. Si recorremos P desde v hasta w empezamos en S y terminamos en $V - S$, de modo que alguna arista e' de P cruza de S a $V - S$.

Consideremos $T' = T - \{e\} \cup \{e'\}$. Con el mismo razonamiento de la demostración de la propiedad del corte, (V, T') es conexo y sin ciclos, así que es un árbol de cubrimiento de G. Y como e es la arista más cara del ciclo C y e' pertenece a C, se tiene $c_{e'} < c_e$, con lo cual T' es más barato que T. □

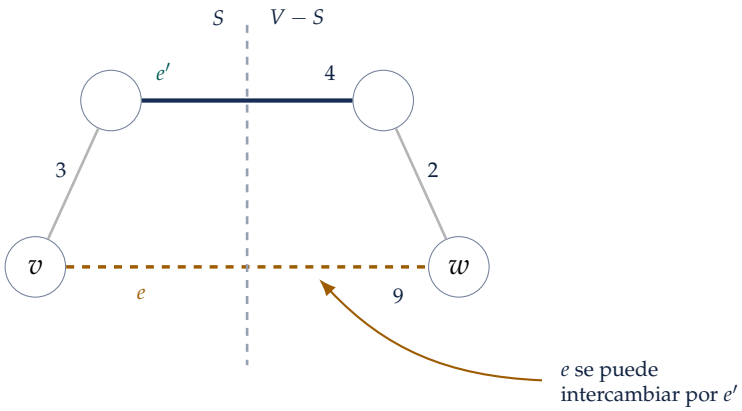


Figura 4.11. El ciclo C del que e es la arista más cara. Al borrar e del árbol T queda el corte $(S, V - S)$, y al recorrer el resto del ciclo desde v hasta w hay que cruzarlo por alguna arista: acá es e', que cuesta menos que e y por lo tanto sirve para el intercambio.

Teorema 4.5.7. El algoritmo de borrado inverso produce un árbol de cubrimiento mínimo de G.

Demostración. Consideremos cualquier arista $e = (v, w)$ que el algoritmo borra. En el momento de borrarla, e pertenece a un ciclo C, porque de lo contrario borrarla desconectaría el grafo; y como es la primera arista de C que el algoritmo encuentra al recorrer las aristas en orden decreciente de costo, es la más cara de C. Por la propiedad del ciclo, e no pertenece a ningún árbol de cubrimiento mínimo.

Falta ver que la salida (V, T) es un árbol de cubrimiento. Es conexa, porque el algoritmo nunca borra una arista si eso desconecta el grafo. Y si contuviera un ciclo C, la arista más cara de C —la primera que el algoritmo habría encontrado— tendría que haber sido borrada, ya que borrarla no desconecta nada; contradicción. □

Nota 4.5.8 (Un principio más general). Combinando la propiedad del corte y la propiedad del ciclo se ve que ocurre algo más general: cualquier algoritmo que construya un árbol de cubrimiento incluyendo aristas cuando lo justifica la propiedad del corte y borrando aristas cuando lo justifica la propiedad del ciclo, en el orden que sea, termina con un árbol de cubrimiento mínimo. Esto permite diseñar otros greedy además de los tres que vimos, y explica por qué tantos algoritmos distintos producen la solución óptima.

4.5.3 Eliminación del supuesto de costos distintos

Hasta acá supusimos que todos los costos son distintos, lo que simplificó varias demostraciones. ¿Qué pasa si algunas aristas tienen el mismo costo?

Hay una manera fácil de resolverlo: perturbamos todos los costos con números distintos y extremadamente pequeños, de modo que pasen a ser todos distintos. Dos costos que originalmente diferían mantienen su orden relativo, porque las perturbaciones son pequeñísimas; y como todos nuestros algoritmos solo comparan costos, las perturbaciones actúan únicamente como criterios de desempate entre costos que antes eran iguales.

Además, todo árbol de cubrimiento mínimo T de la instancia perturbada era también un árbol de cubrimiento mínimo de la instancia original. En efecto, si T costaba más que algún árbol T^* en la instancia original, para perturbaciones suficientemente chicas el cambio de costo de T no alcanza para ponerlo por debajo de T^*. Entonces, corriendo cualquiera de los algoritmos con los costos perturbados obtenemos un árbol T que también es óptimo para la instancia original.

4.5.4 Implementación de Prim

Tanto Prim como Kruskal se pueden implementar, con las estructuras adecuadas, en tiempo $O(m \log n)$. Veamos aquí el caso de Prim y dejemos el de Kruskal para la Sección 4.6. Alcanzar un tiempo comparable con el algoritmo de borrado inverso es difícil, así que no nos ocuparemos de él.

Aunque la demostración de corrección de Prim fue muy distinta de la de Dijkstra, las implementaciones son casi idénticas. Necesitamos decidir qué nodo v agregar a S, para lo cual mantenemos los costos de enganche

$$a(v) \;=\; \min_{e=(u,v)\,:\,u\in S} c_e$$

para cada $v \in V - S$. Como antes, guardamos los nodos en una cola de prioridad con clave $a(v)$: seleccionamos con `ExtraerMin` y actualizamos con `CambiarClave`.

PRIM(G, c, s)

```
1  S ← {s};   T ← ∅
2  Para todo v ≠ s: a(v) ← ∞;  para todo vecino v de s: a(v) ← c_(s,v)
3  while S ≠ V do
4      v ← ExtraerMin de la cola
5      S ← S ∪ {v};   agregar a T la arista que alcanzó a(v)
6      for cada arista e = (v, w) con w ∉ S do
7          if c_e < a(w) then a(w) ← c_e y CambiarClave(w)
8      end for
9  end while
10 return T
```

Estructuras de datos. Es la misma disposición que en Dijkstra: listas de adyacencia con el costo en cada nodo de la lista, arreglo $a[1..n]$ de costos de enganche, arreglo de booleanos $enS[1..n]$, heap binario sobre $V - S$ con clave a y arreglo $pos[1..n]$ para localizar cada nodo dentro del heap, y arreglo $padre[1..n]$ con la arista que enganchó cada nodo. La única diferencia con Dijkstra está en la línea 7: allí la clave es $d(v) + \ell_e$ y aquí es simplemente c_e.

Proposición 4.5.9. Con una cola de prioridad, el algoritmo de Prim se implementa sobre un grafo con n nodos y m aristas en tiempo $O(m)$ más el tiempo de n operaciones `ExtraerMin` y m operaciones `CambiarClave`. Con un heap binario, el tiempo total es $O(m \log n)$.

Nota 4.5.10 (Otros objetivos de diseño). El árbol de cubrimiento mínimo optimiza un objetivo particular: conectar todo con costo total mínimo. Hay otros objetivos razonables que el árbol de cubrimiento mínimo no atiende. Podríamos preocuparnos por las distancias punto a punto dentro del árbol, y no es difícil construir ejemplos en los que el árbol de cubrimiento mínimo no las minimiza. Podríamos preocuparnos por la congestión, y también es fácil encontrar casos en los que concentra mucho tráfico sobre una sola arista. Incluso podríamos preguntarnos si un árbol es el tipo de solución adecuado: borrar cualquier arista de un árbol lo desconecta, así que los árboles no son nada robustos frente a fallas. Todas estas variantes conducen a problemas computacionalmente mucho más difíciles que el del árbol de cubrimiento mínimo.

4.6 Implementación de Kruskal: la estructura Union-Find

Uno de los problemas más básicos sobre grafos es hallar el conjunto de componentes conexas, y en el Capítulo 3 vimos algoritmos lineales con BFS o DFS que lo resuelven.

Consideremos ahora el escenario en que el grafo evoluciona por el agregado de aristas: la población de nodos es fija, pero con el tiempo aparecen aristas entre ciertos pares. El objetivo es mantener el conjunto de componentes conexas a lo largo de esa evolución sin recalcularlas desde cero cada vez que llega una arista. Para eso desarrollaremos una estructura de datos que llamaremos Union-Find, que guarda una representación de las componentes que permite consultarlas y actualizarlas rápido.

Esta es exactamente la estructura que hace eficiente al algoritmo de Kruskal. Cada vez que se considera una arista $e = (v, w)$ hay que averiguar rápidamente la identidad de las componentes conexas que contienen a v y a w: si son distintas, no hay camino de v a w y hay que incluir la arista; si son la misma, ya existe un camino v-w y hay que omitirla. Y cuando se la incluye, la estructura debe permitir fusionar eficientemente las componentes de v y w en una sola.

Definición 4.6.1 (Union-Find)**.** La estructura Union-Find mantiene una colección de conjuntos disjuntos y soporta tres operaciones:

- `IniciarUnionFind`(S), que devuelve una estructura sobre el conjunto S en la que cada elemento está en su propio conjunto. Aspiramos a implementarla en $O(n)$, con $n = |S|$.
- `Find`(u), que devuelve el nombre del conjunto que contiene a u. Aspiramos a $O(\log n)$, y en algunas implementaciones a $O(1)$.
- `Union`(A, B), que fusiona los conjuntos A y B en uno solo. Aspiramos a $O(\log n)$.

Con estas operaciones se mantienen las componentes conexas de un grafo $G = (V, E)$ que va recibiendo aristas: los conjuntos son las componentes, y al agregar una arista (u, v) primero se verifica si `Find`(u) = `Find`(v) y, si no lo son, se invoca `Union`(`Find`(u),`Find`(v)). Es importante notar que la estructura sirve para mantener componentes mientras se agregan aristas, pero no está diseñada para borrarlas, cosa que podría partir una componente en dos.

Sobre el nombre de un conjunto hay bastante libertad: lo único que se pide es consistencia, es decir, que `Find`(v) y `Find`(w) devuelvan lo mismo cuando v y w están en el mismo conjunto y cosas distintas en caso contrario. En nuestras implementaciones nombraremos cada conjunto con uno de sus elementos.

4.6.1 Una primera implementación con arreglos

Quizá la manera más simple de implementar Union-Find sea mantener un arreglo *Comp* que contenga el nombre del conjunto al que pertenece cada elemento. Sea S un conjunto de n elementos denotados $\{1, \ldots, n\}$; armamos un arreglo *Comp* de tamaño n donde $Comp[s]$ es el nombre del conjunto que contiene a s, y para iniciarlo ponemos $Comp[s] = s$ para todo s. Con esto `Find`(v) es una simple consulta al arreglo, en tiempo $O(1)$. En cambio, `Union`(A, B) puede costar $O(n)$, porque hay que actualizar $Comp[s]$ para todos los elementos de A y de B.

Hagamos algunas optimizaciones simples. Primero, conviene mantener explícitamente la lista de elementos de cada conjunto, para no recorrer todo el arreglo buscando cuáles actualizar. Segundo, si al fusionar elegimos como nombre del resultado el nombre de uno de los dos conjuntos, digamos A, solo hay que actualizar los $Comp[s]$ con $s \in B$. Claro que si B es grande esto no ayuda mucho; agreguemos entonces la optimización decisiva: mantener un arreglo adicional *tam* con el tamaño de cada conjunto y, al fusionar, quedarse con el nombre del conjunto más grande, de modo que la cantidad de elementos actualizados sea la menor de las dos.

Incluso así, el peor caso de una `Union` sigue siendo $O(n)$: eso ocurre al fusionar dos conjuntos grandes, cada uno con una fracción constante de los elementos. Pero esos casos malos no pueden ocurrir muy seguido, porque el conjunto resultante es todavía

más grande. Para precisarlo, en lugar de acotar el peor caso de una `Union` aislada, acotamos el tiempo total de una secuencia de k operaciones `Union`.

Proposición 4.6.2. En la implementación con arreglos de Union-Find sobre un conjunto S de tamaño n, donde las fusiones conservan el nombre del conjunto más grande, la operación `Find` cuesta $O(1)$, `IniciarUnionFind`(S) cuesta $O(n)$, y cualquier secuencia de k operaciones `Union` cuesta en total a lo sumo $O(k \log k)$.

Demostración. Las afirmaciones sobre `IniciarUnionFind` y `Find` son inmediatas. Consideremos una secuencia de k operaciones `Union`. Lo único que cuesta más de $O(1)$ en una fusión es actualizar el arreglo *Comp*, así que en lugar de acotar el tiempo de una fusión acotaremos el tiempo total dedicado a actualizar $Comp[v]$ para un elemento v a lo largo de las k operaciones.

Partimos de un estado en el que los n elementos están en conjuntos separados. Cada `Union` involucra a lo sumo dos de esos conjuntos unitarios originales, de modo que después de k fusiones todos los elementos salvo a lo sumo $2k$ quedaron completamente intactos.

Fijemos un elemento v. A medida que su conjunto participa de fusiones, su tamaño crece. En algunas de esas fusiones se actualiza $Comp[v]$ y en otras no; pero como la fusión conserva el nombre del conjunto más grande, cada vez que $Comp[v]$ se actualiza el tamaño del conjunto que contiene a v al menos se duplica. Ese tamaño arranca en 1 y no puede superar $2k$, por lo que $Comp[v]$ se actualiza a lo sumo $\log_2(2k)$ veces. Como a lo sumo $2k$ elementos participan de alguna fusión, el tiempo total dedicado a actualizar *Comp* en una secuencia de k fusiones es $O(k \log k)$. □

4.6.2 Una implementación con punteros

La cota anterior sobre el tiempo promedio de una secuencia alcanza para muchas aplicaciones, incluida la implementación de Kruskal. Aun así, veamos cómo reducir el peor caso de una operación individual, al precio de subir el costo de `Find` a $O(\log n)$.

La estructura usa punteros. Cada nodo $v \in S$ vive en un registro con un puntero al nombre del conjunto que lo contiene. Como antes, los conjuntos se nombran con uno de sus elementos. Para `IniciarUnionFind`(S) se crea un registro por elemento con el puntero apuntando a sí mismo.

Consideremos una `Union` de dos conjuntos A y B, donde A se llama $v \in A$ y B se llama $u \in B$. La idea es que el conjunto fusionado se llame u o v; supongamos que elegimos v. Para dejar registrado que se fusionaron y que el nombre del resultado es v, simplemente hacemos que el puntero de u apunte a v. No se actualizan los punteros de los demás nodos de B.

En consecuencia, para un elemento $w \in B$ distinto de u, el nombre de su conjunto se obtiene siguiendo una secuencia de punteros: primero hasta el nombre viejo u, y desde u hasta el nombre nuevo v.

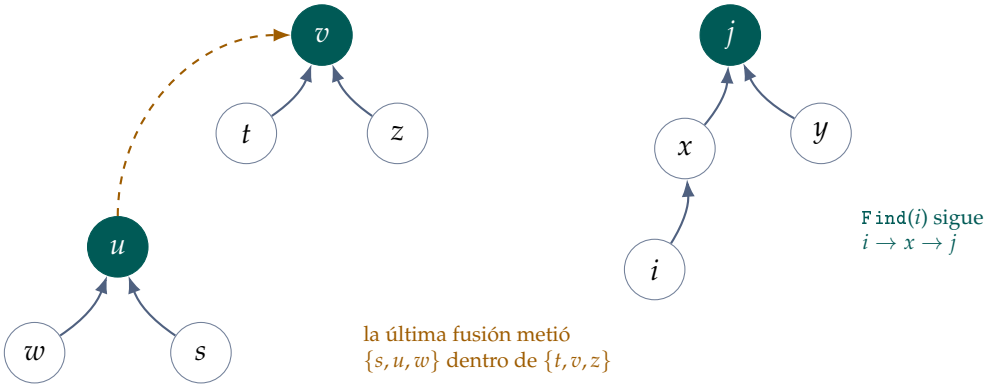


Figura 4.12. Una estructura Union-Find con punteros. Hay dos conjuntos, llamados v y j; la flecha punteada de u a v es el resultado de la última fusión. Para responder una consulta se siguen las flechas hasta llegar a un nodo sin flecha de salida.

Esta estructura implementa `Union` en $O(1)$: solo hay que actualizar un puntero. Pero `Find` deja de ser constante, porque hay que seguir una secuencia de punteros a través de los nombres viejos que fue teniendo el conjunto. La cantidad de pasos de `Find`(u) es exactamente la cantidad de veces que el conjunto que contiene a u cambió de nombre, que es lo mismo que la cantidad de actualizaciones de $Comp[u]$ en la implementación anterior. Sin cuidado con los nombres eso puede llegar a $O(n)$, así que aplicamos la misma optimización: la fusión conserva el nombre del conjunto más grande. Para hacerlo eficientemente, cada registro guarda además el tamaño de su conjunto.

Proposición 4.6.3. En la implementación con punteros de Union-Find sobre un conjunto S de tamaño n, donde las fusiones conservan el nombre del conjunto más grande, `Union` cuesta $O(1)$, `IniciarUnionFind`(S) cuesta $O(n)$ y `Find` cuesta $O(\log n)$.

Demostración. Las afirmaciones sobre `Union` e `IniciarUnionFind` son inmediatas. El tiempo de `Find`(v) es la cantidad de veces que el conjunto que contiene a v cambia de nombre. Como la fusión conserva el nombre del conjunto más grande, cada cambio de nombre al menos duplica el tamaño de ese conjunto. El tamaño arranca en 1 y nunca supera n, de modo que puede duplicarse a lo sumo $\log_2 n$ veces, y por lo tanto hay a lo sumo $\log_2 n$ cambios de nombre. □

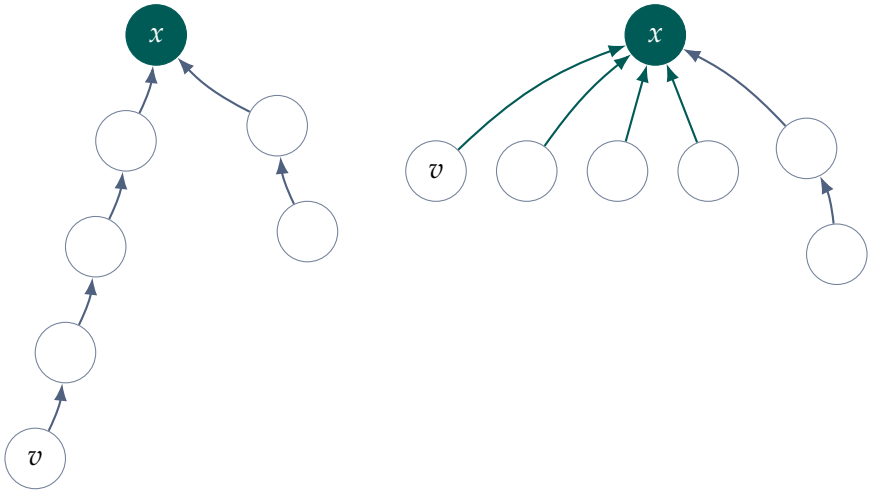
4.6.3 Compresión de caminos

Hay una optimización natural que acelera las operaciones `Find`. Estrictamente no la necesitamos —para todas las aplicaciones que vemos, $O(\log n)$ por operación es suficiente, porque la estructura no es el cuello de botella— pero vale la pena conocerla.

Pensemos en un caso malo. Fusionando repetidamente conjuntos del mismo tamaño se puede llegar a una situación en la que `Find`(v) recorre unos $\log n$ punteros. Si ahora invocamos `Find`(v) muchas veces, cada llamada paga ese costo. Y es un costo redundante:

después de la primera consulta ya conocemos el nombre x del conjunto que contiene a v, y también sabemos que todos los nodos que tocamos en el camino de v a x están en el conjunto x.

La idea de la compresión de caminos es entonces la siguiente: después de cada `Find`, se recorre otra vez el camino recorrido y se hace que todos sus punteros apunten directamente al nombre actual del conjunto. No se pierde información y las consultas posteriores se vuelven más rápidas.



(a) antes de `Find`(v) (b) después de `Find`(v)

Figura 4.13. Compresión de caminos. Todo lo que estaba sobre el camino de v a x pasa a apuntar directamente a x; los nodos que no fueron tocados por la consulta quedan como estaban.

Con compresión, `Union` sigue costando $O(1)$ e `IniciarUnionFind` sigue costando $O(n)$. Algunas operaciones `Find` pueden seguir costando $\log n$, y de hecho para algunas el costo aumenta, porque después de hallar el nombre x hay que volver a recorrer el camino de v a x reajustando los punteros. Pero ese trabajo extra a lo sumo duplica el tiempo, así que `Find` sigue costando $O(\log n)$. La ganancia real está en abaratar las consultas siguientes, y se hace precisa con el mismo tipo de argumento de la Proposición 4.6.2: acotando el tiempo total de una secuencia de n operaciones `Find` en lugar del peor caso de una sola. Sin entrar en los detalles, una secuencia de n operaciones `Find` con compresión requiere un tiempo extremadamente cercano a lineal en n; la cota exacta es $O(n\alpha(n))$, donde $\alpha(n)$ es la función inversa de Ackermann, que crece tan despacio que $\alpha(n) \leq 4$ para cualquier n que pueda aparecer en la práctica.

4.6.4 Implementación de Kruskal

Usemos ahora Union-Find para implementar el algoritmo de Kruskal. Primero hay que ordenar las aristas por costo, lo que cuesta $O(m \log m)$; como hay a lo sumo una arista entre cada par de nodos, vale $m \leq n^2$ y por lo tanto ese tiempo es también $O(m \log n)$.

Después del ordenamiento, usamos la estructura para mantener las componentes conexas de (V, T) a medida que se agregan aristas. Al considerar cada arista $e = (v, w)$ calculamos `Find`(v) y `Find`(w) y los comparamos para saber si v y w están en componentes distintas. Si el algoritmo decide incluir la arista, invocamos `Union`(`Find`(v),`Find`(w)) para fusionar las dos componentes.

KRUSKAL(G, c)

```
1  Ordenar las aristas de E por costo creciente: e_1, e_2, ..., e_m
2  T ← ∅;   IniciarUnionFind(V)
3  for i ← 1 to m do
4      Sea e_i = (v, w)
5      if Find(v) ≠ Find(w) then
6          T ← T ∪ {e_i}
7          Union(Find(v),Find(w))
8      end if
9  end for
10 return T
```

> **Estructuras de datos.** Las aristas se guardan en un arreglo de ternas (v, w, c) que se ordena con mergesort. Las componentes se mantienen en la estructura Union-Find con punteros y compresión de caminos: un arreglo de registros indexado por nodo, cada uno con el puntero al padre y el tamaño de su conjunto. El árbol resultante T se acumula en una lista enlazada de aristas. Se puede cortar el bucle apenas T tiene $n - 1$ aristas.

A lo largo de toda la ejecución se hacen a lo sumo $2m$ operaciones `Find` y $n - 1$ operaciones `Union`. Usando la Proposición 4.6.2 para la implementación con arreglos o la Proposición 4.6.3 para la implementación con punteros, eso da un total de $O(m \log n)$. Implementaciones más eficientes de Union-Find no mejorarían el tiempo de Kruskal, porque el término $O(m \log n)$ del ordenamiento inicial es inevitable.

Proposición 4.6.4. El algoritmo de Kruskal se implementa sobre un grafo con n nodos y m aristas en tiempo $O(m \log n)$.

Ejemplo 4.6.5 (Una red de fibra óptica)**.** Una empresa quiere conectar seis localidades a, b, c, d, e, f y tiene presupuestados los siguientes tendidos posibles, con sus costos en millones:

arista	(a,b)	(b,c)	(a,c)	(c,d)	(b,e)	(d,e)	(d,f)	(e,f)
costo	1	5	3	2	7	4	8	6

Kruskal ordena las aristas por costo: (a,b) con 1, (c,d) con 2, (a,c) con 3, (d,e) con 4, (b,c) con 5, (e,f) con 6, (b,e) con 7 y (d,f) con 8. Agrega (a,b), (c,d), (a,c) y (d,e) sin problema, porque cada una une componentes distintas. Al llegar a (b,c) encuentra que $\texttt{Find}(b) = \texttt{Find}(c)$ y la descarta; agrega (e,f), que era la primera arista hacia f, y con eso el árbol ya tiene cinco aristas y las seis localidades conectadas. Las dos aristas restantes se descartan. El costo total es $1+2+3+4+6 = 16$.

Notemos que la arista (b,c) descartada es la más cara del ciclo *a-b-c-a*, tal como anticipa la propiedad del ciclo, y que la arista (a,b) agregada primero es la más barata de todo el grafo, con lo cual la propiedad del corte aplicada a $S = \{a\}$ garantiza que pertenece a todo árbol de cubrimiento mínimo.

Capítulo 5

Divide y Vencerás

Divide y vencerás designa una familia de técnicas de diseño en las que la entrada se parte en varios trozos, el problema se resuelve recursivamente sobre cada trozo, y las soluciones parciales se combinan en una solución global. En muchos casos es un método simple y muy potente.

Analizar el tiempo de ejecución de un algoritmo de este tipo consiste, casi siempre, en resolver una relación de recurrencia: una desigualdad que acota el tiempo sobre entradas de tamaño n en términos del tiempo sobre entradas más pequeñas. Saber cómo aparecen esas recurrencias en el análisis, y qué métodos hay para extraer de ellas una cota superior explícita, es entonces tan importante como saber diseñar el algoritmo.

Hay además una diferencia con los capítulos anteriores. En los problemas típicos de divide y vencerás el algoritmo de fuerza bruta ya es polinómico, y la técnica sirve para bajar el tiempo a un polinomio de menor grado. El Ejemplo 2.4.4 es un caso de esos: la enumeración de los $\binom{n}{2}$ pares de puntos resuelve el problema del par más cercano en tiempo $\Theta(n^2)$, y la pregunta pasa a ser si se puede hacer algo asintóticamente mejor. El tema de fondo sigue siendo el mismo de siempre —superar la búsqueda por fuerza bruta es el obstáculo conceptual central—, con la diferencia de que la distancia entre la fuerza bruta y la solución mejorada ya no es la distancia entre lo exponencial y lo polinómico.

5.1 Una primera recurrencia: mergesort

Para motivar el enfoque general, empecemos por el algoritmo mergesort. Mergesort ordena una lista de números dividiéndola en dos mitades iguales, ordenando cada mitad por separado mediante recursión, y combinando después los resultados de esas dos llamadas —es decir, las dos mitades ya ordenadas— con el algoritmo lineal de combinación de listas ordenadas del Ejemplo 2.4.2.

Para analizar su tiempo de ejecución conviene abstraer el comportamiento del algoritmo en un esquema, que describe a muchos algoritmos de divide y vencerás a la vez.

Definición 5.1.1 (Esquema básico de divide y vencerás)**.** Dividir la entrada en dos partes de igual tamaño; resolver los dos subproblemas sobre esas partes por separado mediante recursión; y combinar los dos resultados en una solución global, gastando solamente tiempo lineal en la división inicial y en la recombinación final.

En mergesort, como en cualquier algoritmo que siga este esquema, hace falta además un caso base para la recursión: lo habitual es que la recursión "toque fondo" en entradas de tamaño constante. En el caso de mergesort supondremos que, una vez que la entrada se redujo a tamaño 2, la recursión se detiene y los dos elementos se ordenan comparándolos entre sí.

Consideremos cualquier algoritmo que siga el esquema de la Definición 5.1.1 y sea $T(n)$ su tiempo de ejecución en el peor caso sobre instancias de tamaño n. Suponiendo que n es par, el algoritmo gasta tiempo $O(n)$ en dividir la entrada en dos partes de tamaño $n/2$; gasta después tiempo $T(n/2)$ en resolver cada una, porque $T(n/2)$ es el tiempo de peor caso para una entrada de tamaño $n/2$; y gasta finalmente tiempo $O(n)$ en combinar las soluciones de las dos llamadas recursivas. El tiempo $T(n)$ satisface entonces la siguiente relación de recurrencia:

$$T(n) \;\leq\; 2\,T(n/2) + cn \quad \text{para } n > 2, \qquad T(2) \;\leq\; c, \tag{5.1}$$

donde c es una constante.

La estructura de (5.1) es la típica de una recurrencia: una desigualdad que acota $T(n)$ mediante una expresión que involucra $T(k)$ para valores k menores, y un caso base que dice que $T(n)$ es constante cuando n es constante. También se puede escribir de manera más informal como $T(n) \leq 2T(n/2) + O(n)$, ocultando la constante c; sin embargo, al analizar la recurrencia suele ser útil tenerla explícita.

Nota 5.1.2 (Sobre pisos y techos). Para simplificar la exposición vamos a suponer, cada vez que haga falta, que parámetros como n son pares. El uso es impreciso: sin ese supuesto las dos llamadas recursivas serían sobre problemas de tamaño $\lceil n/2 \rceil$ y $\lfloor n/2 \rfloor$, y la recurrencia diría

$$T(n) \leq T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + cn \quad \text{para } n \geq 2.$$

En todas las recurrencias que consideramos acá —y en casi todas las que aparecen en la práctica— las cotas asintóticas no se ven afectadas por la decisión de ignorar pisos y techos, y la manipulación simbólica queda mucho más limpia.

La recurrencia (5.1) no da explícitamente una cota asintótica sobre el crecimiento de T: especifica $T(n)$ de manera implícita, en términos de sus valores sobre entradas menores. Para obtener una cota explícita hay que resolver la recurrencia, es decir, dejar a T únicamente del lado izquierdo de la desigualdad y no también del derecho. Hay dos maneras básicas de hacerlo.

- La más intuitiva es desenrollar la recursión: contabilizar el tiempo de ejecución a lo largo de los primeros niveles, identificar un patrón que se pueda continuar a medida que la recursión se expande, sumar los tiempos sobre todos los niveles —hasta que la recursión toca fondo en subproblemas de tamaño constante— y llegar así a un tiempo total.
- La segunda es partir de una conjetura sobre la solución, sustituirla en la recurrencia y verificar que funciona. Formalmente, la sustitución se justifica con un argumento de inducción sobre n. Hay una variante muy útil de este método en la que uno tiene una forma general para la solución pero no los valores exactos de todos los parámetros; dejándolos sin especificar en la sustitución, muchas veces se los puede deducir sobre la marcha.

5.1.1 Desenrollar la recurrencia

Apliquemos el primer enfoque a (5.1). El argumento está ilustrado en la Figura 5.1.

- Analizar los primeros niveles: en el primer nivel de la recursión hay un único problema de tamaño n, que consume tiempo a lo sumo cn más el tiempo gastado en todas las llamadas recursivas posteriores. En el nivel siguiente hay dos problemas, cada uno de tamaño $n/2$; cada uno consume tiempo a lo sumo $cn/2$, con lo que el total del nivel es a lo sumo cn, otra vez más el tiempo de las llamadas posteriores. En el tercer nivel hay cuatro problemas de tamaño $n/4$, cada uno de costo a lo sumo $cn/4$, para un total de a lo sumo cn.
- Identificar el patrón: en el nivel j de la recursión la cantidad de subproblemas se duplicó j veces, de modo que hay 2^j; cada uno redujo su tamaño a la mitad j veces, de modo que tiene tamaño $n/2^j$ y consume tiempo a lo sumo $cn/2^j$. El nivel j aporta entonces a lo sumo $2^j(cn/2^j) = cn$ al tiempo total.
- Sumar sobre todos los niveles: la misma cota cn vale para el trabajo total realizado en cada nivel. La cantidad de veces que hay que partir la entrada por la mitad para reducir su tamaño de n a 2 es $\log_2 n$. Sumando el trabajo cn sobre $\log_2 n$ niveles se obtiene un tiempo total de $O(n \log n)$.

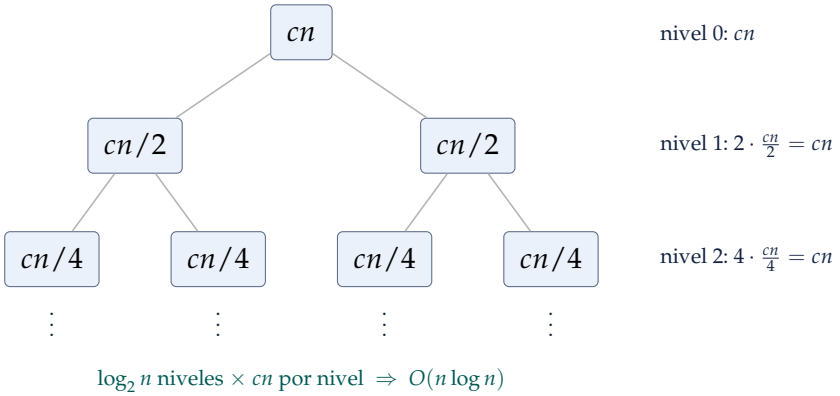


Figura 5.1. Desenrollado de la recurrencia $T(n) \leq 2T(n/2) + O(n)$. La cantidad de subproblemas se duplica de un nivel al siguiente y el tamaño de cada uno se reduce a la mitad, de modo que el trabajo total por nivel se mantiene constante en cn.

Proposición 5.1.3. Toda función $T(\cdot)$ que satisface (5.1) está acotada por $O(n \log n)$ para $n > 1$.

5.1.2 Sustituir una solución en la recurrencia

El argumento anterior sirve para descubrir que $T(n)$ está acotada por $O(n \log n)$. Si en cambio ya se tiene una conjetura sobre el tiempo de ejecución y se la quiere verificar, alcanza con sustituirla en la recurrencia.

Supongamos que creemos que $T(n) \leq cn \log_2 n$ para todo $n \geq 2$. Esto vale claramente para $n = 2$: en ese caso $cn \log_2 n = 2c$, y (5.1) nos dice explícitamente que $T(2) \leq c$. Supongamos ahora, por inducción, que $T(m) \leq cm \log_2 m$ para todo valor m menor que n, y establezcámoslo para $T(n)$. Escribimos la recurrencia para $T(n)$, sustituimos la desigualdad $T(n/2) \leq c(n/2) \log_2(n/2)$ y simplificamos usando $\log_2(n/2) = (\log_2 n) - 1$.

$$\begin{aligned} T(n) &\leq 2\,T(n/2) + cn \\ &\leq 2c(n/2)\log_2(n/2) + cn \\ &= cn\big[(\log_2 n) - 1\big] + cn \\ &= (cn \log_2 n) - cn + cn \\ &= cn \log_2 n. \end{aligned}$$

Esto establece la cota buscada para $T(n)$ suponiendo que vale para los valores $m < n$, con lo que se completa el argumento inductivo.

5.1.3 Sustitución parcial

Existe una forma más débil de sustitución, en la que se conjetura la forma general de la solución sin fijar de antemano el valor exacto de todas las constantes y parámetros.

Supongamos que creemos que $T(n) = O(n \log n)$ pero no estamos seguros de la constante dentro de la notación $O(\cdot)$. Podemos escribir $T(n) \leq kn \log_b n$ para una constante k y una base b que determinaremos más tarde. Nótese que la base y la constante están relacionadas entre sí, ya que cambiar la base del logaritmo equivale a cambiar la constante multiplicativa que lo precede. Probemos un nivel de la inducción:

$$T(n) \leq 2\,T(n/2) + cn \leq 2k(n/2)\log_b(n/2) + cn.$$

Resulta muy tentador elegir la base $b = 2$, porque eso permite aplicar la simplificación $\log_2(n/2) = (\log_2 n) - 1$. Con esa elección,

$$\begin{aligned} T(n) &\leq 2k(n/2)\log_2(n/2) + cn \\ &= 2k(n/2)\big[(\log_2 n) - 1\big] + cn \\ &= kn\big[(\log_2 n) - 1\big] + cn \\ &= (kn\log_2 n) - kn + cn. \end{aligned}$$

Preguntémonos ahora si hay alguna elección de k que haga que esta última expresión quede acotada por $kn\log_2 n$. La respuesta es claramente afirmativa: basta elegir cualquier k que sea al menos tan grande como c, y se obtiene

$$T(n) \leq (kn\log_2 n) - kn + cn \leq kn\log_2 n,$$

lo que completa la inducción. El método de sustitución sirve entonces, además, para deducir las constantes exactas cuando se tiene alguna conjetura sobre la forma general de la solución.

5.2 Otras relaciones de recurrencia

Consideremos ahora una clase de recurrencias que generaliza a (5.1). Se obtiene al considerar algoritmos de divide y vencerás que hacen llamadas recursivas sobre q subproblemas de tamaño $n/2$ cada uno y combinan los resultados en tiempo $O(n)$. Esto corresponde a la recurrencia de mergesort cuando se usan $q = 2$ llamadas recursivas, pero hay algoritmos a los que les resulta útil generar $q > 2$ llamadas, o una sola ($q = 1$).

Si $T(n)$ denota el tiempo de ejecución de un algoritmo diseñado con este estilo, entonces $T(n)$ obedece la siguiente recurrencia, que generaliza a (5.1) reemplazando el 2 por un q:

$$T(n) \;\leq\; q\,T(n/2) + cn \quad \text{para } n > 2, \qquad T(2) \;\leq\; c, \tag{5.2}$$

donde c es una constante. Tratamos los casos $q > 2$ y $q = 1$ por separado, porque son cualitativamente distintos entre sí, y también distintos del caso $q = 2$.

5.2.1 El caso de $q > 2$ subproblemas

Desenrollemos (5.2) para $q > 2$, siguiendo el estilo usado antes. El desenlace es bastante distinto.

- Analizar los primeros niveles: la Figura 5.2 muestra el caso $q = 3$. En el primer nivel hay un único problema de tamaño n, que consume tiempo a lo sumo cn. En el nivel siguiente hay q problemas de tamaño $n/2$, cada uno de costo a lo sumo $cn/2$, para un total de a lo sumo $(q/2)cn$. El nivel siguiente arroja q^2 problemas de tamaño $n/4$, para un total de $(q^2/4)cn$. Como $q > 2$, el trabajo total por nivel crece a medida que avanzamos en la recursión.
- Identificar el patrón: en un nivel arbitrario j hay q^j instancias distintas, cada una de tamaño $n/2^j$. El trabajo total del nivel j es entonces $q^j(cn/2^j) = (q/2)^j\, cn$.
- Sumar sobre todos los niveles: como antes hay $\log_2 n$ niveles, y el trabajo total es la suma sobre todos ellos:

$$T(n) \;\leq\; \sum_{j=0}^{\log_2 n - 1} \left(\frac{q}{2}\right)^j cn \;=\; cn \sum_{j=0}^{\log_2 n - 1} \left(\frac{q}{2}\right)^j.$$

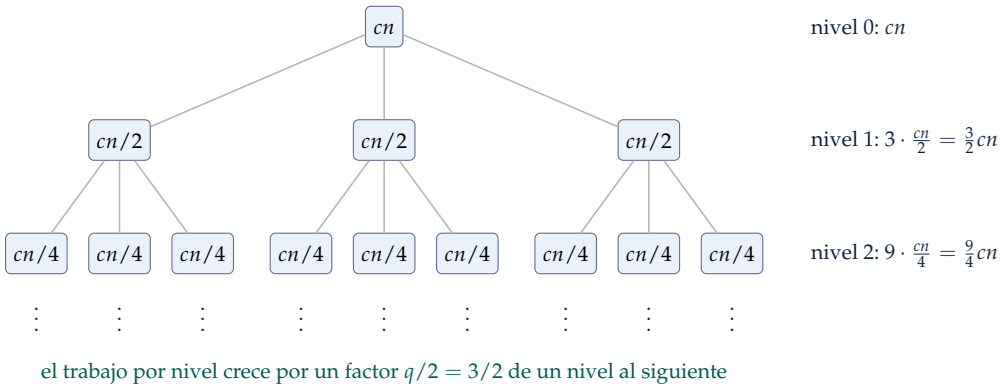


Figura 5.2. Desenrollado de la recurrencia $T(n) \leq 3T(n/2) + O(n)$. Ahora la cantidad de subproblemas se triplica mientras el tamaño de cada uno solo se divide por dos, de modo que el trabajo por nivel crece geométricamente y el grueso del costo se concentra en los niveles de abajo.

Se trata de una suma geométrica de razón $r = q/2$. Usando la fórmula para la suma geométrica con $r > 1$, y aflojando un poco la cota, obtenemos

$$T(n) \;\leq\; cn\left(\frac{r^{\log_2 n} - 1}{r - 1}\right) \;\leq\; cn\left(\frac{r^{\log_2 n}}{r - 1}\right) \;=\; \left(\frac{c}{r - 1}\right) n\, r^{\log_2 n},$$

donde en el último paso separamos el factor $1/(r-1)$, que es simplemente una constante. Falta calcular cuánto vale $r^{\log_2 n}$. Acá usamos una identidad muy cómoda: para $a > 1$ y $b > 1$ vale $a^{\log b} = b^{\log a}$. Por lo tanto

$$r^{\log_2 n} = n^{\log_2 r} = n^{\log_2(q/2)} = n^{(\log_2 q)-1},$$

y en consecuencia

$$T(n) \;\leq\; \left(\frac{c}{r-1}\right) n \cdot n^{(\log_2 q)-1} \;=\; \left(\frac{c}{r-1}\right) n^{\log_2 q} \;=\; O\Big(n^{\log_2 q}\Big).$$

Proposición 5.2.1. Toda función $T(\cdot)$ que satisface (5.2) con $q > 2$ está acotada por $O(n^{\log_2 q})$.

El tiempo de ejecución resulta entonces superlineal, ya que $\log_2 q > 1$, pero sigue siendo polinómico en n. Sustituyendo valores concretos de q: el tiempo es $O(n^{\log_2 3}) = O(n^{1{,}59})$ cuando $q = 3$, y es $O(n^{\log_2 4}) = O(n^2)$ cuando $q = 4$. Que el tiempo crezca con q tiene sentido, por supuesto, porque las llamadas recursivas generan más trabajo cuanto mayor es q.

Sustitución parcial

La aparición de $\log_2 q$ en el exponente salió de manera natural de la resolución de (5.2), pero no es una expresión que uno hubiera adivinado de entrada. Veamos cómo un enfoque basado en sustitución parcial permite descubrir ese exponente de otra manera.

Conjeturemos que la solución de (5.2) con $q > 2$ tiene la forma $T(n) \leq kn^d$ para ciertas constantes $k > 0$ y $d > 1$. Es una conjetura bastante general, porque ni siquiera intentamos especificar el exponente d del polinomio. Empecemos el argumento inductivo y veamos qué restricciones aparecen sobre k y d. Aplicando la hipótesis inductiva a $T(n/2)$,

$$\begin{aligned} T(n) &\leq q\,T(n/2) + cn \\ &\leq qk\Big(\frac{n}{2}\Big)^d + cn \\ &= \frac{q}{2^d}\,kn^d + cn. \end{aligned}$$

Esto está notablemente cerca de funcionar: si elegimos d de modo que $q/2^d = 1$, queda $T(n) \leq kn^d + cn$, que es casi lo que queremos salvo por el término sobrante cn. Hay entonces dos cuestiones a resolver: cómo elegir d para que $q/2^d = 1$, y cómo deshacerse del término cn.

Elegir d es fácil: queremos $2^d = q$, o sea $d = \log_2 q$. El exponente $\log_2 q$ aparece con toda naturalidad apenas nos decidimos a averiguar qué valor de d funciona al sustituirlo en la recurrencia.

Queda deshacernos del término cn. Para eso cambiamos la forma de la conjetura, de modo de restarlo explícitamente: probemos con $T(n) \leq kn^d - \ell n$, donde ya decidimos que $d = \log_2 q$ pero todavía no fijamos las constantes k ni ℓ. Aplicando la nueva fórmula a $T(n/2)$,

$$\begin{aligned} T(n) &\leq qk\Big(\frac{n}{2}\Big)^d - q\ell\Big(\frac{n}{2}\Big) + cn \\ &= \frac{q}{2^d}\, kn^d - \frac{q}{2}\,\ell n + cn \\ &= kn^d - \Big(\frac{q}{2}\,\ell - c\Big)n. \end{aligned}$$

Esto funciona por completo si elegimos ℓ de modo que $\frac{q}{2}\ell - c = \ell$, es decir, $\ell = 2c/(q-2)$, que es positivo justamente porque $q > 2$. Con eso queda cerrado el paso inductivo. Falta el caso base $n = 2$, y para eso usamos que el valor de k todavía no fue fijado: elegimos k suficientemente grande como para que la fórmula sea una cota superior válida en $n = 2$.

5.2.2 El caso de un solo subproblema

Consideremos ahora el caso $q = 1$ de (5.2), que ilustra un desenlace de un tercer sabor. Desenrollamos la recurrencia como antes.

- Analizar los primeros niveles: la Figura 5.3 muestra los primeros niveles. En el primero hay un único problema de tamaño n, de costo a lo sumo cn. El nivel siguiente tiene un problema de tamaño $n/2$, que aporta $cn/2$, y el siguiente uno de tamaño $n/4$, que aporta $cn/4$. A diferencia del caso anterior, el trabajo total por nivel decrece a medida que avanzamos en la recursión.
- Identificar el patrón: en un nivel arbitrario j sigue habiendo una única instancia; tiene tamaño $n/2^j$ y aporta $cn/2^j$ al tiempo de ejecución.
- Sumar sobre todos los niveles: hay $\log_2 n$ niveles, y el trabajo total es

$$T(n) \;\leq\; \sum_{j=0}^{\log_2 n - 1} \frac{cn}{2^j} \;=\; cn \sum_{j=0}^{\log_2 n - 1} \Big(\frac{1}{2^j}\Big).$$

  Esta suma geométrica es muy fácil de acotar: aun continuándola hasta el infinito converge a 2. Por lo tanto $T(n) \leq 2cn = O(n)$.

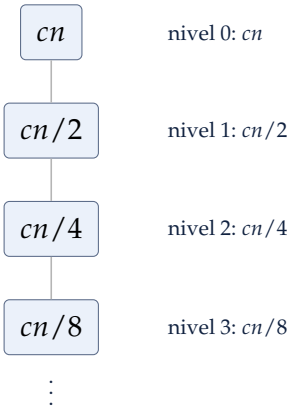


Figura 5.3. Desenrollado de la recurrencia $T(n) \leq T(n/2) + O(n)$. Hay un único subproblema por nivel y su tamaño se divide por dos, de modo que el trabajo por nivel decae geométricamente y la suma de los $\log_2 n$ niveles queda acotada por una constante por cn.

Proposición 5.2.2. Toda función $T(\cdot)$ que satisface (5.2) con $q = 1$ está acotada por $O(n)$.

El resultado es contraintuitivo la primera vez que se lo ve: el algoritmo realiza $\log n$ niveles de recursión y sin embargo el tiempo total sigue siendo lineal en n. Lo que ocurre es que una serie geométrica con exponente decreciente es algo poderoso: la mitad del trabajo del algoritmo se hace en el nivel de arriba de la recursión.

También acá funciona muy bien la sustitución parcial. Conjeturemos, como antes, que la forma de la solución es $T(n) \leq kn^d$, y tratemos de establecerlo por inducción usando (5.2), suponiendo que vale para el valor menor $n/2$:

$$\begin{aligned} T(n) &\leq T(n/2) + cn \\ &\leq k\left(\frac{n}{2}\right)^d + cn \\ &= \frac{k}{2^d}\, n^d + cn. \end{aligned}$$

Si ahora elegimos simplemente $d = 1$ y $k = 2c$, obtenemos

$$T(n) \;\leq\; \frac{k}{2}\, n + cn \;=\; \left(\frac{k}{2} + c\right) n \;=\; kn,$$

lo que completa la inducción.

5.2.3 El efecto del parámetro q

Vale la pena detenerse un momento en el papel del parámetro q en la clase de recurrencias $T(n) \leq qT(n/2) + O(n)$ definida por (5.2). Cuando $q = 1$ el tiempo resultante es lineal; cuando $q = 2$ es $O(n \log n)$; y cuando $q > 2$ es una cota polinómica con un exponente mayor que 1 que crece con q.

La razón de este rango de comportamientos está en dónde se gasta la mayor parte del trabajo de la recursión. Cuando $q = 1$ el tiempo total está dominado por el nivel de arriba, mientras que cuando $q > 2$ está dominado por el trabajo hecho sobre los subproblemas de tamaño constante del fondo de la recursión. Visto así, se aprecia que la recurrencia con $q = 2$ representa un filo: el trabajo hecho en cada nivel es exactamente el mismo, y eso es lo que produce el tiempo $O(n \log n)$.

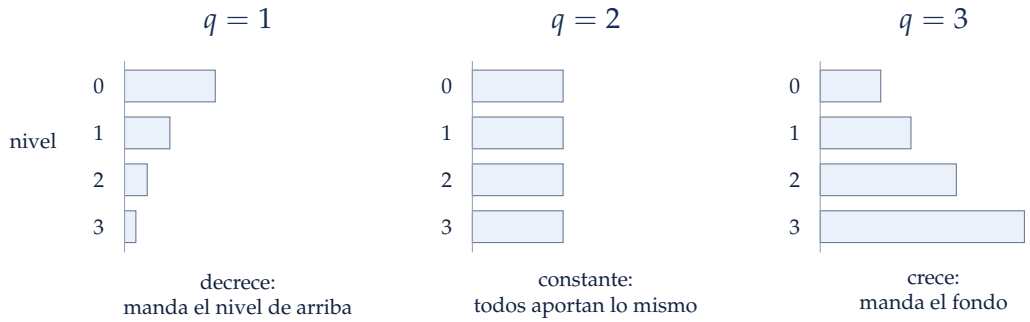


Figura 5.4. Trabajo realizado en cada nivel de la recursión para $T(n) \leq qT(n/2) + O(n)$, con el largo de cada barra proporcional a ese trabajo. El caso $q = 2$ es el filo entre un perfil que decae geométricamente y uno que crece geométricamente, y por eso es el único de los tres en el que la cantidad de niveles, $\log_2 n$, aparece en el resultado.

5.2.4 Una recurrencia relacionada: $T(n) \leq 2T(n/2) + O(n^2)$

Cerramos la discusión con una última recurrencia, ilustrativa tanto como otra aplicación de una suma geométrica decreciente cuanto como contraste interesante con (5.1). Se basa en la siguiente estructura de divide y vencerás: dividir la entrada en dos partes de igual tamaño; resolver los dos subproblemas sobre esas partes por separado mediante recursión; y combinar los dos resultados en una solución global, gastando tiempo cuadrático en la división inicial y en la recombinación final. Un algoritmo con ese perfil tiene un tiempo de ejecución $T(n)$ que satisface

$$T(n) \;\leq\; 2\,T(n/2) + cn^2 \quad \text{para } n > 2, \qquad T(2) \;\leq\; c. \tag{5.3}$$

La primera reacción es conjeturar que la solución será $T(n) = O(n^2 \log n)$, ya que (5.3) se parece muchísimo a (5.1) salvo porque el trabajo por nivel es mayor por un factor igual al tamaño de la entrada. Esa cota superior es correcta —aunque necesitaría un argumento más cuidadoso que la frase anterior—, pero resulta que se puede probar una cota superior más fuerte. Desenrollemos la recurrencia con el molde de siempre.

- Analizar los primeros niveles: en el primer nivel hay un único problema de tamaño n, de costo a lo sumo cn^2. En el nivel siguiente hay dos problemas de tamaño $n/2$, cada uno de costo a lo sumo $c(n/2)^2 = cn^2/4$, para un total de a lo sumo $cn^2/2$. En el tercer nivel hay cuatro problemas de tamaño $n/4$, cada uno de costo a lo sumo $c(n/4)^2 = cn^2/16$, para un total de a lo sumo $cn^2/4$. Ya se ve que algo es distinto de lo que pasaba en (5.1): allí el trabajo por nivel se mantenía constante, y acá decrece.
- Identificar el patrón: en un nivel arbitrario j hay 2^j subproblemas de tamaño $n/2^j$, de modo que el trabajo total del nivel está acotado por $2^j c\,(n/2^j)^2 = cn^2/2^j$.
- Sumar sobre todos los niveles: llegamos casi exactamente a la misma suma que teníamos en el caso $q = 1$ de la recurrencia anterior:

$$T(n) \;\leq\; \sum_{j=0}^{\log_2 n - 1} \frac{cn^2}{2^j} \;=\; cn^2 \sum_{j=0}^{\log_2 n - 1} \left(\frac{1}{2^j}\right) \;\leq\; 2cn^2 \;=\; O(n^2),$$

  donde la segunda desigualdad se sigue de que la suma geométrica es convergente.

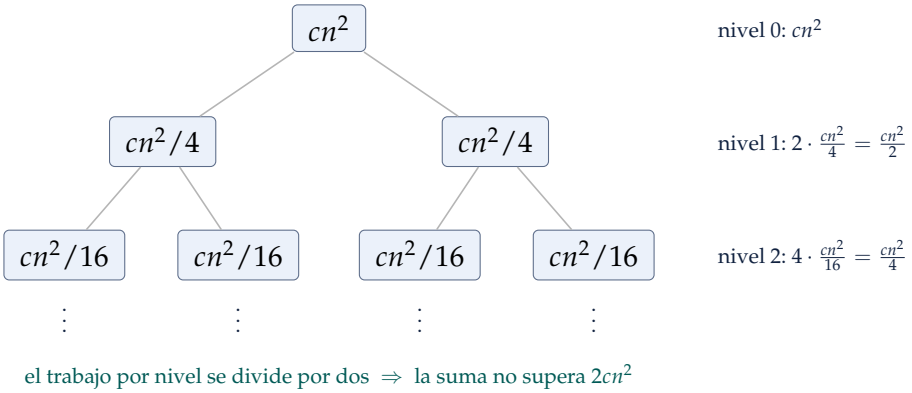


Figura 5.5. Desenrollado de la recurrencia $T(n) \leq 2T(n/2) + O(n^2)$. La cantidad de subproblemas se duplica, pero el costo de cada uno se divide por cuatro al pasar de tamaño n a tamaño $n/2$; el trabajo por nivel decae entonces geométricamente y el total es $O(n^2)$, sin factor logarítmico.

Visto en retrospectiva, la conjetura inicial de $T(n) = O(n^2 \log n)$, basada en la analogía con (5.1), era una sobreestimación por la rapidez con la que decrece n^2 cuando se lo reemplaza por $(\frac{n}{2})^2$, $(\frac{n}{4})^2$, $(\frac{n}{8})^2$, y así siguiendo, en el desenrollado de la recurrencia. Lo que se obtiene es una suma geométrica, y no una suma que crece en una cantidad fija a lo largo de todos los niveles como en la solución de (5.1).

5.3 Contar inversiones

Hasta acá discutimos maneras de resolver algunas recurrencias comunes. Pasemos ahora a aplicar divide y vencerás a un problema concreto, usando lo visto en las secciones

anteriores para acotar el tiempo de ejecución: una variante de la técnica de mergesort resuelve un problema que no está directamente relacionado con ordenar números.

El problema surge en el análisis de rankings. Muchos sitios web usan una técnica conocida como *collaborative filtering*, en la que intentan emparejar las preferencias de una persona —por libros, películas, restaurantes— con las de otras personas. Una vez que el sitio identificó gente con gustos "parecidos" a los de uno, basándose en una comparación de cómo cada uno califica distintas cosas, puede recomendar cosas nuevas que a esas otras personas les gustaron. Otra aplicación aparece en los meta-buscadores, que ejecutan la misma consulta en muchos buscadores distintos y tratan de sintetizar los resultados buscando similitudes y diferencias entre los rankings devueltos.

El problema de fondo en aplicaciones así es el de comparar dos rankings. Uno ordena un conjunto de n películas y el sistema consulta su base de datos buscando otras personas con rankings "parecidos". Ahora bien, ¿cuál es una buena manera de medir numéricamente cuán parecidos son los rankings de dos personas? Claramente un ranking idéntico es muy parecido y uno completamente invertido es muy distinto; queremos algo que interpole a lo largo de la zona intermedia.

Comparemos entonces nuestro ranking con el de un desconocido sobre el mismo conjunto de n películas. Un método natural es etiquetar las películas de 1 a n según nuestro ranking, ordenar después esas etiquetas según el ranking del desconocido, y ver cuántos pares quedan "fuera de orden". Concretamente, nos dan una secuencia de n números $a_1, \ldots, a_n$, que supondremos todos distintos, y queremos definir una medida que diga cuán lejos está esta lista de estar en orden creciente: el valor de la medida debe ser 0 si $a_1 < a_2 < \ldots < a_n$, y debe aumentar a medida que los números se desordenan.

Definición 5.3.1 (Inversión)**.** Dos índices $i < j$ forman una inversión de la secuencia $a_1, \ldots, a_n$ si $a_i > a_j$, es decir, si los dos elementos a_i y a_j están fuera de orden entre sí. El número de inversiones de la secuencia es la cantidad de tales pares.

Ejemplo 5.3.2 (Las inversiones de una secuencia corta)**.** Consideremos la secuencia $2, 4, 1, 3, 5$. Tiene tres inversiones: los pares $(2, 1)$, $(4, 1)$ y $(4, 3)$. Hay además una manera geométrica muy sugestiva de visualizarlas, ilustrada en la Figura 5.6: se dibujan los números de la entrada en el orden en que vienen dados y, debajo, en orden creciente; después se traza un segmento entre cada número de la lista de arriba y su copia en la de abajo. Cada par de segmentos que se cruzan corresponde a un par que aparece en orden opuesto en las dos listas, es decir, a una inversión.

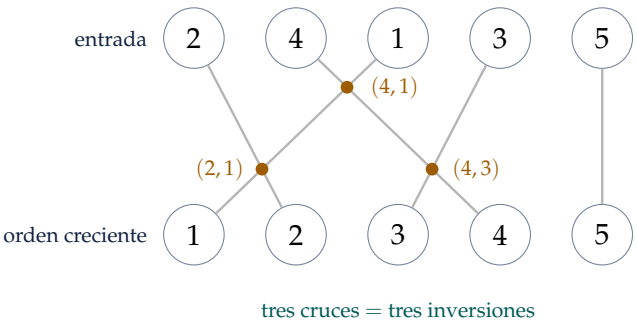


Figura 5.6. Conteo de las inversiones de la secuencia 2, 4, 1, 3, 5. Cada par de segmentos que se cruzan corresponde a un par de elementos que aparece en orden opuesto en la lista de entrada y en la lista creciente, o sea a una inversión.

Nótese que el número de inversiones es una medida que interpola suavemente entre el acuerdo total —cuando la secuencia está en orden creciente no hay ninguna inversión— y el desacuerdo total: si la secuencia está en orden decreciente, todo par forma una inversión y hay $\binom{n}{2}$ de ellas.

5.3.1 Diseño del algoritmo

¿Cuál es el algoritmo más simple para contar inversiones? Claramente podríamos mirar todos los pares de números (a_i, a_j) y determinar si constituyen una inversión; eso llevaría tiempo $O(n^2)$.

Veamos cómo contarlas mucho más rápido, en tiempo $O(n \log n)$. Notemos que, como puede haber una cantidad cuadrática de inversiones, un algoritmo así tiene que poder calcular el total sin mirar nunca cada inversión individualmente. La idea básica es seguir el esquema de la Definición 5.1.1. Fijamos $m = \lceil n/2 \rceil$ y partimos la lista en dos trozos $a_1, \ldots, a_m$ y $a_{m+1}, \ldots, a_n$. Primero contamos las inversiones dentro de cada una de las dos mitades por separado. Después contamos las inversiones (a_i, a_j) en las que los dos números pertenecen a mitades distintas; el truco está en que esta parte hay que hacerla en tiempo $O(n)$ si queremos aplicar la Proposición 5.1.3. Estas inversiones entre la primera y la segunda mitad tienen una forma especialmente cómoda: son exactamente los pares (a_i, a_j) con a_i en la primera mitad, a_j en la segunda y $a_i > a_j$.

Para ayudarnos a contarlas, haremos que el algoritmo también ordene recursivamente los números de las dos mitades. Que el paso recursivo haga un poco más de trabajo —ordenar además de contar— hará más fácil la parte de combinación.

La rutina crucial del proceso es entonces COMBINAR-Y-CONTAR. Supongamos que ya ordenamos recursivamente la primera y la segunda mitad de la lista y contamos las inversiones de cada una. Tenemos ahora dos listas ordenadas A y B con la primera y la segunda mitad respectivamente. Queremos producir una única lista ordenada C con la unión de ambas y, a la vez, contar la cantidad de pares (a, b) con $a \in A$, $b \in B$ y $a > b$. Por lo que discutimos recién, eso es exactamente lo que necesitamos para el paso de combinación.

Esto está muy relacionado con el problema más simple del Ejemplo 2.4.2, que forma el paso de combinación de mergesort: allí teníamos dos listas ordenadas A y B y queríamos combinarlas en una única lista ordenada en tiempo $O(n)$. La diferencia acá es que queremos hacer algo extra: no solo producir una lista ordenada a partir de A y B, sino además contar la cantidad de pares invertidos.

Resulta que se puede hacer con el mismo estilo que usamos para la combinación. La rutina recorre las listas ordenadas A y B quitando elementos del frente y agregándolos a la lista ordenada C. En un paso dado tenemos un puntero actual dentro de cada lista, que indica nuestra posición. Supongamos que esos punteros están sobre los elementos a_i y b_j; en un paso comparamos ambos, quitamos el menor de su lista y lo agregamos al final de C.

Eso resuelve la combinación. ¿Cómo contamos también las inversiones? Como A y B están ordenadas, llevar la cuenta es en realidad muy fácil.

- Cada vez que se agrega a_i a C no se encuentra ninguna inversión nueva, porque a_i es menor que todo lo que queda en B y además viene antes que todos ellos.
- En cambio, si el que se agrega a C es b_j, entonces b_j es menor que todos los elementos que quedan en A y viene después que todos ellos, así que incrementamos la cuenta de inversiones en la cantidad de elementos que quedan en A.

Esta es la idea crucial: en tiempo constante damos cuenta de una cantidad potencialmente grande de inversiones. La Figura 5.7 ilustra el proceso.

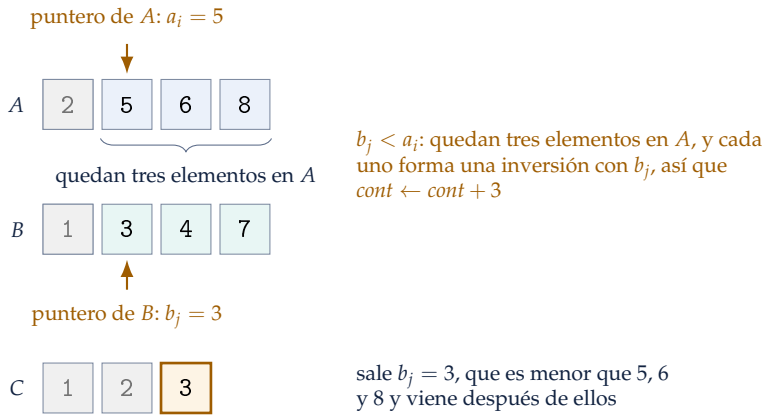


Figura 5.7. Un paso de la combinación de dos listas ordenadas contando a la vez las inversiones entre ellas. Cuando el elemento que sale proviene de B, forma una inversión con cada uno de los elementos que todavía quedan en A, y todas ellas se contabilizan de un saque; cuando el elemento que sale proviene de A no se genera ninguna inversión nueva.

En resumen, tenemos el siguiente algoritmo.

COMBINAR-Y-CONTAR(A, B)

```
1  Puntero actual en cada lista, apuntando al primer elemento
2  cont ← 0;    C ← lista vacía
3  while ambas listas son no vacías do
4      Sean a_i y b_j los elementos señalados por los punteros
5      Agregar el menor de los dos al final de C
6      if (b_j es el menor) then
7          cont ← cont + cantidad de elementos que quedan en A
8      end if
9      Avanzar el puntero de la lista de la que salió el menor
10 end while
11 Agregar al final de C lo que quede de la lista no vacía
12 return cont y la lista combinada C
```

> **Estructuras de datos.** Las listas A, B y C son arreglos de enteros; los "punteros actuales" son dos índices i y j, y la cantidad de elementos que quedan en A se calcula como $|A| - i + 1$ en tiempo constante, sin recorrer nada. La salida C se escribe en un arreglo auxiliar del tamaño de la suma de ambas, que se puede reservar una sola vez al principio de todo y reutilizar en cada llamada. La cuenta *cont* debe ser un entero de 64 bits, ya que puede llegar a $\binom{n}{2}$.

El tiempo de ejecución de COMBINAR-Y-CONTAR se acota con el mismo argumento de cobro del Ejemplo 2.4.2: cada iteración del `while` cuesta tiempo constante y en cada una se agrega a la salida un elemento que no se vuelve a mirar. La cantidad de iteraciones es entonces a lo sumo la suma de los largos iniciales de A y B, y el tiempo total es $O(n)$.

Usamos esta rutina dentro de un procedimiento recursivo que simultáneamente ordena y cuenta las inversiones de una lista L.

ORDENAR-Y-CONTAR(L)

```
1  if L tiene un solo elemento then
2      return 0 y la lista L
3  end if
4  Dividir L en dos mitades: A con los primeros ⌈n/2⌉
   elementos y B con los restantes
5  (r_A, A) ← ORDENAR-Y-CONTAR(A)
6  (r_B, B) ← ORDENAR-Y-CONTAR(B)
7  (r, L) ← COMBINAR-Y-CONTAR(A, B)
8  return r_A + r_B + r y la lista ordenada L
```

> **Estructuras de datos.** La entrada es un arreglo $L[1..n]$ y las "mitades" no se copian: se pasan como un par de índices sobre el mismo arreglo, igual que en mergesort. Con eso, cada nivel de la recursión trabaja sobre el arreglo original más el arreglo auxiliar de la combinación, y el uso de memoria total es $O(n)$.

Ejemplo 5.3.3 (Una corrida completa)**.** Tomemos $L = 5, 8, 2, 6, 1, 7, 3, 4$. La primera mitad, $5, 8, 2, 6$, tiene tres inversiones: $(5, 2)$, $(8, 2)$ y $(8, 6)$. La segunda mitad, $1, 7, 3, 4$, tiene dos: $(7, 3)$ y $(7, 4)$. Las llamadas recursivas devuelven entonces $r_A = 3$ y $r_B = 2$, junto con las mitades ya ordenadas $A = 2, 5, 6, 8$ y $B = 1, 3, 4, 7$.

La combinación final procede así. Sale el 1 de B, y como quedan cuatro elementos en A se suman cuatro inversiones. Sale el 2 de A, sin sumar nada. Sale el 3 de B, y como quedan tres elementos en A —justamente el paso dibujado en la Figura 5.7— se suman tres. Sale el 4 de B, y se suman otras tres. Salen el 5 y el 6 de A, sin sumar nada. Sale el 7 de B, y como solo queda el 8 en A se suma una. Finalmente se agrega el 8. El total del paso de combinación es $r = 4 + 3 + 3 + 1 = 11$.

La cantidad de inversiones de L es entonces $3 + 2 + 11 = 16$. Notemos que el algoritmo nunca enumeró esas 16 inversiones una por una: las once del paso de combinación se contabilizaron en apenas cuatro sumas.

5.3.2 Análisis del algoritmo

Como COMBINAR-Y-CONTAR lleva tiempo $O(n)$, el tiempo de ejecución $T(n)$ del procedimiento completo satisface la recurrencia (5.1), y la Proposición 5.1.3 da la cota.

Proposición 5.3.4. El algoritmo ORDENAR-Y-CONTAR ordena correctamente la lista de entrada y cuenta la cantidad de inversiones; su tiempo de ejecución es $O(n \log n)$ para una lista de n elementos.

5.4 El par de puntos más cercano

Describimos ahora otro problema que se resuelve con un algoritmo del estilo que venimos discutiendo, pero en el que encontrar la manera correcta de combinar las soluciones de los dos subproblemas requiere bastante ingenio.

El problema es muy simple de enunciar: dados n puntos en el plano, hallar el par que está más cerca entre sí. Fue considerado por M. I. Shamos y D. Hoey a principios de los años setenta, como parte de un proyecto para desarrollar algoritmos eficientes para las primitivas básicas de la geometría. Esos algoritmos formaron los cimientos del entonces incipiente campo de la geometría computacional, y encontraron su camino hacia áreas como los gráficos por computadora, la visión artificial, los sistemas de información geográfica y el modelado molecular.

Aunque el problema del par más cercano es uno de los problemas algorítmicos más naturales de la geometría, es sorprendentemente difícil hallar un algoritmo eficiente para él. Es inmediato que existe una solución $O(n^2)$ —calcular la distancia entre cada par de puntos y quedarse con el mínimo, tal como hace el Ejemplo 2.4.4—, y por eso Shamos y Hoey se preguntaron si podría hallarse un algoritmo asintóticamente más

rápido que el cuadrático. Les llevó bastante tiempo resolver la cuestión, y el algoritmo $O(n \log n)$ que damos a continuación es esencialmente el que ellos descubrieron.

5.4.1 Diseño del algoritmo

Empecemos con algo de notación. Denotamos el conjunto de puntos por $P = \{p_1, \ldots, p_n\}$, donde p_i tiene coordenadas (x_i, y_i); y para dos puntos $p_i, p_j \in P$ escribimos $d(p_i, p_j)$ para la distancia euclídea usual entre ellos. El objetivo es hallar un par de puntos p_i, p_j que minimice $d(p_i, p_j)$.

Supondremos que no hay dos puntos de P con la misma coordenada x ni con la misma coordenada y. Esto hace más limpia la discusión, y el supuesto es fácil de eliminar: alcanza con aplicar inicialmente a los puntos una rotación que lo vuelva cierto, o bien con extender levemente el algoritmo.

Es instructivo considerar por un momento la versión unidimensional del problema, que es mucho más simple y cuyo contraste es revelador. ¿Cómo hallaríamos el par de puntos más cercano sobre una recta? Los ordenaríamos, en tiempo $O(n \log n)$, y después recorreríamos la lista ordenada calculando la distancia de cada punto al que le sigue. Es fácil ver que una de esas distancias tiene que ser la mínima.

En dos dimensiones podríamos intentar ordenar los puntos por su coordenada y (o por su coordenada x) con la esperanza de que los dos puntos más cercanos queden próximos entre sí en ese orden. Pero es fácil construir ejemplos en los que quedan muy lejos, lo que impide adaptar el enfoque unidimensional.

El plan será entonces aplicar el estilo de divide y vencerás usado en mergesort: hallamos el par más cercano entre los puntos de la "mitad izquierda" de P y el par más cercano entre los de la "mitad derecha", y usamos esa información para obtener la solución global en tiempo lineal. Si logramos un algoritmo con esa estructura, la solución de (5.1) nos dará un tiempo $O(n \log n)$.

La fase difícil es la última, la de combinación: las distancias que ninguna de las dos llamadas recursivas consideró son precisamente las que ocurren entre un punto de la mitad izquierda y un punto de la mitad derecha; hay $\Theta(n^2)$ distancias así, y sin embargo tenemos que hallar la menor en tiempo $O(n)$ una vez que las llamadas recursivas retornan. Si conseguimos hacerlo, la solución quedará completa: será la menor entre los valores calculados en las llamadas recursivas y esta distancia mínima "de izquierda a derecha".

Preparar la recursión

Saquemos primero de en medio algunas cuestiones fáciles. Va a ser muy útil que toda llamada recursiva, sobre un conjunto $P' \subseteq P$, empiece con dos listas: una lista P'_x con los puntos de P' ordenados por coordenada x creciente, y una lista P'_y con los puntos de P' ordenados por coordenada y creciente. Podemos garantizar que esto siga siendo cierto a lo largo de todo el algoritmo de la siguiente manera.

Antes de que empiece la recursión ordenamos todos los puntos de P por coordenada x y otra vez por coordenada y, produciendo las listas P_x y P_y. A cada entrada de cada

lista le adjuntamos un registro de la posición de ese punto en ambas listas.

El primer nivel de la recursión funciona como sigue, y todos los niveles siguientes de manera completamente análoga. Definimos Q como el conjunto de puntos que ocupan las primeras $\lceil n/2 \rceil$ posiciones de la lista P_x (la "mitad izquierda") y R como el conjunto de los que ocupan las últimas $\lfloor n/2 \rfloor$ posiciones (la "mitad derecha"). Con una única pasada sobre P_x y otra sobre P_y, en tiempo $O(n)$, podemos construir las cuatro listas Q_x, Q_y, R_x y R_y, con los puntos de Q y de R ordenados por coordenada x y por coordenada y; para cada entrada de cada lista registramos, como antes, la posición del punto en las dos listas a las que pertenece.

Determinamos ahora recursivamente un par de puntos más cercano en Q, con acceso a las listas Q_x y Q_y. Supongamos que la llamada devuelve correctamente el par q_0^*, q_1^*. Análogamente determinamos un par más cercano en R, obteniendo r_0^*, r_1^*.

5.4.2 Combinación de las soluciones

La maquinaria general de divide y vencerás nos trajo hasta acá sin entrar realmente en la estructura del problema. Pero nos deja con la dificultad que se veía venir desde el principio: ¿cómo usar las soluciones de los dos subproblemas dentro de una operación de combinación de tiempo lineal?

Sea δ el mínimo entre $d(q_0^*, q_1^*)$ y $d(r_0^*, r_1^*)$. La pregunta verdadera es: ¿hay puntos $q \in Q$ y $r \in R$ con $d(q,r) < \delta$? Si no los hay, entonces ya hallamos el par más cercano en una de las dos llamadas recursivas. Si los hay, entonces el par q, r más cercano de esa forma es el par más cercano de P.

Sea x^* la coordenada x del punto más a la derecha de Q, y sea L la recta vertical de ecuación $x = x^*$. Esta recta "separa" Q de R. Vale entonces el siguiente hecho simple.

Lema 5.4.1. Si existen $q \in Q$ y $r \in R$ con $d(q,r) < \delta$, entonces tanto q como r están a distancia menor que δ de la recta L.

Demostración. Supongamos que existen tales q y r, y escribamos $q = (q_x, q_y)$ y $r = (r_x, r_y)$. Por la definición de x^* sabemos que $q_x \leq x^* \leq r_x$. Entonces

$$x^* - q_x \leq r_x - q_x \leq d(q,r) < \delta \qquad \text{y} \qquad r_x - x^* \leq r_x - q_x \leq d(q,r) < \delta,$$

así que tanto q como r tienen su coordenada x a distancia menor que δ de x^*, y por lo tanto están a distancia menor que δ de la recta L. □

Si queremos hallar un q y un r cercanos podemos entonces restringir la búsqueda a la franja angosta formada por los puntos de P que están a distancia menor que δ de L. Llamemos $S \subseteq P$ a ese conjunto y S_y a la lista con los puntos de S ordenados por coordenada y creciente; con una única pasada sobre la lista P_y podemos construir S_y en tiempo $O(n)$. El Lema 5.4.1 se puede reformular en términos de S.

Lema 5.4.2. Existen $q \in Q$ y $r \in R$ con $d(q,r) < \delta$ si y solo si existen $s, s' \in S$ con $d(s,s') < \delta$.

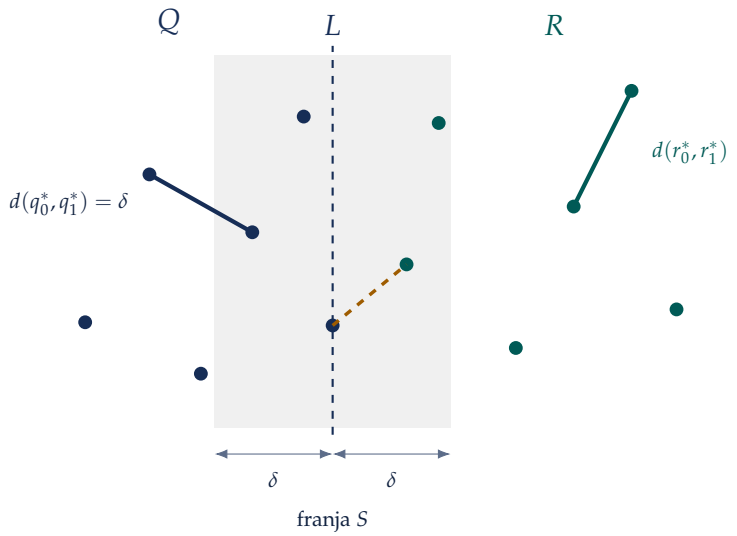


Figura 5.8. Primer nivel de la recursión: la recta L divide a P en la mitad izquierda Q y la mitad derecha R, y en cada una se halla recursivamente el par más cercano. El menor de esos dos valores es δ. Todo par a distancia menor que δ con un punto de cada lado tiene que estar íntegramente dentro de la franja de ancho 2δ centrada en L; en esta instancia el par más cercano de P es justamente uno de esos, dibujado en trazo discontinuo, y por eso ninguna de las dos llamadas recursivas lo encuentra.

Vale la pena notar que S podría ser todo el conjunto P, en cuyo caso los dos lemas anteriores no parecen habernos servido de nada. Pero eso está lejos de ser cierto, como muestra el siguiente hecho notable.

Lema 5.4.3. Si $s, s' \in S$ cumplen $d(s,s') < \delta$, entonces s y s' están a lo sumo a 15 posiciones de distancia en la lista ordenada S_y.

Demostración. Consideremos el subconjunto Z del plano formado por todos los puntos a distancia menor que δ de L. Particionamos Z en cajas: cuadrados con lados horizontales y verticales de longitud $\delta/2$. Una fila de Z consiste en cuatro cajas cuyos lados horizontales tienen las mismas coordenadas y. Esta colección de cajas está dibujada en la Figura 5.9.

Supongamos que dos puntos de S caen en la misma caja. Como todos los puntos de una caja están del mismo lado de L, esos dos puntos pertenecen ambos a Q o ambos a R. Pero dos puntos cualesquiera de una misma caja están a distancia a lo sumo $\delta \cdot \sqrt{2}/2 < \delta$, lo que contradice la definición de δ como la mínima distancia entre dos puntos de Q o entre dos puntos de R. Cada caja contiene entonces a lo sumo un punto de S.

Supongamos ahora que $s, s' \in S$ cumplen $d(s, s') < \delta$ y están a por lo menos 16 posiciones de distancia en S_y. Supongamos sin pérdida de generalidad que s tiene la menor coordenada y. Como hay a lo sumo un punto por caja y cuatro cajas por fila, hay por lo menos tres filas completas de Z entre s y s'. Pero dos puntos de Z separados por al menos tres filas están a distancia por lo menos $3\delta/2$, lo que es una contradicción. □

El valor 15 del Lema 5.4.3 se puede reducir; lo importante para nuestros fines es que sea una constante absoluta.

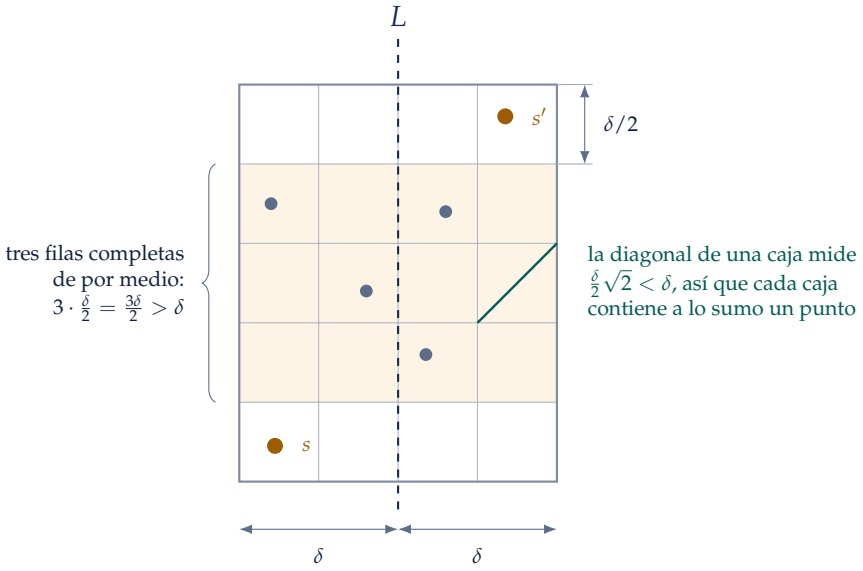


Figura 5.9. La porción del plano cercana a la recta divisoria L, tal como se la analiza en la demostración del Lema 5.4.3. La franja de ancho 2δ se parte en cajas de lado $\delta/2$, cuatro por fila; cada caja contiene a lo sumo un punto, de modo que dos puntos separados por muchas posiciones en el orden por coordenada y quedan forzosamente a distancia mayor que δ.

A la luz del Lema 5.4.3 podemos concluir el algoritmo. Hacemos una única pasada por S_y y, para cada $s \in S_y$, calculamos su distancia a cada uno de los 15 puntos siguientes de S_y. Al hacerlo habremos calculado la distancia de todo par de puntos de S que esté a distancia menor que δ, si es que hay alguno. Podemos entonces comparar la menor de esas distancias con δ y reportar una de dos cosas: el par de puntos más cercano de S, si su distancia es menor que δ; o bien la conclusión correcta de que no hay ningún par de puntos de S a distancia menor que δ. En el primer caso ese par es el par más cercano de P; en el segundo, el par más cercano hallado por las llamadas recursivas es el par más cercano de P.

Nótese el parecido entre este procedimiento y el enfoque que rechazamos al principio, que intentaba hacer una única pasada por P en orden de coordenada y. La razón por la que ahora sí funciona es el conocimiento extra que ganamos de las llamadas recursivas —el valor de δ— y la estructura especial del conjunto S.

5.4.3 Resumen del algoritmo

Una descripción de alto nivel del algoritmo, con la notación que venimos usando, es la siguiente.

PAR-CERCANO(P)

```
1  Construir P_x y P_y    (tiempo O(n log n))
2  return PAR-CERCANO-REC(P_x, P_y)
```

PAR-CERCANO-REC(P_x, P_y)

```
1  if |P| ≤ 3 then
2      return el par más cercano midiendo todas las distancias
3  end if
4  Construir Q_x, Q_y, R_x, R_y    (tiempo O(n))
5  (q0*, q1*) ← PAR-CERCANO-REC(Q_x, Q_y)
6  (r0*, r1*) ← PAR-CERCANO-REC(R_x, R_y)
7  δ ← mín (d(q0*, q1*), d(r0*, r1*))
8  x* ← máxima coordenada x de un punto de Q
9  L ← {(x, y) : x = x*}
10 S ← puntos de P a distancia menor que δ de L
11 Construir S_y    (tiempo O(n))
12 for cada punto s ∈ S_y do
13     Calcular la distancia de s a los 15 puntos siguientes de S_y
14 end for
15 Sean s, s' el par que alcanza el mínimo de esas distancias
16 if d(s, s') < δ then return (s, s')
17 else if d(q0*, q1*) < d(r0*, r1*) then return (q0*, q1*)
18 else return (r0*, r1*)
19 end if
```

> **Estructuras de datos.** Cada punto se guarda una sola vez en un arreglo de registros con sus dos coordenadas y dos campos enteros con su posición en P_x y en P_y; las listas P_x, P_y, Q_x, Q_y, R_x, R_y y S_y son arreglos de índices a ese arreglo, no copias de los puntos. El ordenamiento inicial se hace con mergesort. Gracias a los campos de posición, partir P_y en Q_y y R_y es una sola pasada que consulta, para cada punto, si su posición en P_x cae en la primera o en la segunda mitad; lo mismo vale para construir S_y a partir de P_y. Todo eso es $O(n)$ por nivel, y por eso los ordenamientos no se repiten dentro de la recursión.

5.4.4 Análisis del algoritmo

Probemos primero que el algoritmo produce una respuesta correcta, usando los hechos que fuimos estableciendo al diseñarlo.

Proposición 5.4.4. El algoritmo devuelve un par de puntos más cercano de P.

Demostración. Probamos la corrección por inducción en el tamaño de P, siendo claro el caso $|P| \leq 3$. Dado P, el par más cercano de cada llamada recursiva se calcula correctamente por hipótesis inductiva. Por los Lemas 5.4.2 y 5.4.3, el resto del algoritmo determina correctamente si algún par de puntos de S está a distancia menor que δ y, en caso afirmativo, devuelve el par más cercano de esos. Ahora bien, el par más cercano de P o bien tiene sus dos elementos en Q o en R, o bien tiene un elemento en cada uno. En el primer caso lo halla correctamente la llamada recursiva correspondiente; en el segundo caso está a distancia menor que δ, y lo halla correctamente el resto del algoritmo. □

Proposición 5.4.5. El tiempo de ejecución del algoritmo es $O(n \log n)$.

Demostración. El ordenamiento inicial de P por coordenada x y por coordenada y lleva tiempo $O(n \log n)$. El tiempo de ejecución del resto del algoritmo satisface la recurrencia (5.1), y por lo tanto es $O(n \log n)$ por la Proposición 5.1.3. □

5.5 Multiplicación de enteros

Discutimos ahora una aplicación distinta de divide y vencerás, en la que el algoritmo cuadrático "por defecto" se mejora mediante una recurrencia diferente. El análisis del algoritmo rápido explota una de las recurrencias de la Sección 5.2, en la que se generan más de dos llamadas recursivas por nivel.

El problema que consideramos es extremadamente básico: la multiplicación de dos enteros. En cierto sentido es tan básico que uno podría no pensarlo siquiera como una cuestión algorítmica. Pero de hecho a los escolares se les enseña un algoritmo concreto —y bastante eficiente— para multiplicar dos números x e y de n dígitos: primero se computa un producto parcial multiplicando cada dígito de y por separado por x, y después se suman todos los productos parciales. La Figura 5.10 ayuda a recordarlo. En la escuela siempre lo vemos hecho en base 10, pero funciona exactamente igual en base 2.

```
  12
× 13
  36
 12
 156
```

(a) en base 10

```
   1100
× 1101
   1100
  0000
 1100
1100
10011100
```

(b) en base 2

Figura 5.10. El algoritmo escolar para multiplicar dos enteros, en representación decimal y binaria. Los dos paneles calculan el mismo producto, $12 \times 13 = 156$, ya que 1100 y 1101 son las escrituras binarias de 12 y 13, y 10011100 es la de 156.

Contando una operación sobre un par de bits como un paso primitivo, se necesita tiempo $O(n)$ para calcular cada producto parcial y tiempo $O(n)$ para sumarlo a la suma acumulada de los productos parciales anteriores. Como hay n productos parciales, el tiempo total es $O(n^2)$.

Si uno no volvió a pensar en esto desde la escuela, la perspectiva de mejorar este algoritmo tiene algo de sorprendente. ¿No son todos esos productos parciales "necesarios" de alguna manera? Y sin embargo se puede mejorar el tiempo $O(n^2)$ usando una manera recursiva distinta de realizar la multiplicación.

5.5.1 Diseño del algoritmo

El algoritmo mejorado se basa en una forma más astuta de partir el producto en sumas parciales. Supongamos que trabajamos en base 2 —la base en realidad no importa— y empecemos escribiendo x como $x_1 \cdot 2^{n/2} + x_0$. En otras palabras, x_1 corresponde a los $n/2$ bits de orden alto y x_0 a los $n/2$ bits de orden bajo. Análogamente escribimos $y = y_1 \cdot 2^{n/2} + y_0$. Se obtiene entonces

$$xy \;=\; \big(x_1 \cdot 2^{n/2} + x_0\big)\big(y_1 \cdot 2^{n/2} + y_0\big) \;=\; x_1 y_1 \cdot 2^n + \big(x_1 y_0 + x_0 y_1\big) \cdot 2^{n/2} + x_0 y_0. \quad (5.4)$$

La ecuación (5.4) reduce el problema de resolver una única instancia de n bits —multiplicar los dos números x e y de n bits— al problema de resolver cuatro instancias de $n/2$ bits: calcular los productos x_1y_1, x_1y_0, x_0y_1 y x_0y_0. Tenemos así un primer candidato a solución de divide y vencerás: calcular recursivamente los resultados de esas cuatro instancias de $n/2$ bits y combinarlos con (5.4). La combinación requiere una cantidad constante de sumas de números de $O(n)$ bits, o sea tiempo $O(n)$; el tiempo $T(n)$ queda acotado por la recurrencia

$$T(n) \;\leq\; 4\,T(n/2) + cn$$

para una constante c. ¿Alcanza esto para un tiempo subcuadrático? Podemos averiguarlo observando que es exactamente el caso $q = 4$ de la clase de recurrencias (5.2). Por la Proposición 5.2.1, la solución es $T(n) \leq O(n^{\log_2 q}) = O(n^2)$.

De hecho, entonces, nuestro algoritmo de divide y vencerás con ramificación en cuatro no era más que una manera complicada de volver al tiempo cuadrático. Si queremos hacerlo mejor con una estrategia que reduzca el problema a instancias de $n/2$ bits, deberíamos intentar arreglarnos con solo tres llamadas recursivas. Eso conduce al caso $q = 3$ de (5.2), cuya solución es $T(n) \leq O(n^{\log_2 q}) = O(n^{1,59})$.

Recordemos que el objetivo es calcular la expresión $x_1y_1 \cdot 2^n + (x_1y_0 + x_0y_1) \cdot 2^{n/2} + x_0y_0$ de la ecuación (5.4). Resulta que hay un truco simple que permite determinar todos los términos de esa expresión usando apenas tres llamadas recursivas. El truco es considerar el resultado de la única multiplicación

$$(x_1 + x_0)(y_1 + y_0) \;=\; x_1y_1 + x_1y_0 + x_0y_1 + x_0y_0,$$

que tiene los cuatro productos de antes sumados entre sí, al costo de una sola multiplicación recursiva. Si además determinamos x_1y_1 y x_0y_0 por recursión, obtenemos explícitamente los términos de los extremos, y obtenemos el término del medio restando x_1y_1 y x_0y_0 de $(x_1 + x_0)(y_1 + y_0)$.

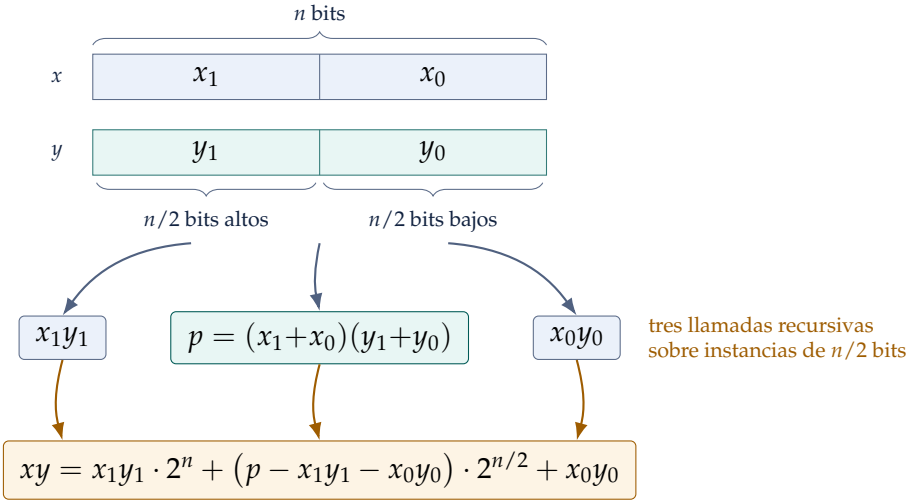


Figura 5.11. La partición de x e y en mitades de $n/2$ bits y las tres llamadas recursivas del algoritmo. El producto del medio, p, no aporta directamente un término de la fórmula: sirve para recuperar la suma $x_1y_0 + x_0y_1$ restándole los otros dos productos, y es eso lo que evita la cuarta llamada recursiva.

El algoritmo completo es entonces el siguiente.

MULTIPLICAR-REC(x, y)

```
1  if n = 1 then
2      return x · y    (producto de dos bits)
3  end if
4  Escribir x = x_1 · 2^{n/2} + x_0  y  y = y_1 · 2^{n/2} + y_0
5  Calcular x_1 + x_0  y  y_1 + y_0
6  p ← MULTIPLICAR-REC(x_1 + x_0, y_1 + y_0)
7  u ← MULTIPLICAR-REC(x_1, y_1)
8  v ← MULTIPLICAR-REC(x_0, y_0)
9  return u · 2^n + (p − u − v) · 2^{n/2} + v
```

> **Estructuras de datos.** Cada entero de n bits se guarda en un arreglo de bits (o de palabras de máquina) con el bit menos significativo en la primera posición. Partirlo en x_1 y x_0 no requiere copiar nada: alcanza con pasar un par de índices sobre el mismo arreglo. Las multiplicaciones por 2^n y $2^{n/2}$ de la línea 9 no son multiplicaciones sino desplazamientos, que se implementan escribiendo el resultado en la posición correcta del arreglo de salida. Las sumas y restas se hacen con el algoritmo usual de acarreo, en tiempo lineal en la cantidad de bits.

Ejemplo 5.5.1 (Multiplicar 12 por 13 con tres productos)**.** Sean $x = 1100$ e $y = 1101$ en binario, es decir, $x = 12$ e $y = 13$ con $n = 4$. La partición da $x_1 = 11 = 3$ y $x_0 = 00 = 0$, así como $y_1 = 11 = 3$ e $y_0 = 01 = 1$; en efecto, $x = 3 \cdot 2^2 + 0 = 12$ e $y = 3 \cdot 2^2 + 1 = 13$.

Las tres llamadas recursivas calculan $p = (3+0)(3+1) = 12$, $u = x_1 y_1 = 9$ y $v = x_0 y_0 = 0$. Notemos que el factor $y_1 + y_0 = 4 = 100$ ya tiene tres bits, uno más que las mitades: las sumas $x_1 + x_0$ e $y_1 + y_0$ pueden tener $n/2 + 1$ bits, y la llamada que calcula p trabaja con ellas tal como vienen. El término del medio se recupera restando: $p - u - v = 12 - 9 - 0 = 3$, que es efectivamente $x_1 y_0 + x_0 y_1 = 3 \cdot 1 + 0 \cdot 3 = 3$. Combinando,

$$xy = 9 \cdot 2^4 + 3 \cdot 2^2 + 0 = 144 + 12 = 156,$$

que coincide con el resultado del algoritmo escolar de la Figura 5.10.

5.5.2 Análisis del algoritmo

Podemos determinar el tiempo de ejecución del algoritmo como sigue. Dados dos números de n bits, realiza una cantidad constante de sumas sobre números de $O(n)$ bits, además de las tres llamadas recursivas. Ignorando por ahora la cuestión de que $x_1 + x_0$ e $y_1 + y_0$ pueden tener $n/2 + 1$ bits en lugar de solamente $n/2$ —lo que resulta no afectar el resultado asintótico—, cada una de esas llamadas recursivas es sobre una

instancia de tamaño $n/2$. En lugar de la recursión con ramificación en cuatro tenemos entonces una con ramificación en tres, con un tiempo de ejecución que satisface

$$T(n) \leq 3\,T(n/2) + cn$$

para una constante c. Este es el caso $q = 3$ de (5.2) que estábamos buscando, y la Proposición 5.2.1 da el resultado.

Proposición 5.5.2. El tiempo de ejecución de MULTIPLICAR-REC sobre dos factores de n bits es $O(n^{\log_2 3}) = O(n^{1,59})$.

Nota 5.5.3 (Cuándo conviene). La cantidad de bits de la entrada tiene que ser suficientemente grande para que este método, o cualquiera de los métodos subcuadráticos conocidos, mejore al algoritmo escolar: la constante escondida en la notación $O(\cdot)$ es mayor, porque cada nivel de la recursión paga varias sumas, restas y desplazamientos de números largos. Para los enteros que entran en una palabra de máquina, el algoritmo escolar —o directamente la instrucción de multiplicación del procesador— es lo que conviene.

Capítulo 6

Programación Dinámica

El estudio de las técnicas de diseño empezó por los algoritmos greedy, que en cierto sentido son el enfoque más natural. Frente a un problema nuevo no cuesta proponer varios algoritmos greedy distintos; la dificultad está en decidir si alguno de ellos resuelve el problema correctamente en todos los casos. Los problemas del Capítulo 4 tenían en común que, al final, sí existía una regla greedy que funcionaba. Eso está lejos de ser cierto en general: en la mayoría de los problemas la verdadera dificultad no es elegir entre varias estrategias greedy, sino que no hay ninguna estrategia greedy natural que funcione.

Para esos problemas hace falta tener otros enfoques a mano. Divide y vencerás sirve a veces como alternativa, pero las versiones del Capítulo 5 no suelen alcanzar para bajar una búsqueda exponencial por fuerza bruta a tiempo polinómico: sus aplicaciones típicas bajaban un tiempo que ya era polinómico a otro polinomio de menor grado.

La programación dinámica es una técnica más potente y más sutil. Su idea básica se inspira en divide y vencerás y es, en esencia, la opuesta a la estrategia greedy: se explora implícitamente el espacio de todas las soluciones posibles, descomponiendo el problema en una serie de subproblemas y construyendo soluciones correctas para subproblemas cada vez más grandes. En cierto modo, la programación dinámica trabaja peligrosamente cerca de la búsqueda por fuerza bruta: recorre sistemáticamente el conjunto exponencialmente grande de soluciones posibles, pero lo hace sin examinarlas nunca a todas de manera explícita. Por ese equilibrio delicado es una técnica a la que cuesta acostumbrarse, y hace falta una cantidad razonable de práctica para sentirse cómodo con ella.

6.1 Planificación de intervalos con pesos: un procedimiento recursivo

En la Sección 4.2 vimos que un algoritmo greedy particular produce una solución óptima para el problema de planificación de intervalos, en el que el objetivo es aceptar un conjunto de intervalos disjuntos tan grande como sea posible. El problema de planificación de intervalos con pesos es una versión estrictamente más general, en la que cada intervalo tiene un valor (o peso) y se busca aceptar un conjunto de valor máximo.

Definición 6.1.1 (Planificación de intervalos con pesos)**.** Tenemos n tareas; la tarea j empieza en s_j, termina en f_j y tiene un valor $v_j > 0$. Dos tareas son compatibles si no se superponen. El objetivo es encontrar un subconjunto $S \subseteq \{1, \ldots, n\}$ de tareas mutuamente compatibles que maximice $\sum_{j \in S} v_j$.

Las tareas son los mismos trabajos de la Sección 4.2, ahora con un valor asociado. Como el problema original es el caso particular en que todos los valores valen 1, ya sabemos que la mayoría de los algoritmos greedy no lo resuelven. Pero incluso el que funcionaba antes —aceptar repetidamente la tarea compatible que termina primero— deja de ser óptimo: la Figura 1.10 muestra una instancia en la que ese algoritmo acepta dos tareas cortas de valor 1 cada una, cuando el óptimo es la tarea larga de valor 10 que se superpone con ambas. No se conoce ningún algoritmo greedy natural para este problema, y eso es lo que motiva el cambio de técnica.

6.1.1 Diseño de un algoritmo recursivo

Supongamos que las tareas están ordenadas por tiempo de finalización no decreciente, $f_1 \leq f_2 \leq \cdots \leq f_n$, y digamos que la tarea i viene antes que la tarea j si $i < j$. Ese es el orden natural, de izquierda a derecha, en el que se recorren las tareas, y para hablar de él conviene una definición.

Definición 6.1.2 (La función p)**.** Para cada tarea j, $p(j)$ es el mayor índice $i < j$ tal que las tareas i y j son compatibles. Si ninguna tarea $i < j$ es compatible con j, se define $p(j) = 0$.

Dicho de otro modo, $p(j)$ es la tarea de índice más alto entre las que terminan antes de que j empiece. Por la forma en que están ordenadas, todas las tareas $1, 2, \ldots, p(j)$ son compatibles con j, y todas las tareas $p(j) + 1, \ldots, j - 1$ se superponen con j.

Ejemplo 6.1.3 (Una instancia con seis tareas)**.** Consideremos las seis tareas de la Figura 6.1, ya numeradas por tiempo de finalización:

j	1	2	3	4	5	6
s_j	0	1	3	2	6	6
f_j	3	4	6	7	9	10
v_j	2	4	4	7	2	1

Consideramos compatibles dos tareas que solo comparten un extremo, es decir, la tarea $i < j$ es compatible con j cuando $f_i \leq s_j$. La tarea 3 empieza en 3; entre las anteriores, la 1 termina en 3 y la 2 termina en 4, así que $p(3) = 1$. La tarea 5 empieza en 6: la 3 termina justo en 6 y la 4 termina en 7, así que $p(5) = 3$, y por el mismo motivo $p(6) = 3$. Las tareas 2 y 4 empiezan antes de que termine la 1, de modo que $p(2) = p(4) = 0$, y $p(1) = 0$ porque no hay tareas anteriores.

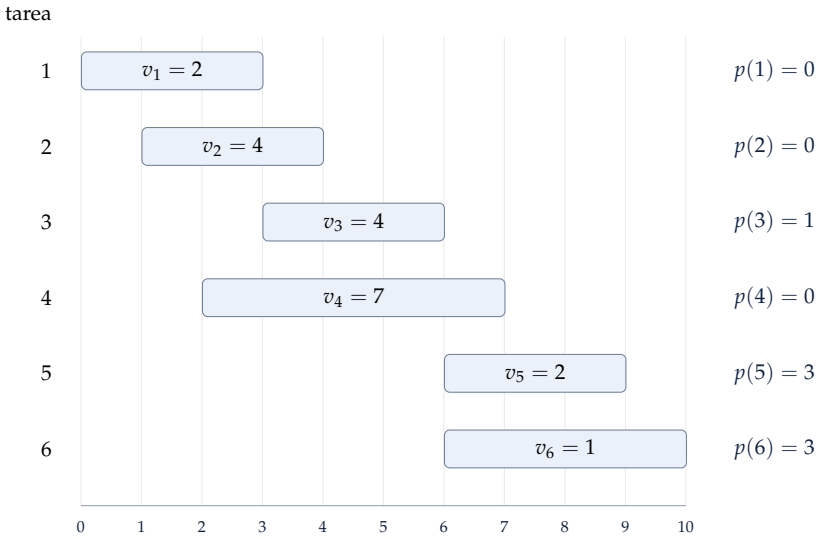


Figura 6.1. La instancia del Ejemplo 6.1.3, con las tareas ordenadas por tiempo de finalización y el valor de $p(j)$ para cada una. Por ejemplo, las tareas 4 y 5 se superponen con la 6, y la última compatible con ella es la 3; por eso $p(6) = 3$.

Consideremos ahora una solución óptima $\mathcal{O}$ de una instancia, ignorando por el momento que no tenemos idea de cuál es. Hay algo completamente obvio que se puede decir de ella: o bien la tarea n, la última, pertenece a $\mathcal{O}$, o bien no pertenece. Exploremos un poco cada lado de esta dicotomía.

- Si $n \in \mathcal{O}$, ninguna tarea con índice estrictamente entre $p(n)$ y n puede estar en $\mathcal{O}$, porque por definición de $p(n)$ las tareas $p(n) + 1, \ldots, n - 1$ se superponen con n. Además, $\mathcal{O}$ tiene que incluir una solución óptima del problema formado por las

tareas $\{1, \ldots, p(n)\}$: si no fuera así, podríamos reemplazar la elección que hace $\mathcal{O}$ entre esas tareas por una mejor, sin riesgo de superponernos con n.

- Si $n \notin \mathcal{O}$, entonces $\mathcal{O}$ es simplemente una solución óptima del problema formado por las tareas $\{1, \ldots, n-1\}$, por un razonamiento totalmente análogo: si no eligiera de manera óptima entre ellas, podríamos reemplazarla por una elección mejor.

Todo esto sugiere que encontrar la solución óptima sobre las tareas $\{1, \ldots, n\}$ pasa por mirar soluciones óptimas de problemas más chicos de la forma $\{1, \ldots, j\}$.

Definición 6.1.4 (Subproblemas de la planificación con pesos)**.** Para cada j entre 1 y n, $\mathcal{O}_j$ denota una solución óptima del problema formado por las tareas $\{1, \ldots, j\}$ y $\text{OPT}(j)$ denota su valor. Se define además $\text{OPT}(0) = 0$, que es el óptimo sobre el conjunto vacío de tareas.

La solución que buscamos es entonces $\mathcal{O}_n$, de valor $\text{OPT}(n)$. Para la solución óptima $\mathcal{O}_j$ sobre $\{1, \ldots, j\}$, el razonamiento anterior —generalizado del caso $j = n$ a cualquier j— dice que o bien $j \in \mathcal{O}_j$, y en ese caso $\text{OPT}(j) = v_j + \text{OPT}(p(j))$, o bien $j \notin \mathcal{O}_j$, y en ese caso $\text{OPT}(j) = \text{OPT}(j-1)$. Como esas son exactamente las dos opciones posibles, vale

$$\text{OPT}(j) \;=\; \begin{cases} 0 & \text{si } j = 0, \\ \max\big(v_j + \text{OPT}(p(j)),\ \text{OPT}(j-1)\big) & \text{si } j \geq 1. \end{cases} \tag{6.1}$$

¿Y cómo se decide si j pertenece a la solución óptima $\mathcal{O}_j$? También es fácil: pertenece si y solo si la primera de las dos opciones es al menos tan buena como la segunda.

Proposición 6.1.5. La tarea j pertenece a una solución óptima del problema sobre $\{1, \ldots, j\}$ si y solo si

$$v_j + \text{OPT}(p(j)) \;\geq\; \text{OPT}(j-1).$$

Estos hechos son el primer componente fundamental sobre el que se apoya una solución por programación dinámica: una ecuación de recurrencia que expresa la solución óptima (o su valor) en términos de las soluciones óptimas de subproblemas más chicos. Aun con lo simple del razonamiento que llevó hasta acá, la recurrencia (6.1) ya es un avance importante, porque da directamente un algoritmo recursivo para calcular $\text{OPT}(n)$, suponiendo que las tareas ya están ordenadas por tiempo de finalización y que ya calculamos $p(j)$ para cada j.

```
COMPUTE-OPT(j)
1  if j = 0 then
2      return 0
3  else
4      return máx(v_j + COMPUTE-OPT(p(j)), COMPUTE-OPT(j − 1))
5  end if
```

> **Estructuras de datos.** Las tareas se guardan en tres arreglos $s[1..n]$, $f[1..n]$ y $v[1..n]$, ya permutados de modo que f quede ordenado de manera no decreciente (por ejemplo con mergesort, en $O(n \log n)$). Los valores de p se guardan en un arreglo de enteros $p[1..n]$. Cada $p[j]$ se obtiene con una búsqueda binaria de $s[j]$ sobre el arreglo ordenado $f[1..j-1]$, buscando el último índice i con $f[i] \leq s[j]$; en total, $O(n \log n)$.

La corrección del algoritmo se sigue directamente por inducción en j.

Proposición 6.1.6. COMPUTE-OPT(j) calcula correctamente OPT(j) para todo $j = 0, 1, \ldots, n$.

Demostración. Por definición $\text{OPT}(0) = 0$, que es lo que devuelve el algoritmo. Tomemos ahora $j > 0$ y supongamos, como hipótesis inductiva, que COMPUTE-OPT(i) calcula correctamente OPT(i) para todo $i < j$. Como $p(j) < j$ y $j - 1 < j$, la hipótesis da COMPUTE-OPT($p(j)$) $=$ OPT($p(j)$) y COMPUTE-OPT($j-1$) $=$ OPT($j-1$), y entonces por (6.1)

$$\text{OPT}(j) \;=\; \text{máx}\big(v_j + \text{OPT}(p(j)),\, \text{OPT}(j-1)\big) \;=\; \text{COMPUTE-OPT}(j). \qquad \square$$

Lamentablemente, si implementáramos COMPUTE-OPT tal como está escrito, en el peor caso tardaría tiempo exponencial. El árbol de llamadas se ensancha muy rápido por la ramificación de la recursión. Un ejemplo extremo es una instancia escalonada como la de la Figura 6.2, en la que $p(j) = j - 2$ para cada $j \geq 2$: ahí COMPUTE-OPT(j) genera llamadas recursivas separadas sobre problemas de tamaño $j - 1$ y $j - 2$. La cantidad total de llamadas crece entonces como los números de Fibonacci, que crecen exponencialmente, y no conseguimos un algoritmo polinómico.

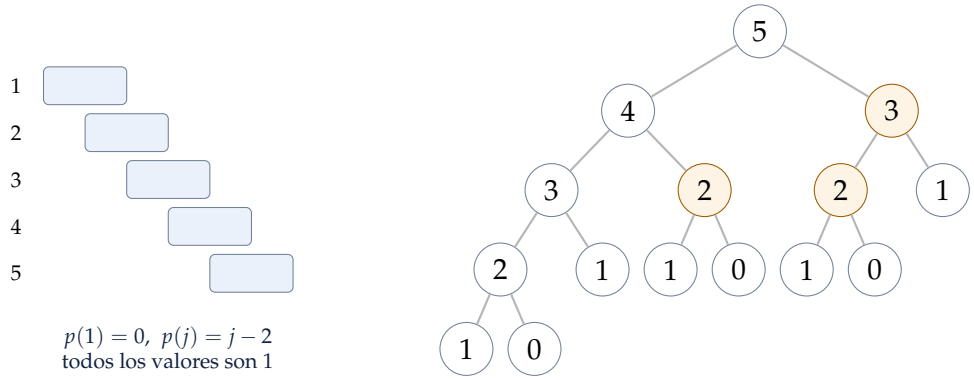


Figura 6.2. A la izquierda, una instancia escalonada en la que cada tarea se superpone solo con sus vecinas, de modo que $p(j) = j - 2$. A la derecha, el árbol de llamadas de COMPUTE-OPT(5) sobre ella; cada nodo j llama a $j - 1$ y a $j - 2$, y no se dibujan las llamadas de los nodos 1 al caso base. Los nodos sombreados repiten un subproblema de tamaño 2 o 3 que ya se había resuelto en otra rama.

6.1.2 Memorización de la recursión

En realidad no estamos tan lejos de un algoritmo polinómico. Una observación fundamental, que es el segundo componente de una solución por programación dinámica, es que el algoritmo recursivo COMPUTE-OPT en realidad solo resuelve $n + 1$ subproblemas distintos: COMPUTE-OPT(0), COMPUTE-OPT(1), ..., COMPUTE-OPT(n). Que tarde tiempo exponencial se debe únicamente a la enorme redundancia con que repite cada una de esas llamadas.

¿Cómo se elimina esa redundancia? Guardando el valor de COMPUTE-OPT(j) en un lugar accesible globalmente la primera vez que se calcula, y usando después ese valor ya calculado en lugar de repetir la llamada recursiva. Esta técnica de guardar los valores ya calculados se llama memorización (en inglés, *memoization*); la estrategia resultante, que parte del problema completo y baja hacia los subproblemas a medida que los necesita, se llama *top-down*.

El procedimiento M-COMPUTE-OPT implementa esta idea con un arreglo global $M[0..n]$. Cada entrada $M[j]$ empieza sin inicializar y pasa a guardar el valor de COMPUTE-OPT(j) apenas se determina por primera vez. Para obtener OPT(n) se ordenan las tareas, se calculan los $p(j)$, se fija $M[0] \leftarrow 0$ y se invoca M-COMPUTE-OPT(n).

M-COMPUTE-OPT(j)
1 **if** $M[j]$ no está inicializado **then**
2 $M[j] \leftarrow \max\big(v_j + \textbf{M-COMPUTE-OPT}(p(j)),\ \textbf{M-COMPUTE-OPT}(j-1)\big)$
3 **end if**
4 **return** $M[j]$

> **Estructuras de datos.** Además de los arreglos de la versión anterior, se usa un arreglo global $M[0..n]$ de enteros (o reales, si los valores lo son). Para marcar las entradas sin inicializar alcanza con llenar $M[1..n]$ al principio con un valor imposible, por ejemplo -1, ya que todos los valores son positivos; así la prueba de la línea 1 cuesta $O(1)$.

6.1.3 Análisis de la versión memorizada

El algoritmo se parece muchísimo a la versión anterior, pero la memorización baja drásticamente el tiempo de ejecución.

Proposición 6.1.7. El tiempo de ejecución de M-COMPUTE-OPT(n) es $O(n)$, suponiendo que las tareas ya están ordenadas por tiempo de finalización y que los valores $p(j)$ ya están calculados.

Demostración. El tiempo que se gasta en una llamada a M-COMPUTE-OPT, sin contar el de las llamadas recursivas que genera, es $O(1)$. Por lo tanto el tiempo total está acotado por una constante por la cantidad de llamadas. Como el código no da ninguna cota explícita para esa cantidad, buscamos una buena medida de progreso.

La medida útil es la cantidad de entradas de M que ya están inicializadas. Al principio vale 0; y cada vez que el procedimiento llega a la línea 2, emitiendo dos llamadas recursivas, llena una entrada nueva y aumenta esa cantidad en 1. Como M tiene solo $n + 1$ entradas, la línea 2 se ejecuta a lo sumo n veces y por lo tanto hay a lo sumo $2n + 1$ llamadas en total. El tiempo de M-COMPUTE-OPT(n) es entonces $O(n)$. □

6.1.4 Calcular la solución además de su valor

Hasta ahora solo calculamos el valor de una solución óptima, pero presumiblemente queremos también el conjunto de tareas. Sería fácil extender M-COMPUTE-OPT para que guarde una solución óptima además de su valor: podríamos mantener otro arreglo S tal que $S[i]$ contenga un conjunto óptimo de tareas entre $\{1, \ldots, i\}$. Mejorar el código de esa forma ingenua multiplicaría el tiempo por un factor $O(n)$ adicional, porque actualizar una posición de M cuesta $O(1)$ pero escribir un conjunto en S cuesta $O(n)$. Ese sobrecosto se evita no guardando S explícitamente y recuperando la solución óptima a partir de los valores guardados en M, una vez calculado el valor óptimo.

La Proposición 6.1.5 dice que j pertenece a una solución óptima sobre $\{1, \ldots, j\}$ si y solo si $v_j + \text{OPT}(p(j)) \geq \text{OPT}(j-1)$. Con esa observación se obtiene un procedimiento simple que recorre el arreglo M hacia atrás para encontrar el conjunto de tareas de una solución óptima.

FIND-SOLUTION(j)
1 **if** $j = 0$ **then**
2 **return** $\emptyset$
3 **else if** $v_j + M[p(j)] \geq M[j-1]$ **then**
4 **return** $\{j\} \cup$ **FIND-SOLUTION**$(p(j))$
5 **else**
6 **return FIND-SOLUTION**$(j-1)$
7 **end if**

> **Estructuras de datos.** Recibe el arreglo $M[0..n]$ ya lleno, junto con $v[1..n]$ y $p[1..n]$. El conjunto devuelto se representa con una lista enlazada de índices: la unión de la línea 4 es insertar j al frente de la lista, en $O(1)$.

Como FIND-SOLUTION se llama recursivamente solo sobre valores estrictamente menores, hace en total $O(n)$ llamadas; y como gasta tiempo constante en cada una, vale lo siguiente.

Proposición 6.1.8. Dado el arreglo M con los valores óptimos de los subproblemas, FIND-SOLUTION(n) devuelve una solución óptima en tiempo $O(n)$.

Nota 6.1.9 (Desempates). En las diapositivas del curso la comparación de la línea 3 aparece con desigualdad estricta, $v_j + M[p(j)] > M[j-1]$. Las dos versiones son correctas: cuando hay empate, las dos opciones llevan a soluciones óptimas del mismo valor, y la desigualdad solo decide cuál de ellas se devuelve. Con $\geq$ se prefiere incluir la tarea j; con $>$ se prefiere dejarla afuera.

6.2 Principios de la programación dinámica: memorización o iteración sobre subproblemas

El algoritmo de la sección anterior sirve para resumir los principios básicos de la programación dinámica y para ofrecer otra perspectiva: iterar sobre los subproblemas en lugar de calcular las soluciones recursivamente.

Obtuvimos una solución polinómica para la planificación de intervalos con pesos diseñando primero un algoritmo recursivo exponencial y convirtiéndolo después, mediante memorización, en un algoritmo recursivo eficiente que consulta un arreglo global M de soluciones óptimas de subproblemas. Para entender de verdad qué está pasando conviene formular una versión esencialmente equivalente del algoritmo. Esta nueva formulación es la que captura de manera más explícita la esencia de la técnica.

6.2.1 Diseño del algoritmo iterativo

La clave del algoritmo eficiente es en realidad el arreglo M. Codifica la idea de que usamos el valor de las soluciones óptimas de los subproblemas sobre las tareas $\{1, \ldots, j\}$ para cada j, y usa (6.1) para definir $M[j]$ a partir de valores que aparecen antes en el arreglo. Una vez que se tiene el arreglo M el problema está resuelto: $M[n]$ contiene el valor óptimo de la instancia completa, y FIND-SOLUTION permite recorrer M hacia atrás para devolver una solución óptima.

Lo que hay que notar, entonces, es que las entradas de M se pueden calcular directamente con un algoritmo iterativo, en lugar de usar recursión memorizada. Alcanza con empezar por $M[0] = 0$ e ir incrementando j: cada vez que hace falta determinar un valor $M[j]$, la respuesta la da (6.1), y los dos valores que usa ya están en el arreglo. Esta estrategia, que desenrolla la recursión a partir de los casos base, se llama *bottom-up*.

BOTTOM-UP$(n, s_1, \ldots, s_n, f_1, \ldots, f_n, v_1, \ldots, v_n)$

```
1  Ordenar las tareas de forma que f_1 ≤ f_2 ≤ ··· ≤ f_n
2  Calcular p(1), p(2), ..., p(n)
3  M[0] ← 0
4  for j = 1 to n do
5      M[j] ← máx(v_j + M[p(j)], M[j − 1])
6  end for
7  return M[n]
```

> **Estructuras de datos.** Los mismos arreglos s, f, v y p de las versiones recursivas, y un arreglo $M[0..n]$ que ahora se llena en orden de índice creciente. No hace falta marcar entradas sin inicializar ni pila de recursión: en la línea 5 las posiciones $p(j)$ y $j - 1$ son menores que j y por lo tanto ya fueron escritas.

6.2.2 Análisis del algoritmo iterativo

Por analogía exacta con la demostración de la Proposición 6.1.6, se prueba por inducción en j que este algoritmo escribe $\text{OPT}(j)$ en la entrada $M[j]$; la recurrencia (6.1) da el paso inductivo. Igual que antes, se puede pasar el arreglo M ya lleno a FIND-SOLUTION para obtener una solución óptima además de su valor. Por último, sin contar el ordenamiento y el cálculo de los $p(j)$, el tiempo de las líneas 3 a 7 es claramente $O(n)$, porque hay n iteraciones y cada una gasta tiempo constante.

Ejemplo 6.2.1 (Corrida sobre la instancia de seis tareas)**.** Volvamos a la instancia del Ejemplo 6.1.3. La Figura 6.3 muestra cómo BOTTOM-UP llena el arreglo M: en cada iteración compara $v_j + M[p(j)]$ con $M[j-1]$ y se queda con el mayor. El resultado es

$$M = [\,0,\ 2,\ 4,\ 6,\ 7,\ 8,\ 8\,],$$

así que el valor óptimo es $\mathrm{OPT}(6) = 8$.

Para recuperar la solución, FIND-SOLUTION(6) compara $v_6 + M[p(6)] = 1 + M[3] = 7$ con $M[5] = 8$; como $7 < 8$, la tarea 6 queda afuera y se sigue por $j = 5$. Ahí $v_5 + M[3] = 2 + 6 = 8 \geq M[4] = 7$, así que se incluye la tarea 5 y se salta a $p(5) = 3$. En $j = 3$, $v_3 + M[1] = 4 + 2 = 6 \geq M[2] = 4$: se incluye la tarea 3 y se salta a $p(3) = 1$. En $j = 1$, $v_1 + M[0] = 2 \geq M[0] = 0$: se incluye la tarea 1 y se salta a $p(1) = 0$, donde el procedimiento termina. La solución es $\{1, 3, 5\}$, de valor $2 + 4 + 2 = 8$, y en la Figura 6.1 se ve que las tres tareas son efectivamente compatibles.

Notemos que la tarea más valiosa, la 4 con $v_4 = 7$, no está en la solución. Un greedy que eligiera primero la tarea de mayor valor la tomaría, y como la tarea 4 se superpone con todas las demás, terminaría con valor $7 < 8$.

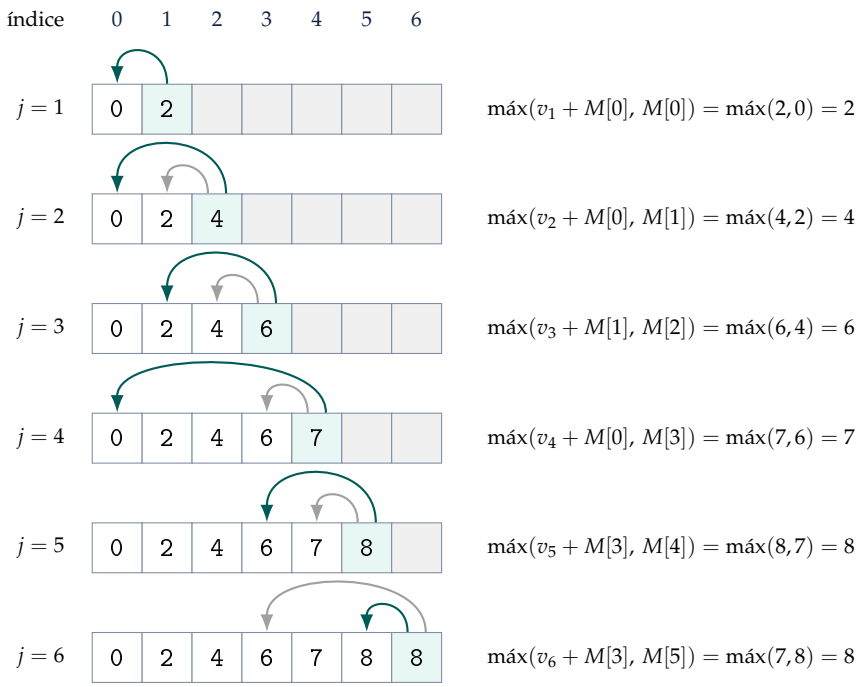


Figura 6.3. Las seis iteraciones de BOTTOM-UP sobre la instancia del Ejemplo 6.1.3. En la fila j la celda nueva es $M[j]$; el arco alto apunta a $M[p(j)]$ y el bajo a $M[j-1]$, y el que se dibuja en color es la opción que ganó el máximo. Solo en la última iteración conviene más dejar la tarea afuera.

6.2.3 Un esquema básico de la programación dinámica

Tenemos entonces un segundo algoritmo eficiente para la planificación de intervalos con pesos. Los dos enfoques —memorización *top-down* e iteración *bottom-up*— tienen claramente mucho en común, porque ambos surgen de la misma idea, contenida en la recurrencia (6.1). En general los algoritmos de programación dinámica son más simples de expresar con el segundo enfoque, la construcción iterativa de los subproblemas; pero siempre hay una manera equivalente de formularlos como recursión memorizada.

Lo más importante es que casi toda la discusión sobre el problema particular de elegir intervalos se puede reformular como una plantilla general, aunque aproximada, para diseñar algoritmos de programación dinámica. Para desarrollar un algoritmo basado en programación dinámica hace falta una colección de subproblemas derivados del problema original que cumpla unas pocas propiedades básicas.

Definición 6.2.2 (Propiedades de los subproblemas)**.** Una colección de subproblemas sirve para programación dinámica si:

(I) hay solamente una cantidad polinómica de subproblemas;

(II) la solución del problema original se calcula fácilmente a partir de las soluciones de los subproblemas (por ejemplo, porque el problema original es uno de ellos);

(III) hay un orden natural de los subproblemas, del "más chico" al "más grande", junto con una recurrencia fácil de calcular —como (6.1) y la Proposición 6.1.5— que permite determinar la solución de un subproblema a partir de las soluciones de algunos subproblemas más chicos.

Por supuesto, son pautas informales. En particular, la noción de "más chico" de la propiedad (iii) depende del tipo de recurrencia que se tenga.

A veces resulta más fácil empezar el diseño formulando un conjunto de subproblemas que parece natural y buscando después una recurrencia que los relacione; pero muchas veces, como pasó con la planificación de intervalos con pesos, conviene definir primero una recurrencia razonando sobre la estructura de una solución óptima, y determinar después qué subproblemas hacen falta para desenrollarla. Esta relación del tipo huevo o gallina entre subproblemas y recurrencias es una cuestión sutil de fondo en la programación dinámica: nunca está claro que una colección de subproblemas vaya a servir hasta encontrar una recurrencia que los vincule, pero puede ser difícil pensar en recurrencias sin tener a mano los subproblemas "más chicos" sobre los que se apoyan.

Nota 6.2.3 (Puntos importantes a recordar). Las diapositivas del curso resumen el esquema en tres etapas, que conviene seguir en ese orden al resolver cualquier problema.

(1) Descomposición del problema. Definir semánticamente una función para descomponer el problema, por ejemplo "OPT(j) = máximo valor de..."; decir cómo se obtiene el valor buscado para el problema completo, por ejemplo "la solución que buscamos es OPT(n)"; y dar una relación de recurrencia para OPT, especificando claramente en qué casos aplica cada rama.

(2) Algoritmo para calcular el valor óptimo. Llenar las entradas de la tabla que corresponden a los casos base; establecer un orden de llenado tal que, al calcular una entrada, todos los términos de la recurrencia que necesita ya estén calculados; y leer de la tabla el valor del problema completo, por ejemplo $M[n]$.

(3) Algoritmo para obtener una solución óptima. Reconstruir las decisiones que llevan al valor óptimo, partiendo del valor del problema completo y avanzando en el orden inverso al que se llenó la tabla. Esta reconstrucción es siempre *top-down*.

Nota 6.2.4 (Memorización o iteración). Ambas versiones calculan exactamente los mismos valores, pero tienen diferencias prácticas. La versión *top-down* solo resuelve los subproblemas que efectivamente se necesitan desde el problema completo, lo que puede ahorrar trabajo cuando la tabla es grande y se usa poco; a cambio paga el costo de las llamadas recursivas y necesita una pila de profundidad hasta n. La versión *bottom-up* obliga a pensar de antemano un orden de llenado válido, pero no tiene recursión y suele permitir ahorrar memoria descartando partes de la tabla que ya no se van a consultar.

6.3 Mínimos cuadrados segmentado: elección de múltiples vías

La planificación de intervalos con pesos tiene una recurrencia basada en una elección fundamentalmente binaria: la tarea n pertenece a una solución óptima o no pertenece. En el problema de esta sección la recurrencia involucra lo que se puede llamar una elección de múltiples vías: en cada paso hay una cantidad polinómica de posibilidades para la estructura de la solución óptima. La programación dinámica se adapta con toda naturalidad a esta situación más general.

Además, el problema es un buen ejemplo de cómo una definición algorítmica limpia puede formalizar una noción que al principio parece demasiado difusa y poco intuitiva

para trabajar con ella matemáticamente.

6.3.1 El problema

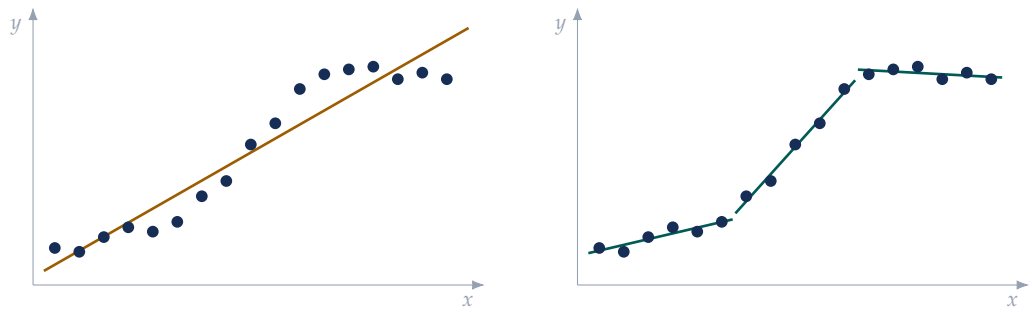
Cuando se miran datos científicos o estadísticos graficados en dos ejes, es habitual trazar una "recta de mejor ajuste" a través de ellos. Es un problema fundacional de la estadística y el análisis numérico, que se formula así. Los datos son un conjunto P de n puntos del plano, $(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)$, con $x_1 < x_2 < \ldots < x_n$. Dada una recta L de ecuación $y = ax + b$, el error de L respecto de P es la suma de sus "distancias" al cuadrado a los puntos de P:

$$\text{Error}(L, P) \;=\; \sum_{i=1}^{n} (y_i - ax_i - b)^2.$$

El objetivo natural es encontrar la recta de error mínimo, y resulta que tiene una solución cerrada que se deduce fácilmente con cálculo diferencial. Sin hacer la deducción, el resultado es que la recta de error mínimo es $y = ax + b$ con

$$a \;=\; \frac{n \sum_i x_i y_i - \left(\sum_i x_i\right)\left(\sum_i y_i\right)}{n \sum_i x_i^2 - \left(\sum_i x_i\right)^2}, \qquad b \;=\; \frac{\sum_i y_i - a \sum_i x_i}{n}. \tag{6.2}$$

Ahora bien, hay un tipo de situación para la que estas fórmulas no fueron pensadas. Muchas veces los datos se parecen a los de la Figura 6.4. En ese caso uno quisiera poder decir algo como "los puntos están aproximadamente sobre una sucesión de tres rectas". ¿Cómo se formaliza esa idea?



(a) una sola recta (b) tres segmentos

Figura 6.4. Los mismos datos ajustados de dos maneras. La recta de mínimos cuadrados de (a) tiene un error grande, mientras que los tres segmentos de (b) siguen los puntos muy de cerca: a simple vista, los datos están sobre tres rectas.

Cualquier recta única a través de esos puntos tiene un error enorme, pero con tres rectas se logra un error muy chico. Podríamos entonces intentar una nueva formulación: en lugar de buscar una sola recta de mejor ajuste, se permite trazar un conjunto arbitrario de rectas y se busca el que minimiza el error. Pero así no queda un buen problema,

porque tiene una solución trivial: con una cantidad arbitraria de rectas, se ajustan los puntos perfectamente trazando una recta distinta entre cada par de puntos consecutivos.

En el otro extremo, podríamos fijar el número tres en el enunciado y buscar el mejor ajuste con a lo sumo tres rectas. Pero eso pierde un aspecto clave de la intuición: no partimos de la idea preconcebida de que los puntos están sobre tres rectas, sino que lo concluimos mirando el dibujo. Intuitivamente hace falta una formulación que obligue a ajustar bien los puntos usando la menor cantidad de rectas posible, es decir, que balancee precisión y parsimonia. El problema de mínimos cuadrados segmentado captura esto de manera muy limpia. Es un ejemplo fundamental de lo que en minería de datos y estadística se llama detección de cambios (*change detection*): dada una sucesión de datos, identificar unos pocos puntos de la sucesión en los que ocurre un cambio discreto, en este caso el paso de una aproximación lineal a otra.

Definición 6.3.1 (Mínimos cuadrados segmentado)**.** Se dan n puntos $p_i = (x_i, y_i)$ con $x_1 < x_2 < \ldots < x_n$ y una constante $C > 0$. Un segmento es un subconjunto de puntos consecutivos $\{p_i, p_{i+1}, \ldots, p_j\}$ para algunos índices $i \leq j$. Hay que particionar los puntos en segmentos y, para cada segmento, ajustar la recta de error mínimo según (6.2). El costo (o penalización) de una partición es

$$\text{Costo} \;=\; E + C\,L,$$

donde E es la suma, sobre todos los segmentos, del error de la recta óptima de cada uno, y L es la cantidad de segmentos. El objetivo es encontrar una partición de costo mínimo.

Esta minimización captura el equilibrio discutido antes. Se permiten particiones con cualquier cantidad de segmentos; al aumentar la cantidad se reduce el término E, pero crece el término CL. La constante C es parte de la entrada, y ajustándola se penaliza más o menos el uso de rectas adicionales.

Hay exponencialmente muchas particiones posibles de P —tantas como subconjuntos de los $n-1$ lugares entre puntos consecutivos donde se puede cortar, es decir 2^{n-1}—, y al principio no está claro que se pueda encontrar la óptima de manera eficiente.

6.3.2 Diseño del algoritmo

Recordemos los ingredientes que necesita un algoritmo de programación dinámica según la Definición 6.2.2: una cantidad polinómica de subproblemas cuyas soluciones den la solución del problema original, y una recurrencia para construir esas soluciones. Igual que en la planificación de intervalos con pesos, conviene pensar en propiedades simples de la solución óptima. Pero no hay una analogía directa: allá se buscaba un subconjunto de n objetos, y acá se busca una partición de n objetos.

Para mínimos cuadrados segmentado la siguiente observación es muy útil: el último punto p_n pertenece a un único segmento de la partición óptima, y ese segmento em-

pieza en algún punto anterior p_i. Este tipo de observación sugiere el conjunto correcto de subproblemas: si conociéramos cuál es el último segmento $p_i, \ldots, p_n$, como en la Figura 6.5, podríamos sacar esos puntos de consideración y resolver recursivamente el problema sobre los puntos restantes $p_1, \ldots, p_{i-1}$.

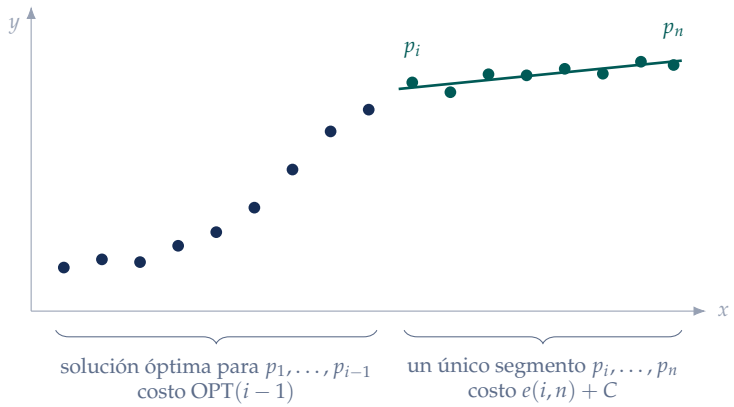


Figura 6.5. Una solución posible: un único segmento ajusta los puntos $p_i, p_{i+1}, \ldots, p_n$, y los puntos restantes $p_1, \ldots, p_{i-1}$ se resuelven de manera óptima por separado.

Definición 6.3.2 (Subproblemas de mínimos cuadrados segmentado)**.** Para $0 \leq j \leq n$, OPT(j) es el costo mínimo de una partición de los puntos $p_1, \ldots, p_j$, con OPT$(0) = 0$ como caso borde. Para $i \leq j$, $e(i,j)$ es el error mínimo de una recta respecto de los puntos $p_i, p_{i+1}, \ldots, p_j$.

La observación anterior dice lo siguiente.

Proposición 6.3.3. Si el último segmento de la partición óptima es $p_i, \ldots, p_n$, entonces el valor de la solución óptima es OPT$(n) = e(i,n) + C + \text{OPT}(i-1)$.

Aplicando la misma observación al subproblema formado por los puntos $p_1, \ldots, p_j$, para obtener OPT(j) hay que encontrar la mejor manera de armar un último segmento $p_i, \ldots, p_j$ —pagando su error más un C adicional por usarlo— junto con una solución óptima OPT$(i-1)$ para los puntos restantes. Queda justificada así la siguiente recurrencia.

Proposición 6.3.4. Para el subproblema sobre los puntos $p_1, \ldots, p_j$,

$$\mathrm{OPT}(j) \;=\; \begin{cases} 0 & \text{si } j = 0, \\ \min\limits_{1 \le i \le j} \big(e(i,j) + C + \mathrm{OPT}(i-1)\big) & \text{si } j \ge 1, \end{cases} \tag{6.3}$$

y el segmento $p_i, \ldots, p_j$ se usa en una solución óptima del subproblema si y solo si el mínimo se alcanza con el índice i.

La parte difícil del diseño ya quedó atrás. A partir de acá basta con construir las soluciones $\mathrm{OPT}(j)$ en orden creciente de j.

SEGMENTED-LEAST-SQUARES$(n, p_1, \ldots, p_n, C)$

```
1  for j = 1 to n do
2      for i = 1 to j do
3          Calcular e(i, j) para el segmento p_i, p_{i+1}, ..., p_j
4      end for
5  end for
6  M[0] ← 0
7  for j = 1 to n do
8      M[j] ← mín_{1≤i≤j} (e(i, j) + C + M[i − 1])
9  end for
10 return M[n]
```

> **Estructuras de datos.** Los puntos llegan en dos arreglos $x[1..n]$ e $y[1..n]$, ya ordenados por x. Los errores se guardan en una matriz $e[1..n][1..n]$ de reales, de la que solo se usa el triángulo con $i \le j$, y los valores óptimos en un arreglo $M[0..n]$. El mínimo de la línea 8 se calcula con un bucle sobre i; si además se guarda en un arreglo de enteros $ini[1..n]$ el índice i que lo alcanzó, la reconstrucción no necesita volver a calcular mínimos.

Por analogía con los argumentos de la planificación de intervalos con pesos, la corrección de este algoritmo se prueba directamente por inducción en j, con (6.3) como paso inductivo. Y, también igual que antes, se puede recorrer hacia atrás el arreglo M para calcular una partición óptima.

FIND-SEGMENTS(j)

```
1 if j = 0 then
2     return ∅
3 else
4     Encontrar un i que minimice e(i, j) + C + M[i − 1]
5     return {{p_i, ..., p_j}} ∪ FIND-SEGMENTS(i − 1)
6 end if
```

> **Estructuras de datos.** Usa la matriz e y el arreglo M ya calculados. Cada segmento se representa por su par de índices (i, j), y la partición es una lista enlazada de esos pares; como los segmentos se descubren de derecha a izquierda, insertando cada uno al frente la lista queda ordenada de izquierda a derecha. Con el arreglo *ini* la línea 4 se reduce a leer $ini[j]$.

6.3.3 Análisis del algoritmo

Primero hay que calcular todos los errores $e(i, j)$. Hay $O(n^2)$ pares (i, j) para los que hace falta, y para cada par las fórmulas (6.2) permiten calcular $e(i, j)$ en tiempo $O(n)$, recorriendo los puntos del segmento para acumular las sumas y después sumando los errores al cuadrado. El tiempo total para calcular todos los $e(i, j)$ es entonces $O(n^3)$.

Después vienen n iteraciones, para $j = 1, \ldots, n$. Para cada j hay que determinar el mínimo de (6.3) para llenar la entrada $M[j]$, lo que cuesta $O(n)$, con un total de $O(n^2)$. El tiempo de ejecución es entonces $O(n^2)$ una vez que se conocen todos los $e(i, j)$, y $O(n^3)$ en total.

Nota 6.3.5 (Bajar el cálculo de los errores a $O(n^2)$). El tiempo total está dominado por el cálculo de los $e(i, j)$, pero en realidad todos esos valores se pueden obtener en tiempo $O(n^2)$, con lo que el algoritmo completo pasa a $O(n^2)$. La idea es no recalcular las sumas desde cero para cada segmento. Una forma de hacerlo es calcular primero, en tiempo $O(n)$, las sumas prefijo

$$X_k = \sum_{t \le k} x_t, \quad Y_k = \sum_{t \le k} y_t, \quad S_k^{xx} = \sum_{t \le k} x_t^2, \quad S_k^{xy} = \sum_{t \le k} x_t y_t, \quad S_k^{yy} = \sum_{t \le k} y_t^2,$$

con todas ellas en 0 para $k = 0$. Las sumas sobre el segmento $p_i, \ldots, p_j$ son entonces diferencias, como $\sum_{t=i}^{j} x_t = X_j - X_{i-1}$, y con ellas se obtienen a y b por (6.2) en tiempo constante. Falta el error, que parece exigir recorrer los puntos; pero en la recta óptima vale la identidad

$$e(i, j) \;=\; \sum_{t=i}^{j} y_t^2 \;-\; a \sum_{t=i}^{j} x_t y_t \;-\; b \sum_{t=i}^{j} y_t,$$

que también se evalúa en tiempo constante. Para un segmento de un solo punto el denominador de a se anula, y se define directamente $e(i, i) = 0$.

Ejemplo 6.3.6 (Seis puntos)**.** Tomemos los seis puntos

$$p_1 = (1,1), \quad p_2 = (2,2), \quad p_3 = (3,3), \quad p_4 = (4,1), \quad p_5 = (5,1), \quad p_6 = (6,1),$$

que se ven en la Figura 6.6. Los primeros tres están sobre la recta $y = x$ y los últimos tres sobre la recta $y = 1$. Los errores $e(i,j)$, redondeados a dos decimales, son los de la tabla siguiente; todo segmento de uno o dos puntos tiene error 0, porque una recta pasa exactamente por ellos.

$e(i,j)$	$j=1$	$j=2$	$j=3$	$j=4$	$j=5$	$j=6$
$i=1$	0	0	0	2,70	3,10	3,14
$i=2$		0	0	1,50	1,50	1,60
$i=3$			0	0	0,67	1,20
$i=4$				0	0	0
$i=5$					0	0
$i=6$						0

Por ejemplo, para el segmento completo, las sumas son $\sum x_i = 21$, $\sum y_i = 9$, $\sum x_i y_i = 29$ y $\sum x_i^2 = 91$, de donde $a = (6 \cdot 29 - 21 \cdot 9)/(6 \cdot 91 - 21^2) = -15/105 = -1/7$ y $b = (9 + 21/7)/6 = 2$. La recta es $y = 2 - x/7$ y su error es $e(1,6) = 22/7 \approx 3{,}14$.

Con $C = 1$ la recurrencia (6.3) da $M[0] = 0$ y luego:

- $M[1] = e(1,1) + 1 + M[0] = 1$; $M[2] = \text{mín}(0+1+0,\ 0+1+1) = 1$; $M[3] = \text{mín}(0+1+0,\ 0+1+1,\ 0+1+1) = 1$. Hasta acá un solo segmento alcanza, porque los tres primeros puntos están alineados.
- $M[4] = \text{mín}(2{,}7+1+0,\ 1{,}5+1+1,\ 0+1+1,\ 0+1+1) = 2$, con empate entre $i = 3$ e $i = 4$.
- $M[5] = \text{mín}(3{,}1+1,\ 1{,}5+1+1,\ 0{,}67+1+1,\ 0+1+1,\ 0+1+2) = 2$, alcanzado por $i = 4$.
- $M[6] = \text{mín}(3{,}14+1,\ 1{,}6+1+1,\ 1{,}2+1+1,\ 0+1+1,\ 0+1+2,\ 0+1+2) = 2$, alcanzado por $i = 4$.

El costo óptimo es 2. FIND-SEGMENTS(6) encuentra el mínimo en $i = 4$, devuelve el segmento $\{p_4, p_5, p_6\}$ y sigue con FIND-SEGMENTS(3), cuyo mínimo está en $i = 1$: el otro segmento es $\{p_1, p_2, p_3\}$. La partición óptima tiene dos segmentos, error total 0 y costo $0 + 2C = 2$.

El papel de C se ve cambiándolo. La partición en dos segmentos cuesta siempre $2C$, mientras que usar un único segmento cuesta $e(1,6) + C = 22/7 + C$; el único segmento pasa a convenir cuando $C > 22/7$. Con $C = 4$, por ejemplo, el óptimo es la recta $y = 2 - x/7$, de costo $22/7 + 4 \approx 7{,}14$, contra 8 de la partición en dos.

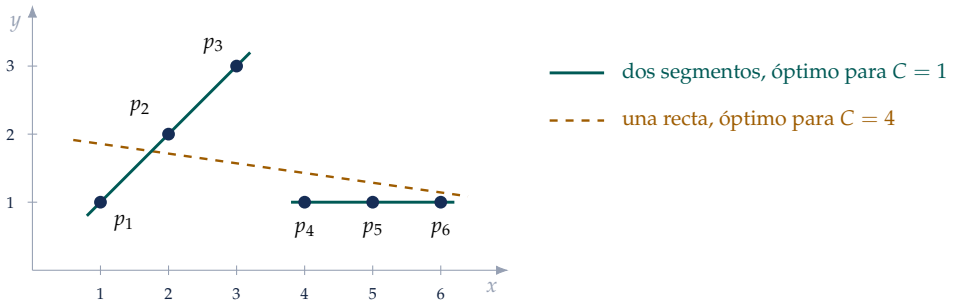


Figura 6.6. Los seis puntos del Ejemplo 6.3.6 y las dos soluciones óptimas según el valor de C. Con C chico conviene pagar un segmento más y tener error nulo; con C grande conviene una sola recta aunque su error sea 22/7.

6.4 Suma de subconjuntos y mochila: agregar una variable

Los problemas de planificación son una fuente muy rica de problemas algorítmicos con motivación práctica. Hasta acá aparecieron problemas en los que cada pedido ocupa un intervalo de tiempo fijo sobre un recurso, y otros en los que cada pedido tiene una duración y un plazo pero no exige un intervalo particular, como el de minimizar la tardanza máxima de la Sección 4.3. El problema de esta sección es una versión del segundo tipo que es difícil de resolver directamente con las técnicas vistas. Se resuelve con programación dinámica, pero con una vuelta de tuerca: el conjunto "obvio" de subproblemas no alcanza, y hay que armar una colección más rica agregando una nueva variable a la recurrencia.

6.4.1 El problema

Tenemos una única máquina que procesa trabajos y un conjunto de pedidos $\{1, 2, \ldots, n\}$. La máquina solo se puede usar durante el período que va del tiempo 0 al tiempo W, para algún número W. Cada pedido corresponde a un trabajo que requiere un tiempo w_i de procesamiento. Si el objetivo es mantener la máquina tan ocupada como sea posible hasta el "corte" W, ¿qué trabajos hay que elegir?

Definición 6.4.1 (Suma de subconjuntos)**.** Se dan n objetos $\{1, \ldots, n\}$, cada uno con un peso no negativo w_i, y una cota W. Se busca un subconjunto S de los objetos tal que $\sum_{i \in S} w_i \leq W$ y, sujeto a esa restricción, $\sum_{i \in S} w_i$ sea lo más grande posible.

Este problema es un caso particular natural de un problema más general, llamado problema de la mochila, en el que cada objeto tiene además un valor.

Definición 6.4.2 (Problema de la mochila)**.** Tenemos n objetos y una mochila. Cada objeto i tiene un peso $w_i > 0$ y un valor $v_i > 0$, y la mochila tiene una capacidad de peso W. El objetivo es cargar la mochila maximizando el valor cargado, es decir, elegir un subconjunto S que maximice $\sum_{i \in S} v_i$ sujeto a $\sum_{i \in S} w_i \leq W$.

La suma de subconjuntos es el caso $v_i = w_i$. El nombre de mochila alude al problema de llenar una mochila de capacidad W tan completa como sea posible, o con tanto valor como sea posible, usando un subconjunto de los objetos. Los problemas de mochila aparecen muchas veces como subproblemas dentro de otros problemas más complejos. Para las cantidades w_i y W se habla indistintamente de peso o de tiempo.

Como el problema se parece a otros problemas de planificación ya estudiados, es natural preguntarse si un algoritmo greedy encuentra la solución óptima. Parece que la respuesta es negativa: al menos, no se conoce ninguna regla greedy eficiente que construya siempre una solución óptima.

- Para la suma de subconjuntos, una idea es ordenar los objetos por peso decreciente —o al menos hacerlo con los de peso a lo sumo W— y elegirlos en ese orden mientras el peso total no pase de W. Pero si W es par y hay tres objetos de pesos $\{W/2 + 1, W/2, W/2\}$, el algoritmo toma el primero y ya no entra nada más, mientras que los otros dos llenan la máquina exactamente.

- Ordenar por peso creciente falla con entradas como $\{1, W/2, W/2\}$: se toman 1 y $W/2$, y el segundo $W/2$ ya no entra, cuando los dos objetos de peso $W/2$ suman W.

Ejemplo 6.4.3 (Ninguna estrategia greedy es óptima)**.** Consideremos la siguiente instancia de la mochila, con capacidad $W = 11$:

i	1	2	3	4	5
v_i	1	6	18	22	28
w_i	1	2	5	6	7

El conjunto $\{1, 2, 5\}$ tiene valor 35 y peso 10; el conjunto $\{3, 4\}$ tiene valor 40 y peso 11; el conjunto $\{3, 5\}$ tendría valor 46, pero pesa 12 y excede la capacidad.

Un greedy por valor agrega sucesivamente el objeto de mayor valor que todavía entre: toma el 5 (peso 7), no puede tomar el 4 ni el 3, y completa con el 2 y el 1, para un total de 35. Un greedy por peso agrega sucesivamente el objeto más liviano: toma el 1, el 2 y el 3 (peso 8), y ya no entra nada más, para un total de 25. Un greedy por razón valor/peso agrega el objeto de mayor v_i/w_i; las razones son 1, 3, 3,6, 3,67 y 4, así que en esta instancia coincide con el greedy por valor y también termina en 35. El óptimo es $\{3, 4\}$, con valor 40.

Recordemos los principios de la programación dinámica: hay que encontrar una cantidad chica de subproblemas de modo que cada uno se resuelva fácilmente a partir de subproblemas "más chicos", y que la solución del problema original se obtenga fácilmente una vez que se conocen las soluciones de todos ellos. La dificultad acá está en dar con un buen conjunto de subproblemas.

6.4.2 Diseño del algoritmo

Un primer intento fallido

Una estrategia general, que funcionó para la planificación de intervalos con pesos, es considerar subproblemas que involucran solo a los primeros i pedidos. Probemos con ella acá. Usamos la notación $\text{OPT}(i)$, igual que antes, para la mejor solución posible usando un subconjunto de los pedidos $\{1, \ldots, i\}$. La clave del método para la planificación con pesos fue concentrarse en una solución óptima $\mathcal{O}$ y considerar dos casos, según que el último pedido n fuera aceptado o rechazado. Igual que allá, la primera parte sale inmediatamente de la definición de $\text{OPT}(i)$.

- Si $n \notin \mathcal{O}$, entonces $\text{OPT}(n) = \text{OPT}(n-1)$.

Falta el caso $n \in \mathcal{O}$. Lo que querríamos es una recursión simple que diga el mejor valor posible para las soluciones que contienen al último pedido. En la planificación con pesos esto era fácil, porque bastaba borrar los pedidos que chocaban con n. Acá no es tan simple: aceptar el pedido n no implica de inmediato que haya que rechazar ningún otro. Lo que sí implica es que para el subconjunto $S \subseteq \{1, \ldots, n-1\}$ que aceptemos queda menos peso disponible: el pedido n consume un peso w_n y solo queda $W - w_n$ para el resto, como en la Figura 6.7. Si no sabemos qué objetos se eligieron antes de n, ni siquiera sabemos si hay lugar para n.

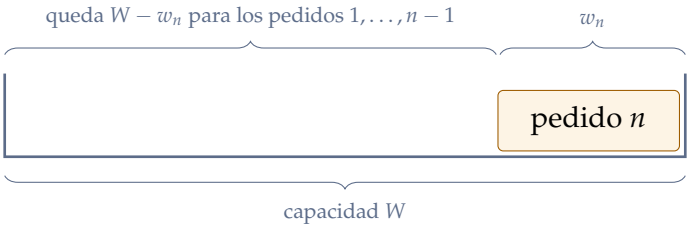


Figura 6.7. Después de incluir el pedido n en la solución, se consumió un peso w_n y quedan disponibles $W - w_n$ unidades para los demás.

Una solución mejor: una variable adicional

Esto sugiere que hacen falta más subproblemas: para conocer el valor de $\text{OPT}(n)$ no solo necesitamos el valor de $\text{OPT}(n-1)$, sino también la mejor solución que se puede obtener con un subconjunto de los primeros $n-1$ objetos y peso total permitido $W - w_n$. Hacen falta entonces muchos más subproblemas: uno para cada conjunto inicial $\{1, \ldots, i\}$ de

objetos y cada valor posible w del peso que queda disponible. Supongamos que W es entero y que todos los pedidos tienen pesos w_i enteros. Hay un subproblema para cada $i = 0, 1, \ldots, n$ y cada entero $0 \leq w \leq W$.

Definición 6.4.4 (Subproblemas de la suma de subconjuntos)**.** $\text{OPT}(i, w)$ es el valor de una solución óptima usando un subconjunto de los objetos $\{1, \ldots, i\}$ con peso máximo permitido w, es decir,

$$\text{OPT}(i, w) \;=\; \max_{S} \sum_{j \in S} w_j,$$

donde el máximo se toma sobre los subconjuntos $S \subseteq \{1, \ldots, i\}$ que cumplen $\sum_{j \in S} w_j \leq w$.

Con este nuevo conjunto de subproblemas se puede expresar $\text{OPT}(i, w)$ como una expresión simple de valores de problemas más chicos, y además $\text{OPT}(n, W)$ es exactamente la cantidad que buscamos. Como antes, sea $\mathcal{O}$ una solución óptima del problema original.

- Si $n \notin \mathcal{O}$, entonces $\text{OPT}(n, W) = \text{OPT}(n-1, W)$, porque el objeto n se puede ignorar.
- Si $n \in \mathcal{O}$, entonces $\text{OPT}(n, W) = w_n + \text{OPT}(n-1, W - w_n)$, porque ahora hay que usar de manera óptima la capacidad restante $W - w_n$ entre los objetos $1, 2, \ldots, n-1$.

Cuando el objeto n es demasiado grande, es decir, $W < w_n$, necesariamente $\text{OPT}(n, W) = \text{OPT}(n-1, W)$. En otro caso, el óptimo permitiendo los n pedidos es la mejor de las dos opciones. El mismo razonamiento para el subproblema con objetos $\{1, \ldots, i\}$ y peso máximo permitido w da la recurrencia siguiente.

Proposición 6.4.5. Para todo $0 \leq i \leq n$ y todo entero $0 \leq w \leq W$,

$$\text{OPT}(i, w) \;=\; \begin{cases} 0 & \text{si } i = 0, \\ \text{OPT}(i-1, w) & \text{si } w_i > w, \\ \max\big(\text{OPT}(i-1, w),\; w_i + \text{OPT}(i-1, w - w_i)\big) & \text{en otro caso.} \end{cases} \tag{6.4}$$

Como antes, buscamos un algoritmo que construya una tabla con todos los valores $\text{OPT}(i, w)$, calculando cada uno a lo sumo una vez.

```
SUBSET-SUM(n, W, w_1, ..., w_n)
1  for w = 0 to W do
2      M[0, w] ← 0
3  end for
4  for i = 1 to n do
5      for w = 0 to W do
6          Calcular M[i, w] con la recurrencia (6.4)
7      end for
8  end for
9  return M[n, W]
```

> **Estructuras de datos.** Los pesos se guardan en un arreglo de enteros $w[1..n]$, y los valores óptimos en una matriz de enteros $M[0..n][0..W]$, representada como arreglo bidimensional para acceder a cualquier entrada en $O(1)$. La tabla se llena por filas, de arriba hacia abajo.

Usando (6.4) se prueba inmediatamente por inducción que el valor devuelto $M[n, W]$ es el valor óptimo para los pedidos $1, \ldots, n$ con peso disponible W.

6.4.3 Análisis del algoritmo

El dibujo de la Figura 6.3, que mostraba cómo se llenaba el arreglo M en la planificación con pesos, tiene acá un análogo bidimensional, que refleja el arreglo bidimensional de subproblemas que se va construyendo. La Figura 6.8 muestra la dependencia: el valor $M[i, w]$ se calcula a partir de los dos valores $M[i-1, w]$ y $M[i-1, w-w_i]$, que están en la fila anterior. Por eso llenar la tabla por filas es un orden válido.

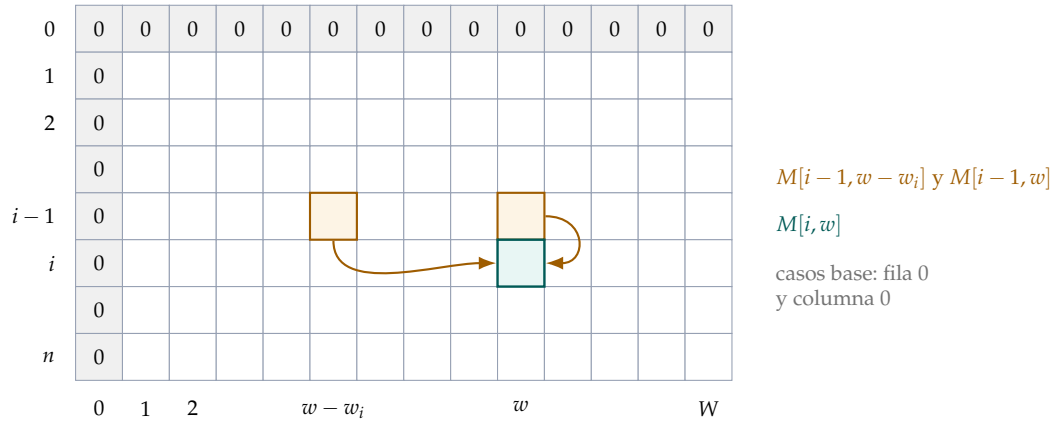


Figura 6.8. La tabla bidimensional de valores de OPT. La fila 0 y la columna 0 valen siempre 0. La entrada $M[i, w]$ se calcula a partir de las dos entradas de la fila anterior marcadas, $M[i-1, w]$ y $M[i-1, w-w_i]$.

Ejemplo 6.4.6 (Suma de subconjuntos con $W = 6$)**.** Consideremos una instancia con límite de peso $W = 6$ y $n = 3$ objetos de pesos $w_1 = w_2 = 2$ y $w_3 = 3$. El algoritmo llena la tabla de la Figura 6.9 fila por fila.

- Fila $i = 1$: para $w < 2$ el objeto no entra y se copia la fila 0; para $w \geq 2$ vale $\max(0,\, 2 + 0) = 2$.
- Fila $i = 2$: para $w \geq 2$ se compara $M[1, w] = 2$ con $2 + M[1, w - 2]$, que vale 2 si $w < 4$ y 4 si $w \geq 4$. La fila queda $0, 0, 2, 2, 4, 4, 4$.
- Fila $i = 3$: para $w \geq 3$ se compara $M[2, w]$ con $3 + M[2, w - 3]$. Así $M[3, 3] = \max(2, 3) = 3$, $M[3, 4] = \max(4, 3) = 4$, $M[3, 5] = \max(4, 3 + 2) = 5$ y $M[3, 6] = \max(4, 3 + 2) = 5$.

El valor óptimo es $\mathrm{OPT}(3, 6) = 5$, que se obtiene usando el tercer objeto y uno de los dos primeros. Para recuperar el conjunto se recorre la tabla hacia atrás desde $(3, 6)$: como $M[3, 6] = 5 \neq M[2, 6] = 4$, el objeto 3 está en la solución y se pasa a $(2, 3)$; ahí $M[2, 3] = M[1, 3] = 2$, así que el objeto 2 se puede dejar afuera y se pasa a $(1, 3)$; como $M[1, 3] = 2 \neq M[0, 3] = 0$, el objeto 1 está en la solución y se pasa a $(0, 1)$, donde se termina. El conjunto es $\{1, 3\}$.

w	0	1	2	3	4	5	6
$i = 0$	0	0	0	0	0	0	0
$i = 1$	0	0	2	2	2	2	2
$i = 2$	0	0	2	2	4	4	4
$i = 3$	0	0	2	3	4	5	5

$w_1 = 2,\ w_2 = 2,\ w_3 = 3$

se incluye el objeto de la fila

no se incluye, o fin

Figura 6.9. La tabla completa de la suma de subconjuntos del Ejemplo 6.4.6. Las celdas marcadas son las que visita la reconstrucción desde $M[3, 6]$.

Pasemos al tiempo de ejecución. Igual que en la planificación con pesos, se construye una tabla de soluciones M y cada valor $M[i, w]$ se calcula en tiempo $O(1)$ a partir de valores anteriores. El tiempo es entonces proporcional a la cantidad de entradas de la tabla.

Proposición 6.4.7. El algoritmo SUBSET-SUM(n, W) calcula correctamente el valor óptimo del problema y su tiempo de ejecución es $O(nW)$.

Este método no es tan eficiente como el de la planificación con pesos. De hecho, su tiempo de ejecución no es una función polinómica de n, sino una función polinómica de n y de W, el mayor entero que interviene en la definición del problema. Los algoritmos

de este tipo se llaman pseudopolinómicos. Pueden ser razonablemente eficientes cuando los números $\{w_i\}$ de la entrada son razonablemente chicos, pero se vuelven menos prácticos a medida que esos números crecen.

Nota 6.4.8 (Por qué $O(nW)$ no es polinómico). El tamaño de una entrada es la cantidad de bits necesaria para escribirla. El número W se escribe con alrededor de $\log_2 W$ bits, así que el tiempo $O(nW) = O(n\,2^{\log_2 W})$ es exponencial en el tamaño de esa parte de la entrada. Duplicar la cantidad de bits de W —agregar un solo bit, de hecho— puede duplicar el tiempo del algoritmo, mientras que agregar un objeto más solo suma una fila. Si en cambio los pesos están acotados por un polinomio en n, el algoritmo sí es polinómico.

Para recuperar un conjunto óptimo S de objetos se recorre la tabla M hacia atrás con un procedimiento parecido a los anteriores, como en el Ejemplo 6.4.6: en la posición (i, w), si $M[i, w] = M[i-1, w]$ el objeto i puede quedar afuera y se sigue en $(i-1, w)$; si no, el objeto i está en la solución y se sigue en $(i-1, w-w_i)$.

Proposición 6.4.9. Dada la tabla M con los valores óptimos de los subproblemas, un conjunto óptimo S se encuentra en tiempo $O(n)$.

6.4.4 Extensión: el problema de la mochila

El problema de la mochila es un poco más complejo que el de planificación anterior, porque cada objeto i tiene, además de su peso no negativo w_i, un valor v_i propio. El objetivo es encontrar un subconjunto S de valor máximo $\sum_{i \in S} v_i$, con la restricción de que el peso total no supere W: $\sum_{i \in S} w_i \leq W$.

No cuesta extender el algoritmo a este problema más general. Se usa el conjunto análogo de subproblemas: $\mathrm{OPT}(i, w)$ es el valor de una solución óptima usando un subconjunto de los objetos $\{1, \ldots, i\}$ con peso máximo disponible w. Consideramos una solución óptima $\mathcal{O}$ e identificamos dos casos, según que n esté o no en $\mathcal{O}$.

- Caso 1: $\mathrm{OPT}(i, w)$ no se alcanza incluyendo al objeto i. Una solución óptima se obtiene eligiendo un subconjunto óptimo de los objetos $\{1, \ldots, i-1\}$ con el mismo límite de peso w.
- Caso 2: $\mathrm{OPT}(i, w)$ se alcanza incluyendo al objeto i. Su valor es la suma del valor v_i del objeto y del máximo valor de un subconjunto de los objetos $\{1, \ldots, i-1\}$ cuyo peso acumulado no supere el límite disponible después de incluir al objeto i, que es $w - w_i$.

El caso 2 solo es posible si $w_i \leq w$; cuando $w_i > w$, $\mathrm{OPT}(i, w)$ no se alcanza incluyendo al objeto i, justamente porque no entra. Esto lleva a la siguiente análoga de (6.4).

Proposición 6.4.10. Para todo $0 \leq i \leq n$ y todo entero $0 \leq w \leq W$,

$$\mathrm{OPT}(i, w) \;=\; \begin{cases} 0 & \text{si } i = 0, \\ \mathrm{OPT}(i-1, w) & \text{si } w_i > w, \\ \max\big(\mathrm{OPT}(i-1, w),\ v_i + \mathrm{OPT}(i-1, w - w_i)\big) & \text{en otro caso.} \end{cases} \tag{6.5}$$

KNAPSACK$(n, W, w_1, \ldots, w_n, v_1, \ldots, v_n)$

```
1  for w = 0 to W do
2      M[0, w] ← 0
3  end for
4  for i = 1 to n do
5      for w = 0 to W do
6          if w_i > w then M[i, w] ← M[i − 1, w]
7          else M[i, w] ← máx(M[i − 1, w], v_i + M[i − 1, w − w_i])
8      end for
9  end for
10 return M[n, W]
```

> **Estructuras de datos.** Dos arreglos de enteros $w[1..n]$ y $v[1..n]$ para la entrada y una matriz $M[0..n][0..W]$ para la tabla. Para reconstruir la solución se recorre la matriz desde $M[n][W]$ y los objetos elegidos se acumulan en una lista enlazada. Si solo interesa el valor, alcanza con guardar dos filas de la matriz —la anterior y la actual—, con memoria $O(W)$.

La recurrencia da un algoritmo de programación dinámica completamente análogo, y de ahí el siguiente resultado.

Proposición 6.4.11. El problema de la mochila se resuelve en tiempo $O(nW)$.

Ejemplo 6.4.12 (La mochila con $W = 11$)**.** Retomemos la instancia del Ejemplo 6.4.3. La Figura 6.10 muestra la tabla completa que llena KNAPSACK. Por ejemplo, en la fila 3 y la columna 7, el objeto 3 pesa 5 y entra, así que $M[3,7] = \text{máx}(M[2,7],\ 18 + M[2,2]) = \text{máx}(7,\ 18+6) = 24$. En la fila 4, la columna 11 compara $M[3,11] = 25$ con $22 + M[3,5] = 22 + 18 = 40$ y se queda con 40.

El valor óptimo es $M[5,11] = 40$. La reconstrucción empieza en $(5,11)$: como $M[5,11] = 40 = M[4,11]$, el objeto 5 queda afuera y se pasa a $(4,11)$. Ahí $M[4,11] = 40 \neq M[3,11] = 25$, así que el objeto 4 se incluye y se pasa a $(3, 11-6) = (3,5)$. Como $M[3,5] = 18 \neq M[2,5] = 7$, se incluye el objeto 3 y se pasa a $(2,0)$; de ahí en adelante todas las entradas valen 0 y no se incluye nada más. La solución es $\{3,4\}$, de valor $18 + 22 = 40$ y peso $5 + 6 = 11$, la misma que se había encontrado a mano y que ninguno de los tres greedy obtenía.

objetos $1, \ldots, i$

w	0	1	2	3	4	5	6	7	8	9	10	11
$\{\}$	0	0	0	0	0	0	0	0	0	0	0	0
$\{1\}$	0	1	1	1	1	1	1	1	1	1	1	1
$\{1,2\}$	0	1	6	7	7	7	7	7	7	7	7	7
$\{1,2,3\}$	0	1	6	7	7	18	19	24	25	25	25	25
$\{1,2,3,4\}$	0	1	6	7	7	18	22	24	28	29	29	40
$\{1,2,3,4,5\}$	0	1	6	7	7	18	22	28	29	34	35	40

i	v_i	w_i
1	1	1
2	6	2
3	18	5
4	22	6
5	28	7

se incluye el objeto
no se incluye, o fin

Figura 6.10. La tabla de KNAPSACK para la instancia con capacidad $W = 11$. Cada fila corresponde a los objetos $1, \ldots, i$ y cada columna a un límite de peso w. Las celdas marcadas son las que visita la reconstrucción, que incluye los objetos 4 y 3.

6.5 Estructura secundaria de ARN: programación dinámica sobre intervalos

En el problema de la mochila se llegó a un algoritmo de programación dinámica agregando una variable nueva. Hay otra manera, distinta pero muy frecuente, de terminar agregando una variable a un programa dinámico. Se empieza pensando en el conjunto de subproblemas sobre $\{1, 2, \ldots, j\}$, para todas las elecciones de j, y no aparece ninguna recurrencia natural. Se pasa entonces al conjunto más grande de subproblemas sobre $\{i, i+1, \ldots, j\}$, para todas las elecciones de i y j con $i \leq j$, y ahí sí aparece una recurrencia natural. De esta forma se agrega la segunda variable i, y el efecto es considerar un subproblema por cada intervalo contiguo de $\{1, 2, \ldots, n\}$.

Hay unos cuantos problemas canónicos que responden a este perfil; quien haya estudiado algoritmos de análisis sintáctico (*parsing*) para gramáticas libres de contexto

probablemente vio al menos un algoritmo de programación dinámica de este estilo. Acá el ejemplo es el problema de predecir la estructura secundaria del ARN, una cuestión fundamental de la biología computacional.

6.5.1 El problema

Como se aprende en los cursos introductorios de biología, Watson y Crick propusieron que el ADN de doble cadena está "cerrado" como un cierre relámpago por un apareamiento complementario de bases. Cada cadena de ADN se puede ver como una tira de bases, cada una tomada del conjunto $\{A, C, G, T\}$, que corresponden a adenina, citosina, guanina y timina. Las bases A y T se aparean entre sí, y las bases C y G se aparean entre sí; son esos apareamientos A-T y C-G los que mantienen unidas a las dos cadenas.

Las moléculas de ARN de cadena simple son componentes clave de muchos procesos que ocurren dentro de la célula, y siguen más o menos los mismos principios estructurales. Pero, a diferencia del ADN de doble cadena, el ARN no tiene una "segunda cadena" a la que pegarse; por eso tiende a enrollarse y formar pares de bases consigo mismo. El conjunto de pares que se forma de esta manera, y la forma resultante, se llama estructura secundaria, y entenderla es esencial para entender el comportamiento de la molécula.

A los efectos del problema, una molécula de ARN de cadena simple es una sucesión $B = b_1 b_2 \cdots b_n$ de n símbolos (bases) del alfabeto $\{A, C, G, U\}$; el símbolo T del ADN se reemplaza por una U, lo que para nosotros no tiene importancia. En una primera aproximación su estructura secundaria se modela como sigue. Como es habitual, A se aparea con U y C con G; cada base se aparea con a lo sumo otra base, es decir, el conjunto de pares forma un emparejamiento; y resulta que las estructuras secundarias son, otra vez en primera aproximación, "sin nudos", lo que se formaliza con una condición de no cruzamiento.

Definición 6.5.1 (Estructura secundaria)**.** Una estructura secundaria sobre $B = b_1 b_2 \cdots b_n$ es un conjunto de pares $S = \{(i, j)\}$, con $i, j \in \{1, \ldots, n\}$, que cumple:

(I) (Sin ángulos filosos.) Los extremos de cada par están separados por al menos cuatro bases intermedias: si $(i, j) \in S$, entonces $i < j - 4$.

(II) (Watson–Crick.) Los elementos de cada par de S son $\{A, U\}$ o $\{C, G\}$, en cualquier orden: A-U, U-A, C-G o G-C.

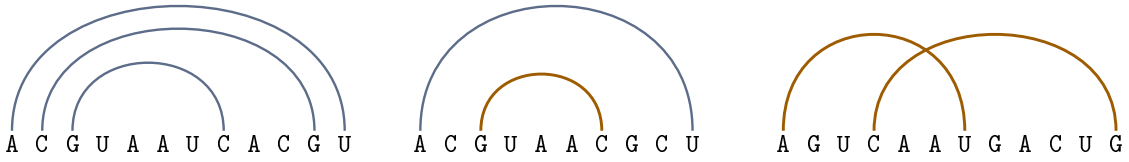
(III) S es un emparejamiento: ninguna base aparece en más de un par.

(IV) (Sin cruces.) Si (i, j) y (k, ℓ) son dos pares de S, no puede ocurrir que $i < k < j < \ell$.

Desde el punto de vista estructural, la condición (i) aparece simplemente porque la molécula no puede doblarse de manera demasiado brusca, y las condiciones (ii) y (iii)

son las reglas fundamentales de Watson y Crick para el apareamiento. La condición (iv) es la llamativa, porque no es obvio por qué tendría que valer en la naturaleza. Aunque en las moléculas reales hay excepciones esporádicas, los llamados pseudonudos, resulta ser una buena aproximación a las restricciones espaciales que sufren las estructuras secundarias reales.

La manera más clara de dibujar una estructura secundaria es "estirar" la cadena en una línea y marcar cada par con un arco por encima de ella, como en la Figura 6.11. En ese dibujo la condición de no cruzamiento dice exactamente que los arcos no se cortan: dos arcos o están uno dentro del otro o están uno al lado del otro.



(a) estructura válida (b) ángulo filoso: solo tres bases entre G y C (c) dos pares que se cruzan

Figura 6.11. Tres conjuntos de pares dibujados como arcos sobre la cadena estirada. El de (a) es una estructura secundaria. En (b) el par interno viola la condición de ángulos filosos, y en (c) los pares $(1,7)$ y $(4,12)$ violan la condición de no cruzamiento, porque $1 < 4 < 7 < 12$.

¿Cuáles de todas las estructuras secundarias posibles de una molécula de ARN son las que probablemente se forman en condiciones fisiológicas? La hipótesis usual es que la molécula forma la estructura secundaria que optimiza la energía libre total. El modelo correcto de energía libre es un tema muy debatido; una primera aproximación es suponer que la energía libre de una estructura secundaria es proporcional simplemente a la cantidad de pares de bases que contiene. Con todo esto, el problema básico se enuncia de manera muy simple.

Definición 6.5.2 (Predicción de la estructura secundaria de ARN)**.** Dada una molécula de ARN de cadena simple $B = b_1 b_2 \cdots b_n$, encontrar una estructura secundaria S con la máxima cantidad posible de pares de bases.

6.5.2 Diseño y análisis del algoritmo

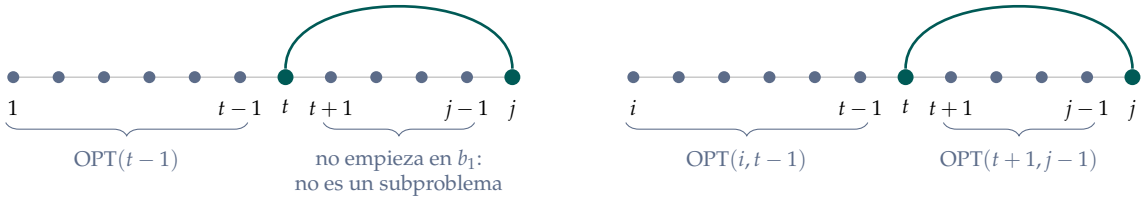
Un primer intento

El primer intento natural de aplicar programación dinámica se basaría presumiblemente en los siguientes subproblemas: $\text{OPT}(j)$ es la máxima cantidad de pares de bases en una estructura secundaria sobre $b_1 b_2 \cdots b_j$. Por la condición de ángulos filosos, $\text{OPT}(j) = 0$ para $j \leq 5$, y $\text{OPT}(n)$ es la solución buscada.

El problema aparece al intentar escribir una recurrencia que exprese $\text{OPT}(j)$ en términos de soluciones de subproblemas más chicos. Se puede avanzar un trecho: en la estructura secundaria óptima sobre $b_1 b_2 \cdots b_j$ ocurre que o bien

- j no está en ningún par, o bien
- j se aparea con t para algún $t < j - 4$.

En el primer caso alcanza con consultar la solución de $\text{OPT}(j-1)$. El segundo caso está en la parte (a) de la Figura 6.12: por la condición de no cruzamiento, ningún par puede tener un extremo entre 1 y $t-1$ y el otro entre $t+1$ y $j-1$. Quedan aislados así dos subproblemas nuevos, uno sobre las bases $b_1 b_2 \cdots b_{t-1}$ y otro sobre las bases $b_{t+1} \cdots b_{j-1}$. El primero lo resuelve $\text{OPT}(t-1)$, pero el segundo no está en nuestra lista de subproblemas, porque no empieza en b_1.



(a) con una variable (b) con dos variables

Figura 6.12. Incluir el par (t, j) separa la cadena en dos subproblemas independientes, porque ningún arco puede cruzar al arco (t, j). Con una sola variable, en (a), el tramo de adentro no es de la forma $b_1 \cdots b_k$; con dos variables, en (b), los dos tramos son intervalos y por lo tanto subproblemas.

Programación dinámica sobre intervalos

Esta es la observación que hace ver que hay que agregar una variable: necesitamos trabajar con subproblemas que no empiezan en b_1, es decir, con subproblemas sobre $b_i b_{i+1} \cdots b_j$ para todas las elecciones de $i \leq j$.

Definición 6.5.3 (Subproblemas sobre intervalos)**.** $\text{OPT}(i, j)$ es la máxima cantidad de pares de bases en una estructura secundaria sobre la subcadena $b_i b_{i+1} \cdots b_j$. Por comodidad de notación se permite también referirse a $\text{OPT}(i, j)$ cuando $i > j$, y en ese caso vale 0. El valor buscado es $\text{OPT}(1, n)$.

Tomada esta decisión, el razonamiento anterior lleva directamente a una recurrencia que funciona. Hay tres casos.

- Caso 1: $i \geq j - 4$. Entonces $\text{OPT}(i, j) = 0$ por la condición de ángulos filosos, porque entre dos bases cualesquiera del intervalo hay a lo sumo tres bases intermedias.

- Caso 2: la base b_j no está apareada. Entonces $\mathrm{OPT}(i,j) = \mathrm{OPT}(i,j-1)$.
- Caso 3: la base b_j está apareada con b_t para algún t con $i \le t < j-4$. Por la condición de no cruzamiento los dos subproblemas que quedan, sobre $b_i \cdots b_{t-1}$ y sobre $b_{t+1} \cdots b_{j-1}$, se optimizan de manera independiente, como en la parte (b) de la Figura 6.12, y $\mathrm{OPT}(i,j) = 1 + \mathrm{OPT}(i,t-1) + \mathrm{OPT}(t+1,j-1)$ para el mejor t posible.

Proposición 6.5.4. Para $i < j-4$,

$$\mathrm{OPT}(i,j) \;=\; \text{máx} \begin{cases} \mathrm{OPT}(i,j-1) \\ \text{máx}_t \big(1 + \mathrm{OPT}(i,t-1) + \mathrm{OPT}(t+1,j-1)\big) \end{cases} \tag{6.6}$$

donde el máximo interno se toma sobre los t tales que $i \le t < j-4$ y b_t es el complemento de Watson–Crick de b_j. Si no hay ningún t así, el máximo interno se omite. Para $i \ge j-4$ vale $\mathrm{OPT}(i,j) = 0$.

Falta asegurarse de entender en qué orden conviene construir las soluciones de los subproblemas. La forma de (6.6) muestra que siempre se invocan soluciones de subproblemas sobre intervalos más cortos: aquellos para los que la longitud $k = j - i$ es menor. Todo funciona entonces sin problemas si se construyen las soluciones en orden creciente de longitud del intervalo.

RNA-SECONDARY-STRUCTURE$(n, b_1, \ldots, b_n)$

```
1  for k = 0 to 4 do
2      for i = 1 to n − k do
3          j ← i + k;    M[i, j] ← 0
4      end for
5  end for
6  for k = 5 to n − 1 do
7      for i = 1 to n − k do
8          j ← i + k
9          Calcular M[i, j] con la recurrencia (6.6)
10     end for
11 end for
12 return M[1, n]
```

Estructuras de datos. La cadena se guarda en un arreglo de caracteres $b[1..n]$ y la tabla en una matriz de enteros $M[1..n+1][0..n]$, de la que solo se usa la parte con $i \le j$; las entradas con $i > j$, que aparecen en la recurrencia cuando $t = i$ o cuando $t+1 > j-1$, se dejan en 0 desde el principio. Saber si $b[t]$ y $b[j]$ son complementarios cuesta $O(1)$ con una tabla booleana de 4×4 indexada por las cuatro letras.

Acotar el tiempo de ejecución es fácil: hay $O(n^2)$ subproblemas para resolver, y evaluar la recurrencia (6.6) cuesta tiempo $O(n)$ para cada uno, porque hay que recorrer los posibles t. El tiempo de ejecución es entonces $O(n^3)$.

Como siempre, se puede recuperar la estructura secundaria misma —no solo su valor— registrando cómo se alcanzan los máximos en (6.6) y recorriendo el cálculo hacia atrás. Desde (i, j): si $M[i, j] = M[i, j-1]$ se sigue por $(i, j-1)$; si no, se busca un t que alcance el máximo, se agrega el par (t, j) y se sigue recursivamente por los dos intervalos $(i, t-1)$ y $(t+1, j-1)$.

Ejemplo 6.5.5 (La cadena ACCGGUAGU)**.** Tomemos $B =$ `ACCGGUAGU`, con $n = 9$:

k	1	2	3	4	5	6	7	8	9
b_k	A	C	C	G	G	U	A	G	U

Como las únicas entradas que pueden ser no nulas son las de $i < j - 4$, alcanza con mirar $i \in \{1, 2, 3, 4\}$ y $j \in \{6, 7, 8, 9\}$; las demás valen 0 por el caso base. La Figura 6.13 muestra la tabla final.

- Longitud $k = 5$: $(1, 6)$, $(2, 7)$, $(3, 8)$ y $(4, 9)$. En $(1, 6)$ el único t posible es $t = 1$, y $b_1 b_6 = AU$ se aparea: $M[1, 6] = 1$. En $(2, 7)$, $b_2 b_7 = CA$ no se aparea y $M[2, 7] = M[2, 6] = 0$. En $(3, 8)$, $b_3 b_8 = CG$ se aparea y $M[3, 8] = 1$. En $(4, 9)$, $b_4 b_9 = GU$ no se aparea y $M[4, 9] = 0$.
- Longitud $k = 6$: en $(1, 7)$ la base $b_7 = A$ solo podría aparearse con una U, y no hay U entre b_1 y b_2; queda $M[1, 7] = M[1, 6] = 1$. En $(2, 8)$, $b_8 = G$ se aparea con $b_2 = C$ o con $b_3 = C$, y cualquiera de los dos da 1: $M[2, 8] = 1$. En $(3, 9)$, $b_9 = U$ no tiene ninguna A entre b_3 y b_4, así que $M[3, 9] = M[3, 8] = 1$.
- Longitud $k = 7$: en $(1, 8)$, $b_8 = G$ puede aparearse con b_2 o b_3, y el mejor valor es 1; también $M[1, 7] = 1$, así que $M[1, 8] = 1$. En $(2, 9)$, $b_9 = U$ no tiene A entre b_2 y b_4, y queda $M[2, 9] = M[2, 8] = 1$.
- Longitud $k = 8$: en $(1, 9)$, $b_9 = U$ puede aparearse con $b_1 = A$ (con $t = 1 < 5$). Esa opción da $1 + \mathrm{OPT}(1, 0) + \mathrm{OPT}(2, 8) = 1 + 0 + 1 = 2$, que supera a $M[1, 8] = 1$. Por lo tanto $M[1, 9] = 2$.

El valor óptimo es $\mathrm{OPT}(1, 9) = 2$. La reconstrucción agrega el par $(1, 9)$ y sigue por el intervalo $(2, 8)$, donde encuentra el par $(2, 8)$; la estructura es $\{(1, 9), (2, 8)\}$, que se ve a la derecha de la figura. Los dos pares están anidados, así que no se cruzan, y en ambos hay al menos cuatro bases intermedias.

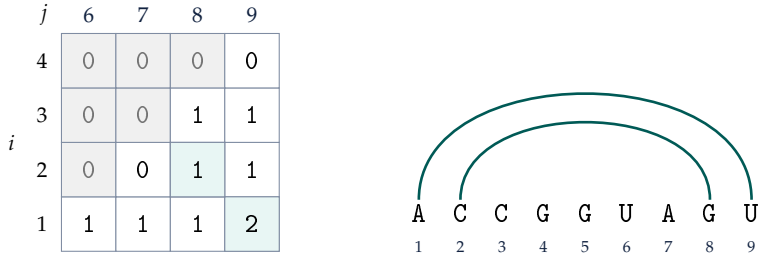


Figura 6.13. La tabla del Ejemplo 6.5.5, con la fila $i = 1$ abajo como en el libro. Las celdas grises son casos base ($i \geq j - 4$). Las celdas en color son las que usa la reconstrucción, y a la derecha está la estructura óptima $\{(1,9),(2,8)\}$.

Nota 6.5.6 (El orden de llenado, visto en la tabla). En la tabla de la Figura 6.13 los intervalos de una misma longitud $k = j - i$ están sobre una misma diagonal. Para $k = 5$ son las celdas $(1,6)$, $(2,7)$, $(3,8)$ y $(4,9)$; para $k = 6$, las tres celdas de la diagonal siguiente; y así hasta la celda $(1,9)$, la única con $k = 8$. Recorrer las diagonales en orden de k creciente es lo que hace el pseudocódigo. En cambio, llenar la tabla por filas en orden creciente de i, como en la mochila, no sirve: la celda (i,j) usa $(t+1, j-1)$, que está en una fila de índice mayor y todavía no se habría calculado. Sí serviría recorrer las filas en orden decreciente de i y, dentro de cada fila, las columnas en orden creciente de j, porque así todo lo que usa la recurrencia ya está calculado.

6.6 Alineamiento de secuencias

El alineamiento de secuencias es un problema fundamental que aparece al comparar cadenas de texto, y tiene una variedad muy amplia de aplicaciones.

6.6.1 El problema

Los diccionarios en línea ofrecen funciones que uno no obtiene de un diccionario impreso: si se busca una definición y se escribe una palabra que no está —por ejemplo, *ocurrance*—, el diccionario responde preguntando si quisimos decir *occurrence*. ¿Cómo lo hace? Para decidir qué quisimos decir probablemente, lo natural es buscar en el diccionario la palabra más "parecida" a la que escribimos. Para eso hay que responder la pregunta de cómo definir la similitud entre dos palabras o cadenas.

Intuitivamente, uno diría que *ocurrance* y *occurrence* son parecidas porque se puede hacer idénticas a las dos palabras agregando una `c` a la primera y cambiando la `a` por una `e`. Como ninguno de esos cambios parece muy grande, concluimos que las palabras son bastante parecidas. Dicho de otra forma, se pueden alinear casi letra por letra, y la Figura 6.14 muestra tres maneras de hacerlo. El guion indica un hueco: una posición en

la que hubo que agregar una letra a la otra palabra para que quedaran alineadas. Una letra enfrentada a otra distinta es una discrepancia.

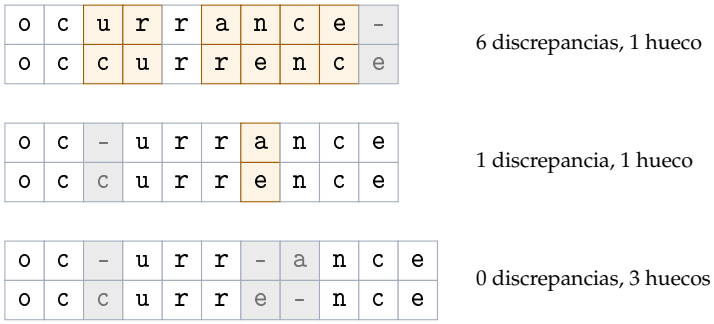


Figura 6.14. Tres alineamientos de *ocurrance* (fila de arriba) con *occurrence* (fila de abajo). Las discrepancias se marcan en color y los huecos en gris.

Queremos un modelo en el que la similitud esté determinada, a grandes rasgos, por la cantidad de huecos y discrepancias en que se incurre al alinear las dos palabras. Por supuesto, hay muchas maneras posibles de alinearlas. ¿Qué es mejor: un hueco y una discrepancia, o tres huecos y ninguna discrepancia? Cuando las cadenas no se parecen a palabras del castellano o del inglés —por ejemplo, `abbbaabbbbaab` y `ababaaabbbbbab`— puede costar bastante decidir si se pueden alinear bien o no.

Las interfaces de diccionarios y los correctores ortográficos no son las aplicaciones más exigentes de este problema. De hecho, determinar similitudes entre cadenas es uno de los problemas computacionales centrales de la biología molecular. El genoma de un organismo se divide en moléculas de ADN lineales gigantes llamadas cromosomas, y no se falsea mucho la realidad pensándolos como una cinta enorme que contiene una cadena sobre el alfabeto $\{A, C, G, T\}$. Esa cadena codifica las instrucciones para construir proteínas. Si se tienen dos cepas de bacterias X e Y muy emparentadas, y se sabe que cierta subcadena del ADN de X codifica un tipo de toxina, encontrar una subcadena muy "parecida" en el ADN de Y permite conjeturar, antes de hacer ningún experimento, que esa porción del ADN de Y codifica una toxina similar. Usar el cómputo para guiar decisiones sobre experimentos biológicos es una de las marcas de la biología computacional.

A principios de los años setenta, los biólogos moleculares Needleman y Wunsch propusieron una definición de similitud que, prácticamente sin cambios, es el estándar que se usa hoy; en la misma época Levenshtein había llegado a una noción equivalente, la distancia de edición. La definición venía con un algoritmo de programación dinámica eficiente para calcularla, de modo que los biólogos descubrieron el paradigma de manera independiente unos veinte años después de que lo formularan matemáticos e informáticos. Además de la bioinformática, tiene aplicaciones en herramientas como `diff` de Unix y en la corrección de errores.

Definición 6.6.1 (Alineamiento)**.** Sean $X = x_1x_2 \cdots x_m$ e $Y = y_1y_2 \cdots y_n$ dos cadenas. Un alineamiento M es un emparejamiento entre $\{1, 2, \ldots, m\}$ y $\{1, 2, \ldots, n\}$ —un conjunto de pares ordenados en el que cada elemento aparece en a lo sumo un par— sin cruzamientos: si $(i, j), (i', j') \in M$ e $i < i'$, entonces $j < j'$. Escribimos x_i–y_j para denotar el par (i, j) de M.

Intuitivamente, un alineamiento dice qué pares de posiciones se enfrentan al escribir una cadena debajo de la otra. Dos pares x_i–y_j y $x_{i'}$–$y_{j'}$ se cruzan si $i < i'$ pero $j > j'$, y eso es lo que se prohíbe.

Definición 6.6.2 (Costo de un alineamiento)**.** Hay un parámetro $\delta > 0$, la penalización por hueco, y para cada par de letras p, q del alfabeto hay una penalización por discrepancia α_{pq}, que se paga por enfrentar p con q. El costo de un alineamiento M es

$$\text{costo}(M) \;=\; \underbrace{\sum_{(x_i, y_j) \in M} \alpha_{x_i y_j}}_{\text{discrepancias}} \;+\; \underbrace{\sum_{i\,:\,x_i \text{ sin emparejar}} \delta \;+\; \sum_{j\,:\,y_j \text{ sin emparejar}} \delta}_{\text{huecos}}.$$

El objetivo es, dadas X e Y, encontrar un alineamiento de costo mínimo. Ese costo mínimo es la medida de similitud entre las cadenas: cuanto menor, más parecidas.

Habitualmente se supone que $\alpha_{pp} = 0$ para toda letra p —enfrentar una letra con otra copia de sí misma no cuesta nada—, aunque nada de lo que sigue lo necesita. Las cantidades δ y $\{\alpha_{pq}\}$ son parámetros externos que hay que cargar en el software de alineamiento, y elegirlos bien es todo un trabajo; para el diseño del algoritmo los tomamos como dados. Volviendo al primer ejemplo, los parámetros son los que deciden entre el segundo y el tercer alineamiento de la Figura 6.14: el segundo es estrictamente mejor si y solo si $\delta + \alpha_{ae} < 3\delta$.

Ejemplo 6.6.3 (Un alineamiento de CTACCG con TACATG)**.** La Figura 6.15 muestra el alineamiento

$$M = \{x_2\text{–}y_1,\ x_3\text{–}y_2,\ x_4\text{–}y_3,\ x_5\text{–}y_4,\ x_6\text{–}y_6\}$$

de $X = \texttt{CTACCG}$ con $Y = \texttt{TACATG}$. Las posiciones x_1 e y_5 quedan sin emparejar, y el único par con letras distintas es x_5–y_4, que enfrenta una `C` con una `A`. El costo es $2\delta + \alpha_{CA}$. Los pares no se cruzan: los índices de X $(2, 3, 4, 5, 6)$ y los de Y $(1, 2, 3, 4, 6)$ crecen juntos.

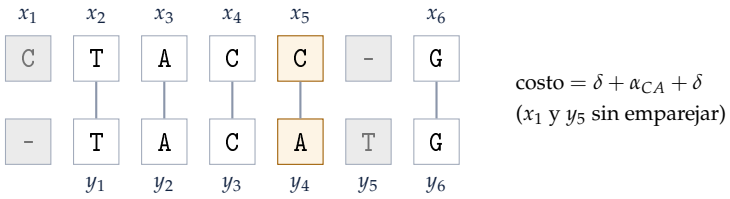


Figura 6.15. El alineamiento del Ejemplo 6.6.3. Cada segmento vertical es un par de M; la discrepancia está en color y los huecos en gris.

6.6.2 Diseño del algoritmo

Tenemos ahora una definición numérica concreta de la similitud entre X e Y: el costo mínimo de un alineamiento entre ellas. Hay que calcular ese costo mínimo, y un alineamiento óptimo que lo alcance. Lo que motiva la programación dinámica es la siguiente dicotomía básica.

- En el alineamiento óptimo M, o bien $(m, n) \in M$, o bien $(m, n) \notin M$. Es decir, los últimos símbolos de las dos cadenas se enfrentan entre sí, o no.

Por sí sola, esta dicotomía es demasiado débil para dar una solución por programación dinámica. Pero se combina con el siguiente hecho básico.

Proposición 6.6.4. Sea M un alineamiento cualquiera de X e Y. Si $(m, n) \notin M$, entonces la posición m de X o la posición n de Y no está emparejada en M.

Demostración. Supongamos, por el absurdo, que $(m, n) \notin M$ y que hay números $i < m$ y $j < n$ tales que $(m, j) \in M$ y $(i, n) \in M$. Esto contradice la definición de alineamiento: los pares (i, n) y (m, j) cumplen $i < m$ pero $n > j$, así que se cruzan. □

Hay una manera equivalente de escribir la Proposición 6.6.4 que deja a la vista tres alternativas y lleva directamente a una recurrencia.

Corolario 6.6.5. En un alineamiento óptimo M vale al menos una de las siguientes:

(I) $(m, n) \in M$;

(II) la posición m de X no está emparejada;

(III) la posición n de Y no está emparejada.

Definición 6.6.6 (Subproblemas del alineamiento)**.** $\mathrm{OPT}(i, j)$ es el costo mínimo de un alineamiento entre $x_1 x_2 \cdots x_i$ e $y_1 y_2 \cdots y_j$. El valor buscado es $\mathrm{OPT}(m, n)$.

Veamos cada caso del corolario, en la versión de las diapositivas.

- Caso 1a: $\mathrm{OPT}(i,j)$ se alcanza dejando x_i sin emparejar. Se paga el hueco de x_i más el costo mínimo de alinear $x_1 \cdots x_{i-1}$ con $y_1 \cdots y_j$: $\delta + \mathrm{OPT}(i-1,j)$.
- Caso 1b: $\mathrm{OPT}(i,j)$ se alcanza dejando y_j sin emparejar. Se paga el hueco de y_j más el costo mínimo de alinear $x_1 \cdots x_i$ con $y_1 \cdots y_{j-1}$: $\delta + \mathrm{OPT}(i,j-1)$.
- Caso 2: $\mathrm{OPT}(i,j)$ se alcanza emparejando x_i–y_j. Se paga la discrepancia de ese par más el costo mínimo de alinear $x_1 \cdots x_{i-1}$ con $y_1 \cdots y_{j-1}$: $\alpha_{x_i y_j} + \mathrm{OPT}(i-1,j-1)$.

Para los casos base, notemos que $\mathrm{OPT}(i,0) = i\delta$ y $\mathrm{OPT}(0,j) = j\delta$, porque la única manera de alinear una palabra de i letras con una palabra vacía es usar i huecos.

Proposición 6.6.7. Los costos mínimos de alineamiento cumplen

$$\mathrm{OPT}(i,j) = \begin{cases} j\delta & \text{si } i = 0, \\ i\delta & \text{si } j = 0, \\ \min \begin{cases} \alpha_{x_i y_j} + \mathrm{OPT}(i-1,j-1) \\ \delta + \mathrm{OPT}(i-1,j) \\ \delta + \mathrm{OPT}(i,j-1) \end{cases} & \text{si } i,j \geq 1. \end{cases} \tag{6.7}$$

Además, (i,j) está en un alineamiento óptimo de este subproblema si y solo si el mínimo se alcanza con el primero de los tres valores.

Con esto el algoritmo de programación dinámica queda claro: se construyen los valores $\mathrm{OPT}(i,j)$ con la recurrencia (6.7). Hay solo $O(mn)$ subproblemas, y $\mathrm{OPT}(m,n)$ es el valor buscado.

SEQUENCE-ALIGNMENT$(m, n, x_1, \ldots, x_m, y_1, \ldots, y_n, \delta, \alpha)$

```
1  for i = 0 to m do  M[i,0] ← iδ  end for
2  for j = 0 to n do  M[0,j] ← jδ  end for
3  for i = 1 to m do
4      for j = 1 to n do
5          M[i,j] ← mín(α[x_i, y_j] + M[i−1, j−1], δ + M[i−1, j], δ + M[i, j−1])
6      end for
7  end for
8  return M[m,n]
```

Estructuras de datos. Las cadenas se guardan en arreglos de caracteres $x[1..m]$ e $y[1..n]$. Las penalizaciones por discrepancia forman una matriz α indexada por las letras del alfabeto (para $\{A,C,G,T\}$, una matriz de 4×4 con las letras traducidas a índices 0.,3), de modo que $\alpha[x_i, y_j]$ se consulta en $O(1)$. La tabla es una matriz $M[0..m][0..n]$ que se llena por filas; cada entrada usa la de arriba, la de la izquierda y la diagonal superior izquierda, que ya están calculadas.

Igual que en los algoritmos anteriores, el alineamiento mismo se construye recorriendo hacia atrás la tabla M con la segunda parte de la Proposición 6.6.7. Desde (i, j): si el mínimo se alcanzó con el primer término, se agrega el par (i, j) y se sigue por $(i-1, j-1)$; si se alcanzó con el segundo, x_i queda sin emparejar y se sigue por $(i-1, j)$; si con el tercero, y_j queda sin emparejar y se sigue por $(i, j-1)$. Al llegar a la fila 0 o a la columna 0, las letras que quedan van contra huecos.

6.6.3 Análisis del algoritmo

La corrección del algoritmo se sigue directamente de la Proposición 6.6.7. El tiempo de ejecución es $O(mn)$, porque la tabla M tiene $O(mn)$ entradas y en el peor caso se gasta tiempo constante en cada una; la reconstrucción cuesta $O(m+n)$, porque cada paso baja i, j o ambos.

Hay una manera gráfica atractiva de pensar este algoritmo. Se arma un grafo en grilla G_{XY} con las filas rotuladas por los símbolos de X y las columnas por los de Y, y aristas dirigidas como en la Figura 6.16. Las filas se numeran de 0 a m y las columnas de 0 a n, y el nodo de la fila i y la columna j se denota (i, j). Cada arista horizontal y cada arista vertical cuesta δ, y la arista diagonal de $(i-1, j-1)$ a (i, j) cuesta $\alpha_{x_i y_j}$.

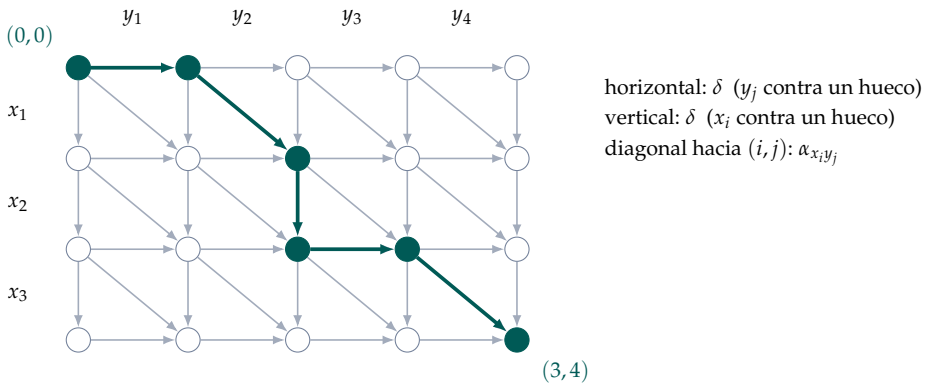


Figura 6.16. El grafo en grilla G_{XY} para $m = 3$ y $n = 4$. Las diagonales del camino marcado corresponden a los pares x_1–y_2 y x_3–y_4 de un alineamiento; su tramo horizontal inicial deja y_1 sin emparejar, el vertical deja x_2 sin emparejar, y el horizontal del medio deja y_3 sin emparejar.

El sentido de esta imagen es el siguiente: la recurrencia (6.7) para $\mathrm{OPT}(i, j)$ es exactamente la recurrencia que se obtiene para el camino de costo mínimo en G_{XY} de $(0, 0)$ a (i, j).

Proposición 6.6.8. Sea $f(i, j)$ el costo mínimo de un camino de $(0, 0)$ a (i, j) en G_{XY}. Entonces $f(i, j) = \mathrm{OPT}(i, j)$ para todo i, j.

Demostración. Por inducción en $i + j$. Si $i + j = 0$, entonces $i = j = 0$ y efectivamen-

te $f(0,0) = \text{OPT}(0,0) = 0$. Si $i = 0$ y $j > 0$, el único camino a $(0,j)$ usa j aristas horizontales y cuesta $j\delta = \text{OPT}(0,j)$; lo mismo vale para la columna 0.

Consideremos ahora valores arbitrarios $i, j \geq 1$ y supongamos que el enunciado vale para todos los pares (i', j') con $i' + j' < i + j$. La última arista de un camino mínimo hasta (i,j) viene de $(i-1, j-1)$, de $(i-1, j)$ o de $(i, j-1)$. Por lo tanto

$$\begin{aligned} f(i,j) &= \text{mín}\big(\alpha_{x_i y_j} + f(i-1,j-1),\ \delta + f(i-1,j),\ \delta + f(i,j-1)\big) \\ &= \text{mín}\big(\alpha_{x_i y_j} + \text{OPT}(i-1,j-1),\ \delta + \text{OPT}(i-1,j),\ \delta + \text{OPT}(i,j-1)\big) \\ &= \text{OPT}(i,j), \end{aligned}$$

donde de la primera línea a la segunda se pasa por la hipótesis inductiva, y de la segunda a la tercera por (6.7). □

El valor del alineamiento óptimo es entonces el largo del camino mínimo en G_{XY} de $(0,0)$ a (m,n); a cualquier camino de $(0,0)$ a (m,n) en G_{XY} lo llamamos camino de esquina a esquina. Además, las aristas diagonales de un camino mínimo corresponden exactamente a los pares de un alineamiento de costo mínimo. Esta conexión con el problema de caminos mínimos en G_{XY} no mejora directamente el tiempo del alineamiento, pero ayuda a la intuición y sugirió algoritmos para variantes más complejas del problema.

Ejemplo 6.6.9 (Alinear mean con name)**.** Alineemos $X = \texttt{mean}$ con $Y = \texttt{name}$. Supongamos que $\delta = 2$, que enfrentar una vocal con otra vocal distinta, o una consonante con otra consonante distinta, cuesta 1, que enfrentar una vocal con una consonante cuesta 3, y que enfrentar una letra consigo misma cuesta 0. La Figura 6.17 muestra la tabla M, con las filas rotuladas por las letras de `mean` y las columnas por las de `name`. Por ejemplo:

- $M[1,1] = \text{mín}(\alpha_{mn} + 0,\ 2+2,\ 2+2) = \text{mín}(1,4,4) = 1$, porque `m` y `n` son consonantes;
- $M[2,2] = \text{mín}(\alpha_{ea} + M[1,1],\ 2 + M[1,2],\ 2 + M[2,1]) = \text{mín}(1+1,\ 2+3,\ 2+3) = 2$;
- $M[3,2] = \text{mín}(\alpha_{aa} + M[2,1],\ 2 + M[2,2],\ 2 + M[3,1]) = \text{mín}(0+3,\ 2+2,\ 2+5) = 3$.

El valor óptimo es $M[4,4] = 6$. Para reconstruir, desde $(4,4)$ se ve que $M[4,4] = 6 = 2 + M[4,3]$ y que la diagonal daría $\alpha_{ne} + M[3,3] = 3 + 5 = 8$: la `e` final de `name` queda contra un hueco. En $(4,3)$ gana la diagonal, $\alpha_{nm} + M[3,2] = 1 + 3 = 4$, y se empareja `n` con `m`. En $(3,2)$ gana la diagonal con $\alpha_{aa} + M[2,1] = 0 + 3$, y se empareja `a` con `a`. En $(2,1)$ el mínimo es $2 + M[1,1] = 3$, así que la `e` de `mean` queda contra un hueco. Finalmente en $(1,1)$ se empareja `m` con `n`. El alineamiento es

```
m e a n -
n - a m e
```

con costo $1 + 2 + 0 + 1 + 2 = 6$.

	-	n	a	m	e
-	0	2	4	6	8
m	2	1	3	4	6
e	4	3	2	4	4
a	6	5	3	5	5
n	8	6	5	4	6

$\delta = 2$
$\alpha = 1$ entre dos vocales o dos consonantes distintas
$\alpha = 3$ entre vocal y consonante

camino de la reconstrucción

Figura 6.17. La tabla M del Ejemplo 6.6.9. Las filas corresponden a los prefijos de `mean` y las columnas a los de `name`; la fila y la columna rotuladas con un guion son los prefijos vacíos. Las celdas en color forman un camino mínimo de esquina a esquina en G_{XY}: los pasos diagonales son los pares del alineamiento.

6.7 Caminos de costo mínimo en un grafo: el algoritmo de Bellman-Ford

6.7.1 El problema

Sea $G = (V, E)$ un grafo dirigido, con n nodos y m aristas. Supongamos que cada arista $(v, w) \in E$ tiene un peso asociado c_{vw}. Los pesos pueden modelar muchas cosas distintas; acá los interpretamos como el costo de ir directamente del nodo v al nodo w.

En la Sección 4.4 vimos el algoritmo de Dijkstra para encontrar caminos mínimos en grafos con largos no negativos. Ahora el problema es más complejo: buscamos caminos de costo mínimo cuando los costos pueden ser negativos. Entre las motivaciones para estudiarlo se destacan dos. La primera es que los costos negativos son cruciales para modelar varios fenómenos con caminos mínimos. Por ejemplo, los nodos pueden representar agentes de un mercado financiero, y c_{vw} el costo de una transacción en la que se le compra al agente v y se le vende inmediatamente al agente w. Un camino representa entonces una sucesión de transacciones, y las aristas de costo negativo representan transacciones que dejan ganancia. La segunda es que el algoritmo para costos negativos resulta ser, en aspectos cruciales, más flexible y descentralizado que el de Dijkstra, y por eso tiene aplicaciones importantes en el diseño de algoritmos de ruteo distribuidos, que determinan el camino más eficiente en una red de comunicaciones.

Hay dos problemas relacionados.

- Dado un grafo G con costos en las aristas, decidir si G tiene un ciclo de costo negativo, es decir, un ciclo dirigido W tal que $c(W) = \sum_{(v,w)\in W} c_{vw} < 0$.
- Si el grafo no tiene ciclos de costo negativo, encontrar un camino P de un nodo de origen s a un nodo destino t de costo total $\sum_{(v,w)\in P} c_{vw}$ mínimo.

El segundo se llama problema del camino de costo mínimo, o del camino más corto. En términos de la motivación financiera, un ciclo de costo negativo es una sucesión rentable de transacciones que vuelve al punto de partida: se le compra a v_1 y se le vende a v_2, se le compra a v_2 y se le vende a v_3, y así sucesivamente, hasta volver a v_1 con una ganancia neta. Los ciclos negativos de una red así son oportunidades de arbitraje.

Tiene sentido estudiar el camino de costo mínimo de s a t suponiendo que no hay ciclos negativos. Como muestra la Figura 6.18, si hay un ciclo negativo W, un camino P_s de s al ciclo y otro camino P_t del ciclo a t, se pueden armar caminos de s a t de costo arbitrariamente negativo: se usa P_s para llegar al ciclo, se da la vuelta a W tantas veces como se quiera, y se usa P_t para llegar a t.

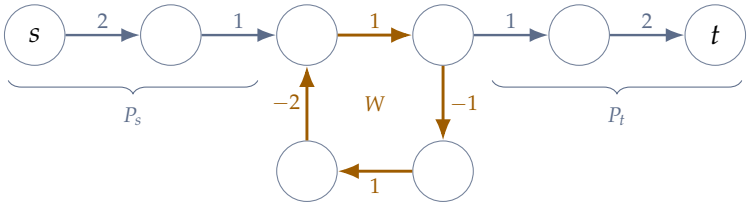


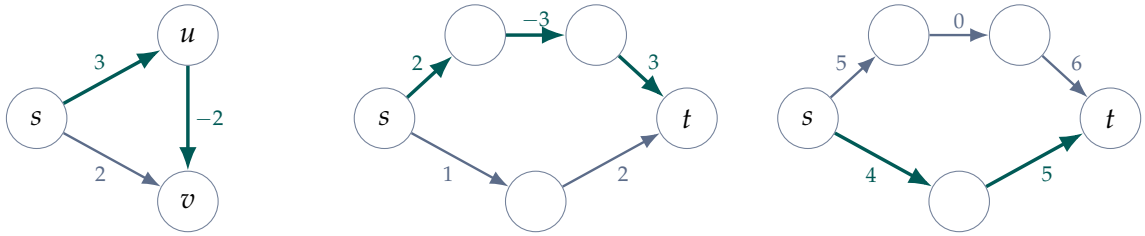
Figura 6.18. El ciclo W tiene costo $1 - 1 + 1 - 2 = -1$. Dar una vuelta más por él baja en 1 el costo de un camino de s a t, así que no existe un camino de costo mínimo si se permite repetir nodos.

6.7.2 Algunos intentos fallidos

Recordemos el algoritmo de Dijkstra, que resuelve el problema cuando no hay costos negativos. Calcula un camino mínimo desde el origen s hasta cada nodo del grafo, esencialmente con un algoritmo greedy: mantiene un conjunto S de nodos cuyo camino mínimo desde s ya se conoce y lo hace crecer de a un nodo. Su primer paso lo muestra bien: se considera la arista más barata que sale de s, digamos (s, v), y se agrega v a S, con la seguridad de que el camino mínimo de s a v es la arista (s, v). Si no hay costos negativos eso es claro, porque cualquier otro camino de s a v empieza con una arista que sale de s y que cuesta al menos tanto como (s, v).

Esa observación deja de valer con costos negativos. Como sugiere la parte (a) de la Figura 6.19, un camino que empieza por una arista cara pero después compensa con aristas de costo negativo puede ser más barato que uno que empieza por una arista barata. El enfoque greedy de Dijkstra no funciona.

Otra idea natural es modificar primero los costos c_{vw} sumándoles a todos una constante grande K, es decir, tomar $c'_{vw} = c_{vw} + K$ para cada arista. Si K es suficientemente grande, todos los costos modificados son no negativos y se puede usar Dijkstra con los costos c'. Pero este enfoque no encuentra los caminos mínimos respecto de los costos originales, porque cambiar c por c' cambia cuál es el camino mínimo: los caminos más largos, en cantidad de aristas, sufren incrementos mayores. Si un camino P de tres aristas es apenas más barato que otro camino P' de dos aristas, después del cambio P' pasa a ser más barato, porque a P' solo se le suma $2K$ y a P se le suma $3K$. Es lo que ocurre en las partes (b) y (c) de la figura.



(a) Dijkstra fija $d(v) = 2$, pero s-u-v cuesta 1

(b) costos originales: P arriba cuesta 2, P' abajo cuesta 3

(c) sumando $K = 3$: P cuesta 11 y P' cuesta 9

Figura 6.19. Dos ideas que fallan con costos negativos. En (a) Dijkstra toma la arista más barata que sale de s y se equivoca. En (b) y (c), sumar la misma constante a todas las aristas cambia el camino mínimo, marcado en color, porque el camino de tres aristas recibe más incremento que el de dos.

6.7.3 Un enfoque por programación dinámica

Intentaremos usar programación dinámica para encontrar un camino mínimo de s a t cuando hay costos negativos pero no ciclos negativos. Se podría probar una idea que ya funcionó: que el subproblema i sea encontrar un camino mínimo usando solo los primeros i nodos. Esa idea no funciona de inmediato, aunque se la puede hacer funcionar con algo de esfuerzo. Hay una solución más simple y más eficiente, el algoritmo de Bellman-Ford. El desarrollo de la programación dinámica como técnica general de diseño se le suele atribuir al trabajo de Richard Bellman en los años cincuenta, y este algoritmo de caminos mínimos fue una de sus primeras aplicaciones.

La solución se apoya en la siguiente observación crucial.

Lema 6.7.1. Si G no tiene ciclos de costo negativo, entonces hay un camino de costo mínimo de v a t que es simple (no repite nodos), y que por lo tanto tiene a lo sumo $n - 1$ aristas.

Demostración. Consideremos un camino P de v a t de costo mínimo y, entre todos los de costo mínimo, uno con la menor cantidad posible de aristas. Si P repitiera un nodo, contendría un ciclo W entre dos visitas consecutivas a ese nodo, como en la Figura 6.20. Como todo ciclo tiene costo no negativo, podríamos quitar W de P y obtener un camino de costo no mayor y con menos aristas, lo que contradice la elección de P. Entonces P no contiene ciclos, es decir, es simple; y un camino simple visita a lo sumo n nodos, de modo que tiene a lo sumo $n - 1$ aristas. □

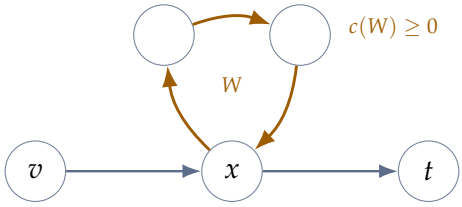


Figura 6.20. Si un camino de v a t pasa dos veces por x, el tramo entre las dos visitas es un ciclo W. Sin ciclos negativos, sacarlo no aumenta el costo y reduce la cantidad de aristas.

Definición 6.7.2 (Subproblemas de Bellman-Ford)**.** $\text{OPT}(i, v)$ es el costo mínimo de un camino de v a t que usa a lo sumo i aristas. Si no hay ningún camino así, $\text{OPT}(i, v) = \infty$.

Por el Lema 6.7.1, si no hay ciclos de costo negativo el problema original consiste en calcular $\text{OPT}(n-1, s)$; más en general, $\text{OPT}(n-1, v)$ es el costo mínimo de un camino de v a t para cada nodo v. Notemos que los subproblemas miden caminos hacia el destino t, no desde el origen s. Se podría diseñar el algoritmo con subproblemas que midan el costo mínimo de un camino de s a v con a lo sumo i aristas, lo que formaría un paralelo más natural con Dijkstra; pero la versión hacia t es la natural para los protocolos de ruteo, en los que cada nodo quiere saber cómo llegar a un destino.

Hace falta ahora expresar $\text{OPT}(i, v)$ en términos de subproblemas más chicos. El enfoque más natural considera muchas opciones distintas; es otro ejemplo del principio de elección de múltiples vías de la Sección 6.3. Fijemos un camino óptimo P que realiza $\text{OPT}(i, v)$, como en la Figura 6.21.

- Caso 1: el costo mínimo se alcanza con un camino que usa menos de i aristas, es decir, a lo sumo $i-1$. Entonces $\text{OPT}(i, v) = \text{OPT}(i-1, v)$.
- Caso 2: el costo mínimo lo alcanza un camino que usa exactamente i aristas. Si (v, w) es la primera arista de ese camino, a continuación se sigue por el mejor camino de w a t que usa a lo sumo $i-1$ aristas, y $\text{OPT}(i, v) = c_{vw} + \text{OPT}(i-1, w)$.

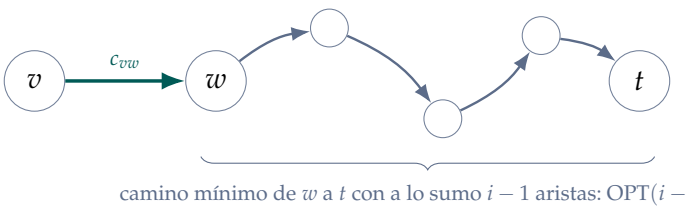


Figura 6.21. Un camino de costo mínimo de v a t con exactamente i aristas: la primera arista (v, w) seguida de un camino mínimo de w a t con a lo sumo $i-1$ aristas.

Esto lleva a la siguiente fórmula recursiva.

Proposición 6.7.3. Los costos $\mathrm{OPT}(i, v)$ cumplen

$$\mathrm{OPT}(i,v) \;=\; \begin{cases} 0 & \text{si } i = 0 \text{ y } v = t, \\ \infty & \text{si } i = 0 \text{ y } v \neq t, \\ \min \begin{cases} \mathrm{OPT}(i-1, v) \\ \min_{(v,w)\in E}\{\mathrm{OPT}(i-1, w) + c_{vw}\} \end{cases} & \text{si } i > 0. \end{cases} \tag{6.8}$$

Con esta recurrencia se obtiene el siguiente algoritmo de programación dinámica para calcular $\mathrm{OPT}(n-1, s)$.

SHORTEST-PATHS(V, E, c, s, t)

```
1  foreach nodo v ∈ V do  M[0, v] ← ∞  end foreach
2  M[0, t] ← 0
3  for i = 1 to n − 1 do
4      foreach nodo v ∈ V do
5          M[i, v] ← M[i − 1, v]
6          foreach arista (v, w) ∈ E do
7              M[i, v] ← mín(M[i, v], M[i − 1, w] + c_vw)
8          end foreach
9      end foreach
10 end for
11 return M[n − 1, s]
```

> **Estructuras de datos.** Los nodos se numeran de 1 a n y el grafo se representa con listas de adyacencia de salida: para cada nodo v, una lista enlazada con los pares (w, c_{vw}) de sus aristas salientes, de modo que el bucle de la línea 6 recorre exactamente las aristas que salen de v. La tabla es una matriz de reales $M[0..n-1][1..n]$, donde ∞ se representa con un valor centinela mayor que el costo de cualquier camino simple (por ejemplo, $n \cdot \max |c_{vw}| + 1$), cuidando que $\infty + c_{vw}$ se trate como ∞.

La corrección del método se sigue directamente por inducción a partir de (6.8). Una primera cota del tiempo es la siguiente: la tabla M tiene n^2 entradas, y calcular cada una puede costar $O(n)$, porque hay a lo sumo n nodos w para considerar.

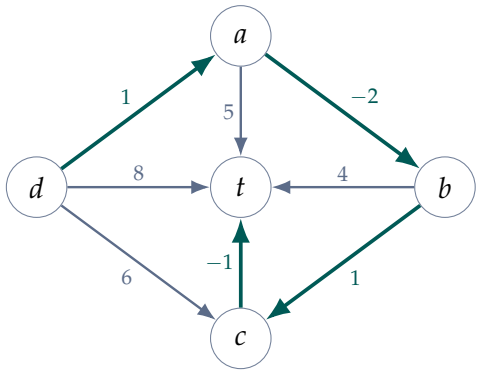
Proposición 6.7.4. El método SHORTEST-PATHS calcula correctamente el costo mínimo de un camino de s a t en cualquier grafo sin ciclos de costo negativo, en tiempo $O(n^3)$.

Dada la tabla M con los valores óptimos de los subproblemas, el camino mínimo que usa a lo sumo i aristas se obtiene en tiempo $O(in)$ recorriendo hacia atrás los subproblemas más chicos: desde v, se busca una arista (v, w) con $M[i, v] = M[i-1, w] + c_{vw}$ y se sigue desde w con $i-1$ (si $M[i, v] = M[i-1, v]$, se sigue desde v con $i-1$).

Ejemplo 6.7.5 (Una corrida de Bellman-Ford)**.** Consideremos el grafo de la Figura 6.22, con cinco nodos y destino t. Como $n = 5$, se calculan las columnas $i = 0, \ldots, 4$ de la tabla, y cada fila de la tabla sigue el costo mínimo de un nodo a t a medida que se le permite usar más aristas.

- Columna $i = 0$: solo t tiene costo 0; los demás valen ∞.
- Columna $i = 1$: cada nodo con una arista directa a t toma su costo: $M[1, a] = 5$, $M[1, b] = 4$, $M[1, c] = -1$ y $M[1, d] = 8$.
- Columna $i = 2$: $M[2, a] = \text{mín}(5,\ -2 + M[1, b]) = \text{mín}(5, 2) = 2$; $M[2, b] = \text{mín}(4,\ 1 + M[1, c]) = 0$; $M[2, d] = \text{mín}(8,\ 1 + M[1, a],\ 6 + M[1, c]) = \text{mín}(8, 6, 5) = 5$. La fila de c no cambia, porque su única arista sale directo a t.
- Columna $i = 3$: $M[3, a] = \text{mín}(2,\ -2 + M[2, b]) = -2$ y $M[3, d] = \text{mín}(5,\ 1 + M[2, a],\ 6 + M[2, c]) = \text{mín}(5, 3, 5) = 3$.
- Columna $i = 4$: $M[4, d] = \text{mín}(3,\ 1 + M[3, a],\ 6 + M[3, c]) = \text{mín}(3, -1, 5) = -1$; ningún otro valor cambia.

El camino mínimo de d a t se actualiza tres veces después de la primera columna: pasa de d-t (costo 8) a d-c-t (costo 5), a d-a-b-t (costo 3) y por último a d-a-b-c-t (costo -1), que usa las cuatro aristas permitidas.



i	0	1	2	3	4
t	0	0	0	0	0
a	∞	5	2	-2	-2
b	∞	4	0	0	0
c	∞	-1	-1	-1	-1
d	∞	8	5	3	-1

Figura 6.22. A la izquierda, el grafo del Ejemplo 6.7.5, con el camino mínimo de d a t en color. A la derecha, la tabla $M[i, v]$: cada fila es un nodo y cada columna una cantidad máxima de aristas. La celda marcada es $\text{OPT}(4, d) = -1$.

6.7.4 Mejoras al algoritmo

Un mejor análisis del tiempo de ejecución

Si el grafo G no tiene demasiadas aristas se puede dar un mejor análisis del tiempo. Un grafo dirigido con n nodos puede tener cerca de n^2 aristas, porque puede haber una

arista entre cada par de nodos, pero muchos grafos son mucho más ralos. Cuando la cantidad de aristas m es bastante menor que n^2 conviene, como ya se hizo varias veces, escribir el tiempo en términos de m y de n, para cuantificar la mejora en grafos con relativamente pocas aristas.

Proposición 6.7.6. El método SHORTEST-PATHS se puede implementar en tiempo $O(mn)$.

Demostración. Consideremos el cálculo de la entrada $M[i, v]$ según (6.8). Antes supusimos que calcular el mínimo interno podía costar $O(n)$, porque hay n nodos w posibles. Pero solo hace falta tomarlo sobre los nodos w a los que v tiene una arista, y el pseudocódigo con listas de adyacencia hace exactamente eso. Si n_v es la cantidad de aristas que salen de v, calcular $M[i, v]$ cuesta $O(n_v + 1)$. Hay que calcular una entrada por cada nodo v y cada índice $1 \leq i \leq n - 1$, lo que da una cota de

$$O\Big(n \sum_{v \in V} (n_v + 1)\Big).$$

Cada arista sale de exactamente un nodo de V, así que cada arista se cuenta exactamente una vez en $\sum_{v \in V} n_v$, y esa suma vale m. Reemplazando, el tiempo es $O(n(m + n))$, que es $O(mn)$ cuando todo nodo tiene al menos una arista (en particular, cuando todo nodo tiene camino a t). □

Mejorar la memoria

Con un cambio chico en la implementación también se puede bajar mucho la memoria. Un problema frecuente de los algoritmos de programación dinámica es el espacio que ocupa la tabla M; tal como está escrito, el arreglo de Bellman-Ford tiene tamaño n^2, y se puede bajar a $O(n)$. En lugar de guardar $M[i, v]$ para cada i, se usa y actualiza un único valor $M[v]$ por nodo, el costo del camino más barato de v a t encontrado hasta el momento. El algoritmo sigue haciendo las iteraciones $i = 1, 2, \ldots, n - 1$, pero ahora i es solo un contador: en cada iteración, y para cada nodo v, se hace la actualización

$$M[v] \leftarrow \text{mín}\Big(M[v], \min_{w\,:\,(v,w) \in E} \big(c_{vw} + M[w]\big)\Big).$$

Vale entonces el siguiente hecho.

Proposición 6.7.7. Durante todo el algoritmo, $M[v]$ es el costo de algún camino de v a t (o ∞), y después de i rondas de actualizaciones el valor $M[v]$ no es mayor que el costo del camino mínimo de v a t con a lo sumo i aristas.

Con esto, el Lema 6.7.1 muestra como antes que alcanza con $n - 1$ iteraciones. Como solo se guarda un arreglo indexado por los nodos, la memoria de trabajo es $O(n)$.

Encontrar los caminos

Una preocupación es si esta versión que ahorra memoria guarda suficiente información para recuperar los caminos mínimos. Acá se los puede recuperar con facilidad. Cada nodo v guarda el primer nodo, después de él mismo, de su camino hacia t; lo denotamos $\mathit{siguiente}(v)$. Ese valor se actualiza cada vez que $M[v]$ decrece estrictamente: cuando $M[v]$ se reemplaza por el mínimo $\min_w(c_{vw} + M[w])$, se fija $\mathit{siguiente}(v)$ en el nodo w que lo alcanza.

> **Estructuras de datos.** En la versión que ahorra memoria alcanzan dos arreglos indexados por los nodos: $M[1..n]$ de reales y $\mathit{siguiente}[1..n]$ de enteros, este último inicializado en un valor nulo (por ejemplo 0). El grafo sigue en listas de adyacencia de salida.

Sea P el "grafo de punteros" dirigido cuyos nodos son los de V y cuyas aristas son los pares $(v, \mathit{siguiente}(v))$. La observación principal es la siguiente.

Lema 6.7.8. Si el grafo de punteros P contiene un ciclo W, ese ciclo tiene costo negativo.

Demostración. Notemos primero que si en algún momento $\mathit{siguiente}(v) = w$, entonces $M[v] \geq c_{vw} + M[w]$. En efecto, los dos lados son iguales justo después de la actualización que fija $\mathit{siguiente}(v)$ en w; y como después $M[w]$ solo puede bajar, la igualdad puede convertirse en desigualdad.

Sean $v_1, v_2, \ldots, v_k$ los nodos del ciclo W en el grafo de punteros, y supongamos que (v_k, v_1) es la última de sus aristas en agregarse. Miremos los valores justo antes de esa última actualización. En ese momento vale $M[v_j] \geq c_{v_j v_{j+1}} + M[v_{j+1}]$ para todo $j = 1, \ldots, k-1$, y además $M[v_k] > c_{v_k v_1} + M[v_1]$, porque estamos por bajar $M[v_k]$ y cambiar $\mathit{siguiente}(v_k)$ a v_1. Sumando todas estas desigualdades, los valores $M[v_j]$ se cancelan y queda

$$0 \;>\; \sum_{j=1}^{k-1} c_{v_j v_{j+1}} + c_{v_k v_1},$$

es decir, W es un ciclo negativo. □

Si G no tiene ciclos negativos, el Lema 6.7.8 implica que el grafo de punteros nunca tiene un ciclo. Para un nodo v, consideremos el camino que se obtiene siguiendo las aristas de P: de v a $\mathit{siguiente}(v) = v_1$, de ahí a $\mathit{siguiente}(v_1) = v_2$, y así sucesivamente. Como el grafo de punteros no tiene ciclos y el destino t es el único nodo sin arista saliente (entre los que tienen M finito), ese camino tiene que llegar a t. Al terminar el algoritmo es, de hecho, un camino mínimo de v a t en G.

Proposición 6.7.9. Supongamos que G no tiene ciclos negativos y consideremos el grafo de punteros P al terminar el algoritmo. Para cada nodo v, el camino de v a t en P es un camino mínimo de v a t en G.

Demostración. Sea v un nodo y $w = \textit{siguiente}(v)$. Como el algoritmo terminó sin más cambios, $M[v] = c_{vw} + M[w]$. Además $M[t] = 0$. Sumando a lo largo del camino que traza el grafo de punteros, su costo es exactamente $M[v]$, que es el costo del camino mínimo de v a t. □

Terminar antes de tiempo

En la versión que ahorra memoria, el camino cuyo costo es $M[v]$ después de i iteraciones puede tener bastantes más de i aristas. Por ejemplo, si el grafo es un único camino de s a t y las actualizaciones se hacen en el orden inverso al de las aristas del camino, los valores finales se obtienen en una sola iteración. Esto no pasa siempre, así que no mejora el tiempo de peor caso, pero conviene aprovecharlo cuando ocurre. Para eso hace falta una señal que diga que ya es seguro terminar antes de la iteración $n - 1$.

La señal sale de una observación simple: si en alguna iteración completa i no cambia ningún valor $M[v]$, entonces ninguno va a cambiar nunca más, porque las iteraciones siguientes empiezan exactamente con los mismos valores. En ese momento se puede detener el algoritmo. No alcanza con que un valor particular $M[v]$ se mantenga igual: para terminar con seguridad, todos los valores tienen que quedar iguales durante una misma iteración.

Nota 6.7.10 (Detectar ciclos negativos). La misma idea sirve para el primero de los dos problemas del principio de la sección. Si el grafo no tiene ciclos negativos, después de $n - 1$ iteraciones ya no cambia ningún valor, por el Lema 6.7.1. Por lo tanto, si al hacer una iteración más, la número n, algún $M[v]$ todavía baja, el grafo tiene un ciclo negativo desde el que se puede llegar a t. Esta prueba también cuesta $O(mn)$.

6.8 Resumen de la técnica

Los seis problemas del capítulo muestran cuatro maneras generales de armar los subproblemas, más dos aplicaciones clásicas.

- Elección binaria, como en la planificación de intervalos con pesos: el último elemento está en la solución o no está, y cada opción lleva a un único subproblema más chico.
- Elección de múltiples vías, como en mínimos cuadrados segmentado: la última decisión tiene una cantidad polinómica de alternativas, y la recurrencia toma el mejor de todos esos subproblemas.
- Agregado de una variable adicional, como en el problema de la mochila: el subproblema natural no guarda suficiente información sobre el estado, y se le agrega un parámetro, en ese caso la capacidad restante.

- Descomposición en subintervalos, como en la estructura secundaria de ARN: los subproblemas son todos los intervalos $[i, j]$, y se resuelven en orden creciente de longitud.

El alineamiento de secuencias y los caminos de costo mínimo en grafos dirigidos sin ciclos de costo negativo son otros ejemplos de la misma técnica. En todos los casos vale la receta de la Nota 6.2.3: definir con palabras qué significa cada subproblema, escribir la recurrencia con sus casos base, llenar la tabla en un orden en el que cada entrada solo use entradas ya calculadas, y reconstruir la solución recorriendo la tabla hacia atrás.

Problema	**Subproblema**	**Opciones**	**Tiempo**	**Técnica**
Intervalos con pesos	$\text{OPT}(j)$	2	$O(n)$	elección binaria
Mínimos cuadrados segm.	$\text{OPT}(j)$	j	$O(n^2)$	múltiples vías
Mochila	$\text{OPT}(i, w)$	2	$O(nW)$	variable adicional
Estructura de ARN	$\text{OPT}(i, j)$	$O(n)$	$O(n^3)$	subintervalos
Alineamiento	$\text{OPT}(i, j)$	3	$O(mn)$	dos prefijos
Bellman-Ford	$\text{OPT}(i, v)$	$n_v + 1$	$O(mn)$	cantidad de aristas

La columna de tiempos no incluye pasos previos como ordenar las tareas y calcular los $p(j)$ en la planificación con pesos, que cuestan $O(n \log n)$, y en mínimos cuadrados segmentado supone el cálculo de los errores en $O(n^2)$ de la Nota 6.3.5.

Capítulo 7

Redes de Flujo

Uno de los problemas planteados en la Sección 1.8 fue el de emparejamiento bipartito (Definición 1.8.8). Recordemos el planteo. Un grafo bipartito $G = (V, E)$ es un grafo no dirigido cuyo conjunto de nodos se puede particionar como $V = X \cup Y$, de modo que toda arista $e \in E$ tiene un extremo en X y el otro en Y. Un emparejamiento en un grafo $G = (V, E)$ es un conjunto de aristas $M \subseteq E$ con la propiedad de que cada nodo aparece en a lo sumo una arista de M, y es un emparejamiento perfecto si cada nodo aparece en exactamente una arista de M.

Los emparejamientos en grafos bipartitos modelan situaciones en las que ciertos objetos se asignan a otros. Un ejemplo natural es aquel en el que los nodos de X representan trabajos, los nodos de Y representan máquinas, y una arista (x_i, y_j) indica que la máquina y_j es capaz de procesar el trabajo x_i. Un emparejamiento perfecto es entonces una forma de asignarle a cada trabajo una máquina que lo pueda procesar, de modo que cada máquina reciba exactamente un trabajo. De la misma manera se pueden representar muchas otras relaciones entre dos conjuntos de objetos, como la que hay entre clientes y tiendas, o entre casas y estaciones de bomberos cercanas.

Determinar el tamaño del emparejamiento más grande de un grafo bipartito es uno de los problemas más antiguos de los algoritmos combinatorios. Como caso particular, G tiene un emparejamiento perfecto si y solo si $|X| = |Y|$ y tiene un emparejamiento de tamaño $|X|$. El problema se resuelve en tiempo polinómico, pero para lograrlo hacen falta ideas bastante distintas de las técnicas vistas hasta ahora. En lugar de atacarlo directamente, conviene formular una clase general de problemas, los problemas de flujo en redes, que lo incluye como caso particular. Aunque la motivación original de estos problemas es el tráfico en una red, sus aplicaciones abarcan áreas sorprendentemente diversas y llevan a algoritmos eficientes para muchos otros problemas además del emparejamiento bipartito.

7.1 El problema de flujo máximo y el algoritmo de Ford-Fulkerson

7.1.1 El problema

Muchas veces se usan grafos para modelar redes de transporte, es decir, redes cuyas aristas llevan algún tipo de tráfico y cuyos nodos funcionan como intercambiadores

que pasan el tráfico de una arista a otra. Pensemos, por ejemplo, en un sistema de rutas en el que las aristas son rutas y los nodos son cruces; en una red de computadoras en la que las aristas son enlaces que transportan paquetes y los nodos son *switches*; o en una red de fluidos en la que las aristas son caños que llevan líquido y los nodos son las uniones entre caños. Los modelos de este tipo tienen varios ingredientes: capacidades en las aristas, que indican cuánto pueden transportar; nodos fuente, que generan tráfico; nodos destino (o sumideros), que absorben el tráfico que les llega; y el tráfico mismo, que se transmite a través de las aristas.

Llamaremos flujo a ese tráfico: una entidad abstracta que se genera en los nodos fuente, se transmite por las aristas y se absorbe en los nodos destino.

Definición 7.1.1 (Red de flujo)**.** Una red de flujo es una cuádrupla $G = (V, E, s, t, c)$ en la que

- (V, E) es un grafo dirigido;
- $s \in V$ es un nodo distinguido llamado fuente y $t \in V$ es otro nodo distinguido llamado destino;
- c asigna a cada arista $e \in E$ una capacidad $c(e) \geq 0$.

Los nodos distintos de s y de t se llaman nodos internos. Se supone además que ninguna arista entra a s, que ninguna arista sale de t, que todo nodo tiene al menos una arista incidente y que todas las capacidades son números enteros.

Estas suposiciones hacen que todo sea más limpio de pensar. Eliminan algunas situaciones patológicas, pero conservan esencialmente todos los aspectos que interesan del problema. La Figura 7.1 muestra una red de flujo con ocho nodos y quince aristas.

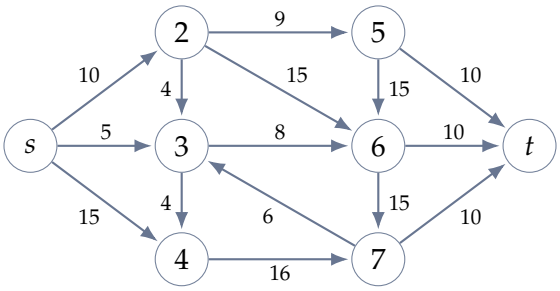


Figura 7.1. Una red de flujo con fuente s y destino t. El número junto a cada arista es su capacidad.

7.1.2 Cortes

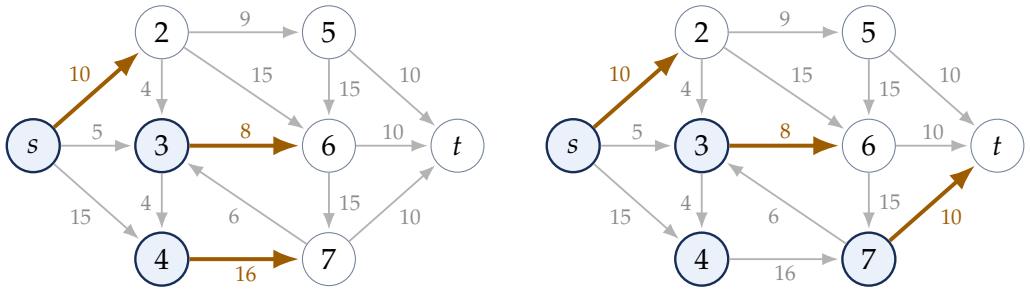
Antes de definir qué es un flujo, vale la pena mirar cómo la estructura de la red limita cuánto se puede transportar de s a t. El obstáculo básico es el siguiente. Si dividimos

los nodos del grafo en dos conjuntos A y B, con $s \in A$ y $t \in B$, intuitivamente todo lo que viaje de s a t tiene que cruzar de A a B en algún momento, y al hacerlo consume parte de la capacidad de las aristas que van de A a B. Cada una de esas divisiones pone entonces una cota a cuánto se puede transportar.

Definición 7.1.2 (Corte)**.** Un corte-st (o simplemente un corte) de una red de flujo es una partición (A, B) de V tal que $s \in A$ y $t \in B$. Su capacidad es la suma de las capacidades de las aristas que salen de A, es decir, de las que van de un nodo de A a un nodo de B:

$$\text{cap}(A, B) = \sum_{e \text{ sale de } A} c(e).$$

Las aristas que van de B a A no cuentan en la capacidad del corte. En la red de la Figura 7.1, el corte más simple es $A = \{s\}$, cuya capacidad es $10 + 5 + 15 = 30$. La Figura 7.2 muestra otros dos.



(a) $A = \{s, 3, 4\}$: capacidad $10 + 8 + 16 = 34$ (b) $A = \{s, 3, 4, 7\}$: capacidad $10 + 8 + 10 = 28$

Figura 7.2. Dos cortes de la red de la Figura 7.1. Los nodos sombreados forman el lado A y las aristas marcadas son las que salen de A. En (a) la arista $(7, 3)$ va de B a A y no se cuenta.

Definición 7.1.3 (Problema de corte mínimo)**.** Dada una red de flujo, encontrar un corte de capacidad mínima.

7.1.3 Flujos

Definición 7.1.4 (Flujo)**.** Un flujo-st (o simplemente un flujo) es una función $f : E \to \mathbb{R}^+$ que asigna a cada arista e la cantidad $f(e)$ de flujo que transporta, y que cumple las dos condiciones siguientes.

(I) Condición de capacidad: para cada $e \in E$, $0 \leq f(e) \leq c(e)$.

(II) Conservación de flujo: para cada nodo $v \in V - \{s, t\}$,

$$\sum_{e \text{ entra a } v} f(e) \;=\; \sum_{e \text{ sale de } v} f(e).$$

El valor de un flujo f es la cantidad de flujo que se genera en la fuente:

$$\text{val}(f) \;=\; \sum_{e \text{ sale de } s} f(e).$$

El flujo de una arista no puede superar su capacidad, y en cada nodo interno la cantidad que entra es igual a la que sale. La fuente no tiene aristas entrantes, pero puede tener flujo saliente: genera flujo. Simétricamente, el destino puede recibir flujo aunque no tenga aristas salientes. Este modelo representa un tráfico que atraviesa la red a ritmo constante, con una única cantidad $f(e)$ por arista; no modela tráfico en ráfagas, que fluctúa con el tiempo.

Para compactar la notación se escribe $f^{\text{out}}(v) = \sum_{e \text{ sale de } v} f(e)$ y $f^{\text{in}}(v) = \sum_{e \text{ entra a } v} f(e)$, y lo mismo para conjuntos de nodos: si $S \subseteq V$, $f^{\text{out}}(S)$ es la suma de $f(e)$ sobre las aristas que salen de S (van de un nodo de S a uno fuera de S) y $f^{\text{in}}(S)$ es la suma sobre las aristas que entran a S. Con esta notación la conservación se escribe $f^{\text{in}}(v) = f^{\text{out}}(v)$ para $v \neq s, t$, y el valor es $\text{val}(f) = f^{\text{out}}(s)$.

Ejemplo 7.1.5 (Un flujo de valor 25)**.** La Figura 7.3 muestra un flujo en la red de la Figura 7.1; cada arista lleva el rótulo $f(e)/c(e)$. Todas las aristas respetan su capacidad. En el nodo 6 entran 5 unidades por $(2,6)$, 0 por $(5,6)$ y 5 por $(3,6)$, y salen 10 por $(6,t)$ y 0 por $(6,7)$: entran 10 y salen 10. El mismo control en los demás nodos internos muestra que la conservación se cumple en todos. De s salen $10 + 5 + 10$ unidades, de modo que el valor del flujo es 25.

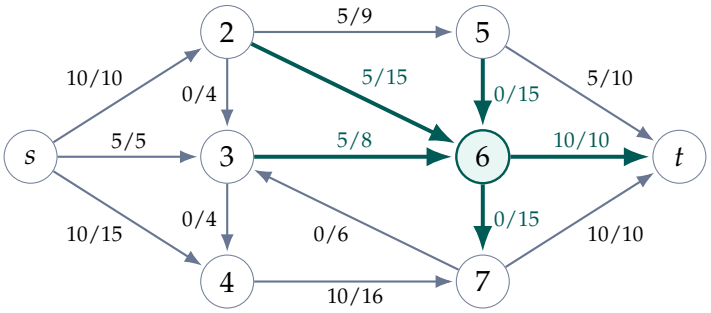


Figura 7.3. Un flujo de valor 25; cada rótulo es $f(e)/c(e)$. Se resaltan las aristas que entran y salen del nodo 6: entran $5 + 0 + 5 = 10$ unidades y salen $10 + 0 = 10$.

Definición 7.1.6 (Problema de flujo máximo)**.** Dada una red de flujo, encontrar un flujo de valor máximo.

El objetivo es organizar el tráfico de modo de aprovechar lo mejor posible la capacidad disponible. Los dos problemas están íntimamente relacionados: todo lo que va de s a t tiene que cruzar cada corte, así que la capacidad de cualquier corte limita el valor de cualquier flujo.

7.1.4 Diseño del algoritmo

¿Cómo encontrar un flujo máximo? Hace falta probar un poco para convencerse de que un enfoque como la programación dinámica no parece funcionar; al menos, no se conoce ningún algoritmo para el problema de flujo máximo que pueda verse como perteneciente de forma natural a ese paradigma. A falta de otras ideas, podemos volver a los enfoques greedy simples y ver dónde fallan.

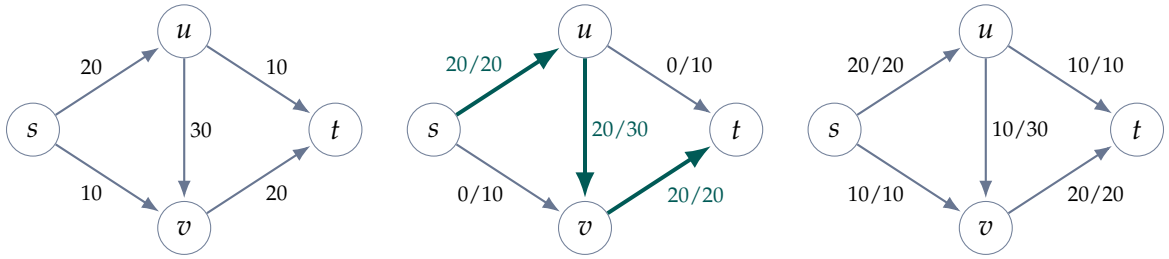
Un primer intento greedy

Empecemos con el flujo nulo, $f(e) = 0$ para toda arista e. Este flujo respeta las condiciones de capacidad y de conservación; el problema es que su valor es 0. Intentemos aumentarlo empujando flujo por un camino de s a t, hasta el límite que impongan las capacidades.

- Empezar con $f(e) = 0$ para toda arista $e \in E$.
- Buscar un camino P de s a t en el que toda arista e cumpla $f(e) < c(e)$.
- Si ese camino existe, aumentar el flujo a lo largo de P tanto como lo permita la arista de P con menos capacidad libre.
- Repetir hasta que no exista un camino así.

Aumentar el flujo sobre todas las aristas de un camino mantiene la conservación, porque cada vez que se aumenta el flujo de una arista que entra a un nodo interno también se aumenta, en la misma cantidad, el de una arista que sale de él.

Ejemplo 7.1.7 (El greedy se estanca)**.** En la red de cuatro nodos de la parte (a) de la Figura 7.4, el algoritmo greedy podría elegir el camino s, u, v, t, cuyas aristas tienen capacidades 20, 30 y 20, y poner 20 unidades en cada una. El flujo resultante, de valor 20, aparece en la parte (b). A partir de ahí no hay ningún camino de s a t en el que todas las aristas tengan capacidad libre: la arista (s, u) y la arista (v, t) están llenas, y todo camino de s a t usa una de las dos. Sin embargo, 20 no es el máximo: el flujo de la parte (c), de valor 30, pone 20 unidades en (s, u), 10 en (s, v), 10 en (u, v), 10 en (u, t) y 20 en (v, t).



(a) capacidades (b) greedy: valor 20 (c) óptimo: valor 30

Figura 7.4. (a) Una red de cuatro nodos. (b) El algoritmo greedy empuja 20 unidades por el camino s, u, v, t, resaltado, y queda sin caminos disponibles. (c) Un flujo de valor 30 en la misma red.

El problema del greedy es que, una vez que aumenta el flujo de una arista, nunca lo disminuye. Para pasar de (b) a (c) hay que hacer algo que el greedy no contempla. Se empujan 10 unidades por (s, v); eso deja demasiado flujo entrando a v. Entonces se deshacen 10 unidades del flujo de (u, v), lo que restablece la conservación en v pero deja demasiado poco flujo saliendo de u. Finalmente se empujan 10 unidades por (u, t), lo que restablece la conservación en u. El resultado es un flujo válido de valor 30.

Esta es una forma más general de empujar flujo: se puede empujar hacia adelante por aristas que tienen capacidad sobrante, y hacia atrás por aristas que ya llevan flujo, para desviarlo en otra dirección. Hace falta entonces algún mecanismo para revertir malas decisiones. El grafo residual es la forma sistemática de buscar operaciones de ida y vuelta como esta.

El grafo residual

Definición 7.1.8 (Grafo residual)**.** Sea G una red de flujo y f un flujo en G. El grafo residual de G respecto de f es $G_f = (V, E_f, s, t, c_f)$, con el mismo conjunto de nodos que G y las aristas siguientes.

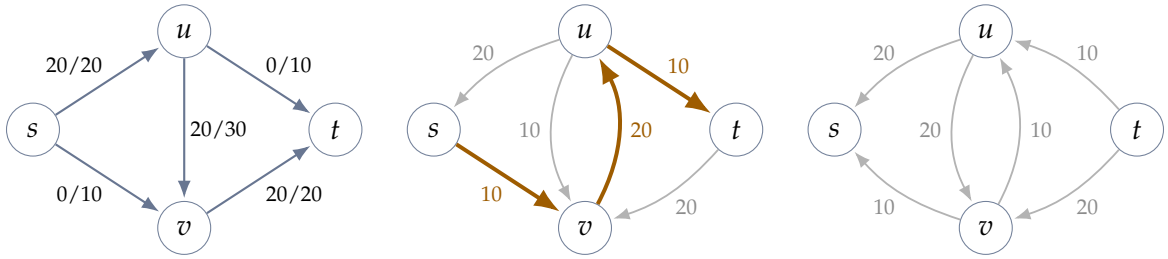
- Aristas directas. Para cada arista $e = (u, v)$ de G con $f(e) < c(e)$ quedan $c(e) - f(e)$ unidades de capacidad libre por las que se podría empujar más flujo hacia adelante. Se incluye en G_f la arista $e = (u, v)$ con capacidad $c(e) - f(e)$.
- Aristas inversas. Para cada arista $e = (u, v)$ de G con $f(e) > 0$ hay $f(e)$ unidades de flujo que se podrían deshacer empujando hacia atrás. Se incluye en G_f la arista inversa $e^R = (v, u)$, con los mismos extremos que e pero con el sentido invertido, y con capacidad $f(e)$.

La capacidad c_f de una arista de G_f se llama capacidad residual, para distinguirla de la capacidad de la arista correspondiente de G:

$$c_f(e) = \begin{cases} c(e) - f(e) & \text{si } e \in E, \\ f(e^R) & \text{si } e^R \in E. \end{cases}$$

Cada arista de G da lugar a una o dos aristas de G_f: si $0 < f(e) < c(e)$, aparecen en G_f tanto la arista directa como la inversa. Por lo tanto G_f tiene a lo sumo el doble de aristas que G. Por ejemplo, una arista (u, v) con capacidad 17 que lleva 6 unidades de flujo da lugar en G_f a una arista directa (u, v) de capacidad residual 11 y a una arista inversa (v, u) de capacidad residual 6.

La parte (b) de la Figura 7.5 muestra el grafo residual del flujo de valor 20 del Ejemplo 7.1.7. La arista (s, u) está llena, así que en G_f solo aparece su inversa (u, s), con capacidad 20; lo mismo pasa con (v, t). La arista (u, v) lleva 20 de sus 30 unidades y aparece dos veces: hacia adelante con capacidad 10 y hacia atrás con capacidad 20. Las aristas (s, v) y (u, t) están vacías y aparecen solo hacia adelante. En G_f sí hay un camino de s a t, que es justamente la operación de ida y vuelta descrita antes: s, v, u, t.



(a) flujo f en G (b) grafo residual G_f (c) G_f tras aumentar 10

Figura 7.5. (a) El flujo de valor 20 que encontró el greedy. (b) Su grafo residual, con la capacidad residual junto a cada arista; el camino s, v, u, t, resaltado, usa la arista (u, v) hacia atrás. (c) El grafo residual después de empujar 10 unidades por ese camino: ninguna arista sale de s.

Caminos de aumento

Hace falta precisar cómo se empuja flujo de s a t en G_f.

Definición 7.1.9 (Camino de aumento y cuello de botella)**.** Un camino de aumento respecto de un flujo f es un camino simple de s a t en el grafo residual G_f, es decir, un camino que no visita ningún nodo más de una vez. El cuello de botella de un camino de aumento P, que se escribe bottleneck(P, f), es la mínima capacidad residual de las aristas de P.

La operación AUGMENT empuja por P tanta cantidad como permite su cuello de botella. Por cada arista de P que es directa, aumenta el flujo en G; por cada arista de P que es inversa, disminuye el flujo de la arista original correspondiente.

```
AUGMENT(f, c, P)
1  b ← bottleneck(P, f)
2  for each arista e ∈ P do
3      if e ∈ E then    // arista directa
4          f(e) ← f(e) + b
5      else             // arista inversa: e^R ∈ E
6          f(e^R) ← f(e^R) − b
7      end if
8  end for
9  return f
```

> **Estructuras de datos.** El flujo se guarda en un arreglo $f[1..m]$ indexado por las aristas de G, y las capacidades en otro arreglo $c[1..m]$. Cada arista de G_f es un registro que guarda el índice de la arista de G de la que proviene y un booleano que dice si es directa o inversa; así la línea 3 se resuelve en $O(1)$. El camino P es una lista enlazada de esos registros, de modo que el cuello de botella de la línea 1 se calcula recorriéndola una vez.

Lo primero es verificar que el resultado de AUGMENT es efectivamente un flujo.

Proposición 7.1.10. Si f es un flujo en G y P es un camino de aumento, el resultado f' de AUGMENT(f, c, P) es un flujo en G.

Demostración. Hay que verificar las condiciones de capacidad y de conservación. Como f' solo difiere de f en aristas correspondientes a P, alcanza con verificar la capacidad en ellas. Sea (u, v) una arista de P y sea $b = \text{bottleneck}(P, f)$, que no es mayor que la capacidad residual de (u, v).

Si $e = (u, v)$ es una arista directa, su capacidad residual es $c(e) - f(e)$, así que

$$0 \leq f(e) \leq f'(e) = f(e) + b \leq f(e) + \big(c(e) - f(e)\big) = c(e).$$

Si (u, v) es la inversa de una arista $e = (v, u) \in E$, su capacidad residual es $f(e)$, así que

$$c(e) \geq f(e) \geq f'(e) = f(e) - b \geq f(e) - f(e) = 0.$$

En los dos casos se cumple la condición de capacidad.

Para la conservación, sea v un nodo interno de P. Como P es simple, P tiene exactamente una arista que entra a v y una que sale de v. Hay cuatro casos según cada una de ellas sea directa o inversa, y en todos la diferencia $f^{\text{in}}(v) - f^{\text{out}}(v)$ no cambia.

- Entra una directa y sale una directa: $f^{\text{in}}(v)$ y $f^{\text{out}}(v)$ aumentan en b.
- Entra una directa y sale una inversa: la arista inversa que sale de v proviene de una arista de G que entra a v y cuyo flujo baja en b. Así, $f^{\text{in}}(v)$ sube b por una arista y baja b por otra, y $f^{\text{out}}(v)$ no cambia.
- Entra una inversa y sale una directa: simétrico al anterior, con $f^{\text{out}}(v)$ bajando y subiendo b.
- Entra una inversa y sale una inversa: las dos aristas originales van en sentido contrario al camino; la que sale de v en G y la que entra a v en G bajan b cada una, de modo que $f^{\text{in}}(v)$ y $f^{\text{out}}(v)$ bajan b.

Como f cumplía la conservación en v, también la cumple f'. □

El algoritmo de Ford-Fulkerson

Con la operación de aumento a disposición, el algoritmo es muy simple: se parte del flujo nulo y se aumenta repetidamente por caminos de aumento mientras existan. Se llama así por L. R. Ford y D. R. Fulkerson, que lo desarrollaron en 1956.

FORD-FULKERSON(G)

1 **for each** arista $e \in E$ **do** $f(e) \leftarrow 0$
2 $G_f \leftarrow$ grafo residual respecto de f
3 while existe un camino P de s a t en G_f **do**
4 $\quad f \leftarrow$ **AUGMENT**(f, c, P)
5 $\quad$ actualizar G_f respecto del nuevo flujo f
6 **end while**
7 **return** f

> **Estructuras de datos.** G_f se representa con listas de adyacencia: para cada nodo v, una lista enlazada con las aristas residuales que salen de v, cada una con su capacidad residual y la referencia a la arista de G de la que proviene. La búsqueda de la línea 3 es un BFS (con una cola) o un DFS (con una pila) desde s que ignora las aristas de capacidad residual 0; mientras recorre guarda en un arreglo $padre[1..n]$ la arista residual por la que llegó a cada nodo, y P se reconstruye siguiendo ese arreglo desde t hasta s.

El algoritmo es realmente muy simple. Lo que no está nada claro es si su ciclo while termina, ni si el flujo que devuelve es máximo. Las respuestas a esas dos preguntas resultan ser bastante sutiles. Antes de responderlas, veamos una ejecución completa.

Ejemplo 7.1.11 (Una ejecución de Ford-Fulkerson)**.** Consideremos la red de la Figura 7.6, con seis nodos y nueve aristas. Una ejecución posible del algoritmo encuentra los cinco caminos de aumento de la tabla siguiente, en ese orden. En cada fila se indica el cuello de botella y el valor del flujo después de aumentar; las aristas usadas hacia atrás, es decir, las aristas inversas del grafo residual, están marcadas con una flecha hacia la izquierda.

Iteración	Camino de aumento	Cuello de botella	Valor
1	$s \to u \to y \to t$	8 (arista (u, y))	8
2	$s \to u \to v \to y \to t$	2 (aristas (s, u), (u, v) y (y, t))	10
3	$s \to v \to y \to x \to t$	6 (arista (y, x))	16
4	$s \to v \leftarrow u \to x \to t$	2 (arista inversa (v, u))	18
5	$s \to v \to y \leftarrow u \to x \to t$	1 (arista (v, y))	19

Los primeros tres caminos solo usan aristas directas, así que el greedy también los podría haber encontrado. Después de la tercera iteración el flujo vale 16, y en G ya no queda ningún camino de s a t con capacidad libre en todas sus aristas; el greedy se habría detenido ahí. El grafo residual en ese momento aparece en la Figura 7.7. En él, el camino de la cuarta iteración va de s a v, vuelve de v a u por la arista inversa de (u, v) —deshaciendo las 2 unidades que la segunda iteración había mandado por esa arista— y sigue por u, x, t. La quinta iteración hace algo parecido con la arista (u, y): desvía una de las 8 unidades que u mandaba a y hacia x, y la reemplaza en y por una unidad que llega desde v.

El flujo final, de valor 19, está en la parte (a) de la Figura 7.8. En su grafo residual, en la parte (b), no hay camino de s a t: desde s solo se llega a v, y desde v no sale ninguna arista residual. El algoritmo termina.

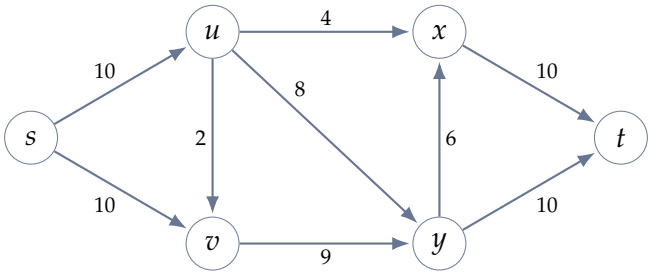
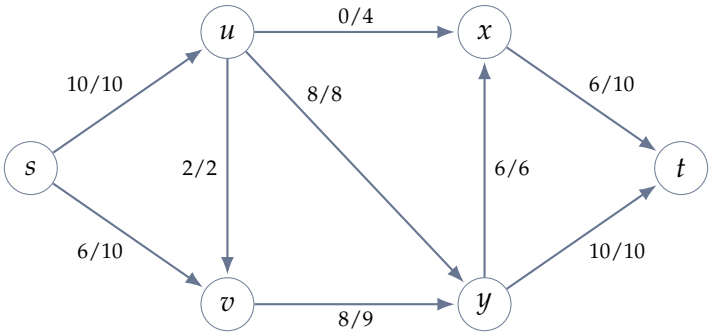
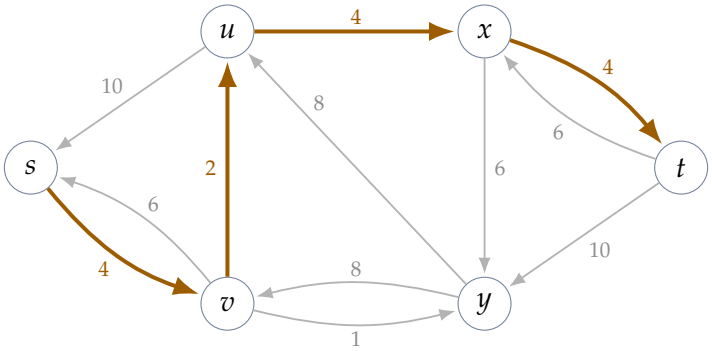


Figura 7.6. La red de flujo del Ejemplo 7.1.11, con las capacidades.

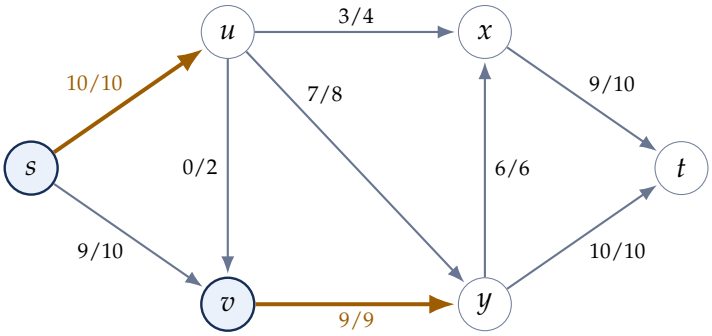


(a) flujo de valor 16

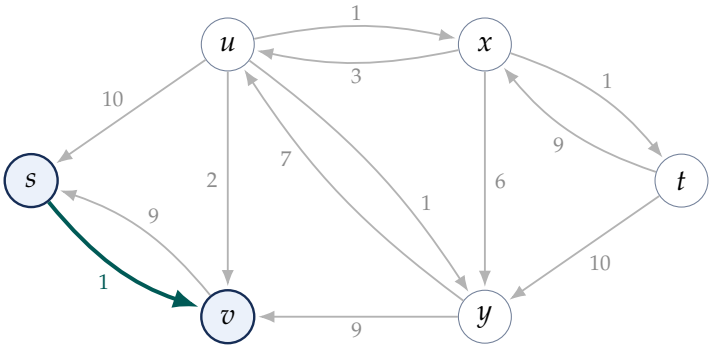


(b) grafo residual G_f

Figura 7.7. (a) El flujo después de las tres primeras iteraciones del Ejemplo 7.1.11; todo camino de s a t en G tiene una arista llena. (b) Su grafo residual. El camino de aumento s, v, u, x, t, resaltado, usa la arista inversa (v, u) y tiene cuello de botella 2.



(a) flujo de valor 19



(b) grafo residual final

Figura 7.8. (a) El flujo que devuelve Ford-Fulkerson en el Ejemplo 7.1.11. (b) Su grafo residual: desde s solo se alcanza v, así que no hay camino de aumento. Los nodos sombreados son los alcanzables desde s; en (a), las aristas resaltadas son las que van de esos nodos al resto, y sus capacidades suman $10 + 9 = 19$.

7.1.5 Análisis del algoritmo: terminación y tiempo de ejecución

Empezamos por algunas propiedades que el algoritmo mantiene y que se prueban por inducción en la cantidad de iteraciones del ciclo while, apoyándose en que todas las capacidades son enteras.

Proposición 7.1.12 (Propiedad de integridad)**.** En todo momento de la ejecución de Ford-Fulkerson, los valores $f(e)$ del flujo y las capacidades residuales de G_f son enteros.

Demostración. Antes de la primera iteración es claro: el flujo es nulo y las capacidades residuales son las capacidades de G. Supongamos que vale después de j iteraciones. Como todas las capacidades residuales de G_f son enteras, el cuello de botella del camino de aumento que se encuentra en la iteración $j + 1$ es entero. Entonces el nuevo flujo f' tiene valores enteros, y también las capacidades del nuevo grafo residual. □

Esta propiedad permite probar que el algoritmo termina. Como en otros puntos de

estas notas, buscamos una medida de progreso que implique la terminación. Primero mostramos que el valor del flujo crece estrictamente con cada aumento.

Proposición 7.1.13. Sea f un flujo en G y P un camino simple de s a t en G_f, y sea f' el resultado de AUGMENT(f, c, P). Entonces $\text{val}(f') = \text{val}(f) + \text{bottleneck}(P, f)$; y como $\text{bottleneck}(P, f) > 0$, vale $\text{val}(f') > \text{val}(f)$.

Demostración. La primera arista e de P sale de s en G_f, y como el camino es simple no vuelve a visitar s. Como en G no entran aristas a s, en G_f no hay aristas inversas que salgan de s, de modo que e es una arista directa. El aumento sube el flujo de e en $\text{bottleneck}(P, f)$ y no cambia el flujo de ninguna otra arista incidente a s. Por lo tanto el valor de f' supera al de f exactamente en $\text{bottleneck}(P, f)$. □

Falta acotar el valor máximo que puede tener un flujo. Una cota es la siguiente: si todas las aristas que salen de s se pudieran saturar, el valor del flujo sería $\sum_{e \text{ sale de } s} c(e)$. Llamemos C a esa suma. Por la condición de capacidad, $\text{val}(f) \leq C$ para todo flujo f. C puede ser una sobreestimación enorme del valor del flujo máximo, pero es una cota finita y fácil de enunciar.

Teorema 7.1.14. Si todas las capacidades de G son enteras y C es una cota superior del valor del flujo máximo, el algoritmo de Ford-Fulkerson termina en a lo sumo C iteraciones del ciclo while.

Demostración. Por la Proposición 7.1.13, el valor del flujo aumenta en cada iteración, y por la Proposición 7.1.12 aumenta al menos en 1. Como empieza en 0 y no puede superar C, el ciclo hace a lo sumo C iteraciones. □

Pasemos al tiempo de ejecución. Sea n la cantidad de nodos de G y m la cantidad de aristas. Como todo nodo tiene al menos una arista incidente, $m \geq n/2$, y por lo tanto $O(m + n) = O(m)$.

Teorema 7.1.15. Si todas las capacidades de G son enteras, el algoritmo de Ford-Fulkerson se puede implementar en tiempo $O(mC)$.

Demostración. Por el Teorema 7.1.14 hay a lo sumo C iteraciones; veamos cuánto trabajo lleva una. El grafo residual tiene a lo sumo $2m$ aristas, porque cada arista de G da lugar a lo sumo a dos. Se mantiene con listas de adyacencia como se indicó junto al pseudocódigo, y un camino de s a t en G_f se busca con BFS o DFS en tiempo $O(m + n) = O(m)$. AUGMENT tarda $O(n)$, porque P tiene a lo sumo $n - 1$ aristas. Con el nuevo flujo f', el nuevo grafo residual se arma en tiempo $O(m)$: para cada arista e de G se construyen las aristas directa e inversa que correspondan en $G_{f'}$. Cada iteración cuesta entonces $O(m)$, y el total es $O(mC)$. □

Nota 7.1.16 (Actualizar G_f sin reconstruirlo). Una versión algo más eficiente no reconstruye el grafo residual en cada iteración, sino que actualiza las listas de aristas de G_f dentro del propio AUGMENT: solo cambian las capacidades residuales de las aristas de P y de sus aristas opuestas, y solo pueden aparecer o desaparecer esas aristas. Para hacerlo en $O(1)$ por arista, cada arista residual guarda un puntero a su arista opuesta (la directa a la inversa y viceversa).

7.2 Flujos máximos y cortes mínimos en una red

El análisis del algoritmo de Ford-Fulkerson ocupa toda esta sección. En el camino no solo se aprende mucho sobre el algoritmo, sino que el análisis da bastante información sobre el problema de flujo máximo en sí.

7.2.1 Análisis del algoritmo: flujos y cortes

El objetivo es mostrar que el flujo que devuelve Ford-Fulkerson tiene el máximo valor posible entre todos los flujos de G. Para eso volvemos a la forma en que la estructura de la red acota el valor de un flujo. Ya tenemos una cota: el valor de cualquier flujo es a lo sumo $C = \sum_{e \text{ sale de } s} c(e)$. A veces esa cota sirve, pero a veces es muy débil. Los cortes de la Definición 7.1.2 dan una manera mucho más general de acotar el valor de un flujo, y el primer paso para verlo es un resultado bastante más fuerte que una simple cota.

Lema 7.2.1 (Valor de flujo)**.** Sea f un flujo y (A, B) un corte. El valor de f es igual al flujo neto que cruza el corte:

$$\text{val}(f) \;=\; f^{\text{out}}(A) - f^{\text{in}}(A) \;=\; \sum_{e \text{ sale de } A} f(e) \;-\; \sum_{e \text{ entra a } A} f(e).$$

El lema dice que mirando cuánto flujo manda f a través de un corte se mide exactamente el valor del flujo: es todo lo que sale de A menos lo que vuelve a entrar a A. Intuitivamente tiene sentido, aunque la prueba requiere manipular un poco las sumas.

Demostración. Por definición $\text{val}(f) = f^{\text{out}}(s)$. Como no entran aristas a la fuente, $f^{\text{in}}(s) = 0$, y se puede escribir $\text{val}(f) = f^{\text{out}}(s) - f^{\text{in}}(s)$. Todo nodo $v \in A$ distinto de s es interno, porque $t \in B$, así que por conservación $f^{\text{out}}(v) - f^{\text{in}}(v) = 0$. Entonces

$$\text{val}(f) \;=\; \sum_{v \in A} \big(f^{\text{out}}(v) - f^{\text{in}}(v)\big),$$

porque el único término no nulo de la suma es el de $v = s$.

Reescribamos la suma de la derecha arista por arista. Si una arista e tiene los dos extremos en A, $f(e)$ aparece una vez con signo $+$ (en el f^{out} de su cola) y una vez con

signo $-$ (en el f^{in} de su cabeza), y los dos términos se cancelan. Si e tiene solo la cola en A, $f(e)$ aparece una sola vez, con signo $+$. Si e tiene solo la cabeza en A, $f(e)$ aparece una sola vez, con signo $-$. Y si e no tiene ningún extremo en A, $f(e)$ no aparece. Por lo tanto

$$\sum_{v \in A} \left(f^{\text{out}}(v) - f^{\text{in}}(v)\right) = \sum_{e \text{ sale de } A} f(e) - \sum_{e \text{ entra a } A} f(e) = f^{\text{out}}(A) - f^{\text{in}}(A).$$

Juntando las dos igualdades se obtiene el lema. □

Cuando $A = \{s\}$ el lema es exactamente la definición del valor del flujo, porque no entran aristas a s.

Ejemplo 7.2.2 (El flujo neto a través de un corte)**.** Tomemos el flujo de valor 25 del Ejemplo 7.1.5 y el corte con $A = \{s, 2, 6\}$, que aparece en la Figura 7.9. Las aristas que salen de A son $(s,3)$, $(s,4)$, $(2,5)$, $(2,3)$, $(6,t)$ y $(6,7)$, y llevan en total $5+10+5+0+10+0 = 30$ unidades. Las que entran a A son $(5,6)$ y $(3,6)$, y llevan $0+5 = 5$. El flujo neto que cruza el corte es $30-5 = 25$, que es justamente el valor del flujo. Las aristas $(s,2)$ y $(2,6)$ tienen los dos extremos en A y no intervienen.

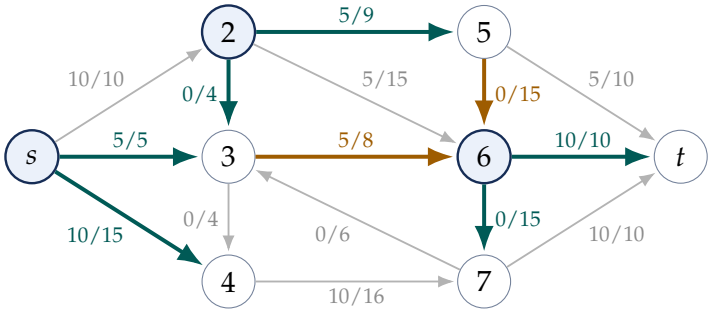


Figura 7.9. El flujo de valor 25 y el corte con $A = \{s, 2, 6\}$ (nodos sombreados). Las aristas que salen de A están en verde y llevan 30 unidades; las que entran a A están en naranja y llevan 5. La diferencia es 25.

Si (A, B) es un corte, las aristas que entran a B son exactamente las que salen de A, y las que salen de B son exactamente las que entran a A. Por lo tanto $f^{\text{out}}(A) = f^{\text{in}}(B)$ y $f^{\text{in}}(A) = f^{\text{out}}(B)$, y el lema se puede reformular así.

Corolario 7.2.3. Sea f un flujo y (A, B) un corte. Entonces $\text{val}(f) = f^{\text{in}}(B) - f^{\text{out}}(B)$.

Con $A = V - \{t\}$ y $B = \{t\}$ queda $\text{val}(f) = f^{\text{in}}(t) - f^{\text{out}}(t)$, y como no salen aristas de t, $\text{val}(f) = f^{\text{in}}(t)$. Es decir, el valor de un flujo se podría haber definido igual de bien desde el destino, como la cantidad de flujo que llega a t.

Una consecuencia muy útil del Lema 7.2.1 es la cota superior siguiente.

Teorema 7.2.4 (Dualidad débil)**.** Sea f un flujo y (A, B) un corte. Entonces $\text{val}(f) \leq \text{cap}(A, B)$.

Demostración.

$$\begin{aligned}
\text{val}(f) &= f^{\text{out}}(A) - f^{\text{in}}(A) && \text{(Lema 7.2.1)} \\
&\leq f^{\text{out}}(A) && \text{(porque } f^{\text{in}}(A) \geq 0\text{)} \\
&= \sum_{e \text{ sale de } A} f(e) \\
&\leq \sum_{e \text{ sale de } A} c(e) && \text{(condición de capacidad en cada término)} \\
&= \text{cap}(A, B).
\end{aligned}$$

□

En cierto sentido el teorema parece más débil que el lema, porque es solo una desigualdad. Pero va a ser extremadamente útil, porque su lado derecho no depende de ningún flujo en particular. Lo que dice es que el valor de todo flujo está acotado por la capacidad de todo corte. Si se exhibe un corte de G de capacidad c^*, se sabe de inmediato que no puede haber un flujo de valor mayor que c^*. Recíprocamente, si se exhibe un flujo de valor v^*, se sabe de inmediato que no puede haber un corte de capacidad menor que v^*. Por ejemplo, en la red de la Figura 7.1 el flujo de valor 25 y el corte de capacidad 28 de la Figura 7.2 dicen que el flujo máximo está entre 25 y 28.

Corolario 7.2.5 (Certificado de optimalidad)**.** Sea f un flujo y (A, B) un corte. Si $\text{val}(f) = \text{cap}(A, B)$, entonces f es un flujo de valor máximo y (A, B) es un corte de capacidad mínima.

Demostración. Para cualquier flujo f', por dualidad débil, $\text{val}(f') \leq \text{cap}(A, B) = \text{val}(f)$, así que f es máximo. Para cualquier corte (A', B'), también por dualidad débil, $\text{cap}(A', B') \geq \text{val}(f) = \text{cap}(A, B)$, así que (A, B) es mínimo. □

Ejemplo 7.2.6 (Un certificado)**.** La Figura 7.10 muestra un flujo de valor $10 + 5 + 13 = 28$ en la red de la Figura 7.1, junto con el corte $A = \{s, 3, 4, 7\}$ de capacidad $10 + 8 + 10 = 28$ que ya habíamos visto. Por el Corolario 7.2.5, el flujo es máximo y el corte es mínimo. Notemos que todas las aristas que salen de A están saturadas y que las dos que entran a A, $(2, 3)$ y $(6, 7)$, no llevan flujo; si no fuera así, el flujo neto a través del corte sería menor que su capacidad.

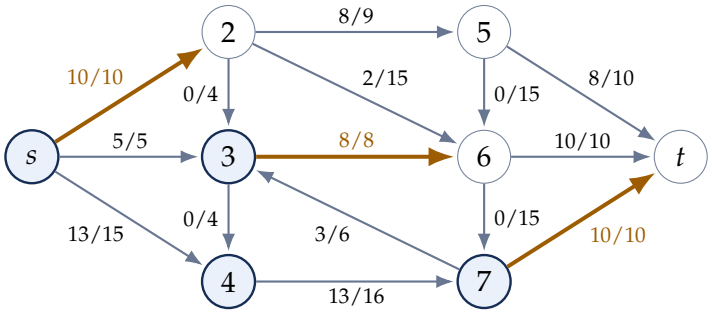


Figura 7.10. Un flujo de valor 28 y un corte de capacidad 28 (lado A sombreado, aristas que salen de A resaltadas). Como los dos valores coinciden, el flujo es máximo y el corte es mínimo.

7.2.2 Análisis del algoritmo: flujo máximo igual a corte mínimo

Sea $\bar{f}$ el flujo que devuelve Ford-Fulkerson. Queremos mostrar que tiene el máximo valor posible, y lo haremos con el método que acabamos de describir: exhibiendo un corte (A^*, B^*) con $\text{val}(\bar{f}) = \text{cap}(A^*, B^*)$. Eso prueba de inmediato que $\bar{f}$ es máximo y que (A^*, B^*) es un corte mínimo. El algoritmo termina cuando el flujo f no tiene camino de s a t en el grafo residual G_f, y resulta que esa es la única propiedad que hace falta para probar que es máximo.

Teorema 7.2.7 (Flujo máximo y corte mínimo)**.** Para todo flujo f las condiciones siguientes son equivalentes.

(I) Existe un corte (A, B) tal que $\text{cap}(A, B) = \text{val}(f)$.

(II) f es un flujo de valor máximo.

(III) No existe ningún camino de aumento respecto de f.

Demostración. Probamos las tres implicaciones en ciclo.

(i) $\Rightarrow$ (ii). Es el Corolario 7.2.5.

(ii) $\Rightarrow$ (iii). Por el contrarrecíproco: si existiera un camino de aumento P respecto de f, por la Proposición 7.1.13 el flujo AUGMENT(f, c, P) tendría valor estrictamente mayor que f, y f no sería máximo.

(iii) $\Rightarrow$ (i). Supongamos que no hay camino de s a t en G_f. Sea A el conjunto de nodos v para los que existe un camino de s a v en G_f, y sea $B = V - A$. Veamos primero que (A, B) es un corte. Claramente es una partición de V. La fuente pertenece a A, porque siempre hay un camino de s a s. Y $t \notin A$ porque no hay camino de s a t en G_f, así que $t \in B$.

Sea ahora $e = (u, v)$ una arista de G con $u \in A$ y $v \in B$, como en la Figura 7.11. Afirmamos que $f(e) = c(e)$. Si no fuera así, e sería una arista directa de G_f con capacidad residual positiva; como $u \in A$, hay un camino de s a u en G_f, y agregándole e se

obtendría un camino de s a v en G_f, lo que contradice $v \in B$.

Sea ahora $e' = (u', v')$ una arista de G con $u' \in B$ y $v' \in A$. Afirmamos que $f(e') = 0$. Si no fuera así, e' daría lugar a una arista inversa (v', u') en G_f; como $v' \in A$, hay un camino de s a v' en G_f, y agregándole esa arista inversa se obtendría un camino de s a u' en G_f, lo que contradice $u' \in B$.

Todas las aristas que salen de A están entonces completamente saturadas, y todas las que entran a A están vacías. Por el Lema 7.2.1,

$$\text{val}(f) = \sum_{e \text{ sale de } A} f(e) - \sum_{e \text{ entra a } A} f(e) = \sum_{e \text{ sale de } A} c(e) - 0 = \text{cap}(A, B). \quad \square$$

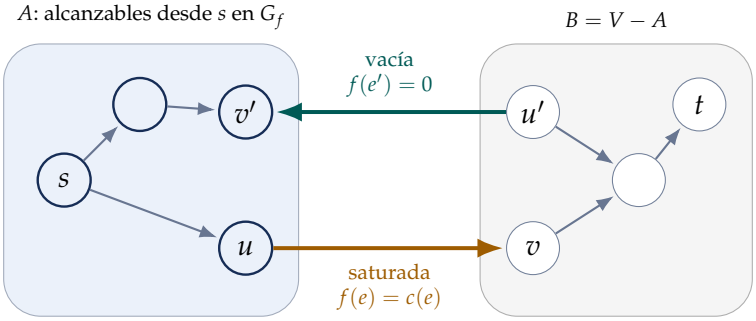


Figura 7.11. El corte (A, B) de la demostración del Teorema 7.2.7. Si una arista (u, v) que sale de A no estuviera saturada, o si una arista (u', v') que entra a A llevara flujo, desde s se alcanzaría un nodo de B en el grafo residual.

La demostración muestra, en retrospectiva, por qué los dos tipos de aristas residuales, directas e inversas, son cruciales: cada uno se encarga de uno de los dos términos de la fórmula del Lema 7.2.1.

Como Ford-Fulkerson termina precisamente cuando no hay camino de s a t en el grafo residual, la implicación (iii) $\Rightarrow$ (ii) da su corrección.

Corolario 7.2.8. El flujo que devuelve el algoritmo de Ford-Fulkerson es un flujo de valor máximo.

El algoritmo se extiende además, sin esfuerzo, para calcular un corte mínimo.

Proposición 7.2.9. Dado un flujo f de valor máximo, se puede calcular un corte de capacidad mínima en tiempo $O(m)$.

Demostración. Se sigue la construcción de la demostración de (iii) $\Rightarrow$ (i): se arma el grafo residual G_f, se hace un BFS o un DFS desde s para determinar el conjunto A de nodos alcanzables, y se devuelve el corte $(A, V - A)$. Como f es máximo, por (ii) $\Rightarrow$ (iii) no

hay camino de aumento, y por (iii) $\Rightarrow$ (i) el corte tiene capacidad igual a val(f); por el Corolario 7.2.5 es mínimo. □

En el Ejemplo 7.1.11, los nodos alcanzables desde s en el grafo residual final son s y v (Figura 7.8), y el corte $(\{s, v\}, \{u, x, y, t\})$ tiene capacidad $c(s, u) + c(v, y) = 10 + 9 = 19$, igual al valor del flujo. La arista (u, v) va de B a A y, como predice la demostración, no lleva flujo.

Nota 7.2.10 (Puede haber muchos cortes mínimos). Una red puede tener varios cortes de capacidad mínima; el procedimiento de la Proposición 7.2.9 encuentra uno en particular, el que tiene el lado A más chico posible. En la red de la Figura 7.4, por ejemplo, el flujo máximo vale 30 y tanto $A = \{s\}$ como $A = \{s, u, v\}$ son cortes de capacidad 30: el primero tiene capacidad $20 + 10$ y el segundo $c(u, t) + c(v, t) = 10 + 20$.

Como bonus, el análisis del algoritmo dio el siguiente hecho notable.

Teorema 7.2.11 (Teorema de flujo máximo y corte mínimo)**.** En toda red de flujo, el valor máximo de un flujo es igual a la capacidad mínima de un corte.

Demostración. Ford-Fulkerson termina y devuelve un flujo f sin caminos de aumento. Por el Teorema 7.2.7 existe un corte (A, B) con val$(f) =$ cap(A, B), y por el Corolario 7.2.5 f es un flujo máximo y (A, B) es un corte mínimo. □

Del mismo teorema se deduce el teorema del camino de aumento: un flujo f es de valor máximo si y solo si no existe ningún camino de aumento respecto de f. Es la equivalencia entre (ii) y (iii).

7.2.3 Flujos con valores enteros

Entre los muchos corolarios que surgen del análisis de Ford-Fulkerson hay otro extremadamente importante. Por la Proposición 7.1.12, el algoritmo mantiene en todo momento un flujo con valores enteros, y por el Corolario 7.2.8 termina con un flujo máximo.

Teorema 7.2.12 (Teorema de integridad)**.** Si todas las capacidades de la red son enteras, existe un flujo máximo f en el que todos los valores $f(e)$ son enteros.

El teorema no dice que todo flujo máximo tenga valores enteros, sino que alguno los tiene. En la red de la Figura 7.4, por ejemplo, cambiemos la capacidad de (u, t) de 10 a 20. El flujo máximo pasa a valer 30 con cualquier reparto que llene (s, u) y (s, v) y mande a t lo que llega: poniendo $f(u, v) = \frac{1}{2}$, $f(u, t) = 19{,}5$ y $f(v, t) = 10{,}5$ se obtiene un flujo máximo con valores no enteros. Aunque el enunciado no menciona el algoritmo, el enfoque algorítmico da probablemente la manera más fácil de demostrarlo.

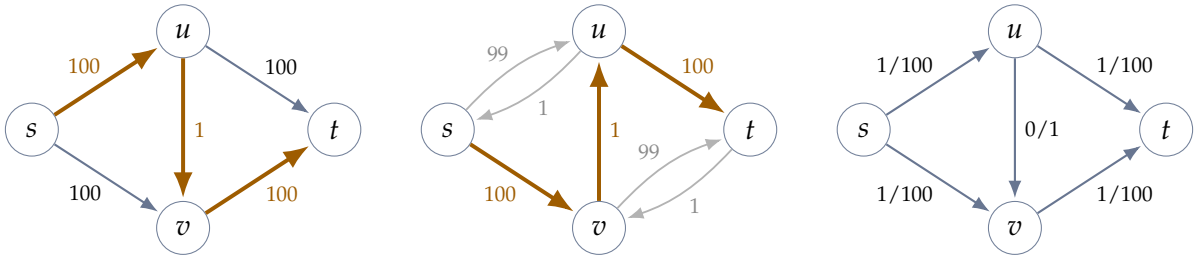
Nota 7.2.13 (¿Capacidades reales?). Admitir capacidades racionales no hace la situación más general: se multiplican todas por el mínimo común múltiplo de sus denominadores y se obtiene un problema equivalente con capacidades enteras. Con capacidades reales, en cambio, el argumento de terminación se cae: la Proposición 7.1.12 era lo que garantizaba que el valor del flujo sube al menos 1 por iteración. Con números reales el valor puede ir subiendo en incrementos cada vez más chicos, y de hecho, con elecciones patológicas de los caminos de aumento, Ford-Fulkerson con capacidades reales puede no terminar nunca. Sin embargo, el Teorema 7.2.11 sigue valiendo con capacidades reales. La implicación (iii) $\Rightarrow$ (i) solo usa que f no tiene camino de s a t en G_f, y para cualquier flujo de valor máximo eso es así; alcanza entonces con probar que en toda red existe un flujo máximo, lo cual se hace por un argumento de compacidad que no depende del algoritmo.

7.3 Elección de buenos caminos de aumento

Cualquier forma de elegir el camino de aumento aumenta el valor del flujo, y eso llevó a la cota de C aumentos, con $C = \sum_{e \text{ sale de } s} c(e)$. Cuando C no es muy grande la cota es razonable, pero cuando C es grande es muy débil.

Ejemplo 7.3.1 (Una elección patológica)**.** Tomemos la red de cuatro nodos de la Figura 7.4 con otras capacidades: (s,u), (s,v), (u,t) y (v,t) tienen capacidad 100, y (u,v) tiene capacidad 1, como en la parte (a) de la Figura 7.12. El flujo máximo vale 200, con $f(e) = 100$ en las cuatro aristas grandes y $f(u,v) = 0$, y se obtiene con dos aumentos, por los caminos s,u,t y s,v,t.

Veamos qué tan mal puede andar Ford-Fulkerson con elecciones patológicas. Supongamos que el primer camino de aumento es $P_1 = s,u,v,t$, con cuello de botella 1. Después de ese aumento $f(u,v) = 1$, así que la arista inversa (v,u) está en el grafo residual. Como segundo camino se elige $P_2 = s,v,u,t$, que usa esa arista inversa y también tiene cuello de botella 1. Después del segundo aumento $f(u,v) = 0$ otra vez y la arista (u,v) vuelve al grafo residual. Si se sigue alternando entre P_1 y P_2, cada aumento tiene cuello de botella 1 y hacen falta 200 aumentos para llegar al flujo de valor 200. Es exactamente la cota del Teorema 7.1.14, porque en este ejemplo $C = 200$.



(a) red y camino P_1 (b) G_f tras P_1, y camino P_2 (c) flujo tras P_1 y P_2

Figura 7.12. Dos iteraciones con una mala elección de caminos. (a) El camino $P_1 = s, u, v, t$ tiene cuello de botella 1. (b) En el grafo residual siguiente, el camino $P_2 = s, v, u, t$ usa la arista inversa (v, u) y también tiene cuello de botella 1. (c) Después de los dos aumentos el valor es apenas 2, y (u, v) vuelve a estar vacía: la situación se repite cien veces.

Nota 7.3.2 (Pseudopolinómico). Como el tiempo de Ford-Fulkerson es $O(mC)$, si C está acotado por un polinomio en m el tiempo es polinómico en el tamaño de la entrada. Pero en general solo se puede decir que el algoritmo es pseudopolinómico, en el sentido de la Nota 6.4.8: su tiempo depende del valor de C y no de la cantidad de bits que hacen falta para escribirlo. En el Ejemplo 7.3.1 las capacidades podrían haber sido 2^{100} en lugar de 100, y la entrada seguiría siendo chica, pero la ejecución patológica haría 2^{101} aumentos. La pregunta natural es si el método se puede implementar en tiempo estrictamente polinómico.

7.3.1 Diseño de un algoritmo más rápido

Se dedicó muchísimo trabajo a encontrar buenas formas de elegir los caminos de aumento para minimizar la cantidad de iteraciones. Acá nos concentramos en uno de los enfoques más naturales. Cada aumento incrementa el valor del flujo en el cuello de botella del camino elegido, así que si se eligen caminos de cuello de botella grande se avanza mucho. Una idea natural es elegir el camino de mayor cuello de botella; pero buscar ese camino puede hacer bastante más lenta cada iteración. Evitamos esa demora sin preocuparnos por elegir el camino de cuello de botella exactamente máximo: mantenemos un parámetro de escalamiento Δ y buscamos caminos cuyo cuello de botella sea al menos Δ.

Sea $G_f(\Delta)$ el subgrafo del grafo residual formado solo por las aristas de capacidad residual al menos Δ. Los valores de Δ que se usan son potencias de 2.

CAPACITY-SCALING(G)

```
1  for each arista e ∈ E do f(e) ← 0
2  Δ ← mayor potencia de 2 que no supera máx{c(e) : e sale de s}
3  while Δ ≥ 1 do
4      G_f(Δ) ← aristas de G_f con capacidad residual ≥ Δ
5      while existe un camino P de s a t en G_f(Δ) do
6          f ← AUGMENT(f, c, P)
7          actualizar G_f(Δ) respecto del nuevo flujo f
8      end while
9      Δ ← Δ/2
10 end while
11 return f
```

> **Estructuras de datos.** Las mismas que en Ford-Fulkerson. No hace falta construir $G_f(\Delta)$ como un grafo aparte: el BFS de la línea 5 recorre las listas de adyacencia de G_f y simplemente ignora las aristas con capacidad residual menor que Δ, que es un entero guardado en una variable.

Ejemplo 7.3.3 (Escalamiento en la red de seis nodos)**.** Ejecutemos CAPACITY-SCALING sobre la red de la Figura 7.6. La mayor capacidad que sale de s es 10, así que se empieza con $\Delta = 8$.

- Fase $\Delta = 8$. Solo cuentan las aristas de capacidad residual al menos 8: (s,u), (s,v), (u,y), (v,y), (x,t) y (y,t). Hay dos caminos, s,u,y,t y s,v,y,t, pero comparten la arista (y,t). Supongamos que el BFS encuentra primero s,v,y,t, con cuello de botella 9. Después del aumento, (y,t) queda con capacidad residual 1 y el otro camino desaparece de $G_f(8)$. Valor: 9.
- Fase $\Delta = 4$. Se encuentra s,u,y,x,t, con cuello de botella 6 (lo limita (y,x)), y después s,u,x,t, con cuello de botella 4. Con eso (s,u) queda saturada, y la única arista residual que sale de s es (s,v), con capacidad residual 1. Valor: 19.
- Fases $\Delta = 2$ y $\Delta = 1$. Desde s solo se llega a v, y de v no sale ninguna arista residual, así que no se encuentra ningún camino.

El resultado es un flujo de valor 19, igual que en el Ejemplo 7.1.11, pero con tres aumentos en lugar de cinco, y distinto de aquel: ahora $f(u,y) = 6$, $f(u,x) = 4$, $f(y,t) = 9$ y $f(x,t) = 10$. Esto ilustra que el flujo máximo no tiene por qué ser único, aunque su valor sí lo es.

7.3.2 Análisis del algoritmo

Lo primero es observar que el algoritmo de escalamiento no es más que una implementación de Ford-Fulkerson. Los ciclos nuevos, el valor Δ y el grafo residual restringido $G_f(\Delta)$ solo sirven para guiar la elección del camino de aumento, con el objetivo de usar aristas de capacidad residual grande mientras se pueda. Por lo tanto todas las propiedades probadas para Ford-Fulkerson valen también para esta versión: el flujo se mantiene entero, y con él las capacidades residuales.

Proposición 7.3.4. Si las capacidades son enteras, durante toda la ejecución de CAPACITY-SCALING el flujo y las capacidades residuales son enteros. En particular, cuando $\Delta = 1$ el grafo $G_f(\Delta)$ coincide con G_f, y por lo tanto cuando el algoritmo termina el flujo f es de valor máximo.

Pasemos al tiempo de ejecución. Una iteración del ciclo exterior, con un valor fijo de Δ, se llama fase de escalamiento Δ. Acotar la cantidad de fases en términos de $C = \sum_{e \text{ sale de } s} c(e)$ es fácil: el valor inicial de Δ es a lo sumo C, se divide por 2 en cada fase y nunca baja de 1.

Lema 7.3.5. El ciclo exterior se repite a lo sumo $1 + \lceil \log_2 C \rceil$ veces.

Lo más difícil es acotar la cantidad de aumentos de cada fase. La idea es que se usan caminos que aumentan mucho el flujo, así que no debería haber demasiados aumentos. Como en la fase Δ solo se usan aristas de capacidad residual al menos Δ, la Proposición 7.1.13 da lo siguiente.

Lema 7.3.6. Durante la fase de escalamiento Δ, cada aumento incrementa el valor del flujo en al menos Δ.

La observación clave es que al terminar la fase Δ el flujo no puede estar demasiado lejos del máximo.

Lema 7.3.7. Sea f el flujo al terminar la fase de escalamiento Δ. Existe un corte (A, B) con $\text{cap}(A, B) \leq \text{val}(f) + m\Delta$, donde m es la cantidad de aristas de G. En consecuencia, el flujo máximo vale a lo sumo $\text{val}(f) + m\Delta$.

Demostración. La prueba es análoga a la de (iii) $\Rightarrow$ (i) en el Teorema 7.2.7. Sea A el conjunto de nodos v para los que hay un camino de s a v en $G_f(\Delta)$, y $B = V - A$. (A, B) es un corte, porque si t estuviera en A la fase no habría terminado.

Sea $e = (u, v)$ una arista de G con $u \in A$ y $v \in B$. Afirmamos que $c(e) < f(e) + \Delta$. Si no, e sería una arista directa de $G_f(\Delta)$, y agregándola a un camino de s a u en $G_f(\Delta)$

se obtendría uno de s a v, contradiciendo $v \in B$. Del mismo modo, si $e' = (u', v')$ es una arista con $u' \in B$ y $v' \in A$, entonces $f(e') < \Delta$: si fuera $f(e') \geq \Delta$, la arista inversa (v', u') estaría en $G_f(\Delta)$ y desde s se llegaría a u'.

Las aristas que salen de A están entonces casi saturadas y las que entran a A están casi vacías. Por el Lema 7.2.1,

$$\begin{aligned} \text{val}(f) &= \sum_{e \text{ sale de } A} f(e) \; - \sum_{e \text{ entra a } A} f(e) \\ &\geq \sum_{e \text{ sale de } A} \big(c(e) - \Delta\big) \; - \sum_{e \text{ entra a } A} \Delta \\ &= \sum_{e \text{ sale de } A} c(e) \; - \sum_{e \text{ sale de } A} \Delta \; - \sum_{e \text{ entra a } A} \Delta \\ &\geq \text{cap}(A, B) - m\Delta, \end{aligned}$$

donde la primera desigualdad usa las cotas sobre las aristas que cruzan el corte y la segunda usa que el grafo tiene m aristas en total. La segunda afirmación del lema sale de la dualidad débil aplicada a este corte. □

Lema 7.3.8. La cantidad de aumentos de una fase de escalamiento es a lo sumo $2m$.

Demostración. En la primera fase es claro: cada arista que sale de s se puede usar en a lo sumo un aumento de esa fase, porque su capacidad es menor que 2Δ y cada aumento le quita al menos Δ de capacidad residual. Consideremos una fase posterior Δ, y sea f_p el flujo al terminar la fase anterior, que usó el parámetro $\Delta' = 2\Delta$. Por el Lema 7.3.7, el flujo máximo f^* cumple $\text{val}(f^*) \leq \text{val}(f_p) + m\Delta' = \text{val}(f_p) + 2m\Delta$. En la fase Δ cada aumento incrementa el flujo en al menos Δ (Lema 7.3.6), así que puede haber a lo sumo $2m$ aumentos. □

Un aumento cuesta $O(m)$, incluyendo el tiempo de armar el grafo y buscar el camino. Hay a lo sumo $1 + \lceil \log_2 C \rceil$ fases y a lo sumo $2m$ aumentos por fase, así que vale lo siguiente.

Teorema 7.3.9. En un grafo con m aristas y capacidades enteras, CAPACITY-SCALING encuentra un flujo máximo con a lo sumo $2m(1 + \lceil \log_2 C \rceil)$ aumentos, y se puede implementar en tiempo $O(m^2 \log C)$.

Cuando C es grande, esta cota es mucho mejor que la cota $O(mC)$ de una implementación arbitraria de Ford-Fulkerson. En el Ejemplo 7.3.1, con capacidades 2^{100}, el algoritmo genérico podría tardar un tiempo proporcional a 2^{100}, mientras que el de escalamiento tarda un tiempo proporcional a $\log_2(2^{100}) = 100$. Una forma de ver la

diferencia es que el algoritmo genérico requiere tiempo proporcional a la magnitud de las capacidades, mientras que el de escalamiento requiere tiempo proporcional a la cantidad de bits que hacen falta para escribirlas. El algoritmo de escalamiento es entonces polinómico en el tamaño de la entrada, es decir, en la cantidad de aristas y en la representación numérica de las capacidades, y cumple el objetivo habitual de conseguir un algoritmo de tiempo polinómico.

7.3.3 Extensiones: algoritmos fuertemente polinómicos

¿Se puede pedir algo cualitativamente mejor? La red del Ejemplo 7.3.1 tiene cuatro nodos y cinco aristas; estaría bien que la cantidad de iteraciones fuera polinómica en 4 y 5, sin depender para nada de las capacidades. Un algoritmo cuya cantidad de operaciones es polinómica solo en $|V|$ y $|E|$, y que trabaja con números de una cantidad polinómica de bits, se llama fuertemente polinómico.

Hay una implementación simple y natural de Ford-Fulkerson que consigue una cota de ese tipo: en cada iteración se elige el camino de aumento con la menor cantidad de aristas, lo que se logra buscando el camino con BFS. Dinitz, e independientemente Edmonds y Karp, probaron que con esa elección el algoritmo termina en $O(mn)$ iteraciones, de modo que el tiempo total es $O(m^2n)$; fueron los primeros algoritmos polinómicos para el problema de flujo máximo. Desde entonces se dedicó muchísimo trabajo a mejorar los tiempos de ejecución, y hay algoritmos que logran $O(mn \log n)$, $O(n^3)$, y $O\big(\text{mín}(n^{2/3}, m^{1/2})\, m \log n \log U\big)$, esta última suponiendo capacidades enteras de valor a lo sumo U.

7.4 Una primera aplicación: el problema de emparejamiento bipartito

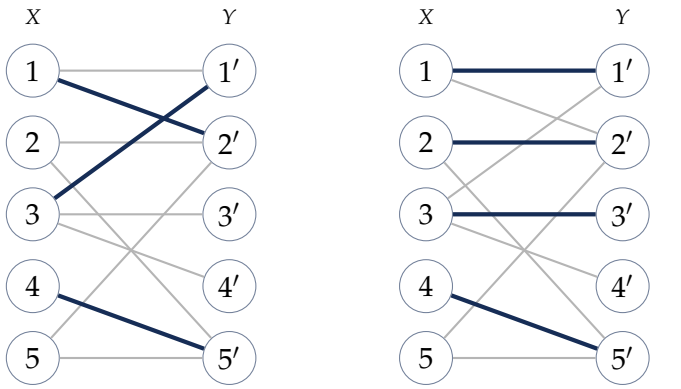
Con algoritmos eficientes para el problema de flujo máximo a disposición, pasamos a desarrollar aplicaciones de los flujos máximos y los cortes mínimos. La primera es el problema de emparejamiento bipartito recordado al comienzo del capítulo, que era una de las motivaciones originales.

7.4.1 El problema

Un grafo bipartito $G = (V, E)$ es un grafo no dirigido cuyo conjunto de nodos se puede particionar como $V = X \cup Y$ de modo que toda arista $e \in E$ tenga un extremo en X y el otro en Y. En las diapositivas del curso los dos lados se llaman L y R, por izquierda y derecha. Un emparejamiento M en G es un subconjunto de aristas $M \subseteq E$ tal que cada nodo aparece en a lo sumo una arista de M.

Definición 7.4.1 (Problema de emparejamiento bipartito)**.** Dado un grafo bipartito $G = (X \cup Y, E)$, encontrar un emparejamiento de cardinalidad máxima.

Ejemplo 7.4.2 (Un grafo bipartito)**.** El grafo de la Figura 7.13 tiene $X = \{1, 2, 3, 4, 5\}$, $Y = \{1', 2', 3', 4', 5'\}$ y diez aristas. El conjunto $\{1\text{-}2', 3\text{-}1', 4\text{-}5'\}$ de la parte (a) es un emparejamiento de tamaño 3. No es máximo: el de la parte (b), $\{1\text{-}1', 2\text{-}2', 3\text{-}3', 4\text{-}5'\}$, tiene tamaño 4.



(a) emparejamiento de tamaño 3 (b) emparejamiento de tamaño 4

Figura 7.13. El grafo bipartito del Ejemplo 7.4.2. Las aristas gruesas forman un emparejamiento.

7.4.2 Diseño del algoritmo

El grafo que define un problema de emparejamiento es no dirigido, mientras que las redes de flujo son dirigidas; aun así, no es difícil usar un algoritmo de flujo máximo para encontrar un emparejamiento máximo.

Definición 7.4.3 (Red asociada a un grafo bipartito)**.** Dado un grafo bipartito $G = (X \cup Y, E)$, su red de flujo asociada $G' = (X \cup Y \cup \{s, t\}, E')$ se construye así.

- Se dirigen todas las aristas de G de X hacia Y.
- Se agrega una fuente s y una arista (s, x) para cada $x \in X$.
- Se agrega un destino t y una arista (y, t) para cada $y \in Y$.
- Todas las aristas de G' tienen capacidad 1.

La Figura 7.14 muestra la red asociada al grafo del Ejemplo 7.4.2. Se calcula un flujo máximo de s a t en G'; el análisis muestra que su valor es el tamaño del emparejamiento máximo de G y que el emparejamiento se puede leer directamente del flujo.

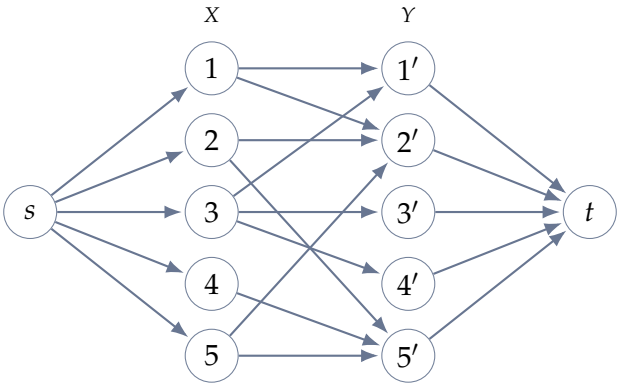


Figura 7.14. La red G' asociada al grafo del Ejemplo 7.4.2. Todas las aristas tienen capacidad 1.

7.4.3 Análisis del algoritmo

El análisis se basa en mostrar que los flujos enteros de G' codifican emparejamientos de G de manera bastante transparente. Supongamos primero que hay en G un emparejamiento de k aristas $(x_{i_1}, y_{i_1}), \ldots, (x_{i_k}, y_{i_k})$. Consideremos el flujo f que manda una unidad por cada camino de la forma s, x_{i_j}, y_{i_j}, t; es decir, $f(e) = 1$ para cada arista de uno de esos caminos y $f(e) = 0$ para las demás. Como los caminos no comparten nodos de X ni de Y, a cada nodo interno le entra a lo sumo una unidad y le sale la misma cantidad, así que f es un flujo de valor k.

Recíprocamente, supongamos que hay en G' un flujo f' de valor k. Por el Teorema 7.2.12 existe un flujo f con valores enteros y valor al menos k; y como todas las capacidades son 1, $f(e)$ vale 0 o 1 en cada arista. Tomando f máximo, sea M' el conjunto de aristas de la forma (x, y) con $f(x, y) = 1$. Hay tres hechos simples sobre M'.

Lema 7.4.4. M' tiene $\text{val}(f)$ aristas.

Demostración. Consideremos el corte (A, B) de G' con $A = \{s\} \cup X$. Por el Lema 7.2.1, el valor del flujo es el total que sale de A menos el total que entra a A. El primer término es exactamente $|M'|$, porque las aristas que salen de A y llevan flujo son las de M', y cada una lleva exactamente una unidad. El segundo término es 0, porque no entran aristas a A. Por lo tanto $|M'| = \text{val}(f)$. □

Lema 7.4.5. Cada nodo de X es la cola de a lo sumo una arista de M', y cada nodo de Y es la cabeza de a lo sumo una arista de M'.

Demostración. Supongamos que $x \in X$ fuera la cola de al menos dos aristas de M'. Como el flujo es entero, de x saldrían al menos dos unidades, y por conservación entrarían al menos dos. Eso es imposible, porque a x solo entra la arista (s, x), de capacidad 1. El razonamiento para $y \in Y$ es simétrico: de y solo sale la arista (y, t), de capacidad 1. □

Mirando M' como un conjunto de aristas del grafo bipartito original, los dos lemas dicen que es un emparejamiento de tamaño $\text{val}(f)$. En resumen, vale lo siguiente.

Teorema 7.4.6. El tamaño del emparejamiento máximo de G es igual al valor del flujo máximo de G'. Además, si f es un flujo máximo entero de G', las aristas de X a Y que llevan flujo forman un emparejamiento máximo de G.

Notemos la forma crucial en que interviene el teorema de integridad: hacía falta saber que hay un flujo máximo en G' que solo toma los valores 0 y 1. Un flujo fraccionario, que mande por ejemplo media unidad por cada una de dos aristas que salen de x, no se corresponde con ningún emparejamiento.

Nota 7.4.7 (Capacidad infinita en las aristas de X a Y). En las diapositivas del curso las aristas de X a Y reciben capacidad infinita en lugar de 1. El flujo máximo es el mismo, porque a cada x le entra a lo sumo una unidad y por lo tanto ninguna arista que sale de x puede llevar más de una. La ventaja de la versión con capacidad infinita aparece al mirar cortes mínimos: un corte de capacidad finita no puede tener aristas de X a Y que vayan de A a B.

Tiempo de ejecución

Sea $n = |X| = |Y|$ y sea m la cantidad de aristas de G. Supondremos que todo nodo del problema original tiene al menos una arista incidente, así que $m \geq n/2$. El tiempo para calcular un emparejamiento máximo está dominado por el de calcular un flujo máximo entero en G', porque convertirlo en un emparejamiento de G es inmediato. En este problema de flujo $C = \sum_{e \text{ sale de } s} c(e) = |X| = n$, porque s tiene una arista de capacidad 1 a cada nodo de X. Por el Teorema 7.1.15, se obtiene lo siguiente.

Teorema 7.4.8. El algoritmo de Ford-Fulkerson encuentra un emparejamiento máximo en un grafo bipartito en tiempo $O(mn)$.

Es interesante que si se usaran las cotas mejores de la Sección 7.3, como $O(m^2 \log C)$, se obtendrían tiempos peores para este problema, como $O(m^2 \log n)$. No hay contradicción: esas cotas están pensadas para ser buenas en todas las instancias, incluso cuando C es muy grande respecto de m y n. En el emparejamiento bipartito $C = n$, y el costo de la sofisticación extra no se justifica. Existen igualmente algoritmos específicos más rápidos: Hopcroft y Karp (1973) resolvieron el problema en tiempo $O(m\sqrt{n})$, y Mądry (2013) en tiempo $\tilde{O}(m^{10/7})$.

Nota 7.4.9 (Emparejamiento en grafos no bipartitos). Si el grafo no es bipartito, el problema de encontrar un emparejamiento de cardinalidad máxima sigue siendo polinómico, pero la estructura es bastante más complicada y la reducción a flujo no funciona. El algoritmo clásico es el algoritmo Blossom de Edmonds (1965), de tiempo $O(n^4)$; el mejor conocido tarda $O(m\sqrt{n})$ (Micali y Vazirani). El trabajo de Edmonds tiene además importancia histórica, porque en él se propone por primera vez identificar los algoritmos eficientes con los de tiempo polinómico.

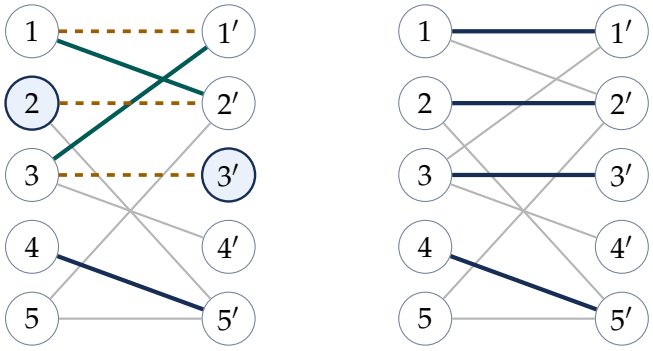
Caminos alternantes

Vale la pena ver qué significan los caminos de aumento en la red G'. Tomemos el emparejamiento $M = \{1\text{-}2', 3\text{-}1', 4\text{-}5'\}$ de la parte (a) de la Figura 7.13, y sea f el flujo correspondiente en G'. Como M no es máximo, f no es un flujo máximo y hay un camino de aumento en el grafo residual G'_f. Uno de ellos es

$$s,\ 2,\ 2',\ 1,\ 1',\ 3,\ 3',\ t,$$

que aparece en la parte (a) de la Figura 7.15. Las aristas $(1, 2')$ y $(3, 1')$ se usan hacia atrás, y todas las demás hacia adelante. Todo camino de aumento tiene que alternar entre aristas usadas hacia atrás y hacia adelante, porque todas las aristas de G' entre X e Y van de X a Y: después de llegar a un nodo de Y por una arista hacia adelante, la única forma de seguir (salvo ir a t) es volver a X por una arista hacia atrás, que es una arista del emparejamiento. Por eso, en el contexto de los emparejamientos, los caminos de aumento se llaman también caminos alternantes.

El efecto del aumento es sacar del emparejamiento las aristas usadas hacia atrás y reemplazarlas por las usadas hacia adelante. Como el camino va de s a t, tiene una arista hacia adelante más que hacia atrás, y el emparejamiento crece en uno. En el ejemplo se sacan 1-2′ y 3-1′ y se agregan 2-2′, 1-1′ y 3-3′, y se obtiene el emparejamiento de tamaño 4 de la parte (b).



(a) M y el camino alternante de 2 a $3'$

(b) emparejamiento después del aumento

Figura 7.15. (a) El emparejamiento M de tamaño 3: la arista 4-$5'$ en azul y, en verde, las dos aristas de M que el camino alternante recorre hacia atrás. Las aristas punteadas son las que recorre hacia adelante. El camino empieza en el nodo libre 2 y termina en el nodo libre $3'$, ambos sombreados. (b) El emparejamiento de tamaño 4 que resulta de intercambiar las aristas del camino.

7.4.4 Extensiones: la estructura de los grafos bipartitos sin emparejamiento perfecto

Algorítmicamente ya sabemos encontrar emparejamientos perfectos: se busca un emparejamiento máximo con el algoritmo anterior y se mira si es perfecto. Hagamos ahora una pregunta algo menos algorítmica. No todos los grafos bipartitos tienen emparejamiento perfecto. ¿Cómo es un grafo bipartito sin emparejamiento perfecto? ¿Hay una forma fácil de convencer a alguien de que un grafo no tiene emparejamiento perfecto, sin que tenga que revisar la traza entera de una ejecución del algoritmo? Sería bueno que el algoritmo, al concluir que no hay emparejamiento perfecto, pudiera producir un certificado corto de ese hecho.

Una forma de pensarlo es la siguiente. G tiene emparejamiento perfecto si y solo si el flujo máximo de G' vale n, y por el Teorema 7.2.11, si el flujo máximo vale menos de n hay un corte de capacidad menor que n. En cierto modo, ese corte ya es un certificado. Pero querríamos un certificado con un significado natural en términos del grafo original G.

Definición 7.4.10 (Vecindario)**.** Si S es un conjunto de nodos, $N(S)$ denota el conjunto de nodos adyacentes a algún nodo de S. En el libro este conjunto se escribe $\Gamma(S)$.

Por ejemplo, si dos nodos $x_1, x_2 \in X$ tienen una sola arista incidente cada uno, y el otro extremo de esas dos aristas es el mismo nodo y, claramente no hay emparejamiento perfecto: x_1 y x_2 tendrían que emparejarse los dos con y. En general, si el grafo tiene un emparejamiento perfecto, cada nodo de un conjunto $S \subseteq X$ tiene que emparejarse con

un nodo distinto de $N(S)$, así que $N(S)$ tiene que ser al menos tan grande como S.

Proposición 7.4.11. Si un grafo bipartito $G = (X \cup Y, E)$ tiene un emparejamiento perfecto, entonces $|N(S)| \geq |S|$ para todo $S \subseteq X$.

Esto sugiere un tipo de certificado de que un grafo no tiene emparejamiento perfecto: un conjunto $S \subseteq X$ con $|N(S)| < |S|$. En el grafo del Ejemplo 7.4.2, el conjunto $S = \{2, 4, 5\}$ tiene vecindario $N(S) = \{2', 5'\}$, como muestra la Figura 7.16; tres nodos de X solo pueden emparejarse con dos de Y, así que no hay emparejamiento perfecto, y de hecho a lo sumo 2 de los nodos de S quedan emparejados, lo que explica por qué el máximo es 4.

¿Vale la recíproca? ¿Siempre que no hay emparejamiento perfecto existe un conjunto S así que lo demuestre? La respuesta es sí, agregando la condición obvia $|X| = |Y|$, sin la cual no puede haber emparejamiento perfecto. Este resultado se conoce como teorema de Hall, aunque varias personas descubrieron versiones de él de manera independiente, quizás primero König, a principios del siglo XX. La demostración da además una forma de encontrar ese conjunto en tiempo polinómico.

Teorema 7.4.12 (Teorema de Hall)**.** Sea $G = (X \cup Y, E)$ un grafo bipartito con $|X| = |Y|$. Entonces G tiene un emparejamiento perfecto si y solo si $|N(S)| \geq |S|$ para todo $S \subseteq X$. Además, un emparejamiento perfecto o un conjunto $S \subseteq X$ con $|N(S)| < |S|$ se puede encontrar en tiempo $O(mn)$.

Demostración. La implicación directa es la Proposición 7.4.11. Para la recíproca supongamos que G no tiene emparejamiento perfecto, y sea $n = |X| = |Y|$. Usamos la red G' con capacidad infinita en las aristas de X a Y (Nota 7.4.7), cuyo flujo máximo coincide con el emparejamiento máximo de G. Como no hay emparejamiento perfecto, el flujo máximo es menor que n, y por el Teorema 7.2.11 hay un corte (A, B) de G' con $\text{cap}(A, B) < n$.

Definamos $X_A = X \cap A$, $X_B = X \cap B$ e $Y_A = Y \cap A$. Las aristas que salen de A son de tres tipos: aristas (s, x) con $x \in X_B$, que aportan $|X_B|$; aristas (y, t) con $y \in Y_A$, que aportan $|Y_A|$; y aristas de X_A a $Y \cap B$, que tienen capacidad infinita. Como la capacidad del corte es finita, no hay aristas del tercer tipo, es decir, $N(X_A) \subseteq Y_A$, y además

$$\text{cap}(A, B) = |X_B| + |Y_A|.$$

Entonces

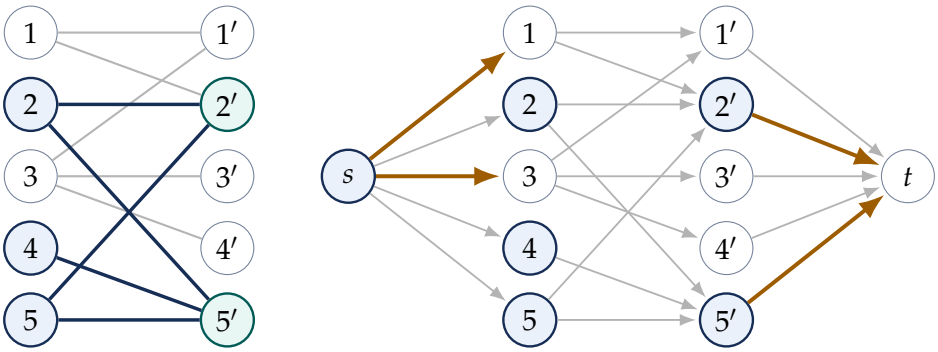
$$|N(X_A)| \leq |Y_A| = \text{cap}(A, B) - |X_B| < n - |X_B| = |X_A|,$$

y $S = X_A$ es el conjunto buscado.

Para el tiempo: se calcula un flujo máximo de G' con Ford-Fulkerson en tiempo $O(mn)$. Si vale n, da un emparejamiento perfecto. Si no, la Proposición 7.2.9 da en

tiempo $O(m)$ un corte mínimo (A, B), cuya capacidad es finita porque, por ejemplo, $A = \{s\}$ tiene capacidad n; y $S = X \cap A$ cumple la condición. □

Ejemplo 7.4.13 (El certificado que devuelve el corte mínimo)**.** En el grafo del Ejemplo 7.4.2, tomemos el flujo máximo correspondiente al emparejamiento {1-1′, 2-2′, 3-3′, 4-5′}. En el grafo residual, desde s solo sale la arista hacia el nodo libre 5. Desde 5 se llega a $2'$ y a $5'$ por aristas hacia adelante; desde $2'$ se vuelve a 2 por la arista inversa de $(2, 2')$, y desde $5'$ se vuelve a 4 por la inversa de $(4, 5')$; desde 2 y 4 no se alcanza ningún nodo nuevo, y las aristas $(2', t)$ y $(5', t)$ están saturadas. El conjunto de nodos alcanzables es $A = \{s, 2, 4, 5, 2', 5'\}$, que aparece en la Figura 7.16. El corte tiene capacidad $|X_B| + |Y_A| = |\{1, 3\}| + |\{2', 5'\}| = 4$, y el certificado es $S = X_A = \{2, 4, 5\}$, con $N(S) = \{2', 5'\}$.



(a) $S = \{2, 4, 5\}$ y $N(S) = \{2', 5'\}$

(b) corte mínimo de G': A sombreado, capacidad $2 + 2 = 4$

Figura 7.16. (a) Un conjunto $S \subseteq X$ con $|N(S)| < |S|$, que certifica que el grafo del Ejemplo 7.4.2 no tiene emparejamiento perfecto. (b) El corte mínimo del Ejemplo 7.4.13: las aristas que salen de A son $(s, 1)$, $(s, 3)$, $(2', t)$ y $(5', t)$, y ninguna arista de X a Y cruza de A a B.

Nota 7.4.14 (La demostración del libro)*.* El libro hace la misma demostración con capacidad 1 en las aristas de X a Y. En ese caso un corte mínimo sí puede tener aristas de X_A a $Y \cap B$, y el libro primero lo modifica: si $y \in N(X_A)$ está en B, se pasa y a A. Eso agrega al corte la arista (y, t), de capacidad 1, pero saca al menos una arista (x, y) con $x \in X_A$, que antes cruzaba el corte; la capacidad no aumenta. Repitiendo, se llega a un corte mínimo con $N(X_A) \subseteq Y_A$, y desde ahí la cuenta es la misma.

Grafos bipartitos regulares

Una aplicación linda del enfoque por flujos es la siguiente. A una hackatón asisten n estudiantes de una facultad y n de otra. Cada estudiante de la primera es amigo de exactamente $k > 0$ estudiantes de la segunda, y cada estudiante de la segunda es amigo de exactamente k de la primera; la amistad es mutua. ¿Se puede organizar la hackatón de modo que cada estudiante programe en pareja con un amigo distinto de la otra facultad? En términos matemáticos, un grafo es k-regular si todos sus nodos tienen grado k, y la pregunta es si todo grafo bipartito k-regular tiene un emparejamiento perfecto.

Teorema 7.4.15. Todo grafo bipartito k-regular $G = (X \cup Y, E)$, con $k > 0$, tiene un emparejamiento perfecto.

Demostración. Contando las aristas desde cada lado, $k|X| = |E| = k|Y|$, así que $|X| = |Y| = n$. Consideremos en la red asociada G' la función f que vale 1 en las aristas (s, x) y (y, t), y $1/k$ en cada arista de X a Y. Cumple la capacidad, y también la conservación: a cada $x \in X$ le entra 1 y le salen k aristas con $1/k$ cada una, y a cada $y \in Y$ le entran k aristas con $1/k$ y le sale 1. Entonces f es un flujo de valor n, y el flujo máximo vale al menos n; como tampoco puede superar $C = n$, vale exactamente n. Por el Teorema 7.4.6, el emparejamiento máximo tiene tamaño n, es decir, es perfecto. □

El flujo f de la demostración no es entero y no corresponde a ningún emparejamiento; lo único que se usa de él es su valor. El teorema de integridad garantiza que también existe un flujo entero con ese valor, y de ese se lee el emparejamiento.

7.5 Caminos disjuntos en grafos dirigidos y no dirigidos

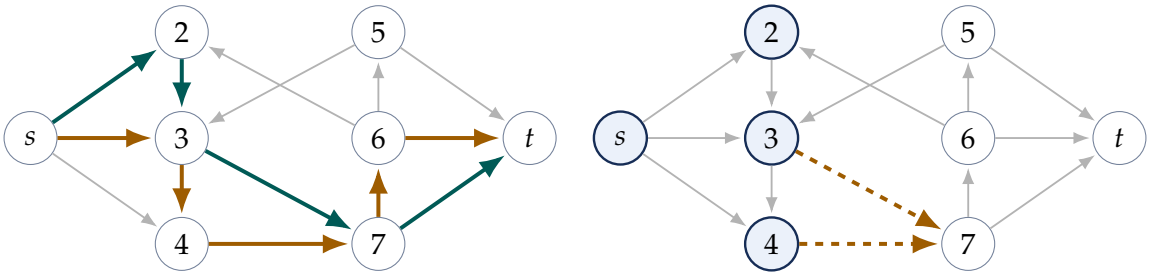
El emparejamiento bipartito usaba flujos enteros con valores 0 y 1 para elegir aristas. La misma idea sirve para elegir caminos.

7.5.1 El problema

Definición 7.5.1 (Caminos disjuntos en aristas)**.** Un conjunto de caminos es disjunto en aristas si ninguna arista pertenece a más de uno de ellos (los caminos sí pueden compartir nodos). Dado un grafo dirigido $G = (V, E)$ con dos nodos distinguidos s y t, el problema de los caminos disjuntos consiste en encontrar la mayor cantidad posible de caminos de s a t disjuntos en aristas.

Un ejemplo típico son las redes de comunicación: si los caminos no comparten enlaces, la conexión entre s y t sobrevive a la caída de cualquier enlace individual mientras quede al menos uno de los caminos.

Ejemplo 7.5.2 (Dos caminos disjuntos)**.** En el grafo dirigido de la Figura 7.17, los caminos $s, 2, 3, 7, t$ y $s, 3, 4, 7, 6, t$ son disjuntos en aristas, aunque los dos pasan por los nodos 3 y 7. No hay tres caminos disjuntos: todo camino de s a t tiene que usar alguna de las aristas $(3, 7)$ o $(4, 7)$, porque son las únicas que salen del conjunto $\{s, 2, 3, 4\}$.



(a) dos caminos disjuntos

(b) dos aristas que desconectan t de s

Figura 7.17. (a) Dos caminos de s a t disjuntos en aristas, en verde y en naranja. (b) Las aristas $(3, 7)$ y $(4, 7)$, punteadas, son las únicas que salen del conjunto sombreado; si se eliminan, no queda ningún camino de s a t.

7.5.2 Diseño y análisis del algoritmo

Tanto el problema de flujo máximo como el de caminos disjuntos piden encontrar algo que va de s a t. La conexión se hace asignándole capacidad 1 a cada arista de G y calculando un flujo máximo.

Teorema 7.5.3. En la red que se obtiene de G poniendo capacidad 1 en cada arista, el valor del flujo máximo es igual a la cantidad máxima de caminos de s a t disjuntos en aristas.

Demostración. ($\leq$ del número de caminos al flujo.) Supongamos que hay k caminos $P_1, \ldots, P_k$ de s a t disjuntos en aristas. Definamos $f(e) = 1$ si e pertenece a alguno de los caminos y $f(e) = 0$ si no. Como ninguna arista está en dos caminos, $f(e) \leq 1$. En cada nodo interno, cada vez que un camino entra por una arista sale por otra, así que se cumple la conservación. Entonces f es un flujo, y como cada camino usa exactamente una arista que sale de s (se puede suponer que los caminos no vuelven a s, quitando ciclos), su valor es k.

($\geq$ del flujo al número de caminos.) Supongamos que el flujo máximo vale k. Por el teorema de integridad hay un flujo f de valor k con valores 0 o 1. Tomemos una arista (s, u) con $f(s, u) = 1$. Si $u = t$, ya tenemos un camino. Si no, por conservación hay una arista (u, v) con $f(u, v) = 1$, y se sigue así, eligiendo siempre una arista que todavía no se usó, hasta llegar a t; esto termina porque hay finitas aristas y la conservación

garantiza que en cada nodo interno al que se entra por una arista no usada todavía queda una arista de salida no usada. El recorrido puede repetir nodos, pero si contiene un ciclo se lo quita y se obtiene un camino simple. Luego se ponen en 0 los valores del flujo en las aristas del recorrido: lo que queda sigue siendo un flujo con valores 0 y 1, de valor $k-1$. Repitiendo k veces se obtienen k caminos disjuntos en aristas. □

El procedimiento de la segunda parte se llama descomposición del flujo, y se puede implementar en tiempo $O(mn)$. Como el valor del flujo máximo es a lo sumo la cantidad de aristas que salen de s, que es a lo sumo n, Ford-Fulkerson resuelve el problema de los caminos disjuntos en tiempo $O(mn)$.

7.5.3 El teorema de Menger

Hay una pregunta dual. ¿Cuántas aristas hay que eliminar para que no quede ningún camino de s a t? Un conjunto de aristas $F \subseteq E$ desconecta t de s si todo camino de s a t usa al menos una arista de F; el problema de conectividad de la red consiste en encontrar el mínimo tamaño de un conjunto así. En el Ejemplo 7.5.2, el conjunto $F = \{(3,7),(4,7)\}$ desconecta t de s.

Teorema 7.5.4 (Teorema de Menger)**.** En todo grafo dirigido con nodos s y t, la cantidad máxima de caminos de s a t disjuntos en aristas es igual a la cantidad mínima de aristas cuya eliminación desconecta t de s.

Demostración. ($\leq$.) Supongamos que eliminar $F \subseteq E$ desconecta t de s, con $|F| = k$. Cada camino de s a t usa al menos una arista de F, y dos caminos disjuntos en aristas no pueden usar la misma, así que hay a lo sumo k caminos disjuntos.

($\geq$.) Supongamos que la cantidad máxima de caminos disjuntos es k. Por el Teorema 7.5.3, el flujo máximo con capacidades 1 vale k, y por el Teorema 7.2.11 hay un corte (A,B) de capacidad k. Sea F el conjunto de aristas que van de A a B; como todas tienen capacidad 1, $|F| = k$. Todo camino de s a t empieza en A y termina en B, así que usa alguna arista de F, y por lo tanto F desconecta t de s. □

7.5.4 Caminos disjuntos en grafos no dirigidos

En un grafo no dirigido $G = (V,E)$ el problema es el mismo: encontrar la mayor cantidad de caminos entre s y t que no compartan aristas. En la versión no dirigida del grafo de la Figura 7.17 hay tres caminos disjuntos, por ejemplo $s,2,6,5,t$, $s,3,7,6,t$ y $s,4,7,t$ (en la versión dirigida el primero no es un camino, porque la arista $(6,2)$ apunta al revés). La reducción natural reemplaza cada arista no dirigida $\{u,v\}$ por dos aristas dirigidas antiparalelas (u,v) y (v,u), las dos de capacidad 1.

Hay que tener un cuidado: dos caminos P_1 y P_2 pueden ser disjuntos en aristas en el grafo dirigido y no serlo en el no dirigido, si P_1 usa (u,v) y P_2 usa su antiparalela (v,u). Esto se resuelve con el lema siguiente.

Lema 7.5.5. En toda red de flujo existe un flujo máximo f tal que, para cada par de aristas antiparalelas e y e', $f(e) = 0$ o $f(e') = 0$. Además, si las capacidades son enteras, ese flujo se puede elegir entero.

Demostración. Partimos de un flujo máximo, entero si las capacidades lo son, y hacemos inducción en la cantidad de pares de aristas antiparalelas que llevan flujo en los dos sentidos. Si $f(e) > 0$ y $f(e') > 0$ para un par así, sea $\delta = \text{mín}\{f(e), f(e')\}$ y reemplacemos $f(e)$ por $f(e) - \delta$ y $f(e')$ por $f(e') - \delta$. En cada extremo del par el flujo que entra y el que sale bajan los dos en δ, así que se mantiene la conservación; las capacidades se siguen respetando, el valor, medido como flujo neto que sale de s, no cambia, el flujo sigue siendo entero, y hay un par menos con flujo en los dos sentidos. □

Con este lema, la misma demostración del Teorema 7.5.3 funciona para grafos no dirigidos: se toma un flujo máximo entero sin flujo en los dos sentidos de ningún par, y los caminos que se extraen no usan una misma arista no dirigida dos veces. También valen varias versiones del teorema de Menger.

Teorema 7.5.6 (Variantes del teorema de Menger)**.**

(a) En un grafo no dirigido con nodos s y t, la cantidad máxima de caminos entre s y t disjuntos en aristas es igual a la cantidad mínima de aristas cuya eliminación desconecta s de t.

(b) En un grafo no dirigido con nodos no adyacentes s y t, la cantidad máxima de caminos entre s y t internamente disjuntos (que no comparten ningún nodo salvo s y t) es igual a la cantidad mínima de nodos internos cuya eliminación desconecta s de t.

(c) Lo mismo vale en un grafo dirigido con nodos no adyacentes s y t, para caminos de s a t internamente disjuntos.

Las versiones con nodos se reducen a la de aristas con un truco habitual: cada nodo interno v se reemplaza por dos nodos v_{in} y v_{out} unidos por una arista $(v_{\text{in}}, v_{\text{out}})$ de capacidad 1; las aristas que entraban a v ahora entran a v_{in} y las que salían de v ahora salen de v_{out}. Así, a lo sumo una unidad de flujo puede atravesar cada nodo.

7.6 Extensiones del problema de flujo máximo

Buena parte del poder del problema de flujo máximo está en que muchos problemas que a primera vista no tienen nada que ver con él se pueden reducir a él. En esta sección se generaliza el modelo en dos direcciones.

7.6.1 Varias fuentes y varios destinos

Supongamos que el grafo dirigido tiene varios nodos fuente $s_1, \ldots, s_p$ y varios nodos destino $t_1, \ldots, t_q$, y queremos el máximo flujo que se puede mandar desde el conjunto de fuentes al conjunto de destinos. Se resuelve agregando una superfuente s con una arista (s, s_i) de capacidad infinita a cada fuente original, y un superdestino t con una arista (t_j, t) de capacidad infinita desde cada destino original. Los flujos de la red original se corresponden uno a uno con los flujos de la red nueva: el flujo de (s, s_i) es lo que genera la fuente s_i y el de (t_j, t) es lo que absorbe el destino t_j.

7.6.2 Circulaciones con demandas

Un modelo más interesante es aquel en el que cada fuente tiene una oferta fija que tiene que despachar, y cada destino una demanda fija que tiene que recibir. Pensemos en una red de distribución de un producto: hay fábricas con cierta producción y comercios con cierto consumo, y la pregunta es si se puede enviar el producto por la red respetando las capacidades de las rutas de modo que cada comercio reciba exactamente lo que necesita.

Definición 7.6.1 (Circulación con demandas)**.** Sea $G = (V, E)$ un grafo dirigido con capacidades $c(e) \geq 0$ en las aristas, y supongamos que cada nodo v tiene una demanda $d(v)$, que puede ser positiva, negativa o nula. Un nodo con $d(v) > 0$ es un nodo de demanda, que quiere recibir $d(v)$ unidades más de las que envía; uno con $d(v) < 0$ es un nodo de oferta, que quiere enviar $-d(v)$ unidades más de las que recibe; y uno con $d(v) = 0$ es un nodo de transbordo. Una circulación es una función $f : E \to \mathbb{R}^+$ que cumple:

(I) capacidad: $0 \leq f(e) \leq c(e)$ para toda $e \in E$;

(II) conservación con demandas: $f^{\text{in}}(v) - f^{\text{out}}(v) = d(v)$ para todo $v \in V$.

El problema de circulación consiste en decidir si existe una circulación.

Ahora no hay una única fuente ni un único destino, sino que cualquier nodo puede tener oferta o demanda, y no se maximiza nada: solo se pregunta si existe una circulación. Hay una condición necesaria simple.

Proposición 7.6.2. Si existe una circulación, entonces $\sum_{v \in V} d(v) = 0$.

Demostración. Sumando la condición de conservación sobre todos los nodos, $\sum_v d(v) = \sum_v \big(f^{\text{in}}(v) - f^{\text{out}}(v)\big)$. Cada arista $e = (u, v)$ aparece en esa suma exactamente dos veces, con $+f(e)$ en el término de v y con $-f(e)$ en el de u, así que la suma vale 0. □

Si existe una circulación, entonces, la oferta total es igual a la demanda total. Llamemos D a ese valor común:

$$D = \sum_{v:\, d(v)>0} d(v) = \sum_{v:\, d(v)<0} -d(v).$$

Ejemplo 7.6.3 (Una circulación)**.** En la red de la parte (a) de la Figura 7.18, los nodos a, b y c ofrecen 7, 8 y 6 unidades, los nodos d y g demandan 10 y 11, y e es de transbordo; $D = 21$. El flujo indicado es una circulación. Por ejemplo, a b le entran 4 unidades y le salen $6 + 6 = 12$, y $4 - 12 = -8 = d(b)$; a d le entran $3 + 6 + 1 = 10$ y no le sale nada, y $10 = d(d)$; a e le entran 6 y le salen $2 + 4 = 6$.

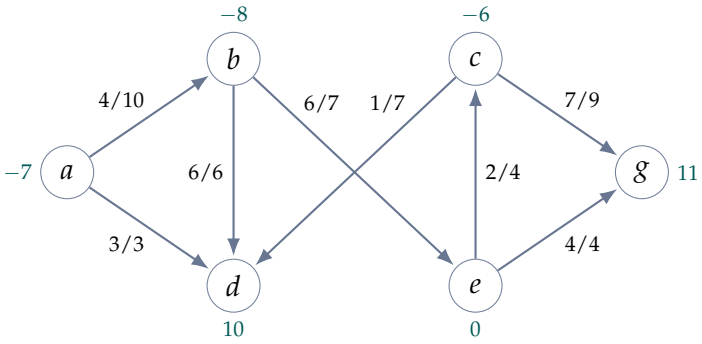
Reducción a flujo máximo

El problema de circulación se reduce a un problema de flujo máximo con una construcción parecida a la de varias fuentes y destinos.

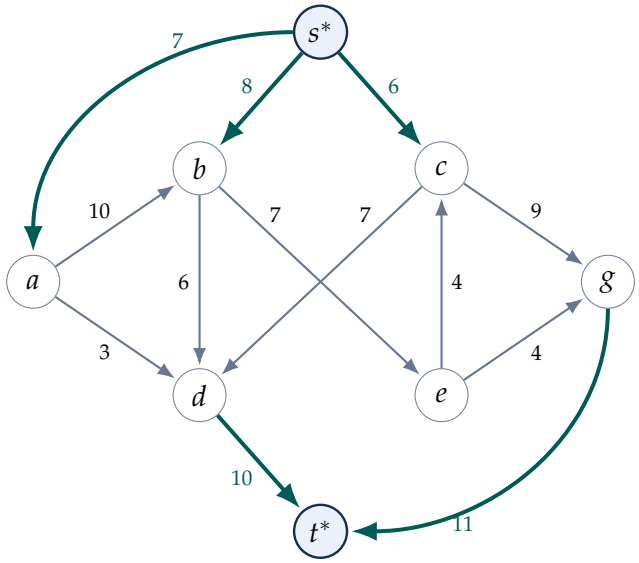
Definición 7.6.4 (Red asociada a un problema de circulación)**.** Dados G, c y d, la red G' se obtiene agregando a G una superfuente s^* y un superdestino t^*, y

- para cada nodo v con $d(v) < 0$, una arista (s^*, v) de capacidad $-d(v)$;
- para cada nodo v con $d(v) > 0$, una arista (v, t^*) de capacidad $d(v)$.

La idea es que s^* le provee a cada nodo de oferta lo que ese nodo tiene que enviar, y t^* le absorbe a cada nodo de demanda lo que ese nodo tiene que recibir. La parte (b) de la Figura 7.18 muestra la red asociada al ejemplo.



(a) una circulación en G



(b) la red G'

Figura 7.18. (a) Una circulación del Ejemplo 7.6.3; junto a cada nodo, en verde, su demanda. (b) La red G' que reduce el problema a flujo máximo: las aristas nuevas, en verde, salen de s^* hacia los nodos de oferta y llegan a t^* desde los nodos de demanda. Una circulación existe si y solo si el flujo máximo de G' satura todas esas aristas, es decir, vale $D = 21$.

Teorema 7.6.5. Existe una circulación en G con demandas d si y solo si el flujo máximo de s^* a t^* en G' vale D. Si todas las capacidades y demandas son enteras y existe una circulación, existe una con valores enteros.

Demostración. Notemos primero que ningún flujo de G' puede valer más que D, porque el corte $(\{s^*\}, V \cup \{t^*\})$ tiene capacidad $\sum_{d(v)<0} -d(v) = D$ (si la suma de las demandas no es 0 no hay circulación, por la Proposición 7.6.2, así que podemos suponer que las dos sumas coinciden).

Si hay una circulación f en G, se la extiende a G' poniendo $f(s^*, v) = -d(v)$ en cada arista desde s^* y $f(v, t^*) = d(v)$ en cada arista hacia t^*. En un nodo v con $d(v) < 0$, el flujo que entra en G' es $f^{\text{in}}(v) - d(v)$ y el que sale es $f^{\text{out}}(v)$, y son iguales porque $f^{\text{in}}(v) - f^{\text{out}}(v) = d(v)$; lo mismo pasa en los nodos de demanda y de transbordo. Entonces es un flujo de s^* a t^*, y vale D.

Recíprocamente, si hay un flujo de valor D en G', todas las aristas que salen de s^* están saturadas, y como el valor también se mide en t^* (Corolario 7.2.3) todas las que llegan a t^* también lo están. Borrando esas aristas queda, en cada nodo v, $f^{\text{in}}(v) - f^{\text{out}}(v) = d(v)$, es decir, una circulación en G.

Si las capacidades y las demandas son enteras, las capacidades de G' son enteras y por el Teorema 7.2.12 hay un flujo máximo entero, del que sale una circulación entera. □

La reducción da un algoritmo, pero también una caracterización de cuándo no existe circulación, en el mismo espíritu que el teorema de Hall.

Teorema 7.6.6. Sea $\sum_v d(v) = 0$. No existe circulación si y solo si existe una partición (A, B) de V tal que

$$\sum_{v \in B} d(v) \;>\; \text{cap}(A, B),$$

donde $\text{cap}(A, B)$ es la suma de las capacidades de las aristas de G que van de A a B.

La desigualdad dice que la demanda neta de los nodos de B supera lo máximo que puede llegarles desde A: aunque todas las aristas de A a B vayan llenas, no alcanza.

Demostración. Si existe una circulación f y (A, B) es cualquier partición, sumando la conservación sobre los nodos de B y cancelando las aristas internas de B, como en la demostración del Lema 7.2.1, se obtiene

$$\sum_{v \in B} d(v) \;=\; \sum_{v \in B} \big(f^{\text{in}}(v) - f^{\text{out}}(v)\big) \;=\; f^{\text{in}}(B) - f^{\text{out}}(B) \;\le\; f^{\text{in}}(B) \;\le\; \text{cap}(A, B).$$

Así que si hay una partición que cumple la desigualdad estricta, no hay circulación.

Recíprocamente, si no hay circulación, por el Teorema 7.6.5 el flujo máximo de G' vale menos que D, y hay un corte (A^*, B^*) de G' con $\text{cap}(A^*, B^*) < D$. Sea $A = A^* - \{s^*\}$ y $B = B^* - \{t^*\}$. Las aristas que salen de A^* son las aristas de G de A a B, las aristas (s^*, v) con $v \in B$ y las aristas (v, t^*) con $v \in A$. Entonces

$$\begin{aligned} \text{cap}(A^*, B^*) &= \text{cap}(A, B) + \sum_{v \in B,\, d(v) < 0} \big(-d(v)\big) + \sum_{v \in A,\, d(v) > 0} d(v) \\ &< D \;=\; \sum_{v \in A,\, d(v) > 0} d(v) + \sum_{v \in B,\, d(v) > 0} d(v). \end{aligned}$$

Cancelando la suma sobre los nodos de demanda de A y pasando de lado la suma sobre los nodos de oferta de B,

$$\text{cap}(A,B) \;<\; \sum_{v\in B,\, d(v)>0} d(v) + \sum_{v\in B,\, d(v)<0} d(v) \;=\; \sum_{v\in B} d(v). \qquad \square$$

7.6.3 Circulaciones con demandas y cotas inferiores

Muchas veces no alcanza con acotar el flujo de una arista por arriba: también se quiere forzar que pase al menos cierta cantidad. Por ejemplo, que una ruta se use sí o sí porque hay un contrato que lo exige.

Definición 7.6.7 (Circulación con cotas inferiores)**.** Sea $G = (V, E)$ un grafo dirigido con demandas $d(v)$ en los nodos y, en cada arista e, una capacidad $c(e)$ y una cota inferior $\ell(e)$ con $0 \le \ell(e) \le c(e)$. Una circulación es una función f que cumple $\ell(e) \le f(e) \le c(e)$ para toda $e \in E$, y $f^{\text{in}}(v) - f^{\text{out}}(v) = d(v)$ para todo $v \in V$. Una arista con cota inferior ℓ y capacidad c se rotula $[\ell, c]$.

La reducción consiste en mandar primero, de manera obligatoria, $\ell(e)$ unidades por cada arista e, y después resolver un problema de circulación sin cotas inferiores con lo que falte. Sea $f_0(e) = \ell(e)$ para toda arista. Esa función cumple las cotas pero en general no la conservación; en cada nodo v deja un desbalance

$$L(v) \;=\; f_0^{\text{in}}(v) - f_0^{\text{out}}(v) \;=\; \sum_{e \text{ entra a } v} \ell(e) \;-\; \sum_{e \text{ sale de } v} \ell(e).$$

Lo que hay que agregarle a f_0 es una función f' que cumpla $0 \le f'(e) \le c(e) - \ell(e)$ y, en cada nodo, $f'^{\text{in}}(v) - f'^{\text{out}}(v) = d(v) - L(v)$.

Definición 7.6.8 (Red sin cotas inferiores)**.** La red G' tiene los mismos nodos y aristas que G, capacidad $c'(e) = c(e) - \ell(e)$ en cada arista, ninguna cota inferior, y demanda $d'(v) = d(v) - L(v)$ en cada nodo.

Por ejemplo, una arista (v, w) con rótulo $[2, 9]$ se convierte en una arista de capacidad 7, y las dos unidades obligatorias se descuentan de las demandas: v ya envía 2, así que su demanda pasa a ser $d(v) + 2$, y w ya recibe 2, así que su demanda pasa a ser $d(w) - 2$. La Figura 7.19 lo muestra.



(a) red G (b) red G'

Figura 7.19. Una arista con cota inferior 2 y capacidad 9 se reemplaza por una arista de capacidad 7 sin cota inferior, corrigiendo las demandas de sus extremos.

Teorema 7.6.9. Existe una circulación en G si y solo si existe una circulación en G'. Además, si todas las demandas, capacidades y cotas inferiores de G son enteras y existe una circulación, existe una con valores enteros.

Demostración. Si f es una circulación en G, sea $f'(e) = f(e) - \ell(e)$. Entonces $0 \leq f'(e) \leq c(e) - \ell(e)$, y en cada nodo

$$f'^{\text{in}}(v) - f'^{\text{out}}(v) \;=\; \big(f^{\text{in}}(v) - f^{\text{out}}(v)\big) - \big(f_0^{\text{in}}(v) - f_0^{\text{out}}(v)\big) \;=\; d(v) - L(v),$$

así que f' es una circulación en G'. Recíprocamente, si f' es una circulación en G', la misma cuenta hacia atrás muestra que $f(e) = f'(e) + \ell(e)$ es una circulación en G. Si los datos de G son enteros, también lo son los de G'; por el Teorema 7.6.5 hay una circulación entera f' en G', y $f = f' + \ell$ es entera. □

7.7 Diseño de encuestas

Muchos problemas se pueden modelar como el de encontrar una circulación con cotas inferiores. Un ejemplo es el diseño de encuestas, que surge en la minería de datos.

7.7.1 El problema

Una empresa quiere diseñar una encuesta para aprender sobre sus clientes. Vende n_2 productos y tiene n_1 clientes, y conoce qué productos compró cada cliente. Las condiciones son las siguientes.

- Al cliente i solo se le puede preguntar por un producto j si lo compró; y se le pregunta a lo sumo una vez por cada uno.
- A cada cliente i se le hacen entre c_i y c'_i preguntas, para que la encuesta no sea ni demasiado corta ni demasiado larga.
- Por cada producto j se les pregunta a entre p_j y p'_j clientes distintos, para tener datos suficientes sobre él.

El problema es decidir si existe una encuesta que cumpla todas las condiciones y, si existe, encontrarla.

7.7.2 Diseño y análisis del algoritmo

Se modela como un problema de circulación con cotas inferiores sobre un grafo bipartito entre clientes y productos, al que se le agregan una fuente y un destino.

- Para cada cliente i y cada producto j que i compró, una arista (i, j) con rótulo $[0, 1]$: la pregunta al cliente i sobre el producto j se hace o no se hace.

- Un nodo s con una arista (s, i) de rótulo $[c_i, c'_i]$ hacia cada cliente i.
- Un nodo t con una arista (j, t) de rótulo $[p_j, p'_j]$ desde cada producto j.
- Una arista (t, s) con rótulo $[0, \infty]$, que devuelve a s todo lo que llega a t.
- Todos los nodos tienen demanda 0.

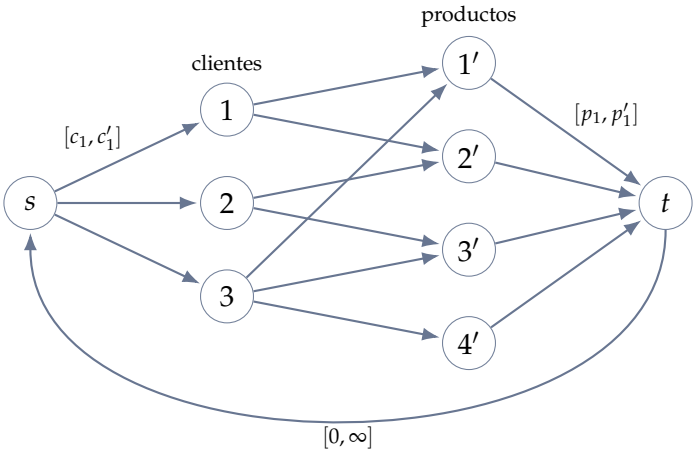


Figura 7.20. La red de circulación del diseño de encuestas con tres clientes y cuatro productos. Las aristas entre clientes y productos tienen rótulo $[0, 1]$ y unen a cada cliente con los productos que compró.

Teorema 7.7.1. Existe una encuesta que cumple las condiciones si y solo si existe una circulación en la red construida. Además, a partir de una circulación entera la encuesta se obtiene preguntándole al cliente i por el producto j exactamente cuando $f(i, j) = 1$.

Demostración. Si existe una encuesta, sea $f(i, j) = 1$ si al cliente i se le pregunta por el producto j y $f(i, j) = 0$ si no; $f(s, i)$ la cantidad de preguntas que se le hacen a i; $f(j, t)$ la cantidad de clientes a los que se les pregunta por j; y $f(t, s)$ la cantidad total de preguntas. Las cotas se cumplen por las condiciones de la encuesta, y la conservación con demanda 0 se cumple en todos los nodos por la forma en que se definieron los valores.

Recíprocamente, si hay una circulación, por el Teorema 7.6.9 hay una circulación entera f. En las aristas entre clientes y productos vale 0 o 1; a cada cliente i se le pregunta por los productos j con $f(i, j) = 1$. Por conservación en i, la cantidad de preguntas es $f(s, i)$, que está entre c_i y c'_i; por conservación en j, la cantidad de clientes a los que se les pregunta por j es $f(j, t)$, que está entre p_j y p'_j. □

El emparejamiento perfecto en un grafo bipartito es el caso particular con $c_i = c'_i = p_j = p'_j = 1$: cada cliente responde exactamente una pregunta y por cada producto se pregunta exactamente una vez.

7.8 Programación de vuelos

Los problemas de programación de las aerolíneas son de los problemas computacionales más complejos que se resuelven en la práctica, y de los mayores consumidores de técnicas algorítmicas de alto nivel. Hay que armar programas eficientes en el uso del equipamiento y del personal, que dejen conformes a los clientes y que resistan imprevistos como el mal tiempo o las averías. Acá se ve una versión de juguete, que igual captura la idea de reutilizar recursos limitados.

7.8.1 El problema

Se tiene un conjunto de k vuelos para un día determinado. El vuelo i sale del aeropuerto o_i a la hora s_i y llega al aeropuerto d_i a la hora f_i. Una misma tripulación puede hacer varios vuelos en el día.

Definición 7.8.1 (Vuelo alcanzable)**.** El vuelo j es alcanzable desde el vuelo i si una tripulación que hizo el vuelo i puede hacer después el vuelo j: por ejemplo, si $d_i = o_j$ y hay tiempo suficiente entre f_i y s_j, o si la tripulación puede trasladarse de d_i a o_j a tiempo.

La relación de alcanzabilidad se puede definir de muchas formas; lo único que usa el algoritmo es que se conoce, para cada par de vuelos, si uno es alcanzable desde el otro. El objetivo es minimizar la cantidad de tripulaciones necesarias para cubrir todos los vuelos. Lo planteamos como problema de decisión: dado c, ¿alcanzan c tripulaciones?

7.8.2 Diseño del algoritmo

- Para cada vuelo i se agregan dos nodos u_i y v_i y una arista (u_i, v_i) con rótulo $[1, 1]$: el vuelo tiene que hacerse, y lo hace exactamente una tripulación.
- Se agrega un nodo s con demanda $-c$, que representa las c tripulaciones al principio del día, y una arista (s, u_i) con rótulo $[0, 1]$ para cada vuelo: una tripulación puede empezar el día con cualquier vuelo.
- Se agrega un nodo t con demanda c y una arista (v_i, t) con rótulo $[0, 1]$ para cada vuelo: una tripulación puede terminar el día con cualquier vuelo.
- Si el vuelo j es alcanzable desde el vuelo i, se agrega una arista (v_i, u_j) con rótulo $[0, 1]$: la misma tripulación hace i y después j.

- Se agrega una arista (s, t) con rótulo $[0, c]$, que corresponde a las tripulaciones que no se usan.
- Los demás nodos tienen demanda 0.

La arista (s, t) permite usar menos de c tripulaciones; sin ella la pregunta sería si se pueden usar exactamente c.

Ejemplo 7.8.2 (Cuatro vuelos)**.** Consideremos los vuelos siguientes, y digamos que j es alcanzable desde i si sale del aeropuerto al que llega i al menos media hora después de que i aterrice.

Vuelo	Origen	Salida	Destino	Llegada
1	Montevideo	8:00	Buenos Aires	9:00
2	Buenos Aires	10:00	Montevideo	11:00
3	Montevideo	10:30	San Pablo	13:00
4	Montevideo	12:00	Buenos Aires	13:00

El vuelo 2 es alcanzable desde el 1 y el 4 es alcanzable desde el 2; no hay más pares. Con $c = 2$ hay una circulación: una tripulación hace 1, 2 y 4 (flujo por $s, u_1, v_1, u_2, v_2, u_4, v_4, t$) y la otra hace el 3 (flujo por s, u_3, v_3, t). Con $c = 1$ no hay circulación, porque la única unidad de flujo que sale de s recorre una cadena de vuelos alcanzables uno desde otro, y el 3 no está en ninguna cadena con el 2: los dos están en el aire entre las 10:30 y las 11:00. La red aparece en la Figura 7.21.

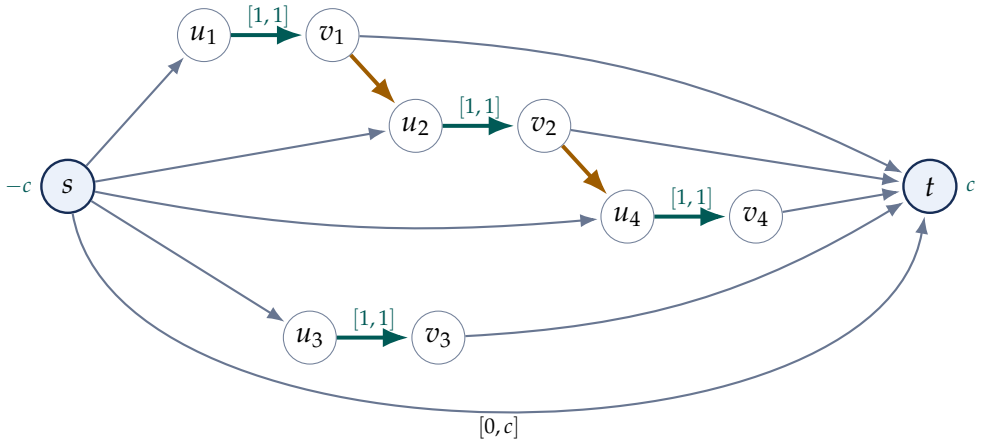


Figura 7.21. La red de circulación del Ejemplo 7.8.2. Las aristas (u_i, v_i) tienen rótulo $[1, 1]$; las demás, salvo (s, t), tienen rótulo $[0, 1]$. En naranja, las aristas (v_i, u_j) que indican que el vuelo j es alcanzable desde el i.

7.8.3 Análisis del algoritmo

Teorema 7.8.3. Los k vuelos se pueden cubrir con a lo sumo c tripulaciones si y solo si existe una circulación en la red construida.

Demostración. Si alcanzan $c' \leq c$ tripulaciones, cada una hace una secuencia de vuelos $i_1, i_2, \ldots, i_r$ en la que cada vuelo es alcanzable desde el anterior. Por cada tripulación se manda una unidad de flujo por el camino $s, u_{i_1}, v_{i_1}, u_{i_2}, \ldots, v_{i_r}, t$, y se mandan las $c - c'$ unidades restantes por la arista (s, t). Como cada vuelo lo hace exactamente una tripulación, cada arista (u_i, v_i) lleva exactamente una unidad; las demás aristas llevan a lo sumo una, porque cada tripulación empieza y termina en un solo vuelo y cada vuelo tiene un solo sucesor en su secuencia. En s salen c unidades y en t entran c, así que es una circulación.

Recíprocamente, si hay una circulación, por el Teorema 7.6.9 hay una circulación entera f, con valores 0 o 1 en todas las aristas salvo (s, t). Salen de s a lo sumo c unidades hacia los nodos u_i. Cada unidad sigue un camino: de u_i pasa a v_i por la única arista que sale de u_i; y de v_i sale por una sola arista de valor 1, que va a t o a algún u_j con j alcanzable desde i. Como el grafo no tiene ciclos (un vuelo alcanzable desde otro sale después de que aquel llega), el camino termina en t. Cada uno de esos caminos es la secuencia de vuelos de una tripulación, y como cada arista (u_i, v_i) lleva exactamente una unidad, cada vuelo queda cubierto por exactamente una tripulación. Se usan a lo sumo c tripulaciones. □

Nota 7.8.4 (Tiempo de ejecución). La red tiene $O(k)$ nodos y $O(k^2)$ aristas. Como nunca hacen falta más de k tripulaciones, el valor óptimo c^* está entre 1 y k y se encuentra con una búsqueda binaria que resuelve $O(\log k)$ problemas de circulación. En cada uno, el flujo de la red auxiliar vale $O(k)$, así que Ford-Fulkerson hace $O(k)$ aumentos de costo $O(k^2)$ cada uno. El tiempo total es $O(k^3 \log k)$. Formulándolo como un problema de flujo mínimo se puede resolver en $O(k^3)$.

Nota 7.8.5 (Qué quedó afuera). Lo resuelto es un problema de juguete. Los modelos reales tienen muchísimos otros factores: normas sindicales, como que una tripulación solo puede volar cierta cantidad de horas en un intervalo dado; la necesidad de planificar varios días y no uno solo; que reasignar una tripulación tiene un costo; que los vuelos no siempre salen o llegan a tiempo; o que conviene optimizar a la vez los horarios de los vuelos y las tarifas. La técnica sirve en general para reutilizar eficientemente recursos limitados, y las técnicas de flujo se usan mucho en la práctica para la programación de vuelos, pero manejar una aerolínea de forma eficiente es un problema muchísimo más difícil que este modelo.

7.9 Segmentación de imágenes

Una aplicación central de los cortes mínimos es la segmentación de imágenes: dividir una imagen en regiones coherentes, como reconocer a cada una de varias personas paradas delante de un fondo complicado, o separar un órgano del tejido que lo rodea en una imagen médica.

7.9.1 El problema

Nos concentramos en la versión más simple: separar el primer plano del fondo. Sea V el conjunto de píxeles de la imagen y E el conjunto de pares de píxeles vecinos; el grafo (V, E) es no dirigido, típicamente una grilla. Para cada píxel i se tienen dos números $a_i \geq 0$ y $b_i \geq 0$: a_i mide qué tan probable es que i pertenezca al primer plano, y b_i qué tan probable es que pertenezca al fondo. Además, para cada par de vecinos $(i, j) \in E$ hay una penalización de separación $p_{ij} \geq 0$ que se paga si uno de los dos queda en el primer plano y el otro en el fondo.

Hay dos objetivos que compiten. Por precisión, si $a_i > b_i$ conviene poner a i en el primer plano, y en el fondo si no. Por suavidad, si muchos vecinos de i están en el primer plano conviene poner también a i ahí, para no pagar las penalizaciones. Se busca una partición (A, B) de los píxeles, con A el primer plano y B el fondo, que maximice

$$q(A,B) \;=\; \sum_{i \in A} a_i \;+\; \sum_{j \in B} b_j \;-\; \sum_{\substack{(i,j) \in E \\ |A \cap \{i,j\}| = 1}} p_{ij}.$$

7.9.2 Formulación como un problema de corte mínimo

El problema se parece al de corte mínimo, pero hay tres diferencias: es de maximización, no hay fuente ni destino, y el grafo es no dirigido. La primera se resuelve con una observación. Sea $Q = \sum_{i \in V} (a_i + b_i)$, que no depende de la partición. Como $\sum_{i \in A} a_i + \sum_{j \in B} b_j = Q - \sum_{i \in A} b_i - \sum_{j \in B} a_j$, maximizar $q(A, B)$ es lo mismo que minimizar

$$q'(A,B) \;=\; \sum_{j \in B} a_j \;+\; \sum_{i \in A} b_i \;+\; \sum_{\substack{(i,j) \in E \\ |A \cap \{i,j\}| = 1}} p_{ij},$$

ya que $q(A, B) = Q - q'(A, B)$. Las otras dos diferencias se resuelven en la construcción de la red G'.

- Hay un nodo por cada píxel, y se agregan una fuente s, que representa el primer plano, y un destino t, que representa el fondo.
- Para cada píxel i se agrega una arista (s, i) de capacidad a_i y una arista (i, t) de capacidad b_i.

- Cada arista no dirigida $\{i, j\} \in E$ se reemplaza por dos aristas dirigidas antiparalelas (i, j) y (j, i), las dos de capacidad p_{ij}.

Teorema 7.9.1. Los cortes $(A \cup \{s\}, B \cup \{t\})$ de G' se corresponden con las particiones (A, B) de los píxeles, y $\text{cap}(A \cup \{s\}, B \cup \{t\}) = q'(A, B)$. En consecuencia, un corte mínimo de G' da una partición óptima.

Demostración. Las aristas que salen del lado de s son de tres tipos. Las aristas (s, j) con $j \in B$ aportan $\sum_{j \in B} a_j$. Las aristas (i, t) con $i \in A$ aportan $\sum_{i \in A} b_i$. Y las aristas entre píxeles que van de A a B aportan p_{ij} por cada par de vecinos separados por la partición: de las dos aristas antiparalelas de ese par, exactamente una va de A a B, así que p_{ij} se cuenta exactamente una vez. La suma es $q'(A, B)$. Minimizar la capacidad del corte es entonces minimizar q', que es maximizar q. □

Ejemplo 7.9.2 (Una fila de tres píxeles)**.** Tomemos tres píxeles en fila, con $a = (6, 2, 1)$, $b = (1, 3, 6)$, $p_{12} = 3$ y $p_{23} = 1$. Mirando cada píxel por separado, el 1 iría al primer plano y el 2 y el 3 al fondo, con $q(\{1\}, \{2, 3\}) = 6 + 3 + 6 - 3 = 12$. Pero la partición $A = \{1, 2\}$, $B = \{3\}$ da $q = 6 + 2 + 6 - 1 = 13$: el píxel 2 prefiere el fondo, pero lo pesa más la penalización de separarse del píxel 1. Revisando las ocho particiones se verifica que 13 es el máximo. En la red de la Figura 7.22, $Q = 19$ y el corte con lado $\{s, 1, 2\}$ tiene capacidad $b_1 + b_2 + a_3 + p_{23} = 1 + 3 + 1 + 1 = 6 = 19 - 13$.

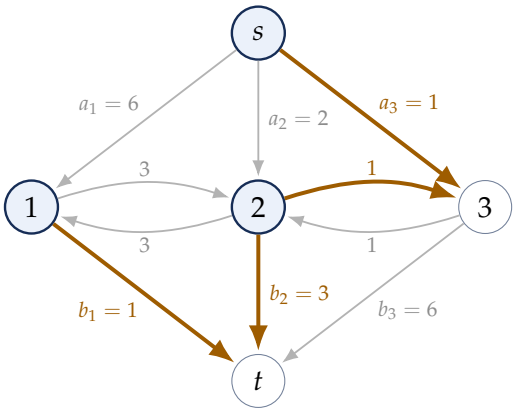


Figura 7.22. La red G' del Ejemplo 7.9.2. El corte mínimo tiene lado $\{s, 1, 2\}$ (nodos sombreados): los píxeles 1 y 2 son el primer plano. Las aristas que salen de ese lado, resaltadas, suman 6.

7.10 Selección de proyectos

Una empresa grande tiene que decidir qué proyectos llevar adelante. Algunos proyectos dejan ganancia y otros solo cuestan, pero hay proyectos que no se pueden hacer sin hacer antes otros: una aplicación que da ganancia puede requerir montar un servidor que solo cuesta.

7.10.1 El problema

Definición 7.10.1 (Selección de proyectos)**.** Hay un conjunto P de proyectos posibles, y cada proyecto v tiene un ingreso p_v, que puede ser positivo o negativo. Hay además un conjunto de prerrequisitos E: si $(v, w) \in E$, no se puede hacer el proyecto v sin hacer también el proyecto w. Un subconjunto $A \subseteq P$ es factible si el prerrequisito de cada proyecto de A también pertenece a A. El problema de selección de proyectos consiste en encontrar un conjunto factible A que maximice el ingreso total $\sum_{v \in A} p_v$.

Los prerrequisitos forman un grafo dirigido sobre P, con una arista (v, w) cuando no se puede hacer v sin w, y un conjunto factible es un conjunto cerrado de ese grafo: si contiene un nodo, contiene todos los nodos a los que ese nodo apunta. Por eso el problema también se conoce como el de cierre de peso máximo.

7.10.2 Formulación como corte mínimo

- Se agrega una fuente s y un destino t.
- Cada arista de prerrequisito (v, w) recibe capacidad infinita.
- Para cada proyecto v con $p_v > 0$ se agrega una arista (s, v) de capacidad p_v.
- Para cada proyecto v con $p_v < 0$ se agrega una arista (v, t) de capacidad $-p_v$.

Sea $C = \sum_{v:\, p_v > 0} p_v$, el ingreso que se obtendría haciendo todos los proyectos rentables sin pagar ningún costo. El corte $(\{s\}, P \cup \{t\})$ tiene capacidad C, así que el corte mínimo tiene capacidad finita.

Teorema 7.10.2. Si (A', B') es un corte de capacidad finita, el conjunto $A = A' - \{s\}$ es factible y $\text{cap}(A', B') = C - \sum_{v \in A} p_v$. En consecuencia, si (A', B') es un corte mínimo, $A' - \{s\}$ es un conjunto óptimo de proyectos.

Demostración. Si A no fuera factible, habría un proyecto $v \in A$ con un prerrequisito $w \notin A$, y la arista (v, w), de capacidad infinita, iría de A' a B'; el corte no tendría

capacidad finita. Así que A es factible, y ninguna arista de prerrequisito cruza de A' a B'. Las aristas que salen de A' son entonces las (s, v) con $v \in B$ y $p_v > 0$, y las (v, t) con $v \in A$ y $p_v < 0$:

$$\begin{aligned} \operatorname{cap}(A', B') &= \sum_{v \notin A,\ p_v > 0} p_v \ + \sum_{v \in A,\ p_v < 0} (-p_v) \\ &= \Big(C - \sum_{v \in A,\ p_v > 0} p_v\Big) - \sum_{v \in A,\ p_v < 0} p_v \ = \ C - \sum_{v \in A} p_v. \end{aligned}$$

Como C es una constante, minimizar la capacidad del corte es maximizar el ingreso de A. Todo conjunto factible A da el corte $(A \cup \{s\},\ (P - A) \cup \{t\})$, que tiene capacidad finita, así que el corte mínimo recorre todos los conjuntos factibles. □

Ejemplo 7.10.3 (Cuatro proyectos)**.** Una empresa evalúa una aplicación a con ingreso 8, un servicio c con ingreso 3, un servidor b con costo 3 y una base de datos d con costo 4. La aplicación necesita el servidor y la base de datos, y el servicio necesita la base de datos. El servicio solo no conviene: $\{c, d\}$ da $3 - 4 = -1$. La aplicación sola con lo que requiere, $\{a, b, d\}$, da $8 - 3 - 4 = 1$. Pero hacer todo da $8 + 3 - 3 - 4 = 4$, porque la base de datos se paga una sola vez. En la red de la Figura 7.23, $C = 8 + 3 = 11$, y el corte mínimo es el que pone los cuatro proyectos del lado de s: tiene capacidad $3 + 4 = 7 = 11 - 4$.

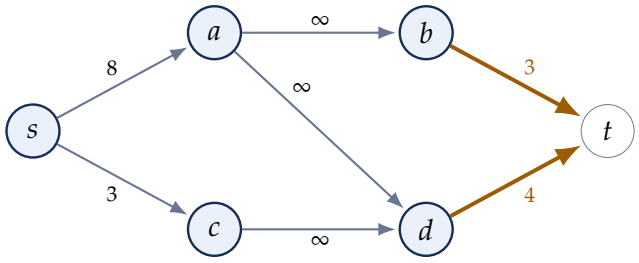


Figura 7.23. La red del Ejemplo 7.10.3. Las aristas de capacidad ∞ son los prerrequisitos. El corte mínimo tiene del lado de s los cuatro proyectos (sombreados), y las aristas que salen de ese lado suman 7.

Nota 7.10.4 (Minería a cielo abierto). Un caso clásico, estudiado desde principios de los años sesenta, es la minería a cielo abierto. Se extraen bloques de tierra de la superficie para recuperar mineral; cada bloque v tiene un valor neto p_v igual al valor del mineral que contiene menos el costo de extraerlo y procesarlo. No se puede sacar un bloque sin sacar antes los que están encima de él, y eso da los prerrequisitos. Elegir qué bloques extraer para maximizar la ganancia es exactamente un problema de selección de proyectos.

7.11 Eliminación en béisbol

Hacia el final de una temporada deportiva, los diarios suelen decir que un equipo está matemáticamente eliminado cuando ya no puede terminar primero pase lo que pase en los partidos que quedan. El razonamiento habitual es que el equipo no puede alcanzar las victorias que el líder ya tiene. Esa condición es suficiente, pero no necesaria, y el problema de decidirlo correctamente se resuelve con flujos.

7.11.1 El problema

Ejemplo 7.11.1 (Una tabla de posiciones)**.** Consideremos cuatro equipos con la situación siguiente, en la que las últimas cuatro columnas dicen cuántos partidos quedan entre cada par.

Equipo	Ganados	Perdidos	Restan	ATL	PHI	NY	MON
Atlanta	83	71	8	—	1	6	1
Filadelfia	80	79	3	1	—	0	2
Nueva York	78	78	6	6	0	—	0
Montreal	77	82	3	1	2	0	—

Montreal termina con a lo sumo 80 victorias y Atlanta ya tiene 83: está eliminado por el argumento habitual. Filadelfia termina con a lo sumo 83 victorias, las que ya tiene Atlanta, así que el argumento habitual no alcanza. Pero Atlanta y Nueva York juegan 6 partidos entre sí: o Atlanta gana alguno y llega a 84, o Nueva York gana los seis y llega a 84. En cualquier caso alguien supera a Filadelfia, que también está eliminado. La respuesta depende entonces no solo de cuántos partidos ganó cada equipo y cuántos le quedan, sino de contra quién los juega.

Definición 7.11.2 (Problema de eliminación)**.** Hay un conjunto de equipos S y un equipo distinguido $z \in S$. Cada equipo x ya ganó w_x partidos, y los equipos x e y tienen que jugar entre sí g_{xy} partidos más; cada partido lo gana uno de los dos. El problema consiste en decidir si hay algún resultado de los partidos que faltan en el que z termine con al menos tantas victorias como cualquier otro equipo.

7.11.2 Formulación como flujo máximo

Si z puede terminar primero, puede hacerlo ganando todos sus partidos restantes. Sea $m = w_z + g_z$, donde g_z es la cantidad total de partidos que le quedan a z. Si algún $w_x > m$, z está eliminado de inmediato. Si no, la pregunta es si los partidos entre los demás equipos se pueden repartir de modo que ningún equipo $x \neq z$ termine con más de m victorias, es decir, que cada x gane a lo sumo $m - w_x$ de sus partidos restantes. Sea $S' = S - \{z\}$ y $g^* = \sum_{\{x,y\} \subseteq S'} g_{xy}$ la cantidad de partidos que quedan entre equipos de S'.

- Se agrega una fuente s y un destino t.
- Para cada par $\{x, y\} \subseteq S'$ con $g_{xy} > 0$ se agrega un nodo de partido x-y y una arista $(s, x\text{-}y)$ de capacidad g_{xy}.
- Para cada equipo $x \in S'$ se agrega un nodo de equipo x y una arista (x, t) de capacidad $m - w_x$.
- Cada nodo de partido x-y tiene aristas de capacidad infinita hacia los nodos de equipo x e y.

Una unidad de flujo que va de s a x-y y de ahí a x representa un partido entre x e y que gana x.

Teorema 7.11.3. El equipo z no está eliminado si y solo si el flujo máximo de la red vale g^*, es decir, si satura todas las aristas que salen de s.

Demostración. Si z no está eliminado, hay un resultado de todos los partidos en el que z gana todos los suyos y cada $x \in S'$ termina con a lo sumo m victorias. Mandando por cada partido entre x e y una unidad por s, x-y, x, t si lo gana x (o por y si lo gana y), se obtiene un flujo de valor g^*: a cada equipo x le llegan tantas unidades como partidos gana, que son a lo sumo $m - w_x$.

Recíprocamente, si el flujo máximo vale g^*, por el teorema de integridad hay un flujo entero de valor g^*. Cada una de las g_{xy} unidades que llegan al nodo x-y va entera a x o a y; se declara ganador del partido correspondiente al equipo al que va. Así se asigna un ganador a cada partido, y la capacidad de (x, t) garantiza que ningún equipo

gana más de $m - w_x$ partidos restantes. Con z ganando todos los suyos, z termina con m victorias y nadie lo supera. □

En el Ejemplo 7.11.1, para $z =$ Filadelfia se tiene $m = 83$. Los partidos entre los demás son Atlanta-Nueva York (6) y Atlanta-Montreal (1), así que $g^* = 7$; las capacidades hacia t son $83 - 83 = 0$ para Atlanta, $83 - 78 = 5$ para Nueva York y $83 - 77 = 6$ para Montreal. Los 6 partidos de Atlanta con Nueva York solo pueden ir a Nueva York, que admite 5, así que el flujo máximo vale $5 + 1 = 6 < 7$ y Filadelfia está eliminado. La Figura 7.24 muestra la red.

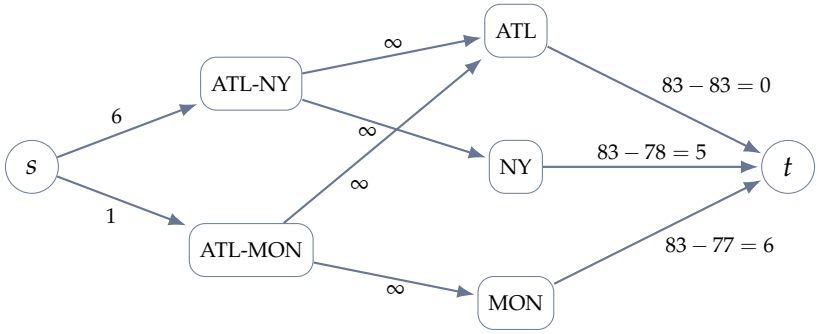


Figura 7.24. La red que decide si Filadelfia está eliminado en el Ejemplo 7.11.1. Las aristas que salen de s suman 7, pero el flujo máximo vale 6.

7.11.3 Un certificado de eliminación

El Teorema 7.11.3 da un algoritmo, pero no una explicación que convenza a un lector de diarios. El razonamiento sobre Filadelfia sugiere cuál es: Atlanta y Nueva York juntos ya tienen $83 + 78 = 161$ victorias y les quedan 6 partidos entre ellos, así que terminan sumando 167 victorias entre los dos, un promedio de 83,5 por equipo; alguno de los dos termina con al menos 84.

Para $T \subseteq S$, sea $w(T) = \sum_{x \in T} w_x$ la cantidad de victorias que ya tienen los equipos de T y $g(T) = \sum_{\{x,y\} \subseteq T} g_{xy}$ la cantidad de partidos que quedan entre ellos.

Teorema 7.11.4 (Hoffman y Rivlin, 1967)**.** El equipo z está eliminado si y solo si existe un conjunto $T^* \subseteq S - \{z\}$ tal que

$$m \;=\; w_z + g_z \;<\; \frac{w(T^*) + g(T^*)}{|T^*|}.$$

Demostración. ($\Leftarrow$) Si existe ese T^*, los equipos de T^* terminan con al menos $w(T^*) + g(T^*)$ victorias entre todos, porque cada partido que queda entre ellos lo gana alguno. En promedio ganan al menos $(w(T^*) + g(T^*))/|T^*|$ partidos, así que alguno termina con más de m victorias, que es lo máximo que puede ganar z.

($\Rightarrow$) Supongamos que z está eliminado. Si algún $w_x > m$, alcanza con $T^* = \{x\}$. Si no, todas las capacidades son no negativas; consideremos un corte mínimo (A, B) de la red, y sea T^* el conjunto de nodos de equipo que están en A. Afirmamos que un nodo de partido x-y está en A si y solo si x e y están los dos en T^*. Por un lado, si x-$y \in A$, las aristas infinitas $(x\text{-}y, x)$ y $(x\text{-}y, y)$ obligan a que x e y estén en A, porque si no el corte sería infinito. Por otro lado, si $x, y \in A$ pero x-$y \notin A$, pasar x-y a A elimina del corte la arista $(s, x\text{-}y)$ sin agregar nada, porque sus aristas salientes van a A; la capacidad bajaría, contradiciendo que el corte es mínimo (si $g_{xy} > 0$).

Las aristas que salen de A son entonces las $(s, x\text{-}y)$ con el par no contenido en T^* y las (x, t) con $x \in T^*$. Como z está eliminado, el flujo máximo es menor que g^*, y por el teorema de flujo máximo y corte mínimo

$$\begin{aligned} g^* \;>\; \operatorname{cap}(A, B) &= \big(g^* - g(T^*)\big) + \sum_{x \in T^*} (m - w_x) \\ &= g^* - g(T^*) - w(T^*) + |T^*|\, m. \end{aligned}$$

Reordenando, $|T^*|\, m < w(T^*) + g(T^*)$; en particular T^* no es vacío, y dividiendo por $|T^*|$ se obtiene la desigualdad buscada. □

Ejemplo 7.11.5 (La liga del este, 30 de agosto de 1996)**.** En la Liga Americana, división este, la situación era la siguiente.

Equipo	Ganados	Perdidos	Restan	NYY	BAL	BOS	TOR	DET
Nueva York	75	59	28	–	3	8	7	3
Baltimore	71	63	28	3	–	2	7	4
Boston	69	66	27	8	2	–	0	0
Toronto	63	72	27	7	7	0	–	0
Detroit	49	86	27	3	4	0	0	–

Detroit termina con a lo sumo $49 + 27 = 76$ victorias, y ningún equipo tiene todavía más de 75; el argumento habitual no lo elimina. Pero con $T^* = \{\text{NYY}, \text{BAL}, \text{BOS}, \text{TOR}\}$ se tiene $w(T^*) = 75 + 71 + 69 + 63 = 278$ y $g(T^*) = 3 + 8 + 7 + 2 + 7 + 0 = 27$, así que esos cuatro equipos terminan con 305 victorias entre todos, un promedio de 76,25. Alguno termina con al menos 77, y Detroit está eliminado.

Capítulo 8

Complejidad Computacional

En los capítulos anteriores se desarrollaron algoritmos eficientes para una variedad amplia de problemas, y hasta se consiguió clasificar informalmente algunos de los problemas que admiten soluciones eficientes: los que se expresan como un corte mínimo en una red, los que admiten una formulación de programación dinámica, los que ceden ante un criterio greedy. Por el camino aparecieron también problemas para los que no se veía cómo hacer lo mismo, pero hasta ahora no hubo ningún intento de caracterizar la familia de problemas que no se pueden resolver de manera eficiente.

En la Sección 2.1 se adoptó el tiempo polinómico como noción de eficiencia (Definición 2.1.2). Una ventaja de trabajar con una definición concreta como esa es que abre la posibilidad de demostrar matemáticamente que ciertos problemas no se pueden resolver en tiempo polinómico. Cuando se empezó a estudiar la complejidad computacional en serio hubo algunos avances en esa dirección para problemas extremadamente difíciles; pero para muchos de los problemas discretos más básicos, que surgen en optimización, en inteligencia artificial, en combinatoria o en lógica, la pregunta resultó demasiado difícil y sigue abierta: no se conocen algoritmos de tiempo polinómico para ellos, y tampoco se sabe demostrar que no existan.

Frente a esa ambigüedad sí hubo progreso. Se caracterizó una clase grande de problemas de esta zona gris y se demostró que son equivalentes en un sentido preciso: un algoritmo de tiempo polinómico para cualquiera de ellos daría un algoritmo de tiempo polinómico para todos. Son los problemas $\mathcal{NP}$-completos, entre los que ya apareció el Conjunto Independiente de la Sección 1.8. Hay literalmente miles de problemas $\mathcal{NP}$-completos, surgidos en áreas muy distintas, así que todas esas preguntas abiertas son en realidad una sola. Desde un punto de vista práctico, que un problema sea $\mathcal{NP}$-completo significa esencialmente que es computacionalmente difícil a todos los efectos prácticos, aunque no se lo pueda demostrar: es una razón de peso para dejar de buscar un algoritmo eficiente, porque encontrarlo equivaldría a encontrar uno para todos los problemas famosos de la clase, con los que ya fracasó muchísima gente.

8.1 Reducciones de tiempo polinómico

La técnica básica para explorar el territorio de los problemas difíciles es comparar la dificultad relativa de dos problemas, para poder afirmar con precisión cosas como "el problema X es al menos tan difícil como el problema Y". La herramienta que formaliza esa comparación es la reducción: se muestra que X es al menos tan difícil como Y argumentando que, si se dispusiera de una caja negra capaz de resolver X, también se podría resolver Y. En otras palabras, X es lo bastante poderoso como para resolver Y.

8.1.1 Definición y consecuencias

Para precisar la idea se agrega al modelo de cómputo la suposición de que X se resuelve en un solo paso: si se escribe la entrada de una instancia de X, la caja negra devuelve en un paso la respuesta correcta. La pregunta es entonces si se pueden resolver instancias arbitrarias de Y con una cantidad polinómica de pasos de cómputo estándar más una cantidad polinómica de llamadas a esa caja negra.

Definición 8.1.1 (Reducción de tiempo polinómico)**.** Un problema Y se reduce en tiempo polinómico a un problema X, y se escribe $Y \leq_P X$, si Y se puede resolver mediante un algoritmo que

- realiza una cantidad de invocaciones a un algoritmo que resuelve X que es polinómica en el tamaño de la instancia de Y, y
- fuera de esas llamadas, hace un tiempo de cómputo adicional también polinómico en el tamaño de la instancia de Y.

Se lee "Y se reduce polinómicamente a X" o "X es al menos tan difícil como Y (respecto del tiempo polinómico)".

En esta definición se sigue pagando el tiempo de escribir la entrada que se le pasa a la caja negra y de leer la respuesta que devuelve. Es como si se le agregara a la computadora un circuito especializado que resuelve instancias de X en un solo paso, y la pregunta fuera cuánto poder extra da ese circuito.

La consecuencia más importante de la definición es la siguiente. Si $Y \leq_P X$ y existe efectivamente un algoritmo de tiempo polinómico para X, la caja negra deja de ser valiosa: se la puede reemplazar por ese algoritmo.

Proposición 8.1.2. Supongamos que $Y \leq_P X$. Si X se puede resolver en tiempo polinómico, entonces Y también se puede resolver en tiempo polinómico.

Demostración. Sea n el tamaño de la instancia de Y. Por la definición de reducción, el algoritmo para Y hace a lo sumo $p(n)$ pasos propios y a lo sumo $p(n)$ llamadas a la caja

negra, para cierto polinomio p. Cada instancia de X que se le pasa a la caja negra hay que escribirla, así que su tamaño es a lo sumo $p(n)$. Si se reemplaza la caja negra por un algoritmo para X que corre en tiempo $q(\cdot)$, con q polinomio, cada llamada cuesta a lo sumo $q(p(n))$, y el tiempo total es a lo sumo $p(n) + p(n)\, q(p(n))$, que es un polinomio en n. □

Este hecho ya se usó, sin nombrarlo, en el capítulo anterior. El problema de emparejamiento bipartito se resolvió con un preprocesamiento polinómico más la resolución de una única instancia del problema de flujo máximo (Sección 7.4); como el flujo máximo se resuelve en tiempo polinómico, el emparejamiento bipartito también. Lo mismo ocurrió con la segmentación de imágenes y el corte mínimo (Sección 7.9). La Proposición 8.1.2 resume una manera muy productiva de diseñar algoritmos eficientes para problemas nuevos: reducirlos a un problema que ya se sabe resolver en tiempo polinómico.

Aquí, en cambio, la reducción se usa para establecer la intratabilidad de los problemas, y para eso interesa la forma contrarrecíproca de la proposición.

Corolario 8.1.3. Supongamos que $Y \leq_P X$. Si Y no se puede resolver en tiempo polinómico, entonces X tampoco se puede resolver en tiempo polinómico.

El corolario es trivialmente equivalente a la proposición, pero pone el énfasis en otro lado: si se tiene un problema Y que se sabe difícil y se muestra que $Y \leq_P X$, la dificultad se contagia a X, que tiene que ser difícil porque, si no, serviría para resolver Y. Como en realidad no se sabe si los problemas que se estudian se pueden resolver en tiempo polinómico, la relación $\leq_P$ se usa para establecer niveles relativos de dificultad entre ellos.

Conviene tener presente para qué sirve cada dirección. Si se tiene entre manos un problema X:

- para encontrar un algoritmo que lo resuelva, se busca un problema Y que se sepa resolver y se reduce X a Y, probando $X \leq_P Y$; es lo que se hizo en los capítulos anteriores;
- para probar que X es difícil, en el sentido de que no se conoce un algoritmo eficiente que lo resuelva, se busca un problema Y que se sepa difícil y se reduce Y a X, probando $Y \leq_P X$.

8.1.2 Reducciones a la Karp

La definición de $\leq_P$ permite muchas llamadas a la caja negra, pero la mayoría de las reducciones que interesan usan una sola, y de una forma muy particular: transforman la instancia de Y en una única instancia de X y devuelven como respuesta, sin modificarla, la respuesta de la caja negra. Este tipo de reducción tiene nombre propio. Es natural en los problemas de decisión, es decir, en los problemas que consisten en responder una pregunta binaria, con respuesta sí o no, a partir de ciertas entradas; por ejemplo, dado un grafo G, ¿es G bipartito?

Definición 8.1.4 (Reducción a la Karp)**.** Sean X e Y problemas de decisión. Una reducción a la Karp de Y a X es un algoritmo A que, dada una instancia y de Y, construye una instancia $x = A(y)$ de X, y que cumple:

(I) A termina en tiempo polinómico en el tamaño de y;

(II) para toda instancia y de Y, la respuesta de Y para y es igual a la respuesta de X para $A(y)$.

Dicho de otra forma, A transforma las entradas de un algoritmo que resuelve Y en entradas de un algoritmo que resuelve X. Una reducción a la Karp es un caso particular de reducción de tiempo polinómico: para resolver y se calcula $A(y)$, que tiene tamaño polinómico porque se escribió en tiempo polinómico, se invoca una única vez la caja negra de X y se devuelve lo que responda. La Figura 8.1 compara este esquema con el de la Definición 8.1.1.

La condición (ii) se suele verificar en dos partes, una por cada sentido: si y es una instancia SÍ de Y, entonces $A(y)$ es una instancia SÍ de X; y si $A(y)$ es una instancia SÍ de X, entonces y es una instancia SÍ de Y. Las instancias x de X que no son la imagen de ningún y no interesan: la reducción no dice nada sobre ellas.

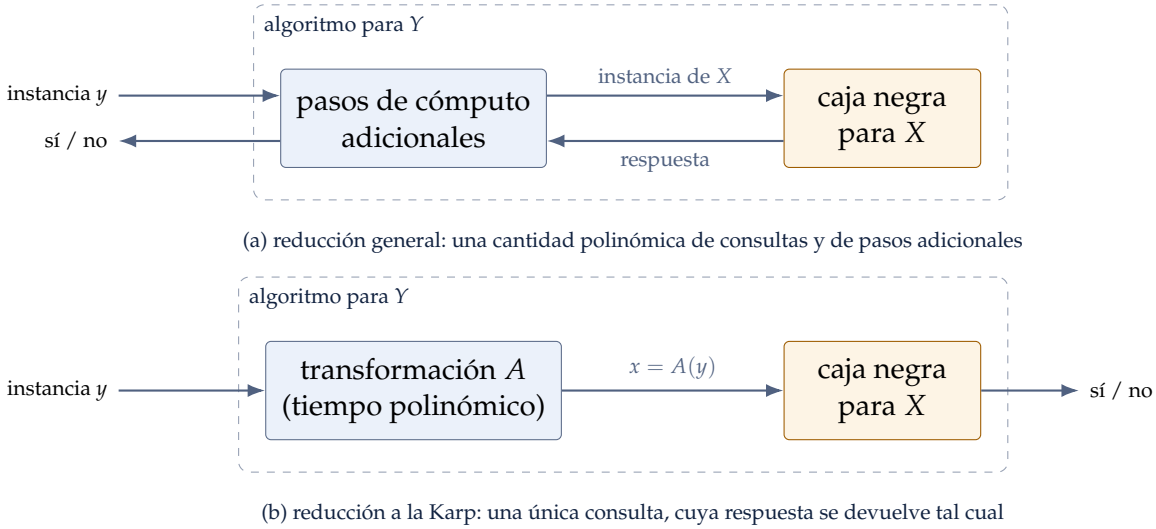


Figura 8.1. Dos formas de resolver Y usando una caja negra para X. En (a) el algoritmo puede consultar la caja negra muchas veces y combinar las respuestas; en (b) transforma la instancia una sola vez y devuelve la respuesta de la caja negra.

Hay trabajos de investigación que estudian la diferencia entre las dos nociones: las reducciones de una sola consulta que devuelven la respuesta tal cual se llaman reducciones de Karp, y las más generales, que pueden consultar muchas veces, reducciones de Cook o reducciones de Turing de tiempo polinómico. Aquí no se profundiza en esa distinción.

8.1.3 *Independent Set* y *Vertex Cover*

El problema del Conjunto Independiente, presentado en la Sección 1.8 (Definición 1.8.9), sirve como primer ejemplo prototípico de problema difícil: no se conoce un algoritmo de tiempo polinómico para resolverlo, pero tampoco se sabe demostrar que no exista. Recordemos que, en un grafo $G = (V, E)$, un conjunto de nodos $S \subseteq V$ es independiente si ningún par de nodos de S está unido por una arista; de manera equivalente, S es independiente si para todo $u \in S$ los nodos adyacentes a u no pertenecen a S. Encontrar conjuntos independientes chicos es fácil ($S = \emptyset$ o un solo nodo ya lo son); lo difícil es encontrar uno grande. Allí el problema se planteó como uno de optimización, pero para estudiar reducciones es mucho más cómodo trabajar con problemas de decisión, así que se lo reformula de esta manera.

Definición 8.1.5 (*Independent Set*)**.** Dado un grafo G y un natural k, ¿existe un conjunto independiente de G de tamaño al menos k?

Desde el punto de vista de la resolubilidad en tiempo polinómico no hay una diferencia importante entre la versión de optimización (encontrar el tamaño máximo

de un conjunto independiente) y la versión de decisión. Resolver la de optimización resuelve automáticamente la de decisión, para cualquier k. Pero también vale una recíproca un poco menos evidente: si se sabe resolver la versión de decisión para todo k, se puede encontrar el tamaño máximo. En un grafo de n nodos basta resolver la versión de decisión para cada k; el mayor k con respuesta afirmativa es el tamaño del conjunto independiente más grande, y con búsqueda binaria alcanza con $O(\log n)$ valores de k. El mismo argumento sirve para cualquier problema cuya versión de optimización tenga como respuesta un número entero entre 0 y n.

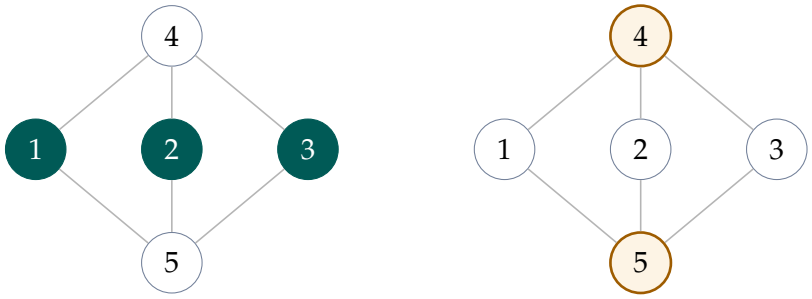
Para ilustrar la estrategia básica de relacionar problemas difíciles entre sí consideremos otro problema de grafos para el que no se conoce ningún algoritmo eficiente.

Definición 8.1.6 (Cobertura de aristas por vértices)**.** Sea $G = (V, E)$ un grafo. Un conjunto de nodos $S \subseteq V$ es una cobertura de aristas por vértices, o simplemente una cobertura, si toda arista $e \in E$ tiene al menos uno de sus extremos en S.

En una cobertura los vértices son los que cubren y las aristas son los objetos cubiertos. Ahora es fácil encontrar coberturas grandes ($S = V$ lo es trivialmente) y lo difícil es encontrar una chica.

Definición 8.1.7 (*Vertex Cover*)**.** Dado un grafo G y un natural k, ¿existe una cobertura de aristas por vértices de G de tamaño no mayor a k?

Ejemplo 8.1.8 (El grafo de cinco nodos)**.** Sea G el grafo de la Figura 8.2, con nodos $1, \ldots, 5$ y aristas $(1,4)$, $(2,4)$, $(3,4)$, $(1,5)$, $(2,5)$ y $(3,5)$. El conjunto $\{1,2,3\}$ es independiente, porque todas las aristas tienen un extremo en $\{4,5\}$; no hay conjuntos independientes de tamaño 4, porque todo conjunto de cuatro nodos contiene a 4 o a 5 y a alguno de 1, 2, 3, que es su vecino. El conjunto $\{4,5\}$ es una cobertura de tamaño 2, y no hay coberturas de tamaño 1: un solo nodo de $\{1,2,3\}$ deja sin cubrir las aristas de los otros dos, y el nodo 4 solo no cubre las aristas que llegan a 5, ni el 5 las que llegan a 4. Así, $(G,3)$ es una instancia SÍ de *Independent Set* y $(G,4)$ una instancia NO, mientras que $(G,2)$ es una instancia SÍ de *Vertex Cover* y $(G,1)$ una instancia NO.



(a) conjunto independiente $S = \{1, 2, 3\}$ (b) cobertura $V \setminus S = \{4, 5\}$

Figura 8.2. El grafo del Ejemplo 8.1.8. (a) Un conjunto independiente de tamaño máximo. (b) Su complemento, que es una cobertura de tamaño mínimo: toda arista tiene un extremo en $\{4, 5\}$.

No se sabe resolver ninguno de los dos problemas en tiempo polinómico, pero sí se puede decir algo de su dificultad relativa: son igual de difíciles. Eso se sigue de la observación que ilustra la Figura 8.2.

Lema 8.1.9. Sea $G = (V, E)$ un grafo. Un conjunto $S \subseteq V$ es independiente si y solo si su complemento $V \setminus S$ es una cobertura de aristas por vértices.

Demostración. Supongamos primero que S es independiente, y sea $e = (u, v)$ una arista cualquiera. Como S es independiente, u y v no pueden estar ambos en S, así que alguno está en $V \setminus S$. Entonces toda arista tiene al menos un extremo en $V \setminus S$, y $V \setminus S$ es una cobertura.

Recíprocamente, supongamos que $V \setminus S$ es una cobertura, y sean u y v dos nodos de S. Si estuvieran unidos por una arista e, ningún extremo de e estaría en $V \setminus S$, contra la hipótesis. Entonces ningún par de nodos de S está unido por una arista, y S es independiente. □

Proposición 8.1.10. *Independent Set* $\leq_P$ *Vertex Cover* y *Vertex Cover* $\leq_P$ *Independent Set*.

Demostración. Sea $n = |V|$. Por el Lema 8.1.9, G tiene un conjunto independiente de tamaño al menos k si y solo si tiene una cobertura de tamaño a lo sumo $n - k$: si S es independiente con $|S| \geq k$, entonces $V \setminus S$ es una cobertura con $|V \setminus S| \leq n - k$, y si T es una cobertura con $|T| \leq n - k$, entonces $V \setminus T$ es independiente con $|V \setminus T| \geq k$. La reducción a la Karp de *Independent Set* a *Vertex Cover* transforma la instancia (G, k) en $(G, n - k)$, lo que se hace en tiempo polinómico. Simétricamente, G tiene una cobertura de tamaño a lo sumo k si y solo si tiene un conjunto independiente de tamaño al menos $n - k$, y la reducción de *Vertex Cover* a *Independent Set* transforma (G, k) en $(G, n - k)$. □

Este tipo de análisis ilustra el plan general: aunque no se sabe resolver eficientemente ni *Independent Set* ni *Vertex Cover*, la Proposición 8.1.10 dice cómo resolver cualquiera de los dos a partir de una solución eficiente del otro, y eso fija su dificultad relativa. Cuando valen las dos reducciones se dice a veces que los problemas son polinómicamente equivalentes.

Decisión, búsqueda y optimización

Para *Vertex Cover* se pueden distinguir tres versiones:

- decisión: ¿existe una cobertura de tamaño no superior a k?;
- búsqueda: encontrar una cobertura de tamaño no superior a k;
- optimización: encontrar una cobertura de tamaño mínimo.

Ya se vio que la versión de decisión permite calcular el tamaño óptimo. Veamos que también permite construir una solución, es decir, que un algoritmo polinómico para la decisión da un algoritmo polinómico para la búsqueda. Sea VC-DECISION una caja negra que responde la versión de decisión. La idea es encontrar un nodo que se pueda poner en la cobertura sin perder la posibilidad de completarla, y seguir recursivamente en el grafo sin ese nodo.

```
BUSCAR-VC(G, k)
1  if G no tiene aristas then return ∅
2  if not VC-DECISION(G, k) then return "no existe"
3  for each nodo v ∈ V do
4      G' ← G − {v}
5      if VC-DECISION(G', k − 1) then
6          S ← BUSCAR-VC(G', k − 1)
7          return S ∪ {v}
8  end for
```

> **Estructuras de datos.** G se representa con listas de adyacencia. El grafo $G - \{v\}$ se construye en $O(n + m)$ copiando las listas y salteando el nodo v y las aristas incidentes a él. La cobertura S se devuelve como una lista encadenada de nodos, y agregarle v en la línea 7 cuesta $O(1)$.

Si G tiene alguna arista y existe una cobertura T de tamaño a lo sumo k, entonces T no es vacía, y para cualquier $v \in T$ el conjunto $T \setminus \{v\}$ es una cobertura de $G - \{v\}$ de tamaño a lo sumo $k - 1$: toda arista de $G - \{v\}$ es una arista de G que no toca a v, así que tiene un extremo en $T \setminus \{v\}$. Por lo tanto el ciclo de la línea 3 siempre encuentra un nodo que cumple la condición de la línea 5. Recíprocamente, si S es una cobertura de $G - \{v\}$, entonces $S \cup \{v\}$ es una cobertura de G, porque las únicas aristas que faltan

cubrir son las que tocan a v; su tamaño es a lo sumo $(k-1)+1=k$. Por inducción en k, el algoritmo devuelve una cobertura de tamaño a lo sumo k siempre que exista.

En cuanto al tiempo, cada nivel de la recursión hace a lo sumo $n+1$ consultas a la caja negra y $O(n(n+m))$ trabajo adicional, y la profundidad de la recursión es a lo sumo $k \leq n$, porque k baja en uno en cada nivel. En total hay $O(n^2)$ consultas y un trabajo adicional polinómico, de modo que la versión de búsqueda se reduce polinómicamente a la de decisión. Finalmente, la versión de optimización se obtiene de la de búsqueda: se encuentra con búsqueda binaria sobre $k \in \{0, \ldots, n\}$ el menor k^* para el que la respuesta es afirmativa, y se llama a BUSCAR-VC(G, k^*).

8.1.4 Reducción a un caso más general: *Vertex Cover* a *Set Cover*

Independent Set y *Vertex Cover* representan dos géneros distintos de problemas. *Independent Set* es un problema de empaquetado: el objetivo es meter la mayor cantidad posible de vértices, sujeta a conflictos (las aristas) que tratan de impedirlo. *Vertex Cover*, en cambio, es un problema de cobertura: el objetivo es cubrir todas las aristas del grafo usando la menor cantidad posible de vértices. *Vertex Cover* es un problema de cobertura expresado en el lenguaje de los grafos; hay un problema de cobertura más general, en el que se busca cubrir un conjunto arbitrario de objetos con una colección de conjuntos más chicos.

Definición 8.1.11 (*Set Cover*, cobertura de conjuntos)**.** Dado un conjunto finito U, un natural r y una colección $S_1, S_2, \ldots, S_\ell$ de subconjuntos de U, ¿es posible elegir no más de r de estos subconjuntos de modo que la unión de todos ellos sea igual a U?

Imaginemos, por ejemplo, que se tienen ℓ programas disponibles y un conjunto U de capacidades que se quiere que tenga un sistema. El programa i ofrece el conjunto $S_i \subseteq U$ de capacidades. En *Set Cover* se busca instalar pocos programas de modo que el sistema tenga todas las capacidades de U.

Intuitivamente, *Vertex Cover* es un caso particular de *Set Cover*: en *Set Cover* se cubre un conjunto arbitrario con subconjuntos arbitrarios, mientras que en *Vertex Cover* se cubren específicamente las aristas de un grafo, con conjuntos formados por las aristas que inciden en cada vértice.

Proposición 8.1.12. *Vertex Cover* $\leq_P$ *Set Cover*.

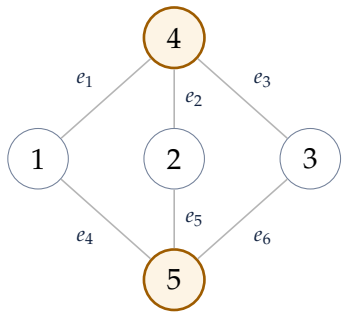
Demostración. Sea (G, k) una instancia de *Vertex Cover*, con $G = (V, E)$. Como el objetivo es cubrir las aristas de E, se construye la instancia de *Set Cover* en la que

$$U = E, \qquad r = k, \qquad S_i = \{e \in E : i \text{ es un extremo de } e\} \text{ para cada } i \in V.$$

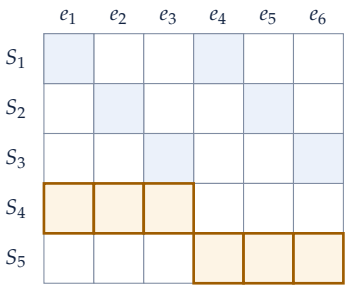
Cada vez que en *Vertex Cover* se elige un vértice i se cubren todas las aristas incidentes a él, que es exactamente lo que representa el conjunto S_i. La construcción recorre una vez las listas de adyacencia, así que se hace en tiempo polinómico.

Veamos que U se puede cubrir con a lo sumo k de los conjuntos $S_1, \ldots, S_n$ si y solo si G tiene una cobertura de tamaño a lo sumo k. Si $S_{i_1}, \ldots, S_{i_j}$, con $j \leq k$, cubren U, entonces toda arista de G incide en alguno de los vértices $i_1, \ldots, i_j$, y $\{i_1, \ldots, i_j\}$ es una cobertura de G de tamaño $j \leq k$. Recíprocamente, si $\{i_1, \ldots, i_j\}$ es una cobertura de G con $j \leq k$, entonces toda arista incide en alguno de esos vértices, y los conjuntos $S_{i_1}, \ldots, S_{i_j}$ cubren U. □

Ejemplo 8.1.13 (Del grafo de cinco nodos a una instancia de *Set Cover*)**.** Para el grafo del Ejemplo 8.1.8, con las aristas nombradas como en la parte (a) de la Figura 8.3, la reducción produce $U = \{e_1, \ldots, e_6\}$ y los conjuntos $S_1 = \{e_1, e_4\}$, $S_2 = \{e_2, e_5\}$, $S_3 = \{e_3, e_6\}$, $S_4 = \{e_1, e_2, e_3\}$ y $S_5 = \{e_4, e_5, e_6\}$. Con $r = 2$ la respuesta es afirmativa: $S_4 \cup S_5 = U$, que corresponde a la cobertura $\{4, 5\}$.



(a) el grafo, con las aristas nombradas



(b) los conjuntos S_i: la celda (S_i, e_j) está pintada si $e_j \in S_i$

Figura 8.3. La reducción de *Vertex Cover* a *Set Cover* sobre el grafo del Ejemplo 8.1.8. Cada vértice i da el conjunto S_i de aristas incidentes a él. La cobertura $\{4, 5\}$ del grafo corresponde a los conjuntos S_4 y S_5, resaltados, que juntos contienen a todas las aristas: cada columna tiene una celda resaltada.

Hay algo que vale la pena notar tanto en esta demostración como en la de la Proposición 8.1.10. Aunque la definición de $\leq_P$ permite muchas llamadas a la caja negra, se hizo una sola, y se usó su respuesta tal cual: las dos son reducciones a la Karp.

8.1.5 *Independent Set* a *Set Packing*

Así como *Set Cover* generaliza a *Vertex Cover*, hay una generalización natural de *Independent Set* como problema de empaquetado de conjuntos arbitrarios.

Definición 8.1.14 (*Set Packing*, empaquetado de conjuntos)**.** Dado un conjunto finito U, un natural r y una colección $S_1, S_2, \ldots, S_\ell$ de subconjuntos de U, ¿es posible elegir al menos r de estos subconjuntos que sean disjuntos dos a dos?

Por ejemplo, U puede ser un conjunto de recursos que no se pueden compartir, y S_i el conjunto de recursos que necesita el proceso i para ejecutar. En *Set Packing* se busca una colección grande de procesos que se puedan ejecutar simultáneamente sin que ningún par de ellos requiera un mismo recurso. La reducción es análoga a la de la Proposición 8.1.12, con los mismos conjuntos.

Proposición 8.1.15. *Independent Set* $\leq_P$ *Set Packing*.

Demostración. Dada una instancia (G, k) de *Independent Set*, con $G = (V, E)$, se construye la instancia de *Set Packing* con

$$U = E, \qquad r = k, \qquad S_i = \{e \in E : i \text{ es un extremo de } e\} \text{ para cada } i \in V,$$

también en tiempo polinómico. La observación clave es que, para dos vértices distintos i y j, los conjuntos S_i y S_j se intersecan si y solo si i y j son adyacentes: la única arista que puede pertenecer a los dos es (i, j).

Si I es un conjunto independiente de G con $|I| \geq k$, los conjuntos S_i con $i \in I$ son al menos k y son disjuntos dos a dos, porque ningún par de vértices de I es adyacente. Recíprocamente, si $S_{i_1}, \ldots, S_{i_j}$, con $j \geq k$, son disjuntos dos a dos, ningún par de los vértices $i_1, \ldots, i_j$ es adyacente, y $\{i_1, \ldots, i_j\}$ es un conjunto independiente de tamaño al menos k. Los vértices aislados no traen problemas: su conjunto es vacío, que es disjunto de cualquier otro, y un vértice aislado se puede agregar a cualquier conjunto independiente. □

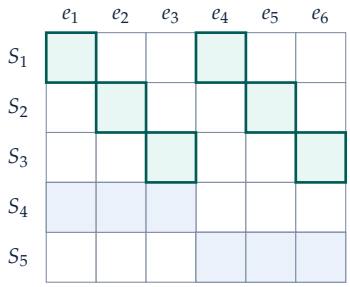


Figura 8.4. La reducción de *Independent Set* a *Set Packing* sobre el grafo de la Figura 8.3 produce los mismos conjuntos. El conjunto independiente $\{1, 2, 3\}$ corresponde a los conjuntos S_1, S_2 y S_3, resaltados, que son disjuntos dos a dos: cada columna tiene a lo sumo una celda resaltada.

8.2 Reducciones mediante artilugios: el problema de satisfacibilidad

Los problemas de esta sección son más abstractos: se formulan en notación booleana y modelan situaciones en las que hay que fijar variables de decisión de modo de satisfacer un conjunto de restricciones, algo muy común, por ejemplo, en inteligencia artificial.

8.2.1 SAT y 3SAT

Sea $X = \{x_1, \ldots, x_n\}$ un conjunto de n variables booleanas, cada una de las cuales puede tomar el valor 0 o 1 (falso o verdadero).

Definición 8.2.1 (Términos, cláusulas y asignaciones)**.** Un término sobre X es una de las variables x_i o su negación $\overline{x}_i$. Una cláusula es una disyunción de términos distintos,

$$C = t_1 \vee t_2 \vee \cdots \vee t_\ell,$$

y su largo es la cantidad ℓ de términos que contiene. Una asignación de verdad para X es una función $\nu : X \to \{0, 1\}$ que le da un valor a cada variable; implícitamente le da a $\overline{x}_i$ el valor opuesto al de x_i. Una asignación satisface una cláusula si al evaluarla con esos valores la cláusula vale 1, es decir, si al menos uno de sus términos recibe el valor 1. Una asignación satisface una colección de cláusulas $C_1, \ldots, C_k$ si las satisface a todas, o sea, si hace valer 1 a la conjunción

$$\Phi = C_1 \wedge C_2 \wedge \cdots \wedge C_k.$$

En ese caso se dice que ν es una asignación que satisface la colección, y que la colección es satisfacible.

Por ejemplo, la única asignación que no satisface la cláusula $x_1 \vee \overline{x}_2$ es $x_1 = 0$, $x_2 = 1$. Consideremos las tres cláusulas

$$(x_1 \vee \overline{x}_2), \qquad (\overline{x}_1 \vee \overline{x}_3), \qquad (x_2 \vee \overline{x}_3).$$

La asignación que les da el valor 1 a todas las variables no las satisface, porque falla la segunda cláusula; en cambio, la que les da el valor 0 a todas sí las satisface. Por otro lado, no hay ninguna asignación que satisfaga

$$\Phi = (x_1 \vee x_2) \wedge (x_1 \vee \overline{x}_2) \wedge (\overline{x}_1 \vee x_2) \wedge (\overline{x}_1 \vee \overline{x}_2) :$$

cada una de las cuatro cláusulas prohíbe exactamente una de las cuatro combinaciones de valores de x_1 y x_2.

Definición 8.2.2 (SAT)**.** Dado un conjunto de cláusulas $C_1, \ldots, C_k$ sobre un conjunto de variables booleanas $X = \{x_1, \ldots, x_n\}$, ¿existe una asignación de verdad para X que satisfaga simultáneamente todas las cláusulas?

Definición 8.2.3 (3SAT)**.** Es la restricción de SAT en la que todas las cláusulas tienen exactamente tres términos, correspondientes a tres variables distintas.

SAT y 3SAT son problemas de búsqueda combinatoria fundamentales, porque contienen los ingredientes básicos de un problema computacionalmente difícil en su forma más despojada. Hay que tomar n decisiones independientes (los valores de las x_i) de modo de satisfacer un conjunto de restricciones. Cada restricción aislada se puede satisfacer de varias maneras, pero las decisiones se tienen que coordinar para que se satisfagan todas a la vez.

8.2.2 SAT y 3SAT son equivalentes

Como toda instancia de 3SAT es en particular una instancia de SAT, se cumple trivialmente 3SAT $\leq_P$ SAT: la transformación es la identidad. Más interesante es que también vale la reducción en el otro sentido, con lo cual la restricción de largo tres no le quita dificultad al problema.

Proposición 8.2.4. SAT $\leq_P$ 3SAT.

Demostración. Hay que definir un algoritmo que, dado un conjunto de cláusulas $\mathcal{C} = \{C_1, \ldots, C_k\}$ sobre las variables X, construya un conjunto de cláusulas de tres términos $\mathcal{C}'$ sobre un conjunto de variables $X' \supseteq X$, de modo que exista una asignación para X

que satisface $\mathcal{C}$ si y solo si existe una asignación para X' que satisface $\mathcal{C}'$, y probar que el algoritmo corre en tiempo polinómico.

Primero se limpia la instancia. Se eliminan las cláusulas que contienen a la vez una variable y su negación, como $x_1 \vee \overline{x}_1 \vee x_2$, que se satisfacen con cualquier asignación, y se eliminan los términos repetidos dentro de cada cláusula, reemplazando por ejemplo $x_1 \vee x_2 \vee x_2$ por $x_1 \vee x_2$. La instancia resultante es satisfacible si y solo si lo era la original, y en ninguna de sus cláusulas se repite una variable. A partir de ella se construye la instancia de 3SAT, empezando con $X' = X$ y $\mathcal{C}'$ vacío.

Se agregan dos variables nuevas u y w, cuyo papel es valer siempre 0, junto con las seis cláusulas

$$\overline{u} \vee w \vee x_1, \quad \overline{u} \vee w \vee \overline{x}_1, \quad \overline{u} \vee \overline{w} \vee x_1,$$
$$\overline{u} \vee \overline{w} \vee \overline{x}_1, \quad \overline{w} \vee u \vee x_1, \quad \overline{w} \vee u \vee \overline{x}_1,$$

donde x_1 es una variable cualquiera de X. Las primeras cuatro recorren las cuatro combinaciones de valores de w y x_1 junto con $\overline{u}$: si $u = 1$, alguna de ellas queda con sus tres términos en 0, así que toda asignación que las satisface tiene $u = 0$. Con $u = 0$, las dos últimas se reducen a $\overline{w} \vee x_1$ y $\overline{w} \vee \overline{x}_1$, que juntas obligan a $w = 0$. Recíprocamente, con $u = w = 0$ las seis cláusulas se satisfacen, cualquiera sea el valor de x_1. Después, cada cláusula C_i de la instancia limpia se reemplaza según su largo r.

- Si $r = 1$, con $C_i = t_1$, se agrega la cláusula $t_1 \vee u \vee w$.
- Si $r = 2$, con $C_i = t_1 \vee t_2$, se agrega la cláusula $t_1 \vee t_2 \vee u$.
- Si $r = 3$, se agrega C_i sin cambios.
- Si $r \geq 4$, con $C_i = t_1 \vee t_2 \vee \cdots \vee t_r$, se agregan a X' las variables nuevas $z_2, z_3, \ldots, z_{r-2}$ (propias de esta cláusula) y a $\mathcal{C}'$ las $r - 2$ cláusulas

$$\begin{aligned} C'_2 &= t_1 \vee t_2 \vee z_2, \\ C'_\ell &= \overline{z}_{\ell-1} \vee t_\ell \vee z_\ell \qquad \text{para } \ell = 3, \ldots, r-2, \\ C'_{r-1} &= \overline{z}_{r-2} \vee t_{r-1} \vee t_r. \end{aligned}$$

Todas las cláusulas nuevas tienen tres términos sobre variables distintas, porque en la instancia limpia ninguna cláusula repite variables y las variables u, w y z_ℓ son nuevas.

Como en toda asignación que satisface $\mathcal{C}'$ se tiene $u = w = 0$, las cláusulas de los dos primeros casos se satisfacen exactamente cuando se satisface la cláusula original. Falta ver que, para una asignación fija de X, la cadena de un caso $r \geq 4$ se puede satisfacer eligiendo valores para las z_ℓ si y solo si C_i se satisface.

Supongamos que algún término t_j vale 1, y sea c el índice de la cláusula de la cadena que lo contiene: $c = 2$ si $j \leq 2$, $c = j$ si $3 \leq j \leq r - 2$ y $c = r - 1$ si $j \geq r - 1$. Se elige $z_\ell = 1$ para $\ell < c$ y $z_\ell = 0$ para $\ell \geq c$. Las cláusulas C'_ℓ con $\ell < c$ se satisfacen por su término z_ℓ, la cláusula C'_c por t_j, y las cláusulas C'_ℓ con $\ell > c$ por su término $\overline{z}_{\ell-1}$, ya que $\ell - 1 \geq c$. Recíprocamente, si todos los t_j valen 0, la cláusula C'_2 obliga a $z_2 = 1$;

entonces C'_3 obliga a $z_3 = 1$, y así sucesivamente hasta $z_{r-2} = 1$; pero entonces C'_{r-1} tiene sus tres términos en 0. Es decir, la cadena no se puede satisfacer.

Juntando todo, $\mathcal{C}$ es satisfacible si y solo si $\mathcal{C}'$ lo es. En cuanto al tiempo, la limpieza es lineal en el tamaño de la instancia, se agregan seis cláusulas fijas, y una cláusula de largo r se reemplaza por a lo sumo $\max(1, r-2)$ cláusulas con a lo sumo $r-3$ variables nuevas. El tamaño de la instancia construida es lineal en el de la original, y el algoritmo corre en tiempo polinómico. □

Ejemplo 8.2.5 (Una cláusula de largo cinco)**.** La cláusula $C = x_1 \vee \overline{x}_2 \vee x_3 \vee x_4 \vee \overline{x}_5$ tiene largo $r = 5$, así que se agregan dos variables nuevas z_2, z_3 y se la reemplaza por

$$x_1 \vee \overline{x}_2 \vee z_2, \qquad \overline{z}_2 \vee x_3 \vee z_3, \qquad \overline{z}_3 \vee x_4 \vee \overline{x}_5.$$

Si una asignación hace $x_3 = 1$ (y, por ejemplo, $x_1 = 0$, $x_2 = 1$, $x_4 = 0$, $x_5 = 1$), el término verdadero está en la cláusula del medio: se elige $z_2 = 1$, que satisface la primera, y $z_3 = 0$, que satisface la tercera. Las variables z_ℓ funcionan como un testigo que avanza por la cadena hasta llegar a la cláusula que contiene un término verdadero. Si ningún término de C es verdadero, la primera cláusula obliga a $z_2 = 1$, la segunda a $z_3 = 1$, y la tercera queda en 0.

8.2.3 3SAT a *Independent Set*

La dificultad computacional de SAT y 3SAT parece de naturaleza muy distinta a la de buscar conjuntos independientes o coberturas en grafos. Relacionarlas no es trivial: 3SAT trata de fijar variables booleanas en presencia de restricciones, mientras que *Independent Set* trata de elegir vértices de un grafo. Para resolver una instancia de 3SAT con una caja negra para *Independent Set* hace falta codificar todas esas restricciones booleanas en los nodos y las aristas de un grafo, de modo que la satisfacibilidad corresponda a la existencia de un conjunto independiente grande. Esto ilustra un principio general para diseñar reducciones $Y \leq_P X$ complejas: construir artilugios (en inglés, *gadgets*) con componentes del problema X que representen lo que pasa en el problema Y.

La clave para pensar la reducción es notar que hay dos formas conceptualmente distintas de ver una instancia de 3SAT.

- Una es la que se describió antes: hay que tomar una decisión independiente, 0 o 1, para cada una de las n variables, y se tiene éxito si se consigue, para cada cláusula, alguna de las tres formas de satisfacerla.
- La otra es la siguiente: hay que elegir un término de cada cláusula y después encontrar una asignación que haga valer 1 a todos los términos elegidos, con lo cual se satisfacen todas las cláusulas. Se tiene éxito si se puede elegir un término de cada cláusula de modo que no haya dos elegidos en conflicto, donde dos términos están en conflicto si

uno es una variable x_i y el otro es su negación $\overline{x}_i$. Si se evitan los conflictos, se puede encontrar una asignación que haga verdaderos a todos los términos elegidos.

Proposición 8.2.6. 3SAT $\leq_P$ *Independent Set*.

Demostración. Sea una instancia de 3SAT con variables $X = \{x_1, \ldots, x_n\}$ y cláusulas $C_1, \ldots, C_k$. La reducción se basa en la segunda forma de verla. Se construye un grafo $G = (V, E)$ con $3k$ nodos agrupados en k triángulos: para cada $i = 1, \ldots, k$ se crean tres nodos v_{i1}, v_{i2}, v_{i3} unidos entre sí por aristas, y se etiqueta v_{ij} con el j-ésimo término de la cláusula C_i.

Antes de seguir, conviene ver cómo son los conjuntos independientes de tamaño k de este grafo: como no se pueden elegir dos nodos de un mismo triángulo, son exactamente las formas de elegir un nodo de cada triángulo. Eso implementa el objetivo de elegir en cada cláusula un término que valga 1, pero todavía no impide elegir dos términos en conflicto. Los conflictos se codifican agregando más aristas: para cada par de nodos cuyas etiquetas son términos en conflicto se agrega una arista entre ellos. La instancia de *Independent Set* es (G, k), y se construye en tiempo $O(k^2)$ comparando las etiquetas de todos los pares de nodos.

Veamos que la instancia de 3SAT es satisfacible si y solo si G tiene un conjunto independiente de tamaño al menos k. Si la instancia es satisfacible, fijemos una asignación que la satisface. Cada triángulo tiene al menos un nodo cuya etiqueta vale 1; sea S un conjunto formado por uno de esos nodos de cada triángulo, que tiene tamaño k. S es independiente: dos nodos de S están en triángulos distintos, y si hubiera una arista entre ellos sus etiquetas estarían en conflicto, lo que es imposible porque las dos valen 1.

Recíprocamente, supongamos que G tiene un conjunto independiente S de tamaño al menos k. Como no puede tener dos nodos de un mismo triángulo, tiene exactamente k nodos, uno de cada triángulo. Se construye una asignación ν que hace valer 1 a todas las etiquetas de S. Para cada variable x_i, si ni x_i ni $\overline{x}_i$ son etiqueta de algún nodo de S, se elige $\nu(x_i) = 1$ arbitrariamente. Si no, aparece exactamente una de las dos: si aparecieran x_i en un nodo de S y $\overline{x}_i$ en otro, habría una arista entre esos nodos, contra la independencia de S. Si aparece x_i se elige $\nu(x_i) = 1$, y si aparece $\overline{x}_i$ se elige $\nu(x_i) = 0$. Con esta asignación todas las etiquetas de S valen 1, y como hay una en cada triángulo, todas las cláusulas se satisfacen. □

Ejemplo 8.2.7 (Una instancia de 3SAT como grafo)**.** Para

$$\Phi = (x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \overline{x}_3 \vee x_4) \wedge (\overline{x}_1 \vee x_2 \vee x_4)$$

la reducción construye el grafo de la Figura 8.5, con tres triángulos y $k = 3$. Hay tres pares de términos en conflicto: el x_1 de C_1 con el $\overline{x}_1$ de C_3, el x_1 de C_2 con el mismo $\overline{x}_1$, y el x_3 de C_1 con el $\overline{x}_3$ de C_2. El conjunto formado por el x_2 de C_1, el x_4 de C_2 y el x_2 de C_3 es independiente y tiene tamaño 3. La asignación que se deduce le da el valor 1 a x_2 y a x_4; x_1 y x_3 no aparecen en el conjunto, así que reciben el valor 1 por convención. Se verifica directamente que satisface las tres cláusulas.

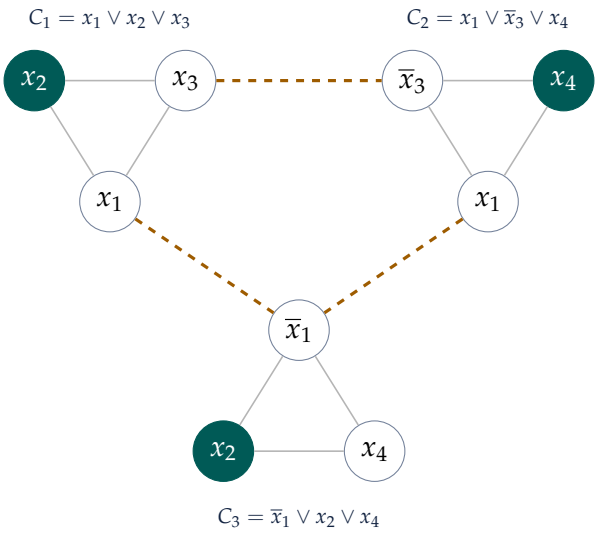


Figura 8.5. El grafo que construye la reducción de 3SAT a *Independent Set* para la fórmula del Ejemplo 8.2.7. Cada cláusula da un triángulo, y las aristas discontinuas unen términos en conflicto. Los nodos resaltados forman un conjunto independiente de tamaño 3, uno por triángulo, y corresponden a una asignación que satisface la fórmula.

8.2.4 Transitividad de las reducciones

Los problemas vistos hasta ahora son de sabores distintos, pero resultaron estar muy relacionados. Se pueden deducir más relaciones gracias a que $\leq_P$ es transitiva.

Proposición 8.2.8. Si $Z \leq_P Y$ e $Y \leq_P X$, entonces $Z \leq_P X$.

Demostración. Dada una caja negra para X, se muestra cómo resolver una instancia de Z: basta componer los dos algoritmos que dan $Z \leq_P Y$ e $Y \leq_P X$. Se ejecuta el algoritmo

para Z que usa una caja negra para Y, y cada vez que se llama a esa caja negra se la simula, con una cantidad polinómica de pasos, mediante el algoritmo que resuelve Y usando la caja negra para X. Como en la demostración de la Proposición 8.1.2, las instancias de Y que se generan tienen tamaño polinómico, y la composición de polinomios es un polinomio, así que el total de pasos y de llamadas a la caja negra para X es polinómico. □

Por ejemplo, como se probó 3SAT $\leq_P$ *Independent Set* $\leq_P$ *Vertex Cover* $\leq_P$ *Set Cover*, se concluye que 3SAT $\leq_P$ *Set Cover*, y como además SAT $\leq_P$ 3SAT, también SAT $\leq_P$ *Set Cover*.

Vale la pena resumir lo visto. Aparecieron tres tipos de problemas: de empaquetado (*Independent Set*, *Set Packing*), de cobertura (*Vertex Cover*, *Set Cover*) y de satisfacción de restricciones (SAT, 3SAT). Y aparecieron tres estrategias de reducción: la equivalencia sencilla, como entre *Independent Set* y *Vertex Cover*; el caso particular, como *Vertex Cover* dentro de *Set Cover*; y la reducción mediante artilugios, como la de 3SAT a *Independent Set*.

8.3 Certificación eficiente y la clase $\mathcal{NP}$

La reducibilidad entre problemas es el primer ingrediente del estudio de la intratabilidad. El segundo es una caracterización de la clase de problemas con la que se está trabajando.

En la Sección 1.8, al presentar el Conjunto Independiente, se señaló algo bueno que se puede decir de él desde el punto de vista computacional: si un grafo contiene un conjunto independiente de tamaño al menos k, es fácil convencer a alguien de ese hecho, simplemente mostrándole el conjunto. Del mismo modo, si una instancia de 3SAT es satisfacible, se lo puede demostrar exhibiendo una asignación que la satisface. Encontrar esa asignación puede ser enormemente difícil, pero una vez hecho el trabajo, cualquiera puede reemplazar los valores en las cláusulas y comprobar que todas se satisfacen.

Lo que aparece acá es el contraste entre encontrar una solución y verificar una solución propuesta. Para *Independent Set* o 3SAT no se conoce un algoritmo de tiempo polinómico que encuentre soluciones, pero verificar una solución propuesta se hace fácilmente en tiempo polinómico. Que esto no es una observación trivial se ve al considerar el problema opuesto: demostrar que una instancia de 3SAT no es satisfacible. ¿Qué evidencia se podría mostrar que convenza a alguien, en tiempo polinómico, de que ninguna asignación la satisface?

8.3.1 Problemas de decisión como conjuntos de cadenas

Para formalizar esta idea se fija un formato de codificación: la entrada de un problema se representa con una cadena binaria finita s, cuyo largo se denota $|s|$. Con esa convención, un algoritmo que resuelve un problema de decisión X es simplemente la implementación de una función $f : \{0,1\}^\star \to \{0,1\}$, donde $\{0,1\}^\star$ es el conjunto de todas las cadenas binarias finitas. Por ejemplo, un algoritmo para *Independent Set* toma como entrada una cadena binaria que representa a un grafo y a un natural k, y devuelve un valor en $\{0,1\}$ (no o sí).

Las instancias SÍ de X son aquellas para las que la respuesta es afirmativa, y un problema de decisión X se identifica con el conjunto de cadenas que representan instancias SÍ; así, $X \subseteq \{0,1\}^\star$, y $s \in X$ significa que s es una instancia SÍ.

Definición 8.3.1 (Algoritmo eficiente para un problema de decisión)**.** Un algoritmo A resuelve el problema de decisión X si toma como entrada una cadena binaria s, devuelve un valor $A(s) \in \{0,1\}$, y cumple que $A(s) = 1$ si y solo si $s \in X$. El algoritmo es eficiente si además existe un polinomio p tal que, para toda cadena s, A termina en a lo sumo $p(|s|)$ pasos, es decir, en tiempo $O(p(|s|))$.

Ejemplo 8.3.2 (Codificación de una instancia de *Independent Set*)**.** Tomemos el grafo del Ejemplo 8.1.8 con $k = 3$. Una codificación posible usa ℓ bits para escribir en binario los números n y k; primero se escribe ℓ en unario, y después n, las filas de la matriz de adyacencia y k.

Parte de la codificación	**Bits**
$\ell = 3$ en unario (bits para representar n y k)	`0001`
$n = 5$ en binario con ℓ bits	`101`
filas 1, 2 y 3 de la matriz de adyacencia	`00011 00011 00011`
filas 4 y 5 de la matriz de adyacencia	`11100 11100`
$k = 3$ en binario con ℓ bits	`011`

Concatenando se obtiene la cadena $s =$ `00011010001100011000111110011100011`, de largo 35. La misma instancia con $k = 4$ se codifica con la cadena s' que coincide con s salvo en los últimos tres bits, que pasan a ser `100`.

Definición 8.3.3 (La clase $\mathcal{P}$)**.** $\mathcal{P}$ es el conjunto de los problemas de decisión para los que existe un algoritmo eficiente.

8.3.2 Certificación eficiente

¿Cómo formalizar la idea de que una solución se puede verificar eficientemente, independientemente de que el problema se pueda resolver eficientemente? Un algoritmo verificador tiene una estructura distinta de la de un algoritmo que resuelve el problema: para verificar una solución necesita la cadena de entrada s y además una cadena separada t, el certificado, que contiene la evidencia de que s es una instancia SÍ.

Definición 8.3.4 (Certificador eficiente)**.** Un algoritmo B es un certificador eficiente para un problema de decisión X si

- toma como entrada dos cadenas binarias s y t y devuelve una respuesta $B(s,t) \in \{0,1\}$;
- termina en tiempo polinómico en $|s| + |t|$;
- existe un polinomio p tal que, para toda cadena $s \in \{0,1\}^\star$,

$$s \in X \quad \text{si y solo si} \quad \text{existe } t \in \{0,1\}^\star,\ |t| \leq p(|s|),\ \text{tal que } B(s,t) = 1.$$

Aquí s representa una instancia de X, y t un certificado que, cuando $s \in X$, comprueba que la respuesta es afirmativa.

Hace falta un rato para asimilar lo que dice esta definición. Conviene ver a un certificador eficiente como alguien que encara el problema X desde un rol gerencial: no intenta decidir por su cuenta si s pertenece a X, sino que está dispuesto a evaluar eficientemente pruebas t de que s pertenece a X, siempre que no sean demasiado largas. Y es correcto en un sentido débil: s pertenece a X si y solo si existe alguna prueba que lo convenza. Para las instancias NO, en cambio, ningún certificado de largo acotado logra que responda 1.

Ejemplo 8.3.5 (Qué exige la definición)**.** Sea B un certificador eficiente para *Independent Set* basado en la codificación del Ejemplo 8.3.2, con polinomio p. Como $(G,3)$ es una instancia SÍ, tiene que existir una cadena t con $|t| \leq p(35)$ tal que $B(s,t) = 1$. Como $(G,4)$ es una instancia NO, para la cadena s' se tiene que cumplir que $B(s',t') = 0$ para toda cadena t' con $|t'| \leq p(35)$.

Un certificador eficiente B se puede usar como núcleo de un algoritmo de fuerza bruta para X: dada s, se prueban todas las cadenas t de largo a lo sumo $p(|s|)$ y se responde 1 si alguna cumple $B(s,t) = 1$. Pero la existencia de B no da ninguna manera evidente de diseñar un algoritmo eficiente para X: sigue haciendo falta encontrar una cadena t que haga que B responda 1, y hay del orden de $2^{p(|s|)}$ candidatas.

Un certificador eficiente para Independent Set

Un certificado natural para *Independent Set* es una lista L de vértices que representa un conjunto $S \subseteq V$. Para el grafo del Ejemplo 8.1.8, con $V = \{1,2,3,4,5\}$, se identifica cada vértice v con la representación binaria de $v-1$ en $\ell = 3$ bits: el vértice 1 con `000`, el 2 con `001`, y así hasta el 5, con `100`. El conjunto $S = \{1,2,3\}$ se representa entonces con el certificado $t =$ `000 001 010`. El certificador recibe la instancia (G,k) y la lista L.

CERTIFICADOR-IS$((G,k), L)$

```
1  for each nodo v ∈ V do enL[v] ← falso
2  cant ← 0
3  for each v en L do
4      if v no es un nodo de G then return 0
5      if not enL[v] then enL[v] ← verdadero; cant ← cant + 1
6  end for
7  if cant < k then return 0
8  for each arista (u, v) ∈ E do
9      if enL[u] and enL[v] then return 0
10 return 1
```

> **Estructuras de datos.** El grafo viene dado por su matriz de adyacencia, como en la codificación, y el recorrido de las aristas de la línea 8 la recorre entera en $O(n^2)$. L es una lista con los identificadores de los vértices, en el orden en que aparecen en el certificado. *enL* es un arreglo booleano indexado por los vértices $1,\ldots,n$: permite saber en $O(1)$ si un vértice ya apareció en L, lo que sirve tanto para contar los vértices distintos como para chequear cada arista.

Proposición 8.3.6. CERTIFICADOR-IS es un certificador eficiente para *Independent Set*.

Demostración. El tiempo de ejecución es $O(n + |L| + n^2)$, polinómico en $|V|$ y en $|L|$ y, por lo tanto, en la suma de los largos de las entradas.

Supongamos que existe un conjunto independiente S de tamaño al menos k. Entonces existe una lista L que representa a S sin elementos repetidos; tiene $|S| \leq n$ elementos de ℓ bits cada uno, así que su largo es polinómico en $|V|$ y, por lo tanto, en $|s|$. Con entradas (G,k) y L el algoritmo devuelve 1: como $|S| \geq k$, la condición de la línea 7 es falsa, y por la definición de conjunto independiente nunca se cumple la condición de la línea 9. En conclusión, si (G,k) es una instancia SÍ, existe un certificado L de largo polinómico en $|s|$ que hace que el algoritmo responda 1.

Supongamos ahora que no existe un conjunto independiente de tamaño al menos k, y sea L una lista cualquiera. Si L contiene algo que no es un vértice, el algoritmo devuelve 0 en la línea 4. Si no, o bien L tiene menos de k vértices distintos, y se devuelve 0 en la

línea 7, o bien tiene al menos k, y entonces existe alguna arista con los dos extremos en L, porque de lo contrario los vértices de L formarían un conjunto independiente de tamaño al menos k. Para esa arista se cumple la condición de la línea 9 y se devuelve 0. En conclusión, si (G, k) es una instancia NO, ninguna lista L hace que el algoritmo responda 1. □

8.3.3 La clase $\mathcal{NP}$

Definición 8.3.7 (La clase $\mathcal{NP}$)**.** $\mathcal{NP}$ es el conjunto de los problemas de decisión para los que existe un certificador eficiente.

El nombre $\mathcal{NP}$ es una sigla de *nondeterministic polynomial time*, tiempo polinómico no determinista: el acto de buscar una cadena t que haga que un certificador eficiente acepte la entrada s se suele ver como una búsqueda no determinista en el espacio de las pruebas posibles. $\mathcal{NP}$ no significa "no polinómico". Una primera observación es inmediata.

Proposición 8.3.8. $\mathcal{P} \subseteq \mathcal{NP}$.

Demostración. Sea $X \in \mathcal{P}$; existe entonces un algoritmo A de tiempo polinómico que resuelve X. Para ver que $X \in \mathcal{NP}$ hay que exhibir un certificador eficiente B para X, y es muy fácil: ante el par (s, t), B devuelve directamente $A(s)$. Es un gerente muy práctico, que ignora la prueba propuesta y resuelve el problema por su cuenta. B corre en tiempo polinómico, porque A lo hace. Si $s \in X$, entonces $B(s, t) = 1$ para todo t, en particular para t vacío; y si $s \notin X$, entonces $B(s, t) = 0$ para todo t. □

Es fácil ver que los problemas de las secciones anteriores pertenecen a $\mathcal{NP}$; basta decir cómo usa el certificado un certificador eficiente para cada uno.

- En SAT y 3SAT el certificado t es una asignación de valores de verdad a las variables, codificada con n bits, y el certificador evalúa cada cláusula con esa asignación, en tiempo lineal en su largo.
- En *Independent Set* el certificado es un conjunto de al menos k vértices, y el certificador controla que ninguna arista una dos de ellos (Proposición 8.3.6).
- En *Vertex Cover* el certificado es un conjunto de a lo sumo k vértices, y el certificador controla que toda arista tenga un extremo en él.
- En *Set Cover* el certificado es una lista de a lo sumo r de los conjuntos dados, y el certificador controla que su unión sea igual a U.
- En *Set Packing* el certificado es una lista de al menos r de los conjuntos dados, y el certificador controla que sean disjuntos dos a dos.

Sin embargo, no se sabe demostrar que alguno de estos problemas requiera más que tiempo polinómico. De hecho, no se sabe demostrar que exista algún problema de $\mathcal{NP}$ que no pertenezca a $\mathcal{P}$. Así que en lugar de un teorema concreto solo queda una pregunta.

Nota 8.3.9 (¿$\mathcal{P} = \mathcal{NP}$?). ¿Existe algún problema en $\mathcal{NP}$ que no pertenezca a $\mathcal{P}$? Dicho de otro modo, ¿vale $\mathcal{P} = \mathcal{NP}$? Es una de las preguntas abiertas más famosas de la computación.

La creencia general es que $\mathcal{P} \neq \mathcal{NP}$, y se la toma como hipótesis de trabajo en toda el área, aunque no hay mucha evidencia técnica sólida a su favor. Se basa más bien en la sensación de que $\mathcal{P} = \mathcal{NP}$ sería demasiado asombroso para ser cierto. ¿Cómo podría haber una transformación general de la tarea de verificar una solución en la tarea, mucho más difícil, de encontrarla? ¿Cómo podría existir un método general para diseñar algoritmos eficientes, capaz de resolver todos estos problemas difíciles, que nadie haya descubierto? Se ha invertido una enorme cantidad de esfuerzo en intentos fallidos de encontrar algoritmos de tiempo polinómico para problemas difíciles de $\mathcal{NP}$, y la explicación más natural de ese fracaso constante es que esos problemas simplemente no se pueden resolver en tiempo polinómico.

8.4 Problemas $\mathcal{NP}$-completos

Ante la falta de avances en la pregunta de si $\mathcal{P} = \mathcal{NP}$, se pasó a una pregunta relacionada pero más abordable: ¿cuáles son los problemas más difíciles de $\mathcal{NP}$? La reducibilidad polinómica da una manera de encararla y de entender mejor la estructura de $\mathcal{NP}$.

Definición 8.4.1 (Problema $\mathcal{NP}$-completo)**.** Un problema de decisión X es $\mathcal{NP}$-completo si

(I) $X \in \mathcal{NP}$, y

(II) $Y \leq_P X$ para todo $Y \in \mathcal{NP}$.

Es decir, todo problema de $\mathcal{NP}$ se puede reducir a X: la clase de los problemas $\mathcal{NP}$-completos está formada por los problemas más difíciles de $\mathcal{NP}$. La expresión "más difíciles" está justificada en un sentido preciso.

Proposición 8.4.2. Sea X un problema $\mathcal{NP}$-completo. Entonces existe un algoritmo eficiente para X si y solo si $\mathcal{P} = \mathcal{NP}$.

Demostración. Si $\mathcal{P} = \mathcal{NP}$, X se resuelve en tiempo polinómico porque pertenece a $\mathcal{NP}$. Recíprocamente, supongamos que X se resuelve en tiempo polinómico. Si Y es cualquier otro problema de $\mathcal{NP}$, se tiene $Y \leq_P X$, y por la Proposición 8.1.2 Y se resuelve en tiempo polinómico. Por lo tanto $\mathcal{NP} \subseteq \mathcal{P}$, y con la Proposición 8.3.8 se obtiene $\mathcal{P} = \mathcal{NP}$. □

Las consecuencias de la proposición son dos caras de la misma moneda:

- si se encuentra un algoritmo eficiente para algún problema $\mathcal{NP}$-completo, todos los problemas de $\mathcal{NP}$ se pueden resolver eficientemente;
- si se demuestra que algún problema de $\mathcal{NP}$ no se puede resolver eficientemente, entonces ningún problema $\mathcal{NP}$-completo se puede resolver eficientemente.

8.4.1 *Circuit Satisfiability*: un primer problema $\mathcal{NP}$-completo

La definición de $\mathcal{NP}$-completitud tiene propiedades muy lindas, pero antes de entusiasmarse conviene notar algo: no es para nada evidente que existan problemas $\mathcal{NP}$-completos. ¿Por qué no podría haber dos problemas incomparables X y X', de modo que ningún problema de $\mathcal{NP}$ fuera al menos tan difícil como los dos? ¿Por qué no podría haber una sucesión infinita de problemas $X_1, X_2, X_3, \ldots$ en $\mathcal{NP}$, cada uno estrictamente más difícil que el anterior? Para probar que un problema es $\mathcal{NP}$-completo hay que mostrar cómo puede codificar cualquier problema de $\mathcal{NP}$, algo mucho más delicado que lo que se hizo en las secciones anteriores, donde se codificaban problemas concretos en términos de otros.

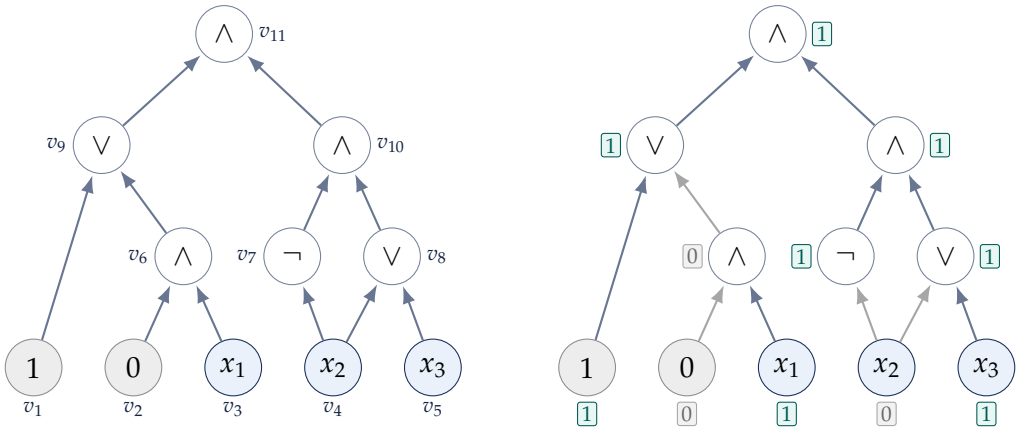
En 1971, Cook y Levin mostraron independientemente cómo hacerlo para problemas muy naturales de $\mathcal{NP}$. Quizás la elección más natural para un primer problema $\mathcal{NP}$-completo sea el de satisfacción de circuitos. Para plantearlo hay que precisar qué es un circuito. Se usan los operadores booleanos con los que se definió SAT: $\wedge$ (AND), $\vee$ (OR) y $\neg$ (NOT). La definición representa un circuito físico hecho de compuertas que implementan esos operadores.

Definición 8.4.3 (Circuito)**.** Un circuito es un grafo dirigido acíclico con las siguientes características.

- Los vértices sin aristas entrantes se llaman fuentes. Cada fuente está etiquetada con el nombre de una variable de $\{x_1, \ldots, x_n\}$, y se dice que es una fuente variable, o con una constante 0 o 1, y se dice que es una fuente constante. Las fuentes variables son las entradas del circuito, y cada variable etiqueta a una sola fuente.
- Los demás vértices están etiquetados con uno de los operadores $\wedge$, $\vee$ o $\neg$. Los vértices de tipo $\wedge$ y $\vee$ tienen dos aristas entrantes, y los de tipo $\neg$ tienen una única arista entrante.
- Todos los vértices tienen al menos una arista saliente, salvo un único vértice, distinguido como la salida del circuito, que no tiene ninguna.

Un circuito calcula una función de sus entradas de la manera natural. Las aristas se piensan como cables que llevan el valor 0 o 1 del vértice del que salen. Cada vértice que no es fuente toma los valores de sus aristas entrantes y les aplica el operador que lo etiqueta; el resultado viaja por las aristas que salen de él. El valor que calcula el circuito es el que calcula el vértice de salida.

Ejemplo 8.4.4 (Evaluación de un circuito)**.** El circuito de la parte (a) de la Figura 8.6 tiene dos fuentes constantes, $v_1 = 1$ y $v_2 = 0$, y tres entradas, x_1, x_2 y x_3. Si a las entradas se les dan los valores 1, 0 y 1, las compuertas de la primera fila calculan $v_6 = 0 \wedge 1 = 0$, $v_7 = \neg 0 = 1$ y $v_8 = 0 \vee 1 = 1$; las de la segunda fila, $v_9 = 1 \vee 0 = 1$ y $v_{10} = 1 \wedge 1 = 1$; y la salida vale $v_{11} = 1 \wedge 1 = 1$. La parte (b) de la figura muestra estos valores. En general, como v_9 vale siempre 1, el circuito calcula $\overline{x}_2 \wedge (x_2 \vee x_3)$, que vale 1 exactamente cuando $x_2 = 0$ y $x_3 = 1$.



(a) el circuito (b) valores con $x_1 = 1$, $x_2 = 0$, $x_3 = 1$

Figura 8.6. (a) Un circuito con dos fuentes constantes, tres entradas y una salida; se numeran sus vértices $v_1, \ldots, v_{11}$. (b) Los valores que calcula cada vértice cuando las entradas valen 1, 0 y 1; en gris, los cables que llevan un 0. La salida vale 1, así que el circuito es satisfacible.

Definición 8.4.5 (*Circuit Satisfiability*, satisfacción de circuitos)**.** Dado un circuito sobre un conjunto de variables $\{x_1, \ldots, x_n\}$, ¿existe alguna asignación de valores binarios a las variables tal que la salida del circuito valga 1?

Si existe, se dice que el circuito es satisfacible, y que esa asignación lo satisface. El Ejemplo 8.4.4 muestra que el circuito de la Figura 8.6 es satisfacible. El teorema de Cook y Levin se puede enunciar así.

Teorema 8.4.6 (Cook, Levin)**.** *Circuit Satisfiability* es $\mathcal{NP}$-completo.

No se da aquí la demostración completa, pero la idea en la que se apoya no es tan difícil de seguir. Hay que probar dos cosas.

La primera es que *Circuit Satisfiability* $\in \mathcal{NP}$, y es sencilla: el certificado es una cadena de n bits con los valores de $x_1, \ldots, x_n$, y el certificador evalúa el circuito con esos

valores, procesando los vértices en un orden topológico (Sección 3.6) de modo que cada compuerta se calcule después de sus entradas. Eso lleva tiempo lineal en el tamaño del circuito.

La segunda es que $X \leq_P$ *Circuit Satisfiability* para todo $X \in \mathcal{NP}$. Se usa el hecho de que cualquier algoritmo que toma como entrada una cantidad fija n de bits y produce una respuesta sí o no se puede representar con un circuito del tipo definido: un circuito equivalente al algoritmo, en el sentido de que su salida vale 1 exactamente en las entradas para las que el algoritmo responde sí. Además, si el algoritmo hace una cantidad de pasos polinómica en n, el circuito tiene tamaño polinómico. La justificación de esta transformación es la parte de la demostración en la que no se entra en detalle, aunque es bastante natural si se piensa que los algoritmos implementados en computadoras reales se reducen, en última instancia, a operaciones sobre compuertas $\wedge$, $\vee$ y $\neg$.

La Figura 8.7 esquematiza el argumento. Se fija el tamaño n de la entrada y se considera un algoritmo B de tiempo polinómico. La cantidad de pasos que hace B con entradas de ese tamaño está acotada por $q(n)$ para cierto polinomio q, y por lo tanto también la cantidad de memoria auxiliar que puede usar. El estado completo de la computación en un momento dado, con el programa, las entradas, la respuesta y la memoria auxiliar, se describe con una cantidad polinómica de bits. Cada paso de cómputo lee unos pocos de esos bits, opera con ellos y escribe el resultado, y se puede implementar con un circuito de una cantidad polinómica de vértices que toma los bits del estado antes del paso y produce los bits del estado después del paso. Encadenando $q(n)$ copias de ese circuito se obtiene la respuesta final para cualquier entrada de tamaño n, con un circuito de tamaño polinómico en n que, a partir de un programa que implementa B, se construye en tiempo polinómico en n. Es importante fijar la cantidad de bits de la entrada, porque refleja una diferencia básica entre algoritmos y circuitos: un algoritmo no suele tener problemas con entradas de largos distintos, mientras que un circuito tiene el tamaño de la entrada fijado en su estructura.

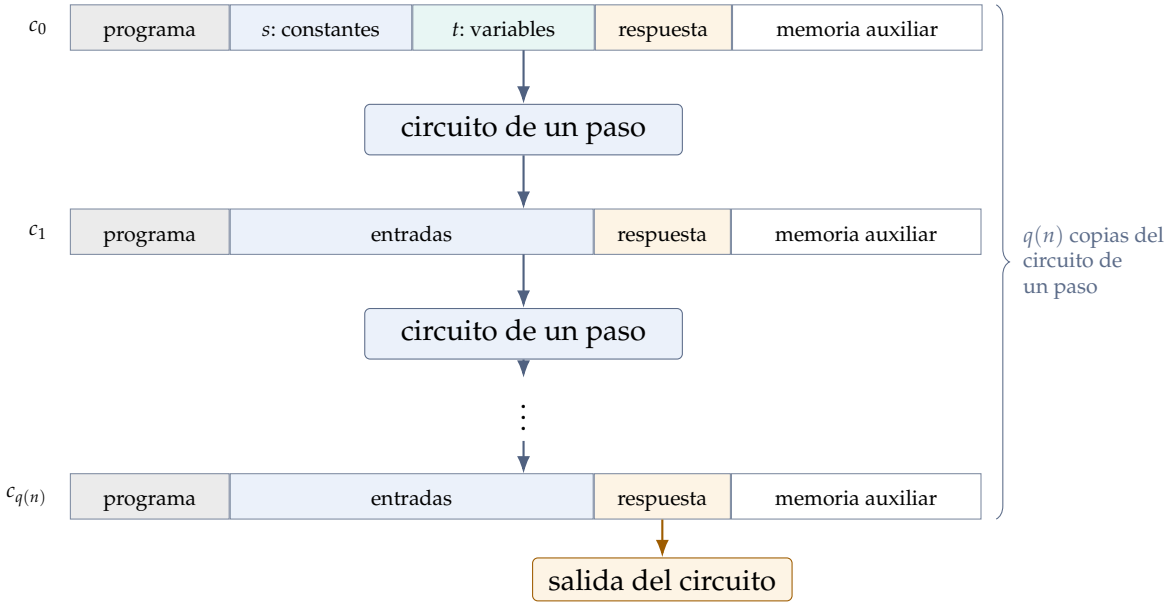


Figura 8.7. Un algoritmo B de tiempo polinómico, con entradas de largo fijo, se simula con un circuito de tamaño polinómico. Cada fila c_i son los bits del estado de la computación después de i pasos, y cada paso se implementa con un circuito de tamaño polinómico. En la fila inicial, los bits de s se fijan como fuentes constantes y los del certificado t quedan como fuentes variables.

Veamos cómo se usa esta relación entre algoritmos y circuitos para probar que $X \leq_P$ *Circuit Satisfiability*. Sea $X \in \mathcal{NP}$ arbitrario y sea B un certificador eficiente para X, con polinomio p. Dada una instancia de X representada por una cadena s de largo n, hay que decidir si existe un t de largo a lo sumo $p(n)$ tal que $B(s,t) = 1$. Como solo interesa la respuesta para esta s concreta, se ve a B como un algoritmo sobre $n + p(n)$ bits, la entrada s y el certificado t, y se lo convierte en un circuito K de tamaño polinómico con $n + p(n)$ fuentes. Las primeras n fuentes se fijan como constantes, con los valores de los bits de s, y las $p(n)$ restantes quedan como fuentes variables, que representan los bits de t y son las entradas de K. Entonces $s \in X$ si y solo si hay una manera de darles valores a las entradas de K que haga que la salida valga 1, es decir, si y solo si K es satisfacible. Así, la respuesta de la caja negra de *Circuit Satisfiability* para K es la respuesta de X para s, y se tiene $X \leq_P$ *Circuit Satisfiability*.

Ejemplo 8.4.7 (Conjuntos independientes de dos nodos como un circuito)**.** Consideremos el problema siguiente: dado un grafo G, ¿tiene un conjunto independiente de dos nodos? El problema pertenece a $\mathcal{NP}$. Veamos cómo se resuelve una de sus instancias construyendo una instancia equivalente de *Circuit Satisfiability*, siguiendo el argumento anterior.

Primero hay que fijar un certificador eficiente. La entrada s es un grafo de n nodos, que se especifica con $\binom{n}{2}$ bits: para cada par de nodos, un bit que indica si están unidos por una arista. El certificado t se especifica con n bits: para cada nodo, un bit que indica si pertenece al conjunto independiente propuesto. El certificador tiene que controlar dos cosas: que al menos dos bits de t valgan 1, y que no haya dos bits de t en 1 que correspondan a los dos extremos de una arista, según el bit correspondiente de s.

Supongamos que interesa la respuesta para el grafo G de tres nodos u, v, w en el que v está unido con u y con w. El circuito equivalente al certificador para grafos de tres nodos se esquematiza en la Figura 8.8. Una parte controla si se eligieron los dos extremos de alguna arista, calculando $(e_{uv} \wedge u \wedge v) \vee (e_{uw} \wedge u \wedge w) \vee (e_{vw} \wedge v \wedge w)$, y la otra controla si se eligieron al menos dos nodos, calculando $(u \wedge v) \vee (u \wedge w) \vee (v \wedge w)$; las dos se arman con compuertas binarias. Las aristas de G se codifican como las constantes $e_{uv} = 1$, $e_{uw} = 0$ y $e_{vw} = 1$ en las primeras tres fuentes, y las otras tres fuentes, que representan la elección de nodos, quedan como variables. Esta instancia de *Circuit Satisfiability* es satisfacible con la asignación $u = 1$, $v = 0$, $w = 1$, que corresponde a elegir los nodos u y w: efectivamente, forman un conjunto independiente de dos nodos en G.

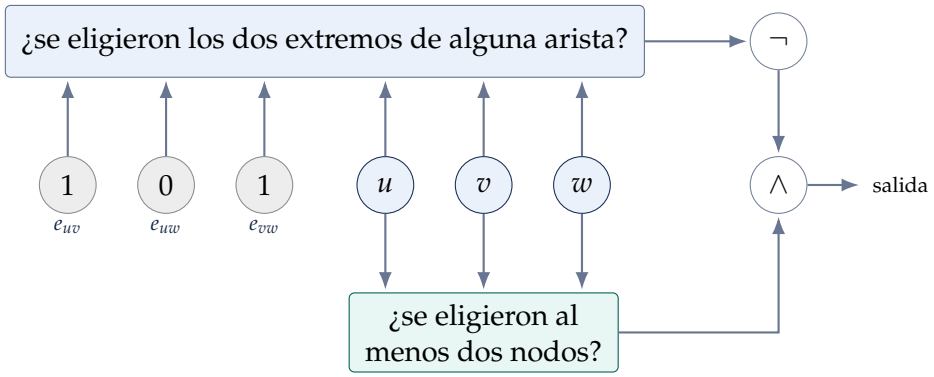


Figura 8.8. Esquema del circuito del Ejemplo 8.4.7, equivalente a un certificador para conjuntos independientes de dos nodos en grafos de tres nodos. Las tres fuentes grises codifican las aristas del grafo G; las fuentes u, v y w son las entradas y representan qué nodos se eligen. La salida vale 1 si se eligieron al menos dos nodos y no se eligieron los dos extremos de ninguna arista.

Con el Teorema 8.4.6 en la mano se abre la puerta a una comprensión mucho más

amplia de los problemas difíciles de $\mathcal{NP}$: una vez que se tiene un primer problema $\mathcal{NP}$-completo, se pueden descubrir muchos más gracias a la observación siguiente.

Proposición 8.4.8. Si Y es un problema $\mathcal{NP}$-completo y X es un problema de $\mathcal{NP}$ tal que $Y \leq_P X$, entonces X es $\mathcal{NP}$-completo.

Demostración. Como $X \in \mathcal{NP}$, solo hay que verificar la condición (ii) de la Definición 8.4.1. Sea Z un problema cualquiera de $\mathcal{NP}$. Por la $\mathcal{NP}$-completitud de Y se tiene $Z \leq_P Y$, y por hipótesis $Y \leq_P X$; por la Proposición 8.2.8, $Z \leq_P X$. □

Así, mientras que probar el Teorema 8.4.6 exigió el trabajo duro de considerar cualquier problema posible de $\mathcal{NP}$, para probar que otros problemas son $\mathcal{NP}$-completos alcanza con una reducción desde un único problema que ya se sabe $\mathcal{NP}$-completo.

8.4.2 SAT es $\mathcal{NP}$-completo

Para establecer la $\mathcal{NP}$-completitud de los problemas de las secciones anteriores hay que conectar *Circuit Satisfiability* con ellos, y lo más fácil es relacionarlo con el problema al que más se parece, SAT.

Proposición 8.4.9. *Circuit Satisfiability* $\leq_P$ SAT.

Demostración. Sea K un circuito con vértices $v_1, \ldots, v_N$. Se construye una instancia de SAT con variables $z_1, \ldots, z_N$, una por vértice, que codifican el valor de verdad que el circuito tiene en cada vértice. El conjunto de cláusulas se construye partiendo de un conjunto vacío y agregando, para cada vértice v_i que no es una fuente variable, cláusulas que dependen de su tipo. Las cláusulas están pensadas para que, si se les asignan a las variables z de las fuentes variables los mismos valores que a las entradas correspondientes del circuito, satisfacerlas obligue a que cada z_i tome el valor que calcula el vértice v_i.

- Si v_i es una fuente constante igual a 0, se agrega la cláusula de un solo término $\overline{z}_i$, y si es una fuente constante igual a 1, la cláusula z_i.
- Si v_i es de tipo $\neg$ y su única entrada viene del vértice v_j, hay que garantizar $z_i = \overline{z}_j$, y para eso se agregan las cláusulas

$$z_i \vee z_j, \qquad \overline{z}_i \vee \overline{z}_j.$$

- Si v_i es de tipo $\vee$ y sus entradas vienen de v_j y v_k, hay que garantizar $z_i = z_j \vee z_k$, y se agregan

$$z_j \vee z_k \vee \overline{z}_i, \qquad \overline{z}_j \vee z_i, \qquad \overline{z}_k \vee z_i.$$

- Si v_i es de tipo $\wedge$ y sus entradas vienen de v_j y v_k, hay que garantizar $z_i = z_j \wedge z_k$, y se agregan
$$\overline{z}_j \vee \overline{z}_k \vee z_i, \qquad z_j \vee \overline{z}_i, \qquad z_k \vee \overline{z}_i.$$
- Además, si v_i es el vértice de salida, cualquiera sea su tipo, se agrega la cláusula z_i.

Las cláusulas de cada compuerta dicen exactamente lo que se quiere. En el caso $\neg$, la primera prohíbe $z_i = z_j = 0$ y la segunda $z_i = z_j = 1$. En el caso $\vee$, la primera dice que si $z_i = 1$ alguna entrada vale 1, y las otras dos que si alguna entrada vale 1 entonces $z_i = 1$. En el caso $\wedge$, la primera dice que si las dos entradas valen 1 entonces $z_i = 1$, y las otras dos que si $z_i = 1$ las dos entradas valen 1. Se agregan a lo sumo cuatro cláusulas por vértice, así que la instancia se construye en tiempo polinómico en el tamaño del circuito.

Veamos que las dos instancias son equivalentes. Si existe una asignación a las entradas que satisface el circuito, se propagan sus valores hasta obtener el valor que calcula cada vértice, como en el Ejemplo 8.4.4, y se le asigna a cada z_i el valor que calcula v_i. Esa asignación satisface todas las cláusulas: las de cada compuerta, porque el valor de cada vértice es el resultado de aplicar su operador a sus entradas; las de las fuentes constantes, por definición; y la de la salida, porque la salida vale 1.

Recíprocamente, supongamos que una asignación satisface las cláusulas, y sean $v_{i_1}, \ldots, v_{i_m}$ las fuentes variables del circuito, asociadas a las variables $x_1, \ldots, x_m$. Se les dan a $x_1, \ldots, x_m$ los mismos valores que a $z_{i_1}, \ldots, z_{i_m}$. Recorriendo los vértices en orden topológico, las cláusulas garantizan que el valor asignado a cada z_i es el que calcula el circuito en v_i. En particular el circuito evalúa en la salida v_f al mismo valor que z_f, y como hay una cláusula formada únicamente por z_f, ese valor es necesariamente 1. La asignación a las entradas satisface el circuito. □

Ejemplo 8.4.10 (Las cláusulas del circuito de la Figura 8.6)**.** Con la numeración de la parte (a) de la Figura 8.6, las fuentes variables son v_3, v_4 y v_5, y las cláusulas que produce la reducción son las siguientes.

Vértice	Tipo y entradas	Cláusulas
v_1	constante 1	z_1
v_2	constante 0	$\overline{z}_2$
v_6	$\wedge$ de v_2 y v_3	$\overline{z}_2 \vee \overline{z}_3 \vee z_6, \quad z_2 \vee \overline{z}_6, \quad z_3 \vee \overline{z}_6$
v_7	$\neg$ de v_4	$z_7 \vee z_4, \quad \overline{z}_7 \vee \overline{z}_4$
v_8	$\vee$ de v_4 y v_5	$z_4 \vee z_5 \vee \overline{z}_8, \quad \overline{z}_4 \vee z_8, \quad \overline{z}_5 \vee z_8$
v_9	$\vee$ de v_1 y v_6	$z_1 \vee z_6 \vee \overline{z}_9, \quad \overline{z}_1 \vee z_9, \quad \overline{z}_6 \vee z_9$
v_{10}	$\wedge$ de v_7 y v_8	$\overline{z}_7 \vee \overline{z}_8 \vee z_{10}, \quad z_7 \vee \overline{z}_{10}, \quad z_8 \vee \overline{z}_{10}$
v_{11}	$\wedge$ de v_9 y v_{10}, salida	$\overline{z}_9 \vee \overline{z}_{10} \vee z_{11}, \quad z_9 \vee \overline{z}_{11}, \quad z_{10} \vee \overline{z}_{11}, \quad z_{11}$

La asignación $z_1 = 1, z_2 = 0, z_3 = 1, z_4 = 0, z_5 = 1, z_6 = 0, z_7 = 1, z_8 = 1, z_9 = 1, z_{10} = 1, z_{11} = 1$, que copia los valores de la parte (b) de la figura, satisface todas las cláusulas.

Teorema 8.4.11. SAT es $\mathcal{NP}$-completo.

Demostración. SAT pertenece a $\mathcal{NP}$: una cadena de n bits define una asignación de valores de verdad para $x_1, \ldots, x_n$, que sirve de certificado, y se puede verificar que cada cláusula se satisface en tiempo lineal en su largo. Además, por la Proposición 8.4.9, *Circuit Satisfiability* $\leq_P$ SAT. Como *Circuit Satisfiability* es $\mathcal{NP}$-completo (Teorema 8.4.6), la Proposición 8.4.8 implica que SAT es $\mathcal{NP}$-completo. □

8.4.3 Consecuencias

Una vez que SAT es $\mathcal{NP}$-completo, la cadena de reducciones de las Secciones 8.1 y 8.2, junto con la Proposición 8.4.8, da la $\mathcal{NP}$-completitud de todos los demás problemas vistos.

Teorema 8.4.12. 3SAT, *Independent Set*, *Vertex Cover*, *Set Cover* y *Set Packing* son $\mathcal{NP}$-completos.

Demostración. Todos pertenecen a $\mathcal{NP}$, como se vio en la Sección 8.3. Por la Proposición 8.4.8 alcanza con aplicar en cadena las reducciones ya probadas, desde un problema que ya se sabe $\mathcal{NP}$-completo:

- SAT es $\mathcal{NP}$-completo y SAT $\leq_P$ 3SAT (Proposición 8.2.4), así que 3SAT es $\mathcal{NP}$-completo;
- 3SAT es $\mathcal{NP}$-completo y 3SAT $\leq_P$ *Independent Set* (Proposición 8.2.6), así que *Independent Set* es $\mathcal{NP}$-completo;
- *Independent Set* es $\mathcal{NP}$-completo e *Independent Set* $\leq_P$ *Vertex Cover* (Proposición 8.1.10), así que *Vertex Cover* es $\mathcal{NP}$-completo;
- *Vertex Cover* es $\mathcal{NP}$-completo y *Vertex Cover* $\leq_P$ *Set Cover* (Proposición 8.1.12), así que *Set Cover* es $\mathcal{NP}$-completo;
- *Independent Set* es $\mathcal{NP}$-completo e *Independent Set* $\leq_P$ *Set Packing* (Proposición 8.1.15), así que *Set Packing* es $\mathcal{NP}$-completo. □

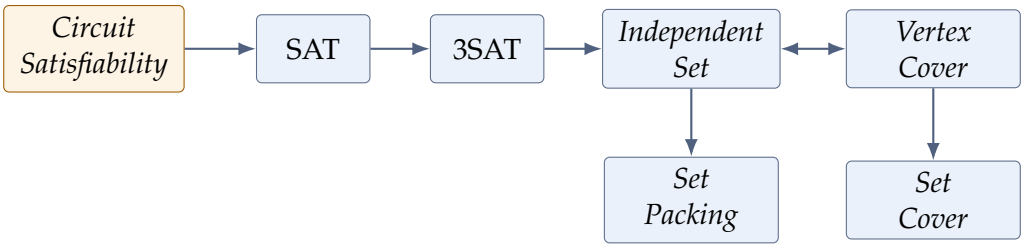


Figura 8.9. Las reducciones del capítulo: una flecha de Y a X indica $Y \leq_P X$. Partiendo de *Circuit Satisfiability* (Teorema 8.4.6), las Proposiciones 8.4.9, 8.2.4, 8.2.6, 8.1.10, 8.1.12 y 8.1.15 propagan la $\mathcal{NP}$-completitud a todos los demás problemas.

Si se acepta la hipótesis de trabajo $\mathcal{P} \neq \mathcal{NP}$, el panorama queda como en la Figura 8.10: dentro de $\mathcal{NP}$ están, por un lado, los problemas de $\mathcal{P}$, que se resuelven eficientemente, y por otro los $\mathcal{NP}$-completos, ninguno de los cuales se podría resolver eficientemente. Por la Proposición 8.4.2, las dos regiones no se tocan: un problema que estuviera en ambas probaría $\mathcal{P} = \mathcal{NP}$.

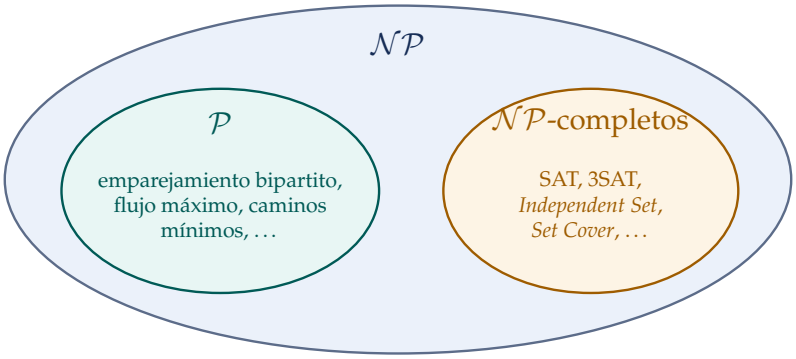


Figura 8.10. El panorama de la clase $\mathcal{NP}$ si $\mathcal{P} \neq \mathcal{NP}$. Los problemas de $\mathcal{P}$ y los $\mathcal{NP}$-completos no tienen ninguno en común; si alguno perteneciera a las dos regiones, se tendría $\mathcal{P} = \mathcal{NP}$.

8.4.4 Estrategia general para probar que un problema es $\mathcal{NP}$-completo

Como $\mathcal{P} \neq \mathcal{NP}$ se cree ampliamente, descubrir que un problema es $\mathcal{NP}$-completo es un indicio fuerte de que no se lo puede resolver en tiempo polinómico. Dado un problema nuevo X, la estrategia básica para probar que es $\mathcal{NP}$-completo es la siguiente.

1. Probar que $X \in \mathcal{NP}$.
2. Elegir un problema Y que se sepa $\mathcal{NP}$-completo.
3. Probar que $Y \leq_P X$.

Como la mayoría de las reducciones son a la Karp, el tercer paso se puede refinar: se considera una instancia arbitraria s_Y de Y y se muestra cómo construir, en tiempo polinómico, una instancia s_X de X que cumpla

(a) si s_Y es una instancia SÍ de Y, entonces s_X es una instancia SÍ de X;

(b) si s_X es una instancia SÍ de X, entonces s_Y es una instancia SÍ de Y.

Es decir, s_Y y s_X tienen la misma respuesta. Es importante no confundir la dirección: la reducción va desde el problema conocido hacia el problema nuevo. Reducir X a un problema $\mathcal{NP}$-completo solo muestra que X no es más difícil que él, lo que ya se sabe para cualquier problema de $\mathcal{NP}$.

En resumen, un problema de decisión es un conjunto de cadenas binarias, las que representan instancias SÍ; un certificado es una cadena de largo polinómico que muestra que una instancia es SÍ; y un certificador eficiente es un algoritmo que valida certificados en tiempo polinómico. Los problemas de $\mathcal{NP}$ son aquellos cuyas instancias SÍ se pueden verificar en tiempo polinómico si se dispone de un certificado, y los $\mathcal{NP}$-completos son los más difíciles de ellos. Si los problemas que se pueden verificar en tiempo polinómico también se pueden resolver en tiempo polinómico es, por ahora, una pregunta sin respuesta.

Bibliografía

Estas notas fueron elaboradas como material de estudio personal para el curso de Programación 3. El contenido sigue de cerca el libro de texto del curso.

- Kleinberg, J. y Tardos, É. *Algorithm Design.* Pearson/Addison-Wesley, 2005.